

1

CIPHER STACK: DEV

AUTHOR:

SOROUSH MALEKI

```
from pathlib import Path
from hashlib import sha256
from datetime import datetime
from collections import defaultdict
import json

class FileInventory:
    def __init__(self):
        self.files = defaultdict(dict)

    def add(self, path: Path) -> None:
        data = path.read_bytes()

        self.files[str(path)] = {
            "size": len(data),
            "hash": sha256(data).hexdigest(),
            "modified": datetime.fromtimestamp(
                path.stat().st_mtime
            ).isoformat()
        }

    def export(self) -> str:
        return json.dumps(
            self.files,
            indent=4,
            sort_keys=True
        )

    def scan_directory(root: str) -> FileInventory:
        inventory = FileInventory()

        for item in Path(root).rglob('*'):
            if item.is_file():
                inventory.add(item)

        return inventory

inventory = scan_directory("/workspace")
report = inventory.export()
Path("report.json").write_text(report)
print(
    f"[+] Indexed "
    f"{len(inventory.files)} files"
)

for name, metadata in list(
    inventory.files.items()
):
    print(
        f"{name[:40]} "
        f"{metadata['size'][:>8]} bytes"
    )
```

BUILD SYSTEMS
AUTOMATE LOGICS
ENGINEER TOOLS



Contents

Python Fundamentals	34
Variables and DataTypes	35
Basic Operations	35
Strings	35
String Split	36
String Interpolation	36
Lists	36
String/List Slices	37
Dictionaries	37
Tuples	37
Sets	38
Booleans	38
Comparison Operators	39
File I/O	39
Conditions	40
Loops	41
For Loops	41
While Loops	41
List Comprehensions	42
Inline If (Ternary)	42
Useful Built-in Operators and Functions	42
Imports	42
Range and String Multiplication	43
Iterating with Index Using range(len())	43
enumerate()	43
zip()	43
in	44
max() and min()	44
randint()	44
input()	44
Methods and Functions	44
Methods	44
Functions	45
Function Parameters	45
Return Values	46
Functions and Logic	46
Default Parameter Values	46
Placeholder Variables	47
Tuple Output of Functions	48
Returning Multiple Values	48
Practical Example: Donation Stats	48

Practical Project: Number Guessing Game	49
*args and **kwargs	50
*args — Variable Positional Arguments	50
**kwargs — Variable Keyword Arguments	51
Variable Scope	51
LEGB Rule	52
Object-Oriented Programming (OOP)	52
Defining a Class	53
map(), filter(), and Lambda Functions	53
map() and filter()	53
Lambda Functions	54
filter() with Lambda	54
Writing a Class	55
More About self	55
Class Definition and Instances	55
Class-Level vs Instance-Level Attributes	56
Classes — Advanced	56
Class Attributes	56
Inheritance	56
Polymorphism	57
Abstract Methods	58
Dunder (Magic) Methods in Python	59
The Book Class	59
__init__ and __del__	60
Working with Python Packages	60
Example Import	60
Installing Third-Party Libraries	61
Bonus: Progress Bars with tqdm	61
Packages and Modules	61
Modules vs Packages	62
Practical Project: Modules and Packages in Action	62
Directory Structure	62
The Module: mymod.py	62
The Main Program: mycode.py	62
The Package Module: jackage/mymod.py	63
Running the Project	63
The __name__ Variable in Packages and Modules	63
How __name__ Works	64
Usage Example	64

What's the Point?	65
Practical Project: Building an Installable Library Management Package	65
Directory Listing	65
Main Program: <code>main.py</code>	65
Package Configuration: <code>setup.py</code>	66
Package Contents: <code>mylibrary_pkg/</code>	67
Package Module: <code>mylibrary_pkg/library.py</code>	67
Running the Project	68
Error Handling in Python	68
Types of Errors	68
Setting Up the Example	69
<code>try</code> and <code>except</code> Blocks	69
Catching Specific Exception Types	69
<code>else</code> and <code>finally</code> Blocks	70
Keeping <code>try</code> Blocks Simple	70
Linters and Code Formatters	71
Installation	71
Example: Messy Code	71
Formatters vs Linters	72
Using <code>black</code> (Formatter)	74
Practical Project: Unit Testing with <code>unittest</code>	75
The Function to Test: <code>unittestInErrorhandling.py</code>	75
The Test File: <code>unittestInErrorhandling-test.py</code>	75
Library vs Framework	76
Running the Tests	76
Decorators: Functions as First-Class Citizens	76
Functions as Objects	77
Passing Functions as Arguments	77
Nested Functions	77
Practical Project: Writing a Decorator	78
A Simple Handler Function	78
Turning It Into a Decorator	78
The Scenario	78
From Handler to Decorator	79
Real-World Uses	80
Generators	80
<code>range</code> as a Generator-Like Object	80
Practical Project: Generators in Action	81
Step 1: Generating and Saving the Dice Rolls — <code>genDice_generatorsINaction.py</code>	81
Step 2: Reading the Dice Rolls One at a Time — <code>getDice_generatorsINaction.py</code>	81
Step 3: Using the Generator in the Game — <code>gamePy-generatorsINaction.py</code> . .	82
Running the Project	82

Practical Script: Custom Counter	82
Running the Script	83
Advanced Modules: collections	83
Counter	83
namedtuple	84
defaultdict	85
The datetime Module	85
Running the Script	86
The math and random Modules	86
The random Module	87
Other Ways to Use random	87
Practical Project: Regular Expressions (regex)	88
File: regex.py	88
Test Data: regex-notebook.txt	89
File: regex2.py	89
Regex Special Characters Reference	89
Reading and Searching the File	90
Grouping and Identifying Card Numbers	90
Compiling Patterns with re.compile	91
Character Groups and Negated Sets	91
Running the Project	91
The decimal Module	91
The sys Module	92
Reading from stdin	92
Command-Line Arguments with sys.argv	93
sys.version and sys.path	93
I/O Redirection	94
Running the Script	94
The os Module	95
Working with Directories	95
Working with Files	95
Listing Directory Contents	96
Environment Variables	96
Running System Commands	96
The subprocess Module	97
Running the Script	97
Web Scraping with requests and bs4	97
Installation	98
The requests Module	98
Extracting Data with Regex	99

Parsing HTML with BeautifulSoup (bs4)	99
Selecting Elements with <code>.select()</code>	99
Extracting Links	100
Selecting by Class and ID	100
Advanced Selectors	100
Introduction to Selenium	101
More Accurate Element Selection	101
Limitations of Static Scraping	101
What Is Selenium?	101
Selenium vs. <code>requests</code>	101
Automating Everyday Tasks	102
Practical Script: Selenium in Action	102
Installation and Setup	102
Setting Up the WebDriver	103
Navigating to a Page and Selecting Elements	103
Interacting with the Page	104
Finding and Clicking Links	104
Taking a Screenshot	104
Other Useful Features	104
Troubleshooting: geckodriver Errors	104
Bonus: Selenium IDE	105
Practical Script: Web Scraping with Selenium (Headless Mode)	105
Background	105
Headless Browser Setup	105
Loading a Page	106
Extracting Links	106
A Note on Selenium’s Capabilities	106
Working with Images — Pillow	107
Opening and Inspecting an Image	107
Reading and Writing Pixels	107
Rotating	108
Cropping and Pasting	108
Transparency and Saving	108
Practical Project: Image Steganography	108
How It Works	108
Encoder	109
Decoder	110
Notes	111
Working with CSV Files	111
fake_phone_book.csv (sample)	111
Reading a CSV File	111
Writing a CSV File	112
csvOutput.csv	112
Working with PDF Files	113

Working with Excel Files (XLSX)	113
Reading an Excel File	113
Writing an Excel File	114
JSON and APIs	114
JSON in Python	114
JSON Method Reference	115
Practical Script: Making API Calls	115
Sending Data with POST	116
Bonus: Building a Web Server with Flask	116
Basic Flask App	116
How It Works	117
We will write a simple API with Flask in JavaScript Section	117
Strings — Advanced	117
Sets — Advanced	118
Reference vs Copy	118
Set Operations	118
Itertools	119
cycle()	120
combinations()	120
combinations_with_replacement()	121
count()	121
accumulate()	122
product()	122
Virtual Environments (venv)	123
Why Do We Need Virtual Environments?	123
What Is a Virtual Environment?	124
How Virtual Environments Help	124
Creating a Virtual Environment	124
Virtual Environment Structure	125
Activating and Deactivating a Virtual Environment	125
Viewing Installed Packages	125
Creating a Requirements File	126
Installing Dependencies from a Requirements File	126
Practical: Current Time Script	126
Python Advanced	127
Data Types And Numbers	128
Converting Data Types	128
Dunder (Magic) Methods for Conversion	128
Practical Conversions	128
Operators and Type Correctness	129

Converting Number Bases	129
Defining Numbers in Different Bases	129
Converting Between Representations	130
Showing Numbers in Different Bases	130
Recap	130
Readable Number Literals	131
Logical Data	131
Logical Value of Objects in Python	131
How Python Determines Truthiness	132
Defining Truthiness for a Custom Class	132
Using the Class	132
and/or Short-Circuit Evaluation in Python	133
Strings	133
Classic Ways to Create Strings	133
Method 1: Using <code>print</code> with Multiple Arguments	134
Method 2: String Concatenation	134
Method 3: <code>str.join()</code>	134
The <code>%</code> Operator for Strings	134
Method 4 — <code>%</code> with a Tuple	135
Method 4.5 — <code>%</code> with a Dictionary	135
Method 5 — Passing the Dictionary Directly	135
The <code>.format()</code> Method for Strings	135
Method 6 — Positional Placeholders	136
Method 7 — Named Placeholders	136
Method 8 — Dictionary Unpacking with <code>**</code>	136
f-Strings	136
Variables and Self-Documenting f-Strings	137
Number Formatting	137
Inline Expressions	137
The Template Class	137
Method 10 — Template with Keyword Arguments	137
Method 11 — Template with Dictionary Unpacking	138
<code>ord()</code> and <code>chr()</code>	138
Uppercase vs Lowercase	138
Naming Techniques	139
The Underscore Variable	139
In Interactive Python Shells	139
The Out and In Dictionaries	139

The Evaluation Trap	140
Using <code>_</code> in Scripts	140
Single Underscore Naming Convention in Python	140
The Module: <code>vehicle.py</code>	141
Using an Underscore at the End	141
Using an Underscore at the Beginning	141
Importing the Module	141
What Happens Here?	142
Why Does This Happen?	142
Accessing Internal Names Explicitly	142
Key Takeaway	143
Double Underscores at the Start of Variable Names	143
Name Mangling in Python	144
Double Underscores at the Start and End	145
Logging in Python	146
Basic Logging	147
Improving the Logging Process	148
Example Output	149
Iterables	149
The <code>map()</code> Function	149
Note	150
The <code>filter()</code> Function	150
The <code>reversed()</code> Function	151
Using <code>reversed()</code>	151
Lazy Evaluation	151
Iterating with <code>reversed()</code>	151
The <code>sorted()</code> Function	151
Basic Sorting	151
Reverse Sorting	152
Sorting with a Key Function	152
More Complex Sorting	152
Sorting Within Groups	152
Tuple-Based Sorting (Multi-Level Sorting)	152
How Tuple Sorting Works	152
Key Idea	153
The <code>enumerate()</code> Function	153
Basic Iteration	153
Manual Indexing (Not Pythonic)	153
Using <code>enumerate()</code>	153
Converting to a List	153
Key Idea	153

The <code>zip()</code> Function	153
Manual Way (Not Recommended)	154
Pythonic Way: Using <code>zip()</code>	154
<code>zip()</code> Output	154
Accessing Zipped Data	154
Unzipping (Reversing <code>zip</code>)	154
Argument Unpacking (<code>*</code> operator)	155
Relationship with <code>enumerate()</code>	155
Why <code>enumerate()</code> is Better	155
Key Idea	155
<code>iter()</code> and <code>next()</code> Functions	155
<code>range()</code> and <code>slice()</code>	157
The <code>slice()</code> Function	157
Using <code>slice()</code> Instead of <code>[]</code>	158
Key Idea	158
Useful DataTypes In Python	158
<code>namedtuple</code> from <code>collections</code>	158
Using a Dictionary	158
Using Tuples with Indexes	158
Named Tuples	159
Accessing Values	159
Immutability	159
Modifying Data	159
Another Example: Point	160
Adding Namedtuples	160
<code>NamedTuple</code> from <code>typing</code>	160
Defining a <code>NamedTuple</code> Class	160
Type Hint Example	161
Creating Instances	161
Accessing Values	161
Using <code>ypstruct</code>	161
Basic Usage	161
Type and Behavior	162
Repeating Structures	162
Multiplication Shortcut	162
<code>Counter</code> from <code>collections</code>	162
Creating a Counter	163
Accessing Counts	163
Most Common Elements	163
Getting Elements Back	163
Updating Counter	163
Total Count	164
Removing Elements	164
Creating Counter in Different Ways	164

Combining Counters	164
Set-Like Operations	164
Intersection (minimum values)	164
Union (maximum values)	164
Difference	165
defaultdict from collections	165
Solution: defaultdict	165
Example with Default str	165
Using defaultdict as a Counter	166
Practical Script: Word Counter (Using defaultdict)	166
Reading Input File	166
Preprocessing Text	166
Extracting Words	166
Counting Words	166
Sorting Results	167
Output Top Results	167
OrderedDict from collections	167
Using OrderedDict	167
Moving Elements	167
Removing Last Item	167
Removing a Specific Key	168
ChainMap from collections	168
Problem	168
Solution: ChainMap	168
Accessing Values	168
Parent Maps	169
Key Idea	169
The deque Data Type	169
A Note on Time Complexity	169
Basic Usage	169
Extending a Deque	170
Inserting and Removing by Value	170
Reversing and Rotating	170
Searching by Index	170
Single-Type Arrays	171
MappingProxyType	172
Useful Python Functions	173
all() and any() — Built-in Boolean Checks	173
min() and max() — Finding Extremes in a Collection	173
Boolean Collections	174
The key Parameter	174

Applying key to Dictionaries	174
sum() and len() — Totalling and Counting	175
The start Parameter	175
Summing a Range	175
Floating-Point Precision with <code>math.fsum()</code>	175
Hashing in Python	175
Password Comparison with Hashes	176
Hashing Tuples	176
Custom <code>__hash__()</code> in Classes	176
Copying Variables — Value Types vs Reference Types	177
Primitive (Value) Types	177
Reference Types	177
Shallow Copy	177
Deep Copy	178
<code>==</code> vs <code>is</code> in Python	178
Accurate Calculations with decimal	178
The Decimal Type	179
Configuring Global Precision with <code>getcontext()</code>	179
Temporary Precision with <code>localcontext()</code>	179
Comprehensions	180
List, Set, and Dictionary Comprehensions	180
List Comprehension	180
Set Comprehension	180
Dictionary Comprehension	180
Generator Comprehensions	181
Consuming a Generator with <code>next()</code>	181
Memory Efficiency	181
Itertools	182
Infinite Iterators — itertools	182
<code>count</code>	182
<code>cycle</code>	182
<code>repeat</code>	183
zip_longest — Zipping by the Longest Iterable	183
<code>zip_longest</code>	183
accumulate — Running Aggregations	184
Default Behaviour — Running Sum	184
The initial Parameter	184
Custom Operations	184
Chaining Iterables with <code>itertools.chain</code>	185

Flattening Lists of Lists	185
Filtering Iterables with <code>filter</code>, <code>filterfalse</code>, <code>compress</code>, <code>dropwhile</code>, and <code>takewhile</code>	186
<code>itertools.filterfalse</code>	186
<code>itertools.compress</code>	186
<code>dropwhile</code> and <code>takewhile</code>	187
Working with Consecutive Pairs Using <code>itertools.pairwise</code>	187
Alternative Implementation with <code>zip</code>	187
Sliding Windows of Three Elements	188
Applying Functions to Argument Tuples with <code>itertools.starmap</code>	188
Cartesian Product with <code>itertools.product</code>	188
Practical Example: Generating Combinations	189
Permutations and Combinations with <code>itertools</code>	189
Permutations	189
Combinations	189
Combinations with Replacement	190
Functions In Python	190
Functions in Python: Definition and Properties	190
Using <code>map</code> with Functions	191
Applying Multiple Functions to Data	191
Passing Functions as Arguments	191
Functions Returning Functions and Closures	192
Returning Functions (Factory Pattern)	192
Checking Callability	192
Making Any Object Callable	192
Simple Function Wrapper (Introduction to Decorators)	193
Callable Classes Using <code>__call__</code>	193
Variable Positional Arguments (<code>*args</code>)	194
Variable Keyword Arguments (<code>**kwargs</code>)	194
Separating Positional and Keyword Arguments	195
Positional-Only Arguments (<code>/</code>)	196
Keyword-Only Arguments (<code>*</code>)	196
Generators In Python	196
Generator Functions with <code>yield</code>	196
Real-World Generator Example (Similar to <code>range</code>)	197
Fibonacci Sequence Generator	198
Generator Expressions (Comprehensions for Generators)	198

Class-Based Generators	199
Memory Usage and Performance: Lists vs Generators	199
Decorators In Python	200
Decorators Introduction	200
Decorators for Functions with Arguments	201
Multiple Decorators	202
Decorators with Parameters	203
Using the <code>decorator</code> Module	204
Functools	204
Caching Function Outputs with <code>functools.cache</code>	204
LRU Cache with <code>functools.lru_cache</code>	205
Partial Functions with <code>functools.partial</code>	205
Another Example	206
Reducing Iterables with <code>functools.reduce</code>	206
Example	206
Combining <code>reduce()</code> with <code>partial()</code>	207
Enum Classes	207
Enum Classes Definition	207
Defining an Enumeration	207
Accessing Enum Members	208
Adding Operators to an Enum Class	208
Using <code>auto()</code> in Enum Classes	209
Unique Enum Values	209
Object Oriented Programming	210
Instance Methods	210
Class Methods and Static Methods	211
Class Methods	211
Static Methods	211
Usage	212
Abstract Classes	212

Properties with the @property Decorator	212
Usage	213
Python Tools	215
Real World Projects: Freecollapse — Payload Generator	216
Imports	216
Functions	216
Script	216
How to Use	219
Real World Projects: hiKrawler — Passive URL Collector	220
Imports	220
Functions	220
Script	220
How to Use	223
Real World Projects: fav — Favicon Hash Generator	223
Imports	223
Functions	224
Script	224
How to Use	224
JavaScript Fundamentals	225
JavaScript and the DOM — Script Placement	226
JavaScript Basics — Variables	226
JavaScript Basics — More About Variables	226
var, let, and const	226
Why Not Just Use var?	227
const	227
JavaScript Basics — Booleans	227
JavaScript Basics — Arrays	228
JavaScript Basics — Objects	228
Browser Console Basics	229
JavaScript Functions	230
Defining and Calling a Function	230
Working with Form Input	230
Template Literals	231
Returning Values from Functions	231
More Ways to Define Functions	231
Function Expressions	232
Arrow Functions	232

Basic Arrow Function Syntax	232
Short Arrow Function Syntax	232
Dynamic Strings with Template Literals	232
Updating Page Content Dynamically	233
Conditional Statements	233
Comparison Operators	233
Comparison Operator Reference	234
Ternary Operator	234
Checking Existence in Conditions	234
A Deeper Look into let and var	235
Comparison with Python	235
Global Variables and Blocks	235
Introduction to the Document Object	236
Accessing Elements in the HTML Document	236
Changing Content Safely with a Function	236
Changing Element Styles	237
Changing Classes Dynamically	237
Explanation	238
Adding Event Listeners	238
More Events and the Event Object	239
Practical Project: Word Counter	239
HTML Structure	239
JavaScript Logic	241
Handling Input Events	241
Splitting Text into Words	242
Design Note	242
String Methods in Depth	242
Length	242
Includes	242
Case Changing	242
Trim	243
Substring	243
Split	243
Converting to Strings	243
Numbers in Depth	243
Checking Types	244
Number Quirks	244
toFixed	244
Operator Priority	244
Converting to Numbers	244

Arrays in Depth	245
Push	245
Includes	245
forEach	245
Booleans in Depth	245
Checking Conditions	246
Converting Other Types to Boolean	246
Arrays of Objects	246
Accessing Values	247
Adding a New Object	247
JSON Compared to JavaScript Objects	247
Nested Objects	247
Simple Math Operators	248
Shorthand Operators	248
Refactoring Functions	249
What Is Refactoring?	249
A Simpler Example Of return KeyWord	250
Adding HTML to Elements	250
Methods for Inserting HTML	251
Hoisting	251
Timers	252
setTimeout	252
setInterval	253
Loops	253
Sample Data	253
Normal for Loop	253
for...of Loop	254
for...in Loop	254
Nested Loops	254
while Loop	255
do...while Loop	255
Fetching Data from a Server	255
Sample Data	255
Using fetch	256
Sending a POST Request	256
Sending the Actual GET Request	257
Using XMLHttpRequest (XHR)	258
Sending a POST Request with XHR	259
Showing API Data on the Page	259

Working with Nested API Data	261
Sample Data	261
HTML Structure	261
HTTP Methods for CRUD Operations	262
Fetching and Displaying the Data	262
Introduction to the DOM	263
HTML Structure	263
Accessing and Modifying Elements	263
Generating Page Content with JavaScript	264
Working with Events: The Target Property	264
Listening for Clicks on a Container	265
Working with the Target Property	265
Practical Project: Feedback Board (CorpComment)	265
HTML Structure	266
JavaScript	267
Flow Summary	272
Methods Used	272
Arrays in More Depth	273
push	273
forEach	273
pop	274
unshift	274
shift	274
splice	274
slice	274
concat	274
Negative Indexes	275
Reference Behavior of Arrays	275
map	275
join	276
some	276
every	276
JavaScript Arrays — find() and filter()	276
JavaScript Objects In More Depth	277
JavaScript — Callback Functions and Methods	277
Callback Functions	277
Methods	278
JavaScript — Default Parameter Values	278
JavaScript — The this Keyword in Objects	278
this in JavaScript vs self in Python	279

JavaScript — Destructuring	279
JavaScript — Spread and Rest Operators	280
Spread Operator (...)	280
Primitives and Reference Values	281
Reference Values	281
undefined vs null	282
Practical Use Case	282
Short-Circuiting with && and 	282
Short-Circuit Evaluation	283
Asynchronous JavaScript	283
What Does “Asynchronous” Mean?	283
A Real-World Analogy	283
Why Is This Useful?	284
Working with Promises	284
Why Does <code>fetch()</code> Return a Promise?	285
Execution Order	285
Async/Await Syntax	285
How to Use <code>async</code> and <code>await</code>	286
Error Handling	286
Practical Project: Async/Await with UI Feedback	287
Review: <code>for...of</code> and <code>while</code> Loops	288
<code>while</code> Loop	288
Iterating Over a String	289
Sets and Maps	289
Set	289
Map	290
Iterating Over a Map	290
Practical Project: JavaScript Modules	291
HTML Setup	291
Exporting.js	292
Default Export	292
Importing.js	292
Switch/Case Statements	293
The Window Object	294
URL-Related Properties	295
Navigating with <code>location.href</code>	295
Reading Query String Parameters	295
Math Functions	295

Rounding Down	296
Rounding Up	296
Rounding to the Nearest Integer	296
Power	296
Square Root	296
Absolute Value	296
Random Numbers	296
Minimum and Maximum	297
Date and Time	297
Getting Date Components	297
Setting Date Components	297
Getting the Current Date and Time	298
Object-Oriented Programming: From Objects to Classes	298
The Problem: Repeating Objects	298
Classes and Inheritance	299
Defining a Class	299
Creating Instances	300
Inheritance	301
Using the Subclass	302
Prototypes	302
Using a Prototype-Based Constructor	302
Built-In Prototypes	303
Promises	303
Introduction to Promises	303
Promises with Async/Await	304
Why and Where to Use Async Code	304
Promise.all and Promise.any	305
A Practical Example: Promise.all with Async Functions	306
The Call Stack and Event Loop	307
What Happens Step by Step	308
The Role of the Web API, Queue, and Event Loop	308
Expected Output Order	308
Regular Expressions: Validation and Matching	309
Flags	309
Testing for a Match	309
Extracting the First Match	309
Matching All Occurrences	309
Useful Regex Patterns	309
Optional Chaining	310
Throwing Errors	311
The Sort Method	311

State in JavaScript	312
Practical Project: Flashlight State	312
The State File	312
The Controls File	312
The HTML Entry Point	313
Vanilla JavaScript Router	314
Practical Project: Vanilla JS Router	314
Navigation	314
The Routes Object	314
Rendering Functions	315
The Router Function	315
Event Listeners	316
Web Server Requirement	316
Local Storage	316
Storing Data	317
Reading Data	317
Removing Data	317
Counting Stored Items	317
postMessage API	317
How It Works	318
Practical Project: postMessage in Action	318
Parent Page	318
Child Page	318
The Vulnerability	319
Recap: Some Real World Used JavaScript Built-in Methods and Functions	320
Array Methods	320
String Methods	320
DOM Methods	320
Async and Browser APIs	321
Practical Script: Flask Job API Server	321
As I promised in Python section, Im going to wite a simple API witch Flask.	
This can be a useful intro and guide to write Backends	321
Endpoint 1 — List Jobs: GET /data.json	322
Endpoint 2 — Single Job: GET /data.json/<job_id>	322
Full Script	323
data.json (Sample)	325
Recap: Python Built-ins and Patterns Used	325
JavaScript Additional Modules	327
JavaScript — Pop-Up Windows	328
JavaScript — Type Casting: String to Number	328
JavaScript — Events	328

JavaScript — Error Handling	328
JavaScript — The DOM	328
Adding and Removing Elements	329
JavaScript Objects and Prototypes	330
Some String Methods	330
Using Prototypes	330
Browser Object Model (BOM)	331
Window and Screen Information	331
Browser History	331
Browser Information	331
Opening and Closing a Window	331
Interval	331
Pause and Resume the Timer	332
JavaScript Tools	333
Real World Projects: G-dorking — Google Dork Generator	334
Script	334
How to Use	335
Real World Projects: JS Path Extractor — jscrawl	335
How It Works	335
Script	335
How to Use	336
Bash Fundamentals	337
Bash Aliases	338
Shells and Shell Scripting	338
A Basic Shell Script	339
Adding Scripts to PATH	339
Shebang	339
Running a Script	340
Variables in Bash	340
Variable Expansion and Quoting	341
Variable Expansion	341
Command Substitution	341
Arithmetic Expansion	341
Brace Expansion	342
Parameter Expansion	342
Globbing (Pathname Expansion)	342

Variables and Quoting	342
Quoting Mechanisms	343
Environment Variables and Shell-Local Variables	343
Environment Variables	344
Shell-Local Variables	344
Scope and Exporting Variables	344
Getting User Input	345
Practical Script: Packet Dropper	345
Positional Parameters	346
Common Positional Parameters	346
Using Positional Parameters	346
Default Values	347
Practical Script: File Compressor	347
protectFromHackers.sh	347
Special Parameters	348
More on Exit Codes	348
The Difference Between \$@ and \$*	348
Intro to Bash Expansions	348
Brace Expansion	349
Brace Expansion Project: Generating Log Directories	349
Tilde Expansion	350
Variable Expansion	350
Command Substitution	351
Arithmetic Expansion	351
Process Substitution	352
Word Splitting	353
Globbering (Filename Expansion)	353
Intro to Shell Operation in Depth	354
Tokenization	354
Command Identification	355
Quote Removal	356

Redirections	357
Shell Operations Recap	357
Conditions	358
Nested Conditions	359
String Comparisons	360
Checking if a String Contains a Substring in Bash	360
Checking Network Connectivity Using Ping in Bash	360
Using the <code>case</code> Statement in Bash	361
Sending Signals to a Process Using Bash	361
Bash Menues and Select Statement	362
Adding Logic to a Bash <code>select</code> Menu	363
Bash System Administration Menu Script	363
Chaining Commands	365
Semicolon ;	365
Ampersand &	365
Logical AND &&	366
Logical OR 	366
Practical Script: DoS Attack Without Root Permission — Fork Bomb	366
For Loops	366
Iterating Over Command Output	367
Iterating Over a Variable	367
C-Style For Loop	367
While Loops	367
Practical Script: Process Monitor	368
While Loops — Using Commands as Conditions	369
File Descriptors	369
Practical Script: Bulk Packet Dropper	370
Using a For Loop	370
IFS and the Read Command	370
Using a While Loop	370
IPS.txt	371
Until Loops	371

Breaking Loops	371
Practical Example: Ping Monitor with Break	372
Continue in Loops	372
Introduction to Bash Arrays	372
Declaring and Accessing Indexed Arrays	373
Adding and Modifying Elements	373
Explicitly Declaring an Array	373
Removing Elements	374
Arrays In Depth	374
Appending to an Array	374
Slicing Arrays	374
Associative Arrays	375
Read-Only Arrays	375
Removing Elements and Arrays	375
Random Numbers in Bash	376
Word Splitting and Arrays	376
users.txt	376
Without Quotations	376
With Quotations	377
Error Handling in Bash	377
Method 1: Parameter Expansion with ?	377
Method 2: Inspecting the Exit Code	377
Method 3: Set Options for Strict Error Handling	378
The readarray Command	378
daysOfWeek.txt	379
Iterating Over Arrays	379
Practical Script: System Account Creation	380
users.txt	380
systemAccountCreationProject.sh	380
Introduction to Bash Functions	381
Passing Arguments to Functions	381
Return Values	382
Variable Scope in Functions	382
Functions Recap	383
Defining and Calling Functions	383
Parameters	383
Variable Number of Arguments	383
Return Values	384
Printing a Function Definition	385

Bonus: AWK basics	385
Fields and Built-in Variables	386
-F : Field Separator	386
-v : Pass a Variable	386
Patterns and Filtering	386
BEGIN and END	387
Useful One-liners	387
Bonus: jq Parsing	387
Basics	387
Arrays	388
Build New Objects / Arrays	388
map and select	388
-arg : Pass Shell Variables to jq	388
Useful One-liners	389
Bash Tools	390
Real World Projects: nice_katana — Active Web Crawler	391
Dependencies	391
Script	391
How to Use	392
Real World Projects: param-maker — Query String Builder	392
Script	392
How to Use	393
Real World Projects: qs2trash — Query String Stripper	393
Script	393
How to Use	393
Real World Projects: wlist_maker — Wordlist Builder	394
Script	394
How to Use	394
Real World Projects: dns-brute-full — Full DNS Bruteforce Pipeline	394
Dependencies	395
Script	395
How to Use	395
Real World Projects: get-asn-details — ASN Lookup Tool	396
Script	396
How to Use	397
Real World Projects: get-certificate-improved — TLS Certificate Inspector	397
Script	397
How to Use	398
Real World Projects: get-chaos-data — Chaos Dataset Downloader	398
Script	399
How to Use	399

Real World Projects: get-ip-asn — IP to ASN Lookup	399
Script	399
How to Use	400
Real World Projects: get-ptr — PTR Record Lookup	400
Script	400
Real World Projects: httpx-full — HTTP Prober	400
Dependencies	400
Script	400
How to Use	401
Real World Projects: crtsh — Certificate Transparency Subdomain Finder	401
Dependencies	401
Script	401
How to Use	402
Real World Projects: OTX — Passive DNS Lookup via AlienVault	402
Script	403
How to Use	403
Real World Projects: HackerTarget — Subdomain Lookup	403
Script	403
How to Use	404
Real World Projects: SecTrails — Subdomain Lookup via SecurityTrails	404
Script	404
How to Use	404
Real World Projects: wayback — Wayback Machine URL Extractor	404
Script	405
How to Use	405
Real World Projects: split-urls — Split URL Lists by Hostname	405
Script	405
How to Use	406
Go Fundamentals	407
Introduction to Go	408
What Kind of Language Is Go?	408
Go Use Cases	408
Programming Language Fundamentals	408
Interpreters and Compilers	408
Dynamic vs. Static Typing	408
Practical Script: Hello World in Go	409
File Structure	409
Importing Packages	409
The Main Function	409

Running and Building	409
Binary Fundamentals	409
Number Bases	410
Binary	410
Bits and Bytes	410
Signed Integers	411
Two's Complement	411
Range of a Signed Byte	411
Comments in Go	411
Core Data Types	412
Integers	412
Floats	412
Byte	412
Rune	412
Bool	412
String	412
Nil	413
Declaring Variables	413
Implicit Assignment in Go	413
Explicit vs. Implicit Assignment	413
When to Use Each	414
Checking a Variable's Type	414
Forcing a Specific Type with Implicit Assignment	414
Console Output in Go	415
The <code>fmt</code> Package	415
<code>fmt.Println</code>	415
<code>fmt.Printf</code>	415
<code>fmt.Sprintf</code>	416
<code>fmt.Sprint</code>	416
Arithmetic Operators in Go	417
Importing Multiple Packages	417
Arithmetic Operators	417
Type Mismatch in Operations	417
Integer Division	417
The <code>math</code> Package	418
String to Integer Conversion with <code>strconv</code>	418
Conditions in Go	419
Comparison Operators	419
Logical Operators	419
If / Else If / Else	420
Switch Statements in Go	420
Basic Switch	420
Matching Multiple Values in One Case	421
Loops in Go	422

The <code>for</code> Loop	422
While-Style Loop	422
Looping Through a String	423
How Strings Work in Go	423
Modifying a Character in a String	423
ASCII vs UTF-8: Why It Matters for Looping	423
Range Syntax for UTF-8 Strings	424
Arrays in Go	425
What Is an Array?	425
Declaring Arrays	425
Multi-Dimensional Arrays	425
Accessing and Modifying Elements	426
Array Length	426
Looping Through Arrays	426
Arrays Are Copied When Passed to Functions	426
Slices in Go	427
What Is a Slice?	427
Creating a Slice from an Existing Array	427
The Three Properties of a Slice	428
Declaring a Slice Without an Existing Array	428
Appending to a Slice	428
Creating a Slice with <code>make</code>	429
Iterating Over a Slice	429
Slices and Functions	429
Maps in Go	430
Declaring and Initialising Maps	430
Adding and Overwriting Entries	431
Deleting an Entry	431
Checking if a Key Exists	431
Practical Example	431
Functions in Go	433
Basic Function Syntax	433
Multiple Return Values	433
Functions as Parameters	433
Anonymous Functions	433
Returning Functions	434
Variadic Functions	434
Named Return Values	435
Structs in Go	436
Defining a Struct	436
Initialising a Struct	437
Accessing and Modifying Fields	437
Passing a Struct to a Function	437
Methods on Structs	437
Embedded Structs	438

Modules and Packages in Go	440
Project Structure	440
Initialising a Module	440
Exported Names	440
Using a Package	441
Interfaces in Go	441
Defining an Interface	441
Implementing an Interface	442
Using an Interface as a Type	442
Interfaces Enable Generic Behaviour	443
Slices of Interface Types	443
Error Handling in Go	444
Runtime Errors and Panics	445
The <code>defer</code> Keyword	445
Recovering from a Panic	445
Returning Errors from Functions	446
Generics in Go	447
The Problem Generics Solve	447
Defining a Generic Function	447
Multiple Type Parameters	448
Using an Interface as a Type Constraint	448
The <code>type</code> Keyword	449
Custom Generic Types	449
Type Aliases	450
Pointers and References in Go	451
How Variables Are Stored in Memory	451
Why Slices Behave Differently	452
The <code>&</code> and <code>*</code> Operators	452
Passing Pointers to Functions	452
Pointer Receivers on Struct Methods	452
User Input in Go	454
Method 1: <code>fmt.Scanln</code>	454
Method 2: <code>bufio.Reader</code>	454
Method 3: Command-Line Arguments	455
Method 4: Reading from a Unix Pipeline	455
Threading and Concurrency	456
How a CPU Executes Operations	456
Processes and Threads	456
Thread Scheduling	457
Concurrency vs Parallelism	457
When Multi-Threading Matters	457
Go Routines	458
What Are Go Routines?	458
The <code>time</code> Package	458

The Worker Functions	458
Launching Go Routines in Main	458
Go Routines — Part 2: Channels	459
The Problem with <code>time.Sleep</code>	459
Why You Cannot Use a Return Value from a Go Routine	459
What Is a Channel?	460
Passing a Channel to a Go Routine	460
Deadlocks	460
Receiving from a Channel (Blocking Code)	460
Recap	460
Go Routines — Part 3: Multiple Go Routines on a Single Channel	461
Receiving from Multiple Go Routines	461
How a Single Receive Works	461
Waiting for All Go Routines	461
The <code>add</code> Function	462
Go Routines — Part 4: Enforcing Predictable Order	462
The Problem with Unpredictable Order	462
Interleaving Launch and Receive	463
Trade-offs	463
Go Routines — Part 5: Multiple Channels and the <code>select</code> Statement	464
Using Multiple Channels	464
The Limitation of Sequential Receives	464
The <code>select</code> Statement	464
Collecting All Results with a Loop	465
Go Routines — Part 6: Directional Channels	466
What Are Directional Channels?	466
Nested Go Routines and Channel Synchronisation	466
Go Routines — Part 7: Buffered Channels	466
Unbuffered Channels and Blocking Sends	466
Buffered Channels	467
Blocking Rules for Buffered Channels	467
Go Routines — Final Part: Mutexes and Wait Groups	468
The Problem: Race Conditions	468
The <code>Counter</code> Struct	468
Fixing the Race Condition with a Mutex	468
Waiting for Go Routines: Three Approaches	469
Wait Groups	469
General Explanation:	470
Go — What’s Next?	471
PHP Fundamentals	472
What is PHP?	473

Getting Started with PHP	473
File Structure	473
Comments	473
Output and Semicolons	473
PHP and HTML	473
Variables in PHP	474
Data Types	474
Output and String Interpolation	474
Type Casting	475
Inspecting Variables with <code>var_dump</code>	475
Arithmetic Operators in PHP	475
Variables in PHP	475
Arithmetic Operators	476
Increment and Decrement	476
Operator Precedence	477
GET and POST in PHP	477
Superglobal Arrays	477
GET Requests	477
POST Requests	477
The HTML Form	478
Reading Request Data	478
Math Functions in PHP	479
Rounding and Absolute Value	479
Power and Square Root	479
Min, Max, and Pi	479
Random Numbers	480
If Statements in PHP	480
Basic If / Elseif / Else	480
Boolean Conditions	481
Logical Operators in PHP	481
AND	481
OR	481
NOT	482
Switch Statements in PHP	482
Basic Switch	482
Using the <code>date</code> Function	482
For Loops in PHP	483
Iterating Over a String	483
While Loops in PHP	484
Arrays in PHP	484
Modifying Elements	484

Array Functions	484
Iterating with <code>foreach</code>	485
Counting Elements	485
Associative Arrays in PHP	485
Iterating with <code>foreach</code>	486
Modifying and Adding Entries	486
Removing Elements	486
Extracting Keys and Values	486
Flipping, Reversing, and Counting	487
isset and empty in PHP	487
Basic Usage	487
Checking GET Parameters Before Use	487
Radio Buttons in PHP	488
The HTML Form	488
Handling the Submission	488
Checkboxes in PHP	489
The HTML Form	489
Handling the Submission	489
Functions in PHP	490
Defining and Calling a Function	490
Return Values and Type Declarations	490
String Functions in PHP	491
Case Conversion and Whitespace	491
Replacing Characters	492
Reversing and Shuffling	492
Comparing, Measuring, and Searching	492
Extracting Substrings	492
Splitting and Joining	492
Output	493
Sanitizing and Validating User Input in PHP	493
The HTML Form	493
Sanitization vs. Validation	493
Reading Input with <code>filter_input()</code>	494
Sanitizing the Submitted Values	494
Validating the Sanitized Email	495
Including Files in PHP	495
header.html	495
footer.html	495
index.php	496
about.php	496
locations.php	496
The <code>\$_SERVER</code> Superglobal in PHP	497

Iterating Over \$_SERVER	497
Useful Keys	497
Cookies in PHP	498
Setting Cookies	498
Reading Cookies	498
Setting Other Headers	498
Sessions in PHP	499
index.php — Starting a Session	499
home.php — Resuming a Session	499
Password Hashing in PHP	500
Hashing a Password	500
Verifying a Password	500
Error Handling and Database Connection in PHP	501
Connecting to a Database	501
Establishing the Connection	501
Catching Errors	502
Checking the Connection	502
Inserting Data into a Database in PHP	502
Including the Database Connection	502
Preparing the Data	502
Writing and Submitting the Query	502
Closing the Connection	503
Retrieving Data from a Database in PHP	503
Including the Database Connection	503
Preparing the Query	503
Writing and Submitting the Query	504
Reading the Result	504
Closing the Connection	504
Object-Oriented Programming in PHP	504
Defining a Class	505
Creating Objects	505
Inheritance	505
Static Properties and Methods	506
Magic Methods	506
Dangerous Functions in PHP	507
Command Execution	507
Code Injection	507
File Inclusion	508
File Operations	508
Serialisation	509
Network Functions	509
Serialisation in PHP	509

Why Unsafe Deserialisation Is Dangerous	509
Recognising Serialised Data	509
Exploitation Approach	510
JSON in PHP	510
Decoding JSON	510
Creating a Simple Object with stdClass	510
Sending and Receiving JSON Between JavaScript and PHP	511
How PHP Receives Request Data	511
Practical Script: Building an API with PHP	511
Other Referenced Files In The Book	514
Nuclei template: SSL.yaml	515
Nuclei template: x9-xss-matcher.yaml	515
Resolvers.txt:	516
Frida Script: linksRecon.js	516
Frida Script: sslpinBypass.js	516
Target Mapping Template	517
mapping website	517
answer these questions	517
what is app used for?	517
does the app have a certain threat model?	517
how does the app pass the data?	517
how does the app handle users?	518
have there been past sec vulnerabilities?	518
does the app use third parties?	518
technologies?	518
authentication	518
findings	518

Python Fundamentals

Python is one of the most powerful and versatile programming languages in modern development.

It is widely used in cybersecurity, automation, scripting, and backend development. Many offensive and defensive security tools are built with Python due to its simplicity, rich ecosystem, and rapid development capabilities.

In this section, we cover the core fundamentals of Python that form the base for writing scripts, automating tasks, and understanding more advanced concepts later in the book.

Variables and DataTypes

A **variable** is a **named** box that holds a value in memory. When you write `x = 10`, Python creates that box, labels it `x`, and puts 10 inside.

int — whole numbers, positive or negative, no limit on size.

```
age = 25
temperature = -3
big = 999999999999
```

float — numbers with a decimal point.

```
height = 5.9
pi = 3.14159
negative = -0.5
```

str or **String** — any text, always inside quotes (single or double).

```
name = "alice"
message = 'hello world'
empty = ""
```

bool — only two possible values: `True` or `False`. Used for conditions and logic.

```
is_admin = True
is_logged_in = False
```

None — represents the absence of a value. Not zero, not empty string — literally nothing.

```
result = None
```

list — an ordered collection of values. Can hold any type, even mixed.

```
numbers = [1, 2, 3]
mixed = [1, "hello", True]
```

dict — key-value pairs, like a mini database.

```
user = {"name": "alice", "age": 25}
user["name"] # "alice"
```

Basic Operations

Strings

A string is just a sequence of characters — like a word or sentence stored in a variable.

```
s = "hello"
# 01234 ← each character has a position, starting from 0
```

Multi-line strings with special characters:

```
"""anything"""
```

String Split

Strings can split up and make Lists

```
likes = "dog,cat,food,computer,code,hack,tech"
likes.split(",")
# ['dog', 'cat', 'food', 'computer', 'code', 'hack', 'tech']
```

String Interpolation

Method 1: C-style formatting

```
print("hello %s %s" % (name, famname))
# hello Sora Mlk
```

Method 2: f-strings (modern, preferred)

```
print(f"hello {name}!")
# hello Sora!
```

Method 3: format() method

```
print("hello {} {}".format(name, famname))
# hello Sora Mlk

print("hello {1} {0}".format(name, famname))
# hello Mlk Sora

print("hello {esm} {famil}".format(esm=name, famil=famname))
# hello Sora Mlk

print("hello {esm} {famil}. you are {sen:1.1f} years old".format(esm=name, famil=famname, sen=23.2))
# hello Sora Mlk. you are 23.2 years old
```

Lists

Lists are mutable and can hold values of different data types.
For example

```
a = [1, 2, 3, 4, 5]
```

Method	Description
.append(value)	Adds a value to the end of the list
.clear()	Removes all elements
.insert(index, value)	Inserts a value at a specified index
.count(value)	Counts occurrences of a value in the list
.pop(index)	Removes and returns an element — defaults to the last element
.sort()	Sorts the list in place
.remove(value)	Removes the first occurrence of a value

Method	Description
<code>.reverse()</code>	Reverses the order of the list

String/List Slices

```
variable[from:to:step]  # standard slice
variable[from:]         # from index to end
variable[::-1]         # reverse order
```

Dictionaries

Dictionaries are mutable and store key-value pairs using curly braces. Values can be of any type, including lists or other dictionaries.

```
pr = {'apple': 10, 'banana': 20}
pr["banana"]
# 20
```

Nested dictionaries:

```
grade = {
    'sora': {
        'math': 20,
        'programming': 18
    },
    'dora': {
        'math': 20,
        'programming': 20
    }
}

grade['sora']['math']
# 20
```

Method	Description
<code>.get(key, default)</code>	Returns the value for a key, or a default if the key does not exist
<code>.keys()</code>	Returns all keys
<code>.values()</code>	Returns all values
<code>.items()</code>	Returns all key-value pairs

Tuples

Tuples are similar to lists but use parentheses and are immutable.

```
result = ('sora', 17)
name, grade = result
name
# 'sora'
grade
# 17
```

Sets

Sets do not allow duplicate values and do not preserve insertion order.

```
myset = set()
myset.add(1)
myset.add(2)
# {1, 2}
```

Only simple, hashable types can be added — lists cannot be added to a set.

```
myset = {1, 2}
yourset = {2, 3}

myset.union(yourset)
# {1, 2, 3}

myset.intersection(yourset)
# {2}
```

A set can be created directly from a list — duplicates are removed automatically:

```
animals = set(['dog', 'cat', 'dog', 'turtle'])
# {'cat', 'dog', 'turtle'}
```

Sets also support `.pop()` and `.remove()`.

Booleans

Booleans represent `True` or `False` and are used in conditions throughout Python:

```
average = 10
average > 12
# False
```

Every decision in a program is ultimately based on a condition that evaluates to `True` or `False`.

Logical operators:

```
and
or
not
```

It is good practice to use parentheses for clarity:

```
(height > 180 and gender == "male") and not somethingSpecific
```

Python also supports chained comparisons:

```
8 > 6 > 5
# True
```

Comparison Operators

> < >= <= == !=

Comparisons also work on strings:

```
"s" > "a"
# True
```

File I/O

Files are a data type in Python. The basic way to open and read a file:

```
f = open("/home/sora/apikey")
f.read()
# 'file content...'

f.read()
# '' — end of file reached

f.seek(0) # go back to the start
f.read()
# 'file content...'

f.close() # always close the file when done
```

Method	Description
<code>.read()</code>	Reads the entire file at once
<code>.readlines()</code>	Reads the file line by line into a list — may fill memory on large files
<code>.readline()</code>	Reads one line at a time

The preferred way to work with files is using a `with` block, which closes the file automatically:

```
with open('/home/sora/apikey') as f:
    lines = f.readlines()
    print(lines)
    print(lines[-1])
```


File open modes:

Mode	Description
'r'	Read — default
'w'	Write (overwrites existing content) — creates the file if it does not exist
'rb'	Read in binary mode
'wb'	Write in binary mode
'a'	Append
'r+'	Read and write without removing existing content
'w+'	Read and write — removes existing content first

```
f = open("/home/sora/apikey", 'a')
f.write("this is a new line here")
f.close()
```

Conditions

Conditions in Python are handled with `if`, `elif`, and `else` blocks. Python evaluates each condition from top to bottom and executes the first block that matches.

```
action = "goal"
score = 0

if action == 'shoot':
    print('shooted')
elif action == 'pass':
    print('passed')
elif action == 'goal':
    print('goooooal!')
    score += 1
else:
    print('>')
```

The `else` block acts as a catch-all — it runs when none of the previous conditions are met.

You can also chain multiple `if` statements independently. Unlike `elif`, each standalone `if` is evaluated regardless of what came before:

```
if score > 0:
    print(f'oooh you won! goals: {score}')
else:
    print('lose')
```

Loops

For Loops

for loops work on any iterable — lists, tuples, dictionaries, and more. Use them when you know how many iterations you need.

```
my_list = [1, 2, 5, 6, 7, 10]
countOfEvens = 0

for this_item in my_list:
    if (this_item % 2) == 0:
        print(this_item)
        countOfEvens += 1
    else:
        print(f'{this_item} is odd')

print(f'there are {countOfEvens} even numbers')
```

When iterating over a list of tuples, you can unpack the values directly in the loop definition:

```
myTuple = (('sora', 23), ('dora', 26))

for name, age in myTuple:
    # print(c)
    # name, age = c    # this is called unpacking... we can do this unpack at first
    print(f'{name} is {age} years old')
```

When iterating over a dictionary, Python returns only the keys by default — equivalent to calling `.keys()`:

```
people = {
    'sora': (23, 184),
    'dora': (26, 172),
}

for person in people:
    print(f'{person}, is {people[person][0]} years old and their height is {people[person][1]}')
```

To get both keys and values, use `.items()`. You also have `.values()` and others available:

```
for person, data in people.items():
    # print(person)
    print(f'{person} -> {data}')
```

While Loops

while loops run as long as a condition is true. Use them when you do not know in advance how many iterations you need:

```
f = 12
print('fuel is low!')

while f <= 100:
    print('taking gas')
    f += 1

print('back to the road!')
```

List Comprehensions

List comprehensions are a shorthand way to create a new list from an existing one. They can include conditions to filter elements:

```
l = [0, 0.45, 9, -1, 4]

# for loop shorthand:
new_l = [x * 2 for x in l if x % 2 == 0]

# or:
# new_l = []

# for n in l:
#     new_l.append(n * 2)

print(new_l)
```

You can also generate a range-based list:

```
a = [e for e in range(10)]
```

Inline If (Ternary)

Python supports a single-line conditional expression:

```
for s in a:
    print('even') if s % 2 == 0 else print('odd') # this is an if shorthand
    # do_this if condition else do_that
    # this format does not support elif directly... so if we have a complex
    # ...if condition, its better to use the old way
```

Useful Built-in Operators and Functions

Imports

```
from random import randint # this is how to add new commands
```

Range and String Multiplication

`range(from, to, step)` generates a sequence of numbers. Multiplying a string by an integer repeats it.

The following commented block prints a multiplication table using nested loops:

```
# for i in range(1, 6):
#     for j in range(1, 6):
#         print(i * j, end='\\t')
#     print()
```

This example uses string multiplication to print a growing and shrinking triangle of asterisks:

```
for i in range(1, 10):
    print(i * "*")

for i in range(10, 1, -1):
    print(i * "*")
```

Iterating with Index Using `range(len())`

```
# l = 'sora'
l = ['sora', 4, 'dora']

for i in range(len(l)):
    print(i, l[i])
```

`enumerate()`

`enumerate()` turns an iterable into pairs of `index:value`, which is cleaner than using `range(len())`:

```
l = ['sora', 4, 'dora']

for i, a in enumerate(l):
    print(i, a)
```

`zip()`

`zip()` pairs two or more iterables together element by element:

```
fname = ['sora', 'dora', 'iggy']
lname = ['mlk', 'eht', 'theDog']
```

```
age = [23, 26, 3]
```

```
for x in zip(fname, lname, age):  
    print(x)
```

in

Checks if a value exists inside an iterable. In dictionaries, it checks against keys:

```
print('sora' in fname)
```

max() and **min()**

```
print(max(1, 2, 5))    # returns the maximum value  
# min() works the same way but returns the minimum
```

randint()

Generates a random integer between two values, inclusive — like rolling a dice:

```
print(randint(1, 6))  
print(randint(1, 6))
```

input()

Gets a value from the user. Note that `input()` always returns a string, so cast it to the appropriate type when needed:

```
ans = randint(1, 10)  
val = int(input('give me a number (1-10) : '))  
print(type(val), val)  
print('its true!') if val == ans else print(f'not correct. the answer is: {ans}')
```

Methods and Functions

Methods

In Python, every data type has a class, and from classes, objects are created. Every class has built-in functions that only work on objects of that class — these are called methods. They are called using dot notation: `.<method_name>()`.

You can always get help on any method:

```
# for example in mylist.pop(), .pop() is a method
l = [1, 2, 3, 4]
help(l.pop)
```

Functions

A function is a block of code that can be executed multiple times in different places. Instead of rewriting the same logic repeatedly, you define it once and call it whenever needed.

```
import random
import time
```

Defining and calling a basic function:

```
def greetings():
    a = input("enter your name: ")
    print(f'hello {a} ! how was your day?')

greetings()
```

Function Parameters

Good programmers name their functions in `snake_case`. Functions can take parameters and also support docstrings — a description written inside triple quotes that becomes the function's help text:

```
def hello(name, n):
    """
    Parameters
    -----
    name : TYPE
        DESCRIPTION.
    n : TYPE
        DESCRIPTION.
    Returns
    -----
    None.
    its better to write some description about functions
    and it becomes a help for function
    (exactly like this, inside three single quotes)
    for example:
        this function says hello to the name, n times
    """
    for i in range(n):
        print('hello', name)

# help(hello)
```

```
hello('jeff', 3)
hello('sora', 2)
```

Return Values

Usually functions do not print anything directly — instead they return a value that can be used elsewhere:

```
a = 5 # int(input('first: '))
b = 6 # int(input('second: '))

def sum_of_two_nums(a, b):
    res = a + b
    return res

print(f'sum of your given numbers are: {sum_of_two_nums(a, b)}')
```

Functions and Logic

The following block is `characters()` function — it animates a string being typed out character by character using selection through a loop

This function iterates through the character set sequentially.

```
def characters(a):
    chars = 'abcdefghijklmnopqrstuvwxyz 123456789`<>!@#%&*( )_-=[]{}.,./\`'
    sentence = ''
    for char in a:
        for c in chars:
            print(sentence + c, end='\r')
            time.sleep(0.005)
            if c == char:
                sentence += c
                break
        print(sentence)
    print(sentence)

characters('hello world, this is Sora!')
```

Default Parameter Values

You can set default values for parameters, which is useful for error handling and optional arguments:

```
def print_times(name, n=1):
    for i in range(n):
        print(name)
```

```
print_times('sora')
```

Also Python's built-in functions use default values:

```
'''
print(*args, sep=' ', end='\n', file=None, flush=False)
even in a python prebuilt function like print, there are default values like:
    file
    end
    sep
    flush
'''
```

More examples with default parameters:

```
def powered(a, p=2):
    res = a ** p
    return res

print(powered(2, 8))

def sum_of_two(a, b=10):
    return a + b

print(sum_of_two(5, 6))
print(sum_of_two('sora ', 'dora')) # python has dynamic types
```

Placeholder Variables

A placeholder variable is a variable used to temporarily hold a value during processing — for example, tracking the largest number seen so far:

```
def largest(nums):
    largest_number = nums[0] # this is a placeholder variable
    for n in nums:
        if largest_number < n:
            largest_number = n
    return largest_number

my_nums = [1, 2, 4, 6, 8, 5, 15, 11]
largest_number = largest(my_nums)
print(largest_number)

# a placeholder can also be an empty list, string, or any other type
# like:    mylist = []
```


Tuple Output of Functions

In Python, a function can return multiple values at once. Under the hood, Python packs them into a tuple, which can then be unpacked into separate variables on the receiving end.

Returning Multiple Values

The following example returns both the count of odd numbers and the odd numbers themselves:

```
def is_even(n):
    return n % 2 == 0

def get_odds(nums):
    odds = []
    count = 0

    for num in nums:
        if not is_even(num):
            odds.append(num)
            count += 1

    return count, odds

my_list = [1, 2, 3, 4, 5, 6, 7, 8, 9]
odds_count, my_odds = get_odds(my_list)
print(odds_count, my_odds, sep=' | ')
```

Another simple example returning two results from one function:

```
def addMultiply(a, b):
    add = a + b
    multiply = a * b
    return add, multiply

print(addMultiply(4, 5))
```

Practical Example: Donation Stats

This example uses placeholder variables and dictionary iteration to calculate donation statistics, then returns multiple values at once:

```
donations = {
    'sora': 20,
    'dora': 25,
    'iggy': 34
}

def donationStatus(d):
```

```

person = ''          # variable declarations (we know we need these vars based on our needs
total = 0
count = len(d.keys())
max_donation = -1    # placeholder

for name, amount in d.items():
    # print(name, amount)
    total += amount
    if amount > max_donation:
        max_donation = amount
        person = name

avg = int(total / count)
return avg, total, person

```

```

avg, total, max_person = donationStatus(donations)
print(f'average: {avg}, total donations: {total}, maximum donation reward goes to: {max_person}')

```

Practical Project: Number Guessing Game

This project brings together functions, loops, conditionals, and return values into a small interactive game. The computer picks a number and the player keeps guessing until they find it.

```

import random

def welcome():
    print('***** welcome to this game *****')
    print('i\'ll pick a number between 1 & 100 and...')
    print('you have to guess it!')
    print()

def finish(num, count):
    print('good game!')
    print(f'my number was {num} and you\'ve found it in {count} guesses')
    print()
    playMore = input("do you wanna keep on playing? (y/n) ")
    if playMore.lower() in ['y', 'yes', 'yeah']:
        # playMore = True
        return True
    else:
        return False

def win(computer_number, guess):
    return computer_number == guess
    # if computer_number == guess:
    #     return True
    # else:
    #     return False

```

```

def get_a_guess():
    ans = int(input('what is your guess? : '))
    return ans

def answer(computer, user):
    if computer > user:
        res = 'my number is larger'
    elif computer < user:
        res = 'my number is smaller'
    else:
        res = 'you won!'
    return res

welcome()
continue_playing = True

while continue_playing:
    computer_number = random.randint(1, 20)
    guess = 0
    count = 0

    while (not win(computer_number, guess)):
        guess = get_a_guess()
        count += 1
        print(answer(computer_number, guess))

    continue_playing = finish(computer_number, count)

```

*args and **kwargs

Consider a function that multiplies numbers together. The simple version takes two parameters, but what if you need to pass three, or a hundred? Adding a parameter for each is not practical.

Python solves this with `*args` and `**kwargs`.

***args** — Variable Positional Arguments

The basic approach with a fixed number of parameters:

```

def multiply(a, b, c=1):
    return a * b * c

print(multiply(3, 5))
print(multiply(2, 3, 4))

```

Using `*args` allows the function to accept any number of positional arguments. The name does not have to be `args` — it can be anything, like `*watermelon`:

```
def multiply2(*args):
    res = 1
    for i in args:
        res *= i
    return res
    # print(args)

print(multiply2(3, 5))      # if the function does not return anything, this will print 'None'
print(multiply2(2, 3, 4))
print(multiply2(1, 2, 3, 4, 5))
```

****kwargs — Variable Keyword Arguments**

****kwargs** accepts named arguments and packs them into a dictionary, where the argument name becomes the key and the value becomes the value.

Positional arguments like `multiply3(3, 5)` would throw an error here — ****kwargs** only accepts keyword arguments:

```
def multiply3(**kwargs):
    print(kwargs)

# print(multiply3(3, 5))    # this will throw an error — kwargs does not accept positional arguments
```

A positional argument is one where the value is assigned by position — `func(1, 2)` maps 1 to `a` and 2 to `b`. Keyword arguments are called by name instead:

```
multiply3(first=3, second=8)    # returns a dict: name -> key, value -> value
```

A practical example using ****kwargs** to calculate area based on what arguments are provided:

```
def calculate_area(**kwargs):
    if 'width' in kwargs:
        return kwargs['width'] * kwargs['height']
    if 'r' in kwargs:
        return kwargs['r'] ** 2 * 3.1415926
    return 1

print(calculate_area(width=6, height=2))
print(calculate_area(r=3))
```

Variable Scope

```
def hello():
    # global name
    name = 'dora'    # enclosing function local
    print(f'in function name is {name}')
```

```
name = 'sora'    # global
print(f'name is {name}')
hello()
print(f'name is {name}')
```

The `name` variable inside `hello()` is a separate local variable — it does not overwrite the global `name`. After the function call, the global `name` remains `'sora'`.

LEGB Rule

Python resolves variable names by searching in this order:

1. Local — inside the current function
2. Enclosing — inside any outer (enclosing) function
3. Global — at the module level
4. Built-in — Python's built-in names like `print`, `len`, etc.

```
'''
if we want our local and enclosing function local variables to be global:
    we have a command named: global
    which is not recommended to use because when we call a function we do not expect our variables
    to be modified.
'''
```

Using the `global` keyword forces a local variable to point to the global scope. It is generally discouraged because it makes functions unpredictable — callers do not expect a function call to silently modify variables outside of it.

Object-Oriented Programming (OOP)

Many times we do not create a program from scratch — we work with existing code and need to understand how it is structured. In Python, almost everything is an object.

```
s = 'sora'
type(s)
# str
```

By typing `.` after an object, we can access operations on it — these are called methods:

```
s.count('a')
# 1

l = [3, 4, 1, 9]
l.sort()
l
# [1, 3, 4, 9]
```

Since almost everything in Python is an object, `'sora'` is an instance of the `str` class. These objects have built-in functions defined in their class — we call them **methods**, accessed with a `.` after the object. Objects also have **attributes** and **properties** — these are also accessed with a `.`, but they are just values, not functions.

A good analogy: a car is the class, a Mercedes is an instance of that class, the color is an attribute, and the function to move it is a method.

Defining a Class

To define a new object type in Python, you first define a class. Classes have special functions like `__init__()` — this is called automatically every time a new object is created from the class. It always takes `self` as its first parameter, which refers to the instance being created:

```
#!/usr/bin/env python3

class ClassName():
    def __init__(self, param1):
        self.param1 = param1
        print('object created')

    def sayHello(self):
        print('hello, im an object from ClassName class')

t = ClassName(5)
print(t.param1)      # this is an attribute
t.sayHello()         # this is a method
print(type(t))
```

map(), filter(), and Lambda Functions

map() and filter()

Both `map()` and `filter()` work on any iterable, not just lists. When used on dictionaries, they operate on the keys. Both take a function name without parentheses.

- **map()** runs a function on every element of an iterable and returns the results.
- **filter()** runs a function on every element and keeps only the elements for which the function returns `True`.

```
def pow2(x):
    return x ** 2

a = [2, 4, 3, 8]
```

The following block shows the manual loop approach — iterating and applying the function element by element:

```
for i in range(len(a)):
    a[i] = pow2(a[i])

print(a)
```

Using `map()` instead is cleaner. Note that the result is a lazy object — it does not compute or fill memory until it is actually consumed:

```
a2 = map(pow2, a)
print(a2)          # this output is whats known as lazy call
print(list(a2))    # the lazy call won't run until it is needed (here)
```

Lambda Functions

Sometimes when working with `map()` or `filter()` we do not want to define a full named function. For these cases, Python provides **lambda functions** — one-liner, anonymous, nameless functions.

```
...
syntax: lambda x: whatever to do with x
...
```

Using `map()` with a lambda:

```
a2 = map(lambda x: x**2, a)    # returns x**2 for each x
print(a2)
print(list(a2))
```

`filter()` with Lambda

The following commented block shows the manual approach to filtering decimal numbers from a list:

```
b = [5, 6.2, 8, 9.1, 8, 4, 2.9]

b2 = []
def is_float(x):
    return not x == int(x)

for i in range(len(b)):
    if is_float(b[i]):
        b2.append(b[i])
print(b2)

# instead of for loop:
print(list(filter(is_float, b)))
```

The same result using a lambda directly:

```
print(list(filter(lambda x: x != int(x), b)))
```

Filtering names by length:

```
names = ['sora', 'dora', 'iggy', 'ghedas']
shortNames = filter(lambda x: len(x) <= 4, names)
print(list(shortNames))
```

Writing a Class

The `__init__` function is called a **constructor** — it is the first function called when an instance is created.

Every time you define an attribute or method inside a class, write `self` first. When calling it, there is no need to mention `self` explicitly.

More About `self`

```
"""
for example we have a class named Human and it contains people.
sora is one of those people.
sora has some attributes and methods like: eat, sleep, code, smoke, and more.
when sora sleeps, we cannot say humankind is sleeping – it is just sora.

that is what the self keyword does.
we define sora and say he is 23 years old. self refers to sora himself, not all humans.
"""
```

Class Definition and Instances

```
class Book():
    # class-level variable – shared across all instances by default
    book_type = 'horror'

    def __init__(self, page):
        self.pages = page
        # the pages attribute stores the value passed to the constructor

    def open(self):
        print('opened the book...')

    def open_last_page(self):
        print(f'opened the book on page: {self.pages}')

mybook = Book(540)
yourbook = Book(12)

print(mybook.pages)
print(type(mybook))
mybook.open()
mybook.open_last_page()
yourbook.open_last_page()
```

Class-Level vs Instance-Level Attributes

Class-level variables are shared across all instances by default:

```
print(mybook.book_type)
print(yourbook.book_type)
```

You can override the class-level variable on a specific instance without affecting others:

```
yourbook.book_type = 'comedy'

print(mybook.book_type)    # still 'horror'
print(yourbook.book_type)  # 'comedy'
```

To change the default for all instances, set it on the class itself. Note that instances with an explicitly set value keep their own value:

```
Book.book_type = 'something else'

print(mybook.book_type)    # 'something else' – follows the class default
print(yourbook.book_type)  # 'comedy' – keeps the value set explicitly on this instance
```

Classes — Advanced

Class Attributes

Class-level attributes are shared across all instances. Referencing them through the class name (e.g. `Circle.pi`) makes it explicit that the value belongs to the class, not a specific instance:

```
class Circle():
    pi = 3.1415926

    def __init__(self, r):
        self.r = r

    def area(self):
        m = self.r ** 2 * Circle.pi    # Circle.pi is a class attribute
        return m

c1 = Circle(10)
print(c1.area())
```

Inheritance

OOP also introduces more advanced concepts. Consider a game where a human can jump and is smart, while a monster has colors and regenerates. **Inheritance** allows objects to inherit attributes and methods from a parent class.

```
...
for example we have a game.
```

*we have a human – a thing that can jump and is smart.
we have a monster – a thing that has colors and regenerates.*

here comes inheritance: humans inherit some attributes and methods from a parent, and so do monster objects can inherit these from each other.

```
class Book():
    def __init__(self, name, page):
        self.name = name
        self.pages = page

    def open(self):
        print(f'opened {self.name}...')

    def open_last_page(self):
        print(f'opened the {self.name} on page: {self.pages}')

b1 = Book('C Programming', 389)
b1.open_last_page()
```

Educational is a type of Book, so it inherits Book's attributes and methods. To set up inheritance:

1. Put the parent class name in parentheses
2. Include the parent's parameters in `__init__`
3. Call the parent's `__init__` inside the child class

```
class Educational(Book):
    def __init__(self, reshte, paye, name, page):
        Book.__init__(self, name, page)  # there is no specific order to receive a
        self.reshte = reshte             # call the parent's init first. its better to use '
        self.paye = paye
        print('a new Educational book added.')

    def open(self):
        print(f'opened an Educational book: {self.name} of {self.reshte}, paye: {self.paye}')

d = Educational('tajrobi', 12, 'riazi', 214)
print(d.pages)
d.open()
d.open_last_page()
```

Polymorphism

Different object types can share the same method name and be used interchangeably — this is **polymorphism**:

```
class Runner():
    def __init__(self, name):
```

```

        self.name = name
    def action(self):
        print(f"{self.name} is running")

class Cyclist():
    def __init__(self, name):
        self.name = name
    def action(self):
        print(f"{self.name} is cycling")

sarah = Runner('sarah')
sarah.action()

sora = Cyclist('sora')
sora.action()
print()

for person in [sora, sarah]:
    person.action()
    ...

    at this point the polymorphism concept becomes clear:
        we have a person concept, object types are different but they both performed the action
    ...

```

You can also pass any of these objects into a generic function:

```

def show_actions(person):
    person.action()

show_actions(sora)

```

Abstract Methods

Sometimes a parent class exists only to define a structure — it does not implement anything itself. Child classes are expected to provide their own implementations:

```

"""
abstract methods:
    sometimes classes are abstract and do not define any behavior
"""

class Human():
    def __init__(self, name):
        self.name = name
    def jump(self):
        raise NotImplementedError("implement the jump")    # user raise to manually throw an e

class Programmer(Human):
    def __init__(self, name):

```

```

        super().__init__(self, name)
    def jump(self):
        print(f'{self.name} jumped')

sora = Programmer('sora')
sora.jump()

```

The practical value of this pattern becomes clear in larger systems:

consider a parent class named `Net`, with child classes for network protocols like `TCP`, `ICMP`, etc. when adding a new protocol (e.g. `sora` protocol), the rules are: - it must be a child of `Net` - it must define its own `.send()` method, because no two protocols work the same way

abstract classes enforce this contract across all child classes.

Dunder (Magic) Methods in Python

Everything in Python is an object. But when you declare a variable like a list and call `len(mylist)`, how does `len` know what to do with it?

For example, when you write `print(mylist)`, it prints exactly like a list. So why doesn't this work the same way for your own objects (like `mybook` from a `Book` class), instead printing something like `<__main__.Book object at ...>`?

In Python, there are special “things” that people refer to by different names: special methods, magic methods, or dunder methods. Dunder stands for “double underscore.”

The Book Class

```

class Book():
    def __init__(self, name='Unknown', page=0):
        self.name = name
        self.pages = page

    def open(self):
        print(f'opened {self.name}...')

    def open_last_page(self):
        print(f'opened the {self.name} on page: {self.pages}')

# at this point we have to define the magic/dunder/special methods for our class

    def __len__(self):
        return self.pages

    def __str__(self):
        r = f'{self.name}, {self.pages}'
        return r

```

```
def __del__(self):
    print(f'Oh! the book {self.name} is vanishing!')

b1 = Book('C Programming', 389)
print(b1)
print(len(b1))
```

__init__ and __del__

The other dunder method we've seen before is `__init__`, which is called a constructor in some languages, because it's executed whenever an instance is created from the class.

There is another one of this kind called `__del__`, which executes whenever the instance is removed. We can delete variables like this:

```
i = 44
print(i) # 44
del i
print(i) # NameError
```

The same thing applies to class instances:

```
b = Book('darsi', 234)
del b # or del(b)
```

Working with Python Packages

There are some packages that we import into our programs and use — things that we haven't written ourselves and that aren't part of the current environment by default. These are called packages, libraries, and so on. Working with packages is one of Python's strong points.

For example, AI is growing fast nowadays, and many people like to work with it. Instead of writing an entire program from scratch to work with AI, in Python you just need to import the required library and use it.

Python has a huge number of libraries for almost everyone — engineers, chemists, game developers, hackers, and many more. All Python libraries can be found on pypi.org.

Example Import

```
import emoji
# or:
from emoji import emoji
print(emoji.emoji('👍')) # if we did "import emoji"
print(emoji('👍'))      # if we did "from emoji import emoji"
```

Imports should be written at the top of the file.

Installing Third-Party Libraries

Before working with a third-party library (like `emoji`), we first have to make sure the package is installed. Installation is usually straightforward — we have a package manager called `pip`, and also `pipx`. The installation command is:

```
pip install emoji
```

Almost every language has its own package manager — for example, in JavaScript we use `npm`.

Depending on the operating system, we might have more than one `pip` or `python` installed. We might be using `pip 3.11` but `python3.12`. At this point it can become a problem, but the solution is simple:

1. We can build a Python virtual environment to work in:

```
python3 -m venv sora-env  
source sora-env/bin/activate
```

Then install the needed packages and libraries inside it.

2. Or we can specify the exact `pip` version we want to use:

```
pip3.12 install emoji --break-system-packages
```

This second approach is not recommended — using a virtual environment (option 1) is much safer and more reliable.

Bonus: Progress Bars with `tqdm`

The `tqdm` library lets us create a simple, clean progress bar:

```
from tqdm import tqdm  
from time import sleep  
  
for i in tqdm(range(10)):  
    sleep(1)
```

Packages and Modules

In the previous part, we've talked about libraries and third-party packages. But we can also write modules ourselves.

We can write all of our functions somewhere else — for example, in a file named `functions.py` — and then import those functions into the main program. What are the benefits?

- The code becomes more readable and clean.
- We can reuse functions wherever we need them.
- And much more.

Modules vs Packages

Modules are simply Python scripts whose names we import and use directly. Packages, on the other hand, are usually larger and made up of multiple modules grouped together.

If we want to create a package, we first have to create a directory with the name of the package. For example, we might name ours `jackage` and move our script (`mymod.py`) into that folder.

The directory also needs a special file named `__init__.py`, which can be empty. However, this file has to exist for Python to recognize the directory as a package.

Practical Project: Modules and Packages in Action

This file demonstrates a working directory structure for modules and packages, along with the terminal output showing how everything connects.

Directory Structure

```
.
├── jackage
│   ├── __init__.py
│   ├── mymod.py
│   └── __pycache__
│       ├── __init__.cpython-313.pyc
│       └── mymod.cpython-313.pyc
├── mycode.py
├── mymod.py
└── __pycache__
    └── mymod.cpython-313.pyc
```

4 directories, 7 files

The Module: `mymod.py`

```
my_pi = 3.15
def my_funky_print():
    print('Oh im so cool!, from mymod.py')
    print('in module name is {}'.format(__name__))
if __name__ == '__main__':
    print("you should use me as a package")
```

Note: `my_pi` here is deliberately set to an arbitrary value (3.15), not the actual value of `pi` — it's just used as an example variable to demonstrate importing.

The Main Program: `mycode.py`

```
import mymod    # import the file/module
mymod.my_funky_print()    # from the module, call the function
#my_funky_print()    # throws an error because what is my_funky_print?
```

```

from mymod import my_funky_print    # directly import the function
my_funky_print()    # directly call the function
from mymod import my_pi    # or even a variable
print('here the pi is {0}'.format(my_pi))
# commented code ^^^^^^

```

The commented-out line `#my_funky_print()` is left in deliberately. It demonstrates that calling `my_funky_print()` directly (without `mymod.` prefix or a direct import) would raise a `NameError`, since the function only exists inside the `mymod` namespace until it's explicitly imported with `from mymod import my_funky_print`.

To use a package, we follow the same idea but with a slightly more nested structure:

```

from jackage import mymod
mymod.my_funky_print()    # this is from a package, not a standalone module
print("*****")
from jackage.mymod import my_funky_print
my_funky_print()
print(f'in main program, __name__ is {__name__}')

```

The Package Module: `jackage/mymod.py`

```

my_pi = 3.15
def my_funky_print():
    print('Oh im so cool!, from mymod.py')
    print('in module name is {}'.format(__name__))

```

This is the same module, but placed inside the `jackage` package directory so it can be imported as `jackage.mymod`.

Running the Project

To run this project, save `mycode.py`, `mymod.py`, and the `jackage/` directory (containing `__init__.py` and `mymod.py`) in the same location, then run:

```
python3 mycode.py
```

The `__name__` Variable in Packages and Modules

We've already talked about `__init__.py` inside a package directory. There's also a useful built-in variable called `__name__`, which exists even if we never declare it ourselves.

To see how it behaves, the following line was added to both `mymod.py` and `mycode.py`:

```
print(f'__name__ is {__name__}')
```

To test this, run `mycode.py` from inside the directory that you just created in previous lesson.

Results:


```
# in module, __name__ is mymod  
# in main program, __name__ is __main__  
# in module name is jackage.mymod      # this is from package: jackage
```

How `__name__` Works

When a program is executed directly, `__name__` is always `__main__`. But when the file is imported as a module, `__name__` becomes the module's name instead.

The `__name__` variable always exists in Python programs. Whenever the code is run directly, it becomes `__main__`; when imported, it becomes the module (or package) name. This lets the program know whether it's being run directly or imported as a module/package.

This gives us a useful feature:

```
if __name__ == '__main__':  
    # do something
```

Usage Example

```
sora@tool [~/python-practice/packagesAndModules] [15:26:35] $ cat mymod.py  
my_pi = 3.15
```

```
def my_funky_print():  
    print('Oh im so cool!, from mymod.py')  
    print('in module name is {}'.format(__name__))  
  
if __name__ == '__main__':  
    print("you should use me as a package")
```

```
*****
```

```
sora@tool [~/python-practice/packagesAndModules] [15:26:22] $ python3 mycode.py  
Oh im so cool!, from mymod.py  
in module name is mymod  
Oh im so cool!, from mymod.py  
in module name is mymod  
here the pi is 3.15  
Oh im so cool!, from mymod.py  
in module name is jackage.mymod  
*****  
Oh im so cool!, from mymod.py  
in module name is jackage.mymod  
in main program, __name__ is __main__  
sora@tool [~/python-practice/packagesAndModules] [15:26:27] $ python3 mymod.py  
you should use me as a package
```

What's the Point?

For example, suppose we have a package that does a lot of useful work — say, drawing charts. We can tell it: “whenever you’re executed directly, get input from the user and do your thing.”

Many Python packages work this way and can be executed directly. One example is the `tqdm` package. Another is `bashplotlib`, which draws charts in the terminal environment.

Practical Project: Building an Installable Library Management Package

(consider this as a task for yourself)

This project demonstrates how to structure a Python project as an installable package using `setuptools`, complete with a main program, a package module, and a `setup.py` file.

Directory Listing

```
total 20K
-rw-rw-r-- 1 hikari hikari 1.4K Mar  4 17:04 main.py
drwxrwxr-x 2 hikari hikari 4.0K Mar  4 17:08 mylibrary_pkg.egg-info/
drwxrwxr-x 3 hikari hikari 4.0K Mar  4 17:08 mylibrary_pkg/
-rw-rw-r-- 1 hikari hikari  392 Mar  4 17:18 README.md
-rw-rw-r-- 1 hikari hikari  667 Mar  4 17:22 setup.py
```

Main Program: `main.py`

```
# main.py

# Import the Library class from the installed package
from mylibrary_pkg.library import Library

def display_menu():
    """Display menu options to the user."""
    print("\n--- Library Management System ---")
    print("1. Add Book")
    print("2. Remove Book")
    print("3. Search Book")
    print("4. Show All Books")
    print("5. Exit")
    print("-----")

if __name__ == "__main__":
    # This block runs only when main.py is executed directly

    my_library = Library()

    while True:
        display_menu()
```

```

choice = input("Enter your choice (1-5): ")

if choice == '1':
    title = input("Enter book title to add: ")
    author = input("Enter book author: ")
    my_library.add_book(title, author)

elif choice == '2':
    title = input("Enter book title to remove: ")
    my_library.remove_book(title)

elif choice == '3':
    title = input("Enter book title to search: ")
    my_library.search_book(title)

elif choice == '4':
    my_library.show_books()

elif choice == '5':
    print("Exiting system. Goodbye!")
    break

else:
    print("Invalid input. Please try again.")

```

The `from mylibrary_pkg.library import Library` line shows how, once a package is installed, its modules can be imported just like any other library — exactly as discussed in earlier files.

Package Configuration: `setup.py`

```

# setup.py
from setuptools import setup, find_packages

setup(
    name="mylibrary_pkg", # Changed name slightly to avoid conflict with directory name 'myli
    version="1.0.0",
    author="Sora",
    author_email="sourallism@gmail.com",
    description="A simple, installable library management system package.",
    long_description="A package containing a modular Library management system.",
    packages=find_packages(), # Finds the 'mylibrary' directory
    classifiers=[
        "Programming Language :: Python :: 3",
        "License :: OSI Approved :: MIT License",
        "Operating System :: OS Independent",
    ],
    python_requires='>=3.6',
)

```

This file tells `setuptools` how to package the project for installation. The `find_packages()` function automatically detects the package directory (`mylibrary_pkg`), and the metadata fields (`name`, `version`, `author`, etc.) describe the package for tools like `pip`. Actually if `find`'s every folder containing `__init__.py`

Package Contents: `mylibrary_pkg/`

`__init__.py` `library.py` `__pycache__`

Package Module: `mylibrary_pkg/library.py`

```
# mylibrary/library.py

class Book:
    """A simple class to represent a book."""
    def __init__(self, title, author):
        self.title = title
        self.author = author

    def __str__(self):
        return f"Title: {self.title}, Author: {self.author}"

class Library:
    """Manages a collection of books."""
    def __init__(self):
        # Using a list of Book objects
        self.books = []

    def add_book(self, title, author):
        """Adds a new book to the collection."""
        new_book = Book(title, author)
        self.books.append(new_book)
        print(f"Success: Book '{title}' added.")

    def remove_book(self, title):
        """Removes a book by title."""
        initial_count = len(self.books)
        self.books = [book for book in self.books if book.title.lower() != title.lower()]

        if len(self.books) < initial_count:
            print(f"Success: Book '{title}' removed.")
        else:
            print(f"Error: Book '{title}' not found.")

    def search_book(self, title):
        """Searches for books matching the title."""
        results = [book for book in self.books if title.lower() in book.title.lower()]

        if results:
```

```

        print(f"Search Results for '{title}':")
        for book in results:
            print(f"  - {book}")
    else:
        print(f"No books found matching '{title}'.")

    def show_books(self):
        """Displays all books in the library."""
        if not self.books:
            print("The library is currently empty.")
            return

        print("\n--- Current Library Collection ---")
        for book in self.books:
            print(f"  - {book}")
        print("-----")

```

The `Book` class represents a single book with a title and author, and defines `__str__` so that printing a `Book` instance produces a readable string instead of the default object representation. The `Library` class manages a list of `Book` objects and provides methods to add, remove, search, and display books.

Running the Project

To run this project, save the files in the structure shown above (with `mylibrary_pkg/` containing `__init__.py` and `library.py`), then run:

```

pip install .
python3 main.py

```

Error Handling in Python

Types of Errors

There are two types of errors:

1. **Logical errors in code** — for example, a program meant to display odd numbers mistakenly treats 4 as an odd number. This error relates to the program's logic, not its syntax.
2. **Syntax errors, name errors, and more** — the programming language does not accept the command as written.

Errors cannot always be avoided. For example, suppose we want to calculate the average of student grades:

```

sum_of_numbers / total_classes

```

This seems fine, but what if a student has no classes? A `ZeroDivisionError` will occur and the program will crash immediately. This is why we need error handling.

Setting Up the Example

```
student1 = [17, 18, 19, 20, 15]    # there are some additional points
def sum_of_numbers(l):
    tempSum = 0
    for i in l:
        tempSum += i
    return tempSum

print(sum_of_numbers(student1))

# somehow, the classes are 0 and ...

# avg = sum_of_numbers(student1) / 0
```

This commented-out line is left in deliberately to show what would happen if we divided the sum of grades by zero classes — it would raise a `ZeroDivisionError` and crash the program, which is exactly the problem that `try/except` blocks are meant to solve.

try and except Blocks

In Python, we use `try` and `except` blocks for error handling:

```
try:
    # try to do these
except:
    # if anything unpredicted happens

def divis(a, b):
    try:
        return a/b
    #except:
    #return None
    except Exception as e:
        return f'Error in returning: {e}'

print(divis(1, 2))
print(divis(3, 0))
```

The commented-out `#except: / #return None` block shows an alternative, simpler way of catching the error — by silently returning `None` without any error message. It's left here for comparison with the more informative version below it, which catches the exception and returns a descriptive error message instead.

Catching Specific Exception Types

We can also catch specific exception types separately, which lets us give more precise error messages depending on what went wrong:

```
def divis2(a=1, b=1):
    try:
        return a/b
```

```

except ZeroDivisionError:
    return f'cannot divide by {b}'
except TypeError:
    return f'cannot divide/divide by not-number types, please enter a number: "{a}" or "{b}"'
except Exception as e:
    return f'Unknown error: {e}'

print(divis2('edds', 7))
print(divis2(6, 'soar'))
print(divis2('edds', 'sora'))
print(divis2('edds'))
print(divis2(7))

```

else and finally Blocks

```

try:
    f = open(input("enter a file path: "), 'r')
    print(f.read())
    f.close()
except FileNotFoundError:
    print("wasnt able to find the file")
except Exception as e:
    print(f'Unknown error: {e}')
else:
    # if no exception happens
    print('finished process with zero errors')
finally:
    # will run in any case
    print('exiting block')

```

The else block runs only if no exception was raised in the try block. The finally block always runs, regardless of whether an exception occurred or not — making it useful for cleanup tasks.

Keeping try Blocks Simple

```

def divis3(a, b):
    try:
        r = a/b
        # its a best practice to do less operation here
        # if print, return and ... are here, makes code more complex
    except Exception as e:
        print(f'had error: {e}')
        r = None
    else:
        print(r)
    finally:
        return r

```

```
divis3(3, 4)
```

It's best practice to keep the code inside a `try` block as minimal as possible. Performing operations like `print` or `return` directly inside the `try` block can make the code harder to follow and debug — it's clearer to separate the calculation from the handling and output logic, as shown above.

Linters and Code Formatters

Linters are not exclusive to Python. A linter looks at code and tells you how “pretty” or well-structured it is.

Installation

```
pip install pylint
```

Or on Debian-based operating systems:

```
sudo apt install pylint
```

Example: Messy Code

```
a=      5
b      =      8
if a!=  0:
    print(    f'a+b ={a+b}')
```



```
else:
    prin("a=0")
```

This code is intentionally messy, and it also contains an error on the last line. Here's the corresponding `pylint` output:

```
sora@tool [~/python-practice] [21:06:12] $ pylint linterInErrorhandling.py
***** Module linterInErrorhandling
linterInErrorhandling.py:11:0: W0311: Bad indentation. Found 9 spaces, expected 4 (bad-indentation)
linterInErrorhandling.py:12:0: C0303: Trailing whitespace (trailing-whitespace)
linterInErrorhandling.py:13:0: C0303: Trailing whitespace (trailing-whitespace)
linterInErrorhandling.py:15:0: C0304: Final newline missing (missing-final-newline)
linterInErrorhandling.py:15:0: W0311: Bad indentation. Found 1 spaces, expected 4 (bad-indentation)
linterInErrorhandling.py:1:0: C0103: Module name "linterInErrorhandling" doesn't conform to snake_case (name)
linterInErrorhandling.py:8:0: C0103: Constant name "a" doesn't conform to UPPER_CASE naming style (name)
linterInErrorhandling.py:9:0: C0103: Constant name "b" doesn't conform to UPPER_CASE naming style (name)
```



```
linterInErrorhandling.py:15:1: E0602: Undefined variable 'prin' (undefined-variable)
```

Your code has been rated at 0.00/10 (previous run: 0.00/10, +0.00)

Formatters vs Linters

A formatter is something different — for example, it can turn `a= 5` into `a = 5`. `black` is a well-known Python formatter.

However, a linter also looks at logic and flags problematic code, not just formatting. `pylint` also has a `--report` switch for more detailed output:

```
sora@tool [~/python-practice] [21:06:12] $ pylint --report y linterInErrorhandling.py:
```

Report
=====
7 statements analysed.

Statistics by type

type	number	old number	difference	%documented	%badname	
=====	=====	=====	=====	=====	=====	
module	1	1	=	100.00	100.00	
-----	-----	-----	-----	-----	-----	
class	0	NC	NC	0	0	
-----	-----	-----	-----	-----	-----	
method	0	NC	NC	0	0	
-----	-----	-----	-----	-----	-----	
function	0	NC	NC	0	0	
-----	-----	-----	-----	-----	-----	

57 lines have been analyzed

Raw metrics

type	number	%	previous	difference	
=====	=====	=====	=====	=====	
code	26	45.61	NC	NC	
-----	-----	-----	-----	-----	
docstring	12	21.05	NC	NC	
-----	-----	-----	-----	-----	
comment	1	1.75	NC	NC	

empty	18	31.58	NC	NC	
-------	----	-------	----	----	--

Duplication

	now	previous	difference	
nb duplicated lines	0	0	0	
percent duplicated lines	0.000	0.000	=	

Messages by category

type	number	previous	difference	
convention	7	6	6	
refactor	0	0	0	
warning	4	2	2	
error	1	1	1	

% errors / warnings by module

module	error	warning	refactor	convention	
linterInErrorhandling	100.00	100.00	0.00	100.00	

Messages

message id	occurrences
trailing-whitespace	3
invalid-name	3
pointless-string-statement	2
bad-indentation	2
undefined-variable	1
trailing-newlines	1

Using **black** (Formatter)

Installation:

```
sudo apt install black
```

Original messy code:

```
a=      5
b      =      8
if a!=      0:
    print(      f'a+b ={a+b}')
```

```
else:
    prin("a=0")
```

Run the formatter:

```
black dirtycode.py
```

Output:

```
All done!
1 file reformatted.
```

Resulting code:

```
a = 5
b = 8
if a != 0:
    print(f'a+b ={a+b}')
```

```
else:
    prin("a=0")
```

As you can see, the logical error (`prin` is undefined) is still present — `black` only makes the code prettier, it does not fix logic or naming errors. That's what a linter like `pylint` is for.

Practical Project: Unit Testing with `unittest`

In small projects, we can test our code simply by giving the program some input and checking the output manually. But in larger programs, this becomes much harder to manage. How do we know that the changes we just made haven't broken something else in the project? Good projects have automatic tests.

The Function to Test: `unittestInErrorhandling.py`

```
def countc(s, c):
    found = 0
    for this_char in s:
        if this_char == c:
            found += 1
    return found
```

This file is named `unittestInErrorhandling.py`. We'll create another file named `unittestInErrorhandling-test.py` to test it. If everything works correctly, the test results will show OK.

The Test File: `unittestInErrorhandling-test.py`

```
import unittest
# unittest is the framework which handles testing stuff
from unittestInErrorhandling import countc
# we also need to import the original function we want to test

# we have to build a class to deal with our test
class TestCountC(unittest.TestCase):
    def test_simple(self): # self refers to TestCountC class instance
        s = 'soraa'
        c = 'a'
        result = 2
        self.assertEqual(countc(s, c), result)
        # assert checks one thing against another, and if there's a mismatch, it stops the program
        # assertEquals checks if one thing is equal to another

    def test_nothing(self): # self refers to TestCountC class instance
        s = ''
        c = 'a'
        result = 0
        self.assertEqual(countc(s, c), result)
        # we can add as many tests as we want

    def test_non_existance(self): # function names should start with test_
```

```

s = 'soralsfdjlkfjslsfl'
c = 'x'
result = 0
self.assertEqual(countc(s, c), result)

def test_all(self):
    s = 'ssssss'
    c = 's'
    result = 6
    self.assertEqual(countc(s, c), result)

if __name__ == '__main__':
    unittest.main()

```

Library vs Framework

`unittest` is a testing framework. But what's the difference between a library and a framework?

- **Library:** we simply import it and start using it however we like.
- **Framework:** when we import a framework, it dictates how we should structure our code around it. For example, `unittest` requires us to create a class, write specific methods, and follow its conventions.

Writing test scripts is itself a topic broad enough for a full course — here we only cover the basics that are likely to be useful. In larger projects, all test files are usually placed in a dedicated directory and run together at once.

Running the Tests

To run this test file, save both `unittestInErrorhandling.py` and `unittestInErrorhandling-test.py` in the same location, then run:

```
python3 unittestInErrorhandling-test.py
```

Decorators: Functions as First-Class Citizens

Let's start with an example. Suppose we have a function `show_home()` that displays the home page of a website, and another one, `sell()`, that displays the selling page.

Sometimes, we need to do different things depending on whether the user is logged in or not. We could always add an `if` statement to handle this, but that means editing every function directly and adding the same check to each one.

This is where decorators come in. We write a special function (shown later) named something like `needs_login`, and then we can decorate each relevant function with `@needs_login`.

Before writing a decorator, though, we need to understand a bit more about how Python functions work.

Functions as Objects

```
def add(a, b):
    result = a + b
    return result

a = add(3, 4)
b = add(1, 2)
print(a)
print(b)
print(add)    # is an object
```

```
def say_hello():
    print('hello world!')

bro = say_hello
print(bro)

del say_hello
print('** SAY HELLO DELETED **')
print(bro)
bro()
```

In Python, functions are first-class citizens. We can assign an existing function to a variable, and that variable becomes the function itself — not its output, and not a copy. We can pass functions around as arguments, return them from other functions, or define them nested inside other functions.

Passing Functions as Arguments

```
def calculate(a, b, c):
    return c(a, b)

print(calculate(1, 5, add))
```

```
def do_twice(anything):
    anything()
    anything()

do_twice(bro)
```

Nested Functions

```
def calculator(a, b, what_to_do):
    def multiply(a, b):
        return int(a * b)
    def add(a, b):
        return int(a + b)

    if what_to_do == "add":
```

```

        return add(a, b)
    elif what_to_do == "multiply":
        return multiply(a, b)

print(calculator(4, 5, 'multiply'))
print(calculator(4, 5, 'jam'))

```

In calculator, the multiply and add functions are defined inside another function. This is what's known as a nested function structure — a key concept that decorators build on.

Practical Project: Writing a Decorator

A Simple Handler Function

```

def run_on_even(f):
    import datetime
    now = datetime.datetime.now()
    minute = now.minute
    if minute % 2 == 0:
        f()
    else:
        print('...')

```

Turning It Into a Decorator

```

# This is a decorator's syntax, just as above, but we put the handler code inside an internal
def run_on_even2(f):
    def wrapper(): # the whole code inside the decorator should be placed into another function
        import datetime
        now = datetime.datetime.now()
        minute = now.minute
        if minute % 2 == 0:
            f()
        else:
            print('...')

    return wrapper # and return it at the end
# we will talk much more about decorators in advanced python section.

```

The Scenario

Imagine being part of a special group whose members are only allowed to talk during even minutes. Before joining this group, there was a function to say hello:

```

def say_hello():
    # after joining the group, talking is not allowed during odd minutes, so...
    import datetime
    now = datetime.datetime.now()

```

```

minute = now.minute
if minute % 2 == 0:
    print('hello! im here!')
else:
    print('...')

```

`say_hello()` *# it doesn't matter whether it's an odd or even minute, but after joining the g*

So the changes were made, and the problem is solved — no more talking during odd minutes. But there's a catch: there are more functions, like `say_bye`, `greetings`, `sing_a_song`, and so on. The same code would need to be added to each of these functions individually.

```

def say_bye():
    print('bye bye!')

def sing_a_song():
    print('what would we do with the drunken sailor?')

def greetings():
    print('hey! how is it going?')

```

This is a real problem — editing every single function isn't practical. We could define a separate function to check whether the condition (even/odd minute) is true, then add an `if` statement to every function, but that's still neither easy nor convenient.

So instead, let's define a single function to solve this once and for all. Here comes the idea: write a function named `run_on_even` that takes another function as input, and if the condition is `True`, runs that function — otherwise, it does something else, or nothing at all.

So now we have a function that solves the problem, but how do we use it? Like this:

```

run_on_even(say_bye)
run_on_even(sing_a_song)

```

From Handler to Decorator

This is still not a decorator — just a handler function. Decorators have some differences and make things even easier to use.

Decorators let us solve the problem without: - changing the function's code, and - changing how the function is called.

We just add a small “decoration” on top of the function, like this:

```

@run_on_even2
def function_name():
    pass

```

So, by defining a function named `run_on_even2` and making a few changes, we can turn it into a decorator. After defining the decorator function, we add a `@` followed by the decorator's name above the functions we want to control — for example, `@run_on_even2`.

Real-World Uses

This concept is useful in many cases — for example, when debugging a function:

```
def decorator_function(f):
    def wrapper():
        # takes all function inputs
        # saves the time
        # runs the function
        # prints the output
        # returns the function output to make it normal

    return wrapper
```

In real-world projects, we don't write many decorators ourselves — when needed, we can usually find an existing one online. But sometimes writing one can be a real lifesaver.

That said, decorators are used constantly without most people noticing. For example, in Flask, there's a decorator that, when placed above a function, ties that function to a specific route — so visiting `/api` triggers the code inside it.

Generators

Sometimes we need functions that give us, for example, prime numbers in the first 7 unsigned integer numbers.

```
def prime_numbers():
    for i in range(1, 8):
        #some code to filter prime numbers
        print(i)

# prime_numbers()
```

The commented-out call `#prime_numbers()` is left here to show that the function definition alone does nothing — it needs to be called to actually run. It's commented out because the filtering logic for primes hasn't been implemented yet; this is just a placeholder showing the structure.

But what if we need 10,000 numbers instead of 7? Memory would come under serious pressure. Imagine there's a 10 terabyte file, and we want to read the whole thing and load it into 10 terabytes of memory — which is practically impossible.

But remember, we also have `readline`, which lets us read a file line by line, on demand, using a `next` command. In the same way, we can write a function that keeps calculating prime numbers and returns them only when needed — as if the function remembers exactly where it left off.

This is a **generator function** — a function that can pause and resume itself.

range as a Generator-Like Object

The `range` function works in a similar way: it doesn't immediately produce all the values we asked for. Instead, it produces them only when they're actually needed.

```
print(range(10), 'this is just a range, not values.')
for i in range(10):
    print(i, 'this is where we need it and returns numbers.')
```

Practical Project: Generators in Action

We’ve already discussed generators. Now imagine we have a dice game where a sudden change in luck mid-game can feel unfair. To solve this, we’re going to write a program that rolls a pair of dice 10,000 times using `random`, and saves the results to a file.

Python has a library called `pickle`, which lets us take an object and “dump” it into a file with `pickle.dump(file)`. Later, in another program, we can “load” it back with `pickle.load(file)` and get the object again.

Here’s the plan:

- We write a generator that produces dice rolls and saves them to a file.
- In a return match, the game loads that saved data and generates dice rolls from the file one at a time.
- This way, both players get the exact same sequence of dice rolls, and luck is no longer a factor between the two games.

Step 1: Generating and Saving the Dice Rolls — `genDice_generatorsINaction.py`

```
# THIS FILE GENERATES DICES
import random
import pickle
data = []
for i in range(10_000):
    this_one = (random.randint(1, 6), random.randint(1, 6))
    data.append(this_one)

#print(data)
dice_out = open('dices.pickle', 'wb')    # write binary mode
pickle.dump(data, dice_out)             # creates a file to store data
dice_out.close()
```

The commented-out `#print(data)` line is left in as a debugging aid — uncommenting it would let you inspect all 10,000 generated dice rolls in the terminal before they’re written to the pickle file.

Now we can write a separate module that generates one dice roll at a time from this saved data.

Step 2: Reading the Dice Rolls One at a Time — `getDice_generatorsINaction.py`

```
# THIS FILE WORKS AS A GENERATOR FUNCTION AND GIVES US ONE DICE AT A TIME
# USES dices.pickle FILE WHICH IS MADE BY genDice_generatorsINaction.py
import pickle
```

```
def get_dice():
    with open('dices.pickle', 'rb') as data_file:
        data = pickle.load(data_file)

    for dice in data:
        yield dice    # yield makes it work like a generator, one value at a time
                     # we use yield in generator functions, and unlike return, it lets the function return
                     # instead of all at once
                     # with this feature, get_dice becomes an iterable and can be used in loops
                     # or with functions like next()
```

Step 3: Using the Generator in the Game — **gamePy-generatorsINaction.py**

```
from getDice_generatorsINaction import get_dice
dast = 0
for dice in get_dice():
    print(dice)
    if dast > 10:
        break
    else:
        dast += 1
```

This program imports the `get_dice` generator and pulls dice rolls from it one at a time, printing each one. The loop stops after `dast` exceeds 10 — meaning only the first 12 dice rolls from the saved file get printed.

Running the Project

To run this project, save all three files in the same location and run them in order:

```
python3 genDice_generatorsINaction.py
python3 gamePy-generatorsINaction.py
```

The first command generates and saves the dice rolls to `dices.pickle`. The second command reads them back one at a time and prints the first 12 rolls.

Practical Script: Custom Counter

This script counts how many times each item appears in a list and identifies the most common item.

```
l = [1, 2, 3, 3, 4, 4, 3]
#l = []

def custom_counter(l):
    counts = {}
    for item in l:
        if item in counts:
            counts[item] += 1
        else:
```

```

        counts[item] = 1
    #return counts
    most_common = None
    max_count = -1
    for item, count in counts.items():
        if count > max_count:
            max_count = count
            most_common = item
    print(f'list: {l}')
    print(f'occurring counts: {counts}')

    if most_common is not None:
        print(f'most common item: {most_common} with count: {max_count}')
    else:
        print('list is empty')

#print(custom_counter(l))
#print(type(custom_counter(l)))

custom_counter(l)

```

The commented-out `#l = []` line is left in as a quick way to test the empty-list case — uncommenting it (and commenting out the original `l`) would trigger the `else` branch, printing “list is empty”.

The commented-out `#return counts` line shows an earlier version of the function that simply returned the counts dictionary instead of printing a full summary. It’s kept here to show the progression from a simple counting function to one that also analyzes the result.

The last two commented-out lines, `#print(custom_counter(l))` and `#print(type(custom_counter(l)))`, are leftover debugging lines. Since `custom_counter` doesn’t return anything (it only prints), calling `print()` on its result would output `None`, and `type()` would return `<class 'NoneType'>`. These lines are kept to illustrate that point.

Running the Script

To run this script, save it as `counterPractice.py` and run:

```
python3 counterPractice.py
```

Advanced Modules: **collections**

In Python, the most common and useful libraries are built in, so there’s no need to install them with `pip`. In other words, Python comes with “batteries included.” One of these built-in libraries is `collections`, which gives us some useful new data types.

Counter

```
#import collections
from collections import Counter

mylist = [1, 2, 3, 3, 5, 4, 4, 4]
print(mylist)
```

The commented-out `#import collections` line is left in to show an alternative import style — importing the whole `collections` module (and then accessing `collections.Counter`) instead of importing `Counter` directly.

Imagine we wanted to count how many times each item appears in this list. Of course, we could write a function to do that ourselves:

- Inside the function, first declare an empty dictionary, for example `counts`.
- Loop through the items in the list, and for each one:
 - If the item is already in `counts`, increment its count.
 - Otherwise, set its count to 1.
- Declare placeholder variables `most_common = None` and `max_count = 0`.
- Loop through item, count in `counts.items()`, and if `count > max_count`, update `max_count = count` and `most_common = item`.
- Finally, deal with the output and print the results.

But this doesn't look like the easiest solution — and Python has this functionality built in already.

```
c = Counter(mylist)
print(c)
print(c.get(4))
print(c.most_common())
print(c.most_common(1))
```

namedtuple

```
print(20*' '*')
from collections import namedtuple
```

Another useful type from `collections` is `namedtuple`. A `namedtuple` sits somewhere between a class and a tuple — it doesn't do anything unusual, and can be used like a normal tuple, but it also accepts named fields, which makes accessing values much easier.

```
Kelas = namedtuple('Kelas', ['amount', 'average', 'teacher']) # declaring a namedtuple which
print(Kelas) # it looks like a new type and may be similar to classes and OOP
```

We can create new instances from this:

```
physics = Kelas(amount=40, average=12.5, teacher='sora')
print(physics)
print(physics.amount) # accessed by name
print(physics[0]) # accessed like a normal tuple
```

defaultdict

```
print(20*' '*')
a = dict()
a['name'] = 'sora'
print(a)
print(a['name'])
#print(a['familyname'])    # exception: key error.
print(a.get('familyname'), 'key does not exists')
```

The commented-out `#print(a['familyname'])` line is left in to show what would happen if we tried to access a key that doesn't exist using square-bracket notation — it would raise a `KeyError` and crash the program. The line below it, using `.get()`, shows the safer alternative, which returns `None` instead of raising an error.

It can become a real problem when we don't know in advance which keys exist and which don't — our program could crash due to unhandled errors. Fortunately, there's a solution for this too:

```
from collections import defaultdict
# we need to pass it a function that creates default values (the first argument must be callable)
# functions are callable -> lambda functions are a kind of function too

d = defaultdict(lambda : 'Key Does Not Exist')
d['name'] = 'sora'
print(d['name'])
print(d['familyname'])
```

`defaultdict` lets us specify a default value (via a callable, such as a `lambda`) to return whenever a missing key is accessed, instead of raising a `KeyError`.

The datetime Module

```
import datetime

nkh = datetime.date(2026, 1, 1)
print(nkh.day)
print(nkh.month)
print(nkh.weekday())    # Monday=0, ..., Sunday=6; January 1, 2026 is a Thursday, so this returns 3
print(nkh.ctime())

t = datetime.time(10, 25, 40)
print(t.hour)
print(t.minute)
print(t.second)

soraBirth = datetime.datetime(2002, 3, 28, 7, 30, 0)
print(type(soraBirth))
print(soraBirth.ctime())
```



```
print(math.radians(90))
# print(help(math))    # shows help for the math module
```

The `round()` function uses “banker’s rounding” (round-half-to-even) in Python 3, which is why `round(4.5)` returns 4 and `round(5.5)` returns 6 — both round toward the nearest even number rather than always rounding up.

The commented-out `#print(help(math))` line is left in to show how to view the full documentation for the `math` module interactively, without actually running it (since `help()` produces a large block of output).

The random Module

```
import random
print(random.randint(1, 6))

# random.random() returns a floating point number between 0 and 1
print(random.random())
```

Computers can’t generate truly random numbers, because everything works based on an algorithm. The numbers we get are called **pseudo-random numbers**.

Pseudo-random numbers are generated using a formula. When a Python program runs, it takes data from the current time (and other sources) and uses it to generate the next random numbers. This starting data is called a **seed**. We can also specify our own seed in Python.

```
# random.seed(5)
print(random.randint(1, 10))
print(random.randint(1, 10))
print(random.randint(1, 10))
print(random.randint(1, 10))
print(random.randint(1, 10))
```

The commented-out `#random.seed(5)` line is left in to demonstrate seeding. No matter how many times the program runs, if `random.seed(5)` is set before these calls, the sequence of numbers will always be the same — for example: 10, 5, 6, 9, 1.

This is why every time the program runs, it should use a fresh seed, and the default seed behavior generally shouldn’t be manipulated. If an attacker manages to intercept many hundreds of consecutive random numbers, there’s a chance they could work out the seed. Someone reportedly exploited a gambling system this way — by analyzing patterns in the output, they figured out the seed, allowing them to predict future numbers and win repeatedly.

Other Ways to Use random

```
nums = range(1,10)
nums = list(nums)
print(nums)
```



```
randomNum = random.choice(nums)
print(randomNum)
```

`.choice()` is used to pick a single random element from an iterable. It can be useful in games such as rock-paper-scissors.

```
print(random.choices(nums, k=2))
```

`.choices()` returns a list of random elements, with the number of elements specified using the `k=` keyword argument. Note that `.choices()` can return duplicate elements.

```
print(random.sample(nums, k=2))
```

If we want the chosen elements to be unique (no duplicates), `.sample()` should be used instead of `.choices()`.

```
print(nums)
random.shuffle(nums)
print(nums)
print(nums)
```

`.shuffle()` randomly reorders the elements of a list in place — note that it modifies `nums` directly and does not return a new list.

Practical Project: Regular Expressions (regex)

File: `regex.py`

Regular expressions let us search for and refer to specific parts of a text using patterns.

```
# regular expression - re
# we can refer to a specified part of a text
s = 'sora has a phone number: +9812312312312, im sora'
print('9812312312312' in s)    # check if the phone number exists here
print(s.index('9812312312312'))
```

But what if we have a list of people with their phone numbers, and we want to extract all of them? We could write a program that scans the whole input, searches for `+` signs followed by digits, and writes them to another file — but this is exactly the kind of problem regex makes easy.

Regex isn't limited to numbers — almost any pattern can be found with it. We give regex a pattern, and it searches for that pattern in the content.

```
import re

print(re.search('sora', s))    # search only matches the first occurrence of the pattern
print(re.search('ora', s))

r = re.search('sora', s)
print(r.start())
print(r.end())
print(r.group())
print(r.span())
```

If we want to find all occurrences instead of just the first one, we use `.findall`:

```
print(re.findall('sora', s))
```

If we want the results as an iterable, there's also `.finditer`:

```
iterRegex = re.finditer('sora', s)
print(iterRegex)    # it's a callable iterator
for match in iterRegex:
    print(match)     # since it's a re.Match object, we can call methods like .start(), .end(),
```

This covers the basics of regex, but there are also special characters that tell regex to look for specific kinds of patterns:

```
'''
\d -> digit (numbers)
\w -> word (alphanumeric characters)
\s -> space, tab, newline, ...
'''

s = 'sora has a phone number: +9812312312312, im sora'
print(re.findall(r'\d', s))    # like f-strings, we have to put an 'r' before our pattern - th
```

Test Data: regex-notebook.txt

```
sora +093792874 6219-2634-3465-3456
dora +094750385 @doradora
iggy +097486418 @iggyigg2 5859-9349-8347-7483
ghedas 6219-9430-2345-2346
```

File: regex2.py

This file reads `regex-notebook.txt` and applies a series of more advanced regex patterns to it.

```
import re
```

Regex Special Characters Reference

<code>\d</code>	digit
<code>\D</code>	anything but digit
<code>\w</code>	word - alphanumeric
<code>\W</code>	anything but alphanumeric
<code>\s</code>	space, tab, newline
<code>\S</code>	anything but white space
<code>.</code>	any character
<code>*</code>	zero or more of that char, for example: <code>.*</code> means: everything, any number of times, even 0
<code>?</code>	optional - doesn't matter if it exists or not, but if it does, there can only be one
<code>+</code>	one or more of that char
<code>{x}</code>	exactly x occurrences of that char
<code>{x,y}</code>	only from x to y occurrences (no space in {x,y})

- `^` only at the start of the line
- `$` only at the end of the line, for example: `pattern$`
- `()` grouping a wanted part of the regex pattern
- `|` or `- a|b` means either is acceptable
- `[...]` matches if even one of the characters inside the brackets matches
- `[^...]` matches anything except these characters

If we want to match literal characters like `+`, `.`, `?`, and so on, we have to escape them with a backslash (`\`).

Reading and Searching the File

```
with open('regex-notebook.txt', 'r') as f:
    text = f.read()
    print(text)
    print(re.findall(r'\+\d+', text))    # \d
    print(re.findall(r'\+\d*', text))    # \d
    print(re.findall(r'\+\w+', text))    # \w
    print(re.findall(r'\+\d{4}', text))
    print(re.findall(r'\+\d{2,6}', text))
    print(re.findall(r'\@w+', text))
    print(re.findall(r'\^w{4}s', text))    # only returns the first four-character word at the
    print(re.findall(r'\^w{4}s', text, re.MULTILINE))
    # the 're.MULTILINE' flag is needed for regex to recognize \n as a new line
    # in this case, we don't want to include the white space at the end, so we can refine the
    print(re.findall(r'(^w{4})s', text, re.MULTILINE))
    # () says this pattern must be matched, but only the specified part should be returned
    print('*****')
```

Grouping and Identifying Card Numbers

```
matches = re.finditer(r'(\d+)-\d+-\d+-\d+', text) # working with card numbers here
# the goal is to identify cards based on their first four digits
# so a group is specified with () and we can work with that value separately

for match in matches:
    print(match)
    print(match.group(1))    # target group
    card_identifier = match.group(1)
    # group(0) is the entire match, group(1) is the first (...), group(2) is the second (...)
    if card_identifier == '6219':
        print('saman bank:')
    elif card_identifier == '5859':
        print('tejarat bank:')
    else:
        print('unknown card:')

    print(match.group())    # the entire match
```

```
print('*****')
```

Compiling Patterns with `re.compile`

```
# we can build a regex pattern and reuse it later with re.compile
pattern = re.compile(r'\+(\d+)')
iter_regex = re.finditer(pattern, text)
print(iter_regex)

for match in iter_regex:
    print(match.group(1))
    print(match.group())
```

Character Groups and Negated Sets

```
print('*****')
MyVar = 'hello this is l'st sora 09301234567, ghedas is a duck'
print(re.findall(r'th[kjihg]s', MyVar))    # this is a character group
print(re.findall(r'[0-9]+', MyVar))
print(re.findall(r'd[^i]ck', MyVar))
```

The first pattern, `th[kjihg]s`, matches “this” because `i` is one of the characters inside the bracket group. The last pattern, `d[^i]ck`, matches “duck” because `[^i]` matches any character except `i`, and `u` satisfies that condition.

Running the Project

To run this project, save `regex.py`, `regex2.py`, and `regex-notebook.txt` in the same location, then run:

```
python3 regex.py
python3 regex2.py
```

The `decimal` Module

Decimal numbers are not handled in the way you might expect by computers.

```
print( 0.1 + 0.2 == 0.3 )    # how can it be False?
print( 0.1 + 0.2 )          # 0.30000000000000004 != 0.3
```

This can cause serious problems when a program is working with numbers that need to be precise.

```
from decimal import *
# this library makes calculations easier and more accurate
print(getcontext())
'''
Context(prec=28, rounding=ROUND_HALF_EVEN,
Emin=-999999, Emax=999999, capitals=1, clamp=0,
flags=[], traps=[InvalidOperation, DivisionByZero,
```

```
Overflow])
'''
```

We can change this context at any time:

```
# we can change this context anytime
getcontext().prec = 3    # sets precision to 3 significant digits
print(Decimal(1)/Decimal(3))
print(Decimal('0.1') + Decimal('0.2') == Decimal('0.3'))
print(Decimal(3.14))
# this can be a little confusing, but we can use this form as a number
print(Decimal('3.14') > 2)    # true

print('1.2 2.3 3.4 4.5 5.6'.split())
```

We can also use `Decimal` on a larger collection of numbers:

```
decNumbers = list(map(Decimal, '1.2 2.3 3.4 4.5 5.6'.split()))
print(decNumbers)
print(min(decNumbers))
print(max(decNumbers))    # works just like a normal number!
print(sum(decNumbers))
```

`getcontext()` returns the current arithmetic context for `Decimal` operations, including `prec` (precision), the rounding mode, and other settings. Setting `getcontext().prec` changes how many significant digits are used in subsequent `Decimal` calculations — this is different from setting a fixed number of decimal places.

The `sys` Module

The `sys` module helps us interact with the environment in which Python is running.

```
#!/usr/bin/python3
import sys

# you can forward your stdout to a file with sys.stdout like this:
sys.stdout = open('SYS-OUTPUT.TXT', 'w')    # DON'T FORGET TO CLOSE THE FILE AT THE END
# you can do the same with stderr
sys.stderr = open('SYS-ERRORLOG.TXT', 'w')

#importlsfjkl    # making an error to test the stderr file handling
```

The commented-out `#importlsfjkl` line is left in deliberately — it's an intentional typo/invalid statement used to test that errors get correctly redirected to the error log file (`SYS-ERRORLOG.TXT`) instead of the terminal.

Reading from `stdin`

Another useful feature of this module is reading input from a pipeline using `stdin`:

```
input1 = sys.stdin.readline().strip()
input2 = sys.stdin.readline().strip()
```

Here's how this looks in practice with more than one input value:

```
sora@tool [~/python-practice] [03:03:20] $ cat SYS-OUTPUT.TXT
your first input was: sora and second input was: dora
```

```
if input1 and input2:
    print(f'your first input was: {input1} and second input was: {input2}')
else:
    print('no data recieved from stdin')
```

Command-Line Arguments with `sys.argv`

The simplest way to get user input is via command-line arguments, rather than the `input()` function:

```
try:
    print(f'hello my name is {sys.argv[1]} and im {sys.argv[2]} years old')
    # if you run this program with two arguments – for example: SYS.py sora 23 – it will print
except IndexError:
    print('enter arguments: SYS.py arg1 arg2')
    # you can even handle program exit with:
    sys.exit(0)
```

Example runs:

```
sora@tool [~/python-practice] [02:08:55] $ python3 SYS.py sora 23
hello my name is sora and im 23 years old
```

```
sora@tool [~/python-practice] [02:10:31] $ python3 SYS.py
enter arguments: SYS.py arg1 arg2
```

`sys.version` and `sys.path`

```
print(help(sys))
print(sys.version)    # 3.13.5 (main, Jun 25 2025, 18:55:22) [GCC 14.2.0]    returns the current
for path in sys.path:
    print(path)
```

`sys.path` returns the module search path — the list of directories Python searches through when looking for modules to import:

```
['/usr/lib/python313.zip', '/usr/lib/python3.13',
 '/usr/lib/python3.13/lib-dynload', '', '/home/sora/.local/lib/python3.13/site-packages',
 '/home/sora/python-practice/libraryTask', '/usr/local/lib/python3.13/dist-packages',
 '/usr/lib/python3/dist-packages']
```

This is the same `libraryTask` directory and `setup.py` discussed in an earlier project — that's why its path appears in `sys.path`.

We can also append additional paths at runtime:

```
#import mymod          # error because this module is not in sys.path
#print(mymod.my_pi)    # also error because mymod is not defined
sys.path.append('/home/sora/python-practice/packagesAndModules')
```

These two commented-out lines show what would happen if we tried to import `mymod` before adding its directory to `sys.path` — both lines would raise an error, since Python wouldn't be able to find the module yet.

If we append a new path like this, it becomes part of `sys.path`. If we want to remove it afterward, we can use `.pop()` — or simply leave it, depending on the use case:

```
import mymod
print('*****')
print(mymod.my_pi)
print('*****')

#sys.path.pop()

for path in sys.path:
    print(path)
```

The commented-out `#sys.path.pop()` line is left in to show the optional cleanup step — removing the appended path after it's no longer needed.

```
# print(sys.modules)    # returns a dictionary of all loaded modules
# i comment this line to keep the outputs readable
```

`sys.modules` returns a dictionary of all currently loaded modules. This line is commented out simply because its output would be very large and clutter the terminal.

I/O Redirection

Another useful feature of this module is I/O management — `stdout` and `stderr` can be redirected to files. This is useful for saving program output to a file, or logging errors separately.

```
sys.stdout.close()
sys.stderr.close()
```

Closing these file objects at the end ensures the redirected output is properly written and saved to disk.

Running the Script

To run this script, save it as `SYS.py`, make it executable, and run it with command-line arguments:

```
chmod +x SYS.py
./SYS.py sora 23
```

Or run it directly with `python3`:

The os Module

The `os` module handles interaction with system operations — executing commands, creating directories, and much more.

```
import os
```

```
print(os.name)    # identifies the operating system
```

`posix` indicates Linux or macOS (Unix-like systems), while `nt` indicates Windows. For more detailed information, there's also the `platform` module:

```
import platform
print(platform.system())    # kernel
print(platform.release())  # release version
print(platform.version())  # more detailed information
```

Working with Directories

```
print(os.getcwd())    # returns the current working directory
```

We can change directories with:

```
# we can change directory with:
os.chdir('/home/sora/')
print(os.getcwd())
```

We can also create new folders:

```
# or make new folders:
os.mkdir('pythonDir')    # creates a directory
os.chdir('./pythonDir')  # changes into the directory we just created
print(os.getcwd())       # shows the current working directory
os.chdir('../')          # goes back to the parent folder
# and remove them with:
os.rmdir('pythonDir')    # only works if the directory is empty
```

If the folder isn't empty, `rmdir` won't work, and we'd have to list the files and remove them one by one. If we want to remove a directory recursively, `os` can't do this on its own — we need the `shutil` module instead:

```
import shutil
shutil.rmtree('/path/to/test_folder')    # works exactly like rm -rf in bash
```

`shutil` can also perform many high-level file operations, such as copying and moving files.

Working with Files

```
f = open('testPythonfile.txt', 'w+')    # creates a file
f.write('hello this is some test text\nand this is a new line\n')    # writes content to the file
```



```
f.seek(0)    # moves the cursor back to position 0
print(f.read())    # reads the file
f.close()    # closes the file

os.remove('testPythonfile.txt')    # removes the file
```

Listing Directory Contents

```
# to get a list of directory and path contents, we have os.listdir:
files = os.listdir('.')    # . means current directory

for file in files:
    #print(file)
    if os.path.isfile(file):
        print(file+'    *is a file')
    elif os.path.isdir(file):
        print(file+'    /is a directory')
```

The commented-out `#print(file)` line is left in to show a simpler alternative — printing each entry’s raw name without checking whether it’s a file or a directory.

Environment Variables

```
print('*****')
```

The `os` module also exposes environment variables such as `PATH`, `HOME`, and more. We can read these values, modify them, or even create new ones.

```
print(os.environ['HOME'])    # returns the current user's home directory
print(os.environ['PATH'])    # the system's executable search paths
```

It’s also easy to change environment variables or set new ones:

```
os.environ['HTTP_PROXY'] = 'http://127.0.0.1:8080'
print(os.environ['HTTP_PROXY'])
```

Running System Commands

One of the most important features of the `os` module is the ability to execute system commands:

```
os.system('cd python-practice')
os.system('ls')
```

However, `os.system` is stateless and doesn’t preserve state between calls. Each time `os.system` runs a command, it opens a new shell, executes the command, and immediately closes that shell. If we want behavior that depends on a previous command’s state (such as a changed directory), we have to chain all the related commands into a single line, separated by semicolons:

```
os.system('ls -ltrh; cd python-practice/; echo "this is python-practice dir: "; ls -ltrh')
```

`popen()` and the `subprocess` module follow the same underlying pattern: open a shell, run a command, then close the shell. `os.system` only executes the given command and does not return its output. To capture the output, we should use `popen()` or the `subprocess` module instead.

```
SYSTEM = os.system('ls')
POPEN = os.popen('ls')
print(SYSTEM)
print(POPEN.read())    # behaves like a file object, giving us access to the output
POPEN.close()
```

The `subprocess` Module

The `subprocess` module works similarly to `os.system`, but it's more advanced and flexible.

```
import subprocess
print(subprocess.call(['ls']))    # a simple usage example
print(subprocess.call('ls -l; whoami', shell=True))    # instead of passing the command as a list
print(subprocess.run('ls -ltrh', shell=True))    # also returns a status: CompletedProcess(...)

print()
print('*****')
print()

result = subprocess.run(['ls', '-l'], capture_output=True, text=True)
print(result.stdout)
```

`subprocess.call()` runs a command and returns its exit code. `subprocess.run()` is the more modern and recommended approach — it returns a `CompletedProcess` object containing details such as the return code, and, when `capture_output=True` is set, the command's standard output and error streams as well.

```
'''
in my opinion, this module is more complicated to use than os.system, and might feel unnecessary
'''
```

That said, `subprocess` is generally preferred over `os.system` for anything beyond trivial commands, since it gives much finer control over input, output, and error handling, and avoids some of the security risks associated with `shell=True`.

Running the Script

To run this script, save it as `Subprocess.py` and run:

```
python3 Subprocess.py
```

Web Scraping with `requests` and `bs4`

Web scraping is the process of extracting data from web pages programmatically. This is an advanced and particularly useful technique for security researchers and developers alike. For example, we could write a program that opens a car-selling website, visits each

car's page one by one, and extracts the listed prices — automating a task that would otherwise take hours to do manually.

Web pages are designed to be human-readable, but not directly machine-readable. The tools covered in this module help bridge that gap, allowing a program to parse and extract structured data from raw HTML.

Before diving in, it's useful to have some basic knowledge of HTML tags and how websites are structured, since scraping involves navigating the HTML document to find the specific elements containing the data we want.

Installation

Before starting, we need to install two libraries: `requests`, for making HTTP requests to fetch web pages, and `bs4` (Beautiful Soup), for parsing and navigating the resulting HTML.

```
pip install requests
pip install bs4
```

`requests` handles the network side — sending HTTP requests and retrieving the raw HTML content of a page. `bs4` then takes that raw HTML and provides convenient methods for searching through it, such as finding all elements with a particular tag, class, or attribute, making it much easier to locate and extract the specific data we're after.

The `requests` Module

There are different levels at which a program can make network requests. At the lowest level, there's the `socket` module, which works directly with hardware and network interfaces. A level above that is `urllib`, which provides a higher-level interface over sockets. For this module, we'll be using `requests` — a high-level library designed specifically for making HTTP requests easily.

```
import requests
```

There are multiple HTTP request methods, such as GET, POST, HEAD, PUT, PATCH, DELETE, and more. The `requests` library can handle all of these.

```
target = 'https://target.com/'
r = requests.get(target)
```

HTTP response codes are grouped into ranges that indicate the type of result:

```
2XX    OK
3XX    REDIRECT
4XX    CLIENT-SIDE ERROR
5XX    SERVER-SIDE ERROR
```

```
print(r)      # <Response [200]>
print(type(r)) # <class 'requests.models.Response'>
print(r.status_code) # 200
print(r.url)   # the target URL
print(r.text)  # the actual response text, including all HTML tags (page source)
```

Extracting Data with Regex

At this point, we can try to extract specific information from the response using regex. For example, let's try to extract all URLs found on this page:

```
import re
#print(re.findall(r'http[s]?://[^\s"'>]+', r.text))
links = re.findall(r'http[s]?://[^\s"'>]+', r.text)
print('***')
print('all urls in this page:')
print()
for link in links:
    print(link)
    print('-----')
```

The commented-out `#print(re.findall(...))` line is left in as an earlier, simpler version of the next line — it shows the result printed directly in one step, before it was split into a separate `links` variable for further processing in the loop below.

However, relying on complicated regex patterns like this isn't ideal for parsing HTML. This is exactly why we need BeautifulSoup — a parser that turns the raw HTML into a DOM-like object, making it much easier to navigate and extract specific elements.

Parsing HTML with BeautifulSoup (bs4)

bs4 (Beautiful Soup) can parse HTML text and extract data based on specific needs.

```
import requests

target = 'https://target.com/'
res = requests.get(target)

import bs4
soup = bs4.BeautifulSoup(res.text)

print(soup)
```

Note: in practice, it's recommended to explicitly specify a parser, e.g. `bs4.BeautifulSoup(res.text, 'html.parser')`, to avoid warnings and ensure consistent parsing behavior.

Now Python can understand the HTML structure as a navigable object.

Selecting Elements with `.select()`

The `.select()` method looks for elements matching a given CSS-style selector:

```
print(soup.select('title'))    # returns a list containing the page's <title> tag
print(soup.select('title')[0]) # returns the first <title> tag by index 0 (there's usually
```

We can store these results in variables and work with them however we want:

```
Title = soup.select('title')[0]
print(type>Title))
# this is not a string – it's an object Python understands as: <class 'bs4.element.Tag'>
```

```
# essentially, it represents a single tag/element within bs4
# and each of these objects has many attributes and methods
print(Title.getText())    # returns the text inside the tag
# Title.get_text() does the same thing, just with different spelling – or use the .text attrib
```

Extracting Links

```
Links = soup.select('a')
print(len(Links))
for link in Links:
    print(f'{link.text} : {link.attrs["href"]}')    # prints the link text and its href attrib
```

To access tag attributes, there are a few different approaches. The simplest is to access an attribute directly by name, such as `link.get("href")`. Additionally, every element object in bs4 has an `.attrs` attribute, which is a dictionary containing all of that element's attributes.

Selecting by Class and ID

We can also search by class name or ID, not just by tag name:

```
textPrimaryDark = soup.select('.text-primary-dark')    # select by class name
# selecting by ID would be: soup.select('#targetElementId')
for t in textPrimaryDark:
    print(t.text)
```

Advanced Selectors

Using `.select()` like this returns all matching elements. If we want, for example, a `<span>` that's nested inside a `<p>`, we can select it this way:

```
soup.select('p span')
```

The same applies to class names and IDs too:

```
soup.select('div .someclass')
```

And if we want only a specific tag with a given class name — for example, `<p>` tags — we can write:

```
soup.select('div p.someclass')
```

This returns all `<p>` tags that are inside a `<div>` and have the class `someclass`.

If we want to select only direct (first-level) child elements, we can use the `>` operator. For example:

```
soup.select('div > span')
```

This would return only the elements marked with `*` below — the direct child `<span>` elements of the `<div>`, but not the `<span>` nested further inside them:

```
<div>
  <span>*
```

```
<span> something </span>
</span>
<span>*
    hello there
</span>
</div>
```

Introduction to Selenium

More Accurate Element Selection

For more accurate element selection on web pages, there's a useful technique: in the browser, right-click on the target element, choose "Inspect Element," then copy its CSS selector.

Imagine a site that sells products, where every product has its own dedicated page. If these single-product pages are structurally similar to each other (which is the case on well-built sites), then the exact selector we copied from one product page will point to the same element — such as the price — on every other product page too.

So, if we want to read data from all of these pages using a single selector, we can access the prices of all products with that one selector. We could write a program that loads every single-product page and reads the title and price from each one. There's a website called Torob that works in a similar way.

There are also browser add-ons for CSS selectors that make this process even easier and more accurate — some of these plugins let us select multiple items at once for better precision.

Limitations of Static Scraping

These methods do have some limitations. Some sites load their data via AJAX, and others restrict or limit request access.

This is where tools like Selenium come in — they can open pages like a real browser, load the site's data, and let us interact with elements directly.

What Is Selenium?

Selenium is a powerful tool built for web scraping and browser automation. We can control Selenium through code to behave like a real user at a computer:

- open web pages
- click on buttons
- enter text
- take screenshots of the website
- extract dynamic data generated by JavaScript
- perform complex operations like logging in, and more

Selenium vs. requests

	REQUESTS	SELENIUM
	raw HTTP request cannot execute JS	controls an actual browser can execute JS, since it has a browser engine
	gets the initial HTML	gets HTML after JS manipulations
	easier to use faster for static pages	more complicated slower, since it runs a browser (though it can also run headless)
	for quickly retrieving static info only the <code>requests</code> package	web automation, testing, scraping dynamic pages Selenium + a browser WebDriver package

After retrieving data with Selenium (instead of `requests`), we can still continue using `bs4` for parsing the resulting HTML.

Automating Everyday Tasks

Selenium can also be very useful for automating everyday tasks, such as:

- open the browser
- search on google.com
- search for “download some music”
- go to a result page
- download a song
- ...and so on

When we write a Python program that performs a sequence of tasks like this, we can call it a “bot.”

This file is more theoretical, covering what Selenium is and what it can do. The actual commands and usage examples will be shown in a separate file. Essentially, with a Python script using Selenium, we can do whatever a real browser can do.

Practical Script: Selenium in Action

Installation and Setup

Before running this script, Selenium needs to be installed, along with a WebDriver. A WebDriver is an executable file that lets Selenium control a browser. Depending on the browser being used, the matching WebDriver must be installed — this example uses Firefox, so `geckodriver` is required.

```
sudo apt update && sudo apt install firefox-esr 'or' firefox
pip install selenium
```

```
# download geckodriver from GitHub, then:
tar -xvzf geckodriver-v0.34.0-linux64.tar.gz
```

```
# this creates a file named: geckodriver
sudo mv geckodriver /usr/local/bin/
sudo chmod +x /usr/local/bin/geckodriver
```

Setting Up the WebDriver

```
from selenium import webdriver      # import webdriver from selenium

from selenium.webdriver.firefox.service import Service
# this is used to tell the system exactly where geckodriver is located

from selenium.webdriver.common.by import By      # By is needed for selecting elements
from selenium.webdriver.common.keys import Keys   # Keys is used to simulate keyboard input
import time      # used to give pages enough time to load, or to normalize timing

geckodriver_path = '/usr/local/bin/geckodriver'
service = Service(executable_path=geckodriver_path)
# after specifying the executable path, we pass service=Service(executable_path=...) to Firefox

driver = webdriver.Firefox(service=service)
# opens a Firefox window; if using Chrome with chromedriver, this would be webdriver.Chrome()
```

Navigating to a Page and Selecting Elements

```
driver.get('https://target.com/')    # sends a GET request to the given address
time.sleep(1)    # wait a second

#search_box = driver.find_element(By.CSS_SELECTOR, "div.bg-bgSearchBox.relative.flex.h-[48px]
search_box = driver.find_element(By.XPATH, "/html/body/div[2]/div/div[2]/div/div/div[1]/div/di
```

The commented-out CSS selector line shows an alternative way of locating the same element. It's kept here for comparison with the XPath approach used on the line below.

To select elements, XPath is another option that works even when `NAME`, `ID`, `CLASS_NAME`, or `CSS_SELECTOR` aren't available. XPath is based on the nested tag structure leading to the target element, similar to a file path:

```
/html/body/div[2]/div/div[2]/div/div/div[1]/div/div[3]/div/div/div[1]/input
```

Here, `div[3]` means the third `<div>` element. The default index is 1 (equivalent to writing the element with no brackets at all), and indexing starts from 1, not 0. Attributes can also be used instead of numeric indexes, for example: `element[@id='some_id']` or `element[@name='some_name']`. The easiest way to get an XPath is to right-click the element in the browser, choose “Inspect Element,” then copy the XPath.

For links specifically, `find_element(By.LINK_TEXT, 'this is a link')` can be used, along with `PARTIAL LINK TEXT`, which matches part of the link's text.

Interacting with the Page

```
search_box.click()    # click on the element
time.sleep(1)
search_box.send_keys('hello world!')    # type some input
time.sleep(1)    # wait a second
search_box.send_keys(Keys.RETURN)    # press Enter on this field

time.sleep(5)
```

Finding and Clicking Links

```
res_links = driver.find_elements(By.TAG_NAME, 'a')    # find_elements (plural) returns all matches
for link in res_links:
    href = link.get_attribute('href')    # get_attribute retrieves the requested attribute from the element
    if href and 'wikipedia.org' in href:    # if the attribute exists and contains the target text
        link.click()
        break
```

Taking a Screenshot

```
screenshot_filename = "selen_results.png"
time.sleep(10)
driver.save_screenshot(screenshot_filename)    # takes a screenshot of the browser window
print(f"Screenshot saved: {screenshot_filename}")
```

Other Useful Features

```
print(driver.current_url)    # the current URL address
print(driver.page_source)    # the page's HTML source, which can be passed to bs4 for parsing
```

There are several other useful features available for web scraping with Selenium, which will be covered in a separate file — particularly around running the browser in headless mode for more efficient scraping.

```
driver.quit()    # always close the browser when finished
```

Troubleshooting: geckodriver Errors

If you see errors like this:

```
sora@tool [~/python-practice] [14:49:17] $ python3 SeleniumInAction.py
Problem reading geckodriver versions: error sending request for url (https://raw.githubusercontent.com/mozilla/geckodriver/releases/latest)
Exception managing firefox: error sending request for url (https://github.com/mozilla/geckodriver/releases/latest)
```

This means Selenium is trying to automatically download geckodriver or check for its latest version via GitHub. If there's no internet connection, this will fail and raise errors.

The solution is to manually download geckodriver, place it somewhere in the system PATH,

and explicitly tell Selenium where it is via the `Service` object, instead of letting Selenium try to locate it automatically:

```
from selenium.webdriver.firefox.service import Service

geckodriver_path = '/usr/local/bin/geckodriver'
service = Service(executable_path=geckodriver_path)

driver = webdriver.Firefox(service=service)
```

Selenium then uses this prepared file directly. This is exactly why these changes were made in the script above.

Bonus: Selenium IDE

There's also a browser extension called Selenium IDE, which can record browser interactions and convert them directly into Selenium code.

Practical Script: Web Scraping with Selenium (Headless Mode)

Background

Earlier, we worked with `requests` combined with `bs4`. We know that `requests` is excellent for working with APIs and static pages, and it's extremely fast. However, `requests` cannot handle dynamic content, which is very common on modern websites. There are workarounds — such as specifying exact URLs for underlying API endpoints — but in many cases, what we actually need is something that can execute JavaScript and read the page after it's been built into the DOM.

Selenium uses a real browser engine to interact with web pages, which makes it powerful and reliable for this kind of task.

This file demonstrates web scraping with Selenium, and also covers how to run it using a headless browser for better speed and lower memory usage.

Headless Browser Setup

To run Selenium in headless mode, the following changes need to be made:

1. Import `Options` from `selenium.webdriver.firefox.options`, alongside the other imports.
2. Create an `Options()` instance.
3. Call `options.add_argument("--headless")` — this is the line that actually enables headless mode.
4. Pass `options=options` when creating the `webdriver.Firefox(...)` instance.

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.firefox.service import Service
from selenium.webdriver.firefox.options import Options    # step 1 of headless setup
```

```

geckodriver_path = '/usr/local/bin/geckodriver'

options = Options()      # step 2 of headless setup
options.add_argument("--headless")    # step 3 of headless setup

service = Service(executable_path=geckodriver_path)

driver = webdriver.Firefox(service=service, options=options)    # step 4 of headless setup

```

Loading a Page

```

driver.get('https://target.com/')

print('current url:')
print(driver.current_url)    # returns the current URL, i.e. the URL in the address bar

print('response text: ')
print(driver.page_source)    # works similarly to r = requests.get('https://site.com/') with r.text
# this output can also be passed to bs4 for parsing

```

Extracting Links

```

links = driver.find_elements(By.TAG_NAME, 'a')    # find_elements stores multiple matched elements

print()
print('iterable object of matched elements: ')
print(links)
print()

for link in links:
    linkText = link.text
    href = link.get_attribute('href')    # get_attribute retrieves the requested attribute from the element
    print(f'{linkText} : {href}')
    print('-----')
    print(link.get_attribute('outerHTML'))    # returns the entire tag, including its HTML
    print('-----')

driver.quit()    # always remember to close the driver when finished, or resources will remain open

```

A Note on Selenium’s Capabilities

Selenium cannot send raw HTTP requests directly — it doesn’t have built-in methods for GET, POST, PUT, PATCH, DELETE, and so on. Selenium was built for automating browser interactions. For example, to send a POST request via Selenium, the page would need to be opened, a form located, input entered into its fields, and the submit button clicked — and that’s where Selenium’s strength lies.

That said, Selenium can always be used as a way to receive responses from websites that load their data dynamically via JavaScript, even if it can’t replace `requests` for direct

HTTP calls.

Working with Images — Pillow

Pillow is a Python library for image processing. Install it with `pip install Pillow`.

```
'''
pillow is a python library to work with images
'''

from PIL import Image
```

Opening and Inspecting an Image

```
img = Image.open('/home/sora/Pictures/ScreenshotLXDE.png')    # opens the image
print(img)
# img.show()      # displays the image

print(img.info)        # {'Software': 'gnome-screenshot'}
print(type(img))       # <class 'PIL.PngImagePlugin.PngImageFile'>
print(img.size)        # (1366, 768)
print(img.filename)    # /home/sora/Pictures/ScreenshotLXDE.png
print(img.format)      # PNG
print(img.mode)        # RGB

print('exif data:')
print(img.getexif())
```

Reading and Writing Pixels

Read the RGB value of a specific pixel:

```
print(img.getpixel((100, 100)))    # returns something like (10, 10, 10) – an RGB tuple
```

Write a single custom pixel:

```
img.putpixel((100, 100), (255, 0, 0))    # places a red pixel at position 100,100
```

Write to a range of pixels — useful for larger pixel-level manipulation:

```
for i in range(10):
    for j in range(10):
        img.putpixel((100+i, 100+j), (i*9, j*12, i*15))
# img.show()
```

Rotating

Rotate an image by a given number of degrees. Negative values rotate in the opposite direction:

```
img2 = img.rotate(90)    # rotates 90 degrees counter-clockwise; use rotate(-90) for clockwise
# img2.show()
```

Cropping and Pasting

To crop, specify two points: (x1, y1) as the top-left corner and (x2, y2) as the bottom-right corner:

```
cropped_img = img.crop((400, 0, 500, 700))
# cropped_img.show()
```

The cropped section can be pasted onto the original image or any other image:

```
img.paste(cropped_img, (0, 0))
# img.show()
```

Transparency and Saving

Set image transparency with `putalpha()` — the value ranges from 0 (fully transparent) to 255 (fully opaque):

```
img.putalpha(200)
# img.show()
```

Save the modified image to disk:

```
img.save('chert.png')
```

Practical Project: Image Steganography

Steganography is the practice of hiding secret data inside an ordinary, non-secret file. This project hides a text message inside an image by encoding each character's ASCII value into the red channel of pixels sampled across the image.

How It Works

Each character of the secret message is converted to its ASCII integer value using `ord()`. These values are then written into the red (R) channel of pixels spaced 10 pixels apart across the image. On decoding, the red channel values are read back and converted to characters using `chr()`. A value of 0 in the red channel signals the end of the message.

Encoder

Save the following into a file called `stegCoder.py` and run it with `python3 stegCoder.py`:

```
#!/usr/bin/python3
```

```
from PIL import Image

image_path = input('enter image path: \n')
img = Image.open(image_path)

secret = input('enter your secret message: \n')
print(f'your secret message is: {secret}')
print(f'image size is: {img.size}')

coded_nums = []

if secret:
    for s in secret:
        print(s)
        coded_nums.append(ord(s))
else:
    print('secret is empty')

print(coded_nums)

def get_char():    # a generator that yields one character value at a time
    for char in coded_nums:
        yield char

char_generator = get_char()

img_size = img.size
X = img_size[0]
Y = img_size[1]
print(f'img width is: {X}')
print(f'img height is: {Y}')

img_format = img.format.lower()

for i in range(0, X, 10):
    for j in range(0, Y, 10):

        try:
            current_num = next(char_generator)
        except:
            current_num = 0    # 0 signals end of message

        r, g, b = img.getpixel((i, j))
        new_color = (current_num, g, b)
```

```
img.putpixel((i, j), new_color)

img.save(f'codedIMG.{img_format}')
```

Decoder

Save the following into a file called `stegDecoder.py` and run it with `python3 stegDecoder.py`:

```
#!/usr/bin/python3
```

```
from PIL import Image
```

```
image_path = input('enter image path: \n')
img = Image.open(image_path)
```

```
decoded_nums = []
img_size = img.size
X = img_size[0]
Y = img_size[1]
stop = False
```

```
print(f'img width is: {X}')
print(f'img height is: {Y}')
```

```
for i in range(0, X, 10):
    for j in range(0, Y, 10):

        r, g, b = img.getpixel((i, j))
        print(chr(r))

        if r != 0:
            decoded_nums.append(chr(r))
        else:
            stop = True    # red channel value of 0 marks end of message
            break

    if stop:
        break
```

```
print(decoded_nums)
```

```
message = ''
for i in decoded_nums:
    message += i
```

```
print()
print('your secret message: ')
```

```
print(message)
```

Notes

- The image must be large enough to hold the entire message. With a step of 10 pixels in both directions, a 1366x768 image can hold up to $(1366/10) * (768/10) \approx 10,488$ characters.
- This technique modifies the red channel of sampled pixels. Visually, the changes are imperceptible to the human eye.
- The encoded image is saved as `codedIMG.png` in the working directory.
- This is a basic implementation for educational purposes. Real-world steganography tools use more sophisticated encoding schemes to avoid detection.

Working with CSV Files

CSV (Comma-Separated Values) is a popular and widely used data format. A phonebook in CSV format looks like this:

```
sora,0912123123,Tehran,sora@gmail.com
dora,0937123123,NYC,dora@gmail.com
```

In this section we use Python's built-in `csv` module. For more advanced use cases, `pandas` is a significantly more powerful alternative.

fake_phone_book.csv (sample)

```
row_number,name,social_id_number,lastname,gender,phone_number,address
1,Douglas,636713361,Chen,male,001-474-278-3469x202,"379 Turner Oval Apt. 004, Jeffreyside, NV 48
2,Michelle,723508723,Wilson,female,(333)447-2206x32349,"992 Isabella Summit, Sarahburgh, MI 162
3,Kevin,731492430,Hunter,female,(813)437-7615,"14768 Richard Park Suite 780, Port Kristinburgh,
```

Reading a CSV File

```
import csv

myfile = open('fake_phone_book.csv', encoding='utf-8')    # encoding specified in case the fil
csvData = csv.reader(myfile)    # reader() reads the .csv file and returns an iterable line by

print(csvData)
for line in csvData:
    print(line)    # prints each line

print('*****')

for line in csvData:
    print(line)
```



```
# this will not print anything – csvData acts like a file pointer  
# once it has been read to the end, it has no seek() method to reset
```

To read the data again, the file must be reopened:

```
myfile.close()  
myfile = open('fake_phone_book.csv', encoding='utf-8')  
csvData = csv.reader(myfile)  
# alternatively: csv.DictReader(myfile) reads each row as a dictionary  
  
lines = list(csvData)    # load all data into a list  
  
print(lines)  
print('*****')  
print(lines[-1])        # last person's information  
print(lines[-1][4])      # last person's gender  
print('*****')
```

Counting how many entries exist for each gender:

```
sex = {}  
for thisLine in lines[1:]:    # skip the header row at index 0  
    # print(thisLine[4])    # prints the gender column for each row  
    s = thisLine[4]  
    if s in sex:  
        sex[s] += 1  
    else:  
        sex[s] = 1  
  
print(sex)
```

Writing a CSV File

csv.writer allows writing rows to an output file. The `delimiter` parameter sets the character used to separate values — here we use `|` instead of the default comma:

```
outfile = open('csvOutput.csv', 'w', newline='')  
csv_writer = csv.writer(outfile, delimiter='|')  
csv_writer.writerow([1, 2, 3])  
csv_writer.writerow(['sora', 'dora', 'iggy'])  
outfile.close()
```

csvOutput.csv

```
1|2|3  
sora|dora|iggy
```

Working with PDF Files

Python can read and write PDF files using the PyPDF2 library. Install it with `pip install PyPDF2`.

```
import PyPDF2

# open the pdf file
mypdf = PyPDF2.PdfReader('/path/to/pdf/file')

len(mypdf.pages)          # get the total page count
firstPage = mypdf.pages[0] # select a page by index
firstPage.extract_text()   # extract and return the text content of the page
```

PyPDF2 also supports writing and merging PDF files, as well as more advanced operations like extracting images and encrypting documents. Refer to the official documentation at <https://pypdf2.readthedocs.io> for further details.

Working with Excel Files (XLSX)

Python can read and write Excel files using the `openpyxl` library. Install it with `pip install openpyxl`.

```
import openpyxl
import datetime
```

Reading an Excel File

```
wb = openpyxl.load_workbook('someExcelFile.xlsx') # load a workbook
print(wb)

# a workbook can have multiple sheets, so always select one before working with it
ws = wb.active # select the active sheet
print(ws)      # <Worksheet "Sheet1">
```

Accessing individual cells by their address:

```
print(ws['A4'].value)
print(ws['B5'].value)
```

```
thisCell = ws['E2']
print(thisCell.value)
```

Iterating over a range of cells to aggregate values:

```
totalAge = 0
for i in range(2, 6):
    this_cell = ws[f'F{i}']
    totalAge += this_cell.value
```

```
print(totalAge)
```

Writing an Excel File

```
newWb = openpyxl.Workbook()    # create a new workbook
Ws = newWb.active              # select the active worksheet

Ws['A1'] = 'sora'
Ws['A2'] = datetime.datetime.now()
Ws['F5'] = 'dora'

newWb.save('createdWithPython.xlsx')    # save the workbook to disk
```

JSON and APIs

Applications often need to communicate with each other — one application requests data from another and expects a response. This concept is called an **API** (Application Programming Interface). REST APIs are the most common implementation of this today, typically operating over HTTP/HTTPS by sending requests to a URL.

Data can be transferred in many formats. One of the most common is **JSON** (JavaScript Object Notation), which is supported by almost every programming language, making it ideal for cross-application communication.

JSON in Python

JSON looks very similar to Python dictionaries:

```
import json

data = {
    'name': 'sora',
    'age': 23,
    'fun': 'hacking',
    'coolness': 5
}

print(type(data))
```

When sending data to another application, the receiving end may be written in a completely different language. Converting to JSON solves this — it is a universal format understood across languages:

```
jdata = json.dumps(data)    # converts a Python dict to a JSON string
print(jdata, '              json form')
```

When receiving JSON data from an external source, it arrives as a string and cannot be used directly in Python. Use `json.loads()` to convert it back into a Python dict:

```
newData = json.loads(jdata)

print(newData, '      python form')
print(newData['age'])
```

JSON Method Reference

```
'''
json.dumps    takes a dict and turns it into a json string
               data_python = {'name': 'Bob', 'city': 'Tehran'}
               json_string = json.dumps(data_python)

json.dump     takes a dict and writes it as json directly to a file
               with open('output.json', 'w') as f:
               json.dump(data_python, f, ensure_ascii=False)

json.loads    takes a json string and turns it into a python dict
               json_string = '{"name": "Alice", "age": 30}'
               data_python = json.loads(json_string)

json.load     takes a json file and turns it into a python dict
               with open('output.json', 'r') as f:
               data_python = json.load(f)

response.json() from the requests library – works like json.loads on an HTTP response body
'''
```

Practical Script: Making API Calls

The `requests` library is the standard way to make HTTP requests in Python. Install it with `pip install requests`.

This example calls a public prayer times API and parses the JSON response:

```
import requests
import json

r = requests.get('https://example.com')
j = json.loads(r.text)

# requests can also parse json directly, which is equivalent to the line above:
# j = r.json()

print(j)
```

```

print('*****')
if r.status_code == 200:
    for item in j:
        print(f'{item} is : {j[item]}')
else:
    print('failed to send request')
print('*****')

```

Checking `r.status_code` before processing the response is good practice — a status code of 200 means the request was successful.

Sending Data with POST

To send data to an API endpoint, use `requests.post()` with a JSON payload:

```

'''
url = 'https://example.com'

data = {
    'id': 5,
    'name': 'sora',
    'age': 23
}

res = requests.post(url, json=data)

print(res.json())
'''

```

Bonus: Building a Web Server with Flask

In the previous section we consumed data from a server. This section covers the other side — being the server.

Flask is a lightweight Python web framework. There are more powerful alternatives like Django, but Flask is minimal, fast, and easy to get started with. Install it with `pip install flask`.

Basic Flask App

Save the following into a file called `FlaskBasics.py` and run it with `python3 FlaskBasics.py`:

```

from flask import Flask, request

app = Flask(__name__)    # flask requires this - __name__ tells flask where to look for resources
# flask is a micro-framework, similar to unittest in how it dictates structure

```

```

@app.route('/')    # declares a route – this is a decorator
def hello_world(): # the function tied to this route returns what the client sees
    return '<h1>Hello, World!</h1>'

@app.route('/test')
def testPage():
    user_input = request.args.get('input', '') # reads the 'input' query parameter from the URL
    return f'You entered: {user_input}'

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=8080) # 0.0.0.0 listens on all interfaces; default would be localhost
    # run the server with: python3 FlaskBasics.py

```

How It Works

- `@app.route('/')` is a **decorator** that maps a URL path to a Python function. Every time a client requests that path, Flask calls the corresponding function and returns its output.
- `request.args.get('input', '')` reads a **query parameter** from the URL. For example, visiting `http://localhost:8080/test?input=hello` would return `You entered: hello`. The second argument is the default value if the parameter is not provided.
- Setting `host='0.0.0.0'` makes the server accessible from other machines on the network, not just localhost.

We will write a simple API with Flask in JavaScript Section

Strings — Advanced

```

s = 'sora is here'

print(s.upper())      # SORA IS HERE
print(s.capitalize()) # Sora is here
print(s.title())      # Sora Is Here
print(s.lower())      # sora is here
print(s.count('e'))    # counts occurrences of 'e'
print(s.find('a'))     # returns the index of first 'a', or -1 if not found
print(s.center(50))    # centers the string in a field of 50 characters
print(s.center(50, '*')) # same but pads with '*' instead of spaces
print(s.startswith('so')) # True
print(s.endswith('re'))  # True
print(s.split())        # splits on whitespace by default – takes an optional delimiter
print(s.split('i'))     # splits on 'i'
print(s.partition('i')) # splits into a 3-tuple: (before, delimiter, after)

```

`join()` is the reverse of `split()` — it combines a list of strings into one, using the calling string as the separator:

```
mylist = ['sora', 'dora', 'iggy']
print(','.join(mylist))    # sora,dora,iggy
print('|'.join(mylist))    # sora|dora|iggy
```

Sets — Advanced

```
s = set()
s.add(1)
s.add(2)
print(s)
s.add(2)    # duplicates are silently ignored
print(s)
s.remove(2) # removes 2 – throws a KeyError if the value does not exist
print(s)
# s.remove(100)    # would throw an error – 100 is not in the set
s.discard(100)    # removes 100 if present, does nothing if not
s.clear()
print(s)
```

Reference vs Copy

Assigning a set with = does not create a copy — both variables point to the same object:

```
s1 = {1, 2, 3}
s2 = s1    # s2 is not a copy – it is the same object as s1
s1.add(100)
print(s1)    # {1, 2, 3, 100}
print(s2)    # also {1, 2, 3, 100}
print("*****")
```

To create an independent copy, use `.copy()`:

```
s1 = {1, 2, 3}
s2 = s1.copy()
s1.add(100)
print(s1)    # {1, 2, 3, 100}
print(s2)    # {1, 2, 3} – unaffected
print("*****")
```

Set Operations

Difference — elements in one set but not the other:

```
s1 = {1, 2, 3}
s2 = {3, 4, 5}
```

```

print(s1.difference(s2))    # {1, 2} – in s1 but not s2
print(s2.difference(s1))    # {4, 5} – in s2 but not s1
s1.difference_update(s2)    # s1 becomes s1 - s2 in place
print(s1)
print("*****")

```

Intersection — elements present in both sets:

```

s1 = {1, 2, 3}
print(s1)
print(s2)
print(s1.intersection(s2))  # {3}
s1.intersection_update(s2)   # s1 becomes the intersection in place
print("*****")

```

Disjoint — checks whether two sets share no elements:

```

s1 = {1, 2, 3}
print(s1)
print(s2)
print(s1.isdisjoint(s2))    # False – they share 3
s1.remove(3)
print(s1.isdisjoint(s2))    # True – no common elements
print("*****")

```

Subset and Superset:

```

s1 = {1, 2, 3}
s2 = {1, 2, 3, 4}
print(s1.issubset(s2))      # True – all of s1 is in s2
print(s2.issuperset(s1))    # True – s2 contains all of s1
s1.add(8)
print(s1.issubset(s2))      # False – 8 is not in s2

```

Union — all elements from both sets combined:

```

print(s1.union(s2))         # returns a new set with all elements
s1.update(s2)               # s1 becomes s1 + s2 in place
print(s1)

```

Itertools

We already know what an iterator is.

For example, if we write a generator, we can iterate over it by calling `next(generator)`.

Another example is:

```

for i in range(0, 10):
    pass

```

This is iteration.

The `itertools` module lets us perform operations on iterators to create more complex iterators.

Import the module:

```
import itertools as it
```

cycle()

Suppose we need a program that repeatedly returns values such as 1, 2, and 3.

```
a = it.cycle([1, 2, 3])
```

```
print(next(a))
print(next(a))
print(next(a))
print(next(a))
print(next(a))
print(next(a))
print(next(a))
print(next(a))
print(next(a))
```

Output:

```
1
2
3
1
2
3
1
2
```

A practical example would be a load balancer.

If we have three IP addresses and want to rotate requests between them, `cycle()` can continuously loop through the available addresses.

combinations()

Suppose we have four numbers and need something that gives us every possible two-number selection.

```
nums = [1, 2, 3, 4]
```

```
combos = it.combinations(nums, 2)
```

```
for i, j in list(combos):
    print(f'{i} - {j}')
```

Output:

```
1 - 2
1 - 3
1 - 4
2 - 3
2 - 4
3 - 4
```

`combinations(iterable, r)` returns all possible groups of `r` elements without repeating an element within the same group.

`combinations_with_replacement()`

This function works similarly to `combinations()`, but elements can appear more than once in a group.

```
combos = it.combinations_with_replacement(nums, 2)

for i, j in list(combos):
    print(f'{i} - {j}')
```

Output:

```
1 - 1
1 - 2
1 - 3
1 - 4
2 - 2
2 - 3
2 - 4
3 - 3
3 - 4
4 - 4
```

Notice that pairs such as 1 - 1 and 2 - 2 are now included.

`count()`

`count()` creates an infinite counter.

Start counting from 3 with a step of 10.

```
counter = it.count(3, 10)

print(next(counter))
print(next(counter))
print(next(counter))
print(next(counter))
print(next(counter))
```

Output:

```
3
13
23
33
43
```

accumulate()

`accumulate()` produces running totals.

```
nums = [1, 2, 3, 4]
```

```
print(nums)
```

```
accumulate = it.accumulate(nums)
```

```
print(list(accumulate))
```

Output:

```
[1, 3, 6, 10]
```

Calculation:

```
1
1 + 2 = 3
1 + 2 + 3 = 6
1 + 2 + 3 + 4 = 10
```

product()

Now suppose we have a list of card values and a list of suits.

```
nums = [1, 2, 3, 4]
```

```
khals = ['heart', 'diamond', 'spear', 'idk']
```

```
print(list(it.product(nums, khals)))
```

`product()` generates every possible pairing between the provided iterables.

Output:

```
(1, 'heart')
(1, 'diamond')
(1, 'spear')
(1, 'idk')
(2, 'heart')
...
```

We can also iterate over the generated pairs:

```
for n, kh in list(it.product(nums, khals)):
    print(f'{n} {kh}')
```

Example output:

```
1 heart
1 diamond
1 spear
1 idk
2 heart
...
```

This operation is called the **Cartesian product**. It generates every possible combination between the elements of multiple iterables.

Virtual Environments (venv)

When we try to install a package with `pip`, Python may not allow us to install it directly into the system environment because it can cause problems with system packages and dependencies.

Python usually gives us two options:

An unsafe option that takes responsibility for modifying system packages:

```
pip install somePackage --break-system-packages
```

And the recommended option:

```
venv
```

But what is a virtual environment?

Why Do We Need Virtual Environments?

Somewhere in the file system, Python packages are stored. When we import a module, Python searches for it in locations listed in `sys.path`.

When a package is not installed, we usually install it with `pip`.

This can create problems.

For example:

- One application may require Flask 1.2.
- Another application may require Flask 1.4.

If Flask is installed system-wide, version conflicts can occur and there may be no simple way to satisfy both applications.

Another common issue is when a project requires many third-party packages. Installing everything globally can make the system harder to maintain and may introduce dependency conflicts.

Moving the project to another machine becomes difficult because that machine also needs all of the required dependencies installed.

What Is a Virtual Environment?

The modern solution is to create a virtual environment for each project's dependencies.

A virtual environment is a directory that contains the packages and libraries required by a specific project.

When a virtual environment is created:

- A specific Python interpreter is used.
- A dedicated `pip` installation is provided.
- Packages installed with `pip` are placed inside the virtual environment instead of the system environment.

As a result, projects remain isolated from one another.

How Virtual Environments Help

A virtual environment behaves similarly to application isolation.

For example:

- You create a project on Linux using your preferred Python version.
- You create and activate a virtual environment.
- You install the required dependencies.
- When you finish working, you deactivate the environment.

Later, you share the project with a friend who uses Windows and may have a different Python version installed.

Your friend can:

- Create or activate a virtual environment.
- Install the dependencies from `requirements.txt`.
- Run the project in an isolated environment.

Because the project dependencies are separated from the operating system, they will not interfere with system packages.

Creating a Virtual Environment

To start a new project:

```
mkdir myproject
cd myproject
```

```
python3 -m venv your-env
```

The package responsible for this functionality is **venv**, which is usually included with Python.

Virtual Environment Structure

After creating a virtual environment, several files and directories are generated:

```
$ python3 -m venv myEnv
$ ls
myEnv

$ cd myEnv/
$ ls
bin  include  lib  lib64  pyvenv.cfg
```

The **bin** directory contains executable files such as:

- **python**
- **pip**
- **activate**

and other related executables.

Activating and Deactivating a Virtual Environment

On Unix-like systems:

```
source your-env/bin/activate
```

To deactivate the environment:

```
deactivate
```

Example:

```
$ source myEnv/bin/activate
```

```
(myEnv) $ deactivate
```

When activated, the environment name appears at the beginning of the shell prompt.

Viewing Installed Packages

To see the packages installed in the current environment:

```
pip list
```

This produces human-readable output.

For machine-readable output:

```
pip freeze
```

Creating a Requirements File

When a project is complete, it is common to save all installed dependencies into a file:

```
pip freeze > requirements.txt
```

This creates a list of all installed packages and their exact versions.

Installing Dependencies from a Requirements File

Another user can install the exact same dependencies with:

```
pip install -r requirements.txt
```

This helps ensure that everyone working on the project uses the same package versions.

Practical: Current Time Script

```
import datetime

def get_time():
    return datetime.datetime.now().strftime("%H:%M:%S")

print(get_time())
```

You can use strftime method (on a time object obviously) with some format like: HH:MM:SS

Python Advanced

In this section, we move beyond the basics and start exploring more advanced concepts of Python.

We introduce additional modules and libraries compared to the fundamentals section, and we go deeper into their internal behavior and usage patterns.

The goal here is not only to learn new tools, but also to understand how and why they work, enabling you to build more complex and efficient Python-based solutions.

Data Types And Numbers

Converting Data Types

```
s = "Hello!"
print(type(s)) # returns str
```

`str`, like other types, can also be used as a constructor to convert between types:

```
n = str(10)
print(n)
print(type(n))
print(type(10))
```

At this point, `n` is a string, and it cannot be used for numerical operations — doing so would raise a `TypeError`.

```
print(str(12.23)) # a string, not a number
print(str(True)) # a string that says: True
print(str(5 < 8)) # a string that says: True
print(str(None)) # a string that says: None
```

Dunder (Magic) Methods for Conversion

`10`, before being converted to a string, was an integer. Looking at `int`'s methods reveals how this conversion actually works:

```
print(dir(int)) # dir() returns an object's available methods
```

Many of the names returned look like data types — these are **dunder** (double underscore) or “magic” methods, used to convert between types. They are not exclusive to `int`; every convertible type has equivalents like `__str__` or `__bool__`.

```
nn = (10).__str__()
print(nn)
print(type(nn))
# this is equivalent to str(10)
# even dir() itself is a dunder call: (10).__dir__()
```

Practical Conversions

User input is always received as a string by default. If it represents a number, it must be explicitly converted before it can be used in math operations — unlike some languages where this happens automatically:

```
a = '10'
a = int(a)
print(a + 3)
```

`int()` (or `.__int__()`) converts to an integer. Similarly, `float()` (or `.__float__()`) converts to a float:

```
print(float('12.23') + 1)
```

And `bool()` (or `.__bool__()`) converts to a boolean:

```
print(bool(1))
print(bool(0))
print((-1).__bool__())
```

Operators and Type Correctness

Converting between types is useful to ensure correctness before performing an operation, or simply to switch between representations. Choosing the correct operator also matters — for example, `4/2` returns a float (`2.0`). To get an integer result instead, use the integer division operator `//`, which always returns an integer:

```
print(4//2)
```

Converting Number Bases

The base used in everyday life and in most regular work is base 10 — numbers go 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, In computing, bases 2, 8, and 16 are far more important.

```
a = 10
```

Defining Numbers in Different Bases

Python provides literals for defining numbers directly in other bases.

In base 2 (binary):

```
b = 0b1010    # 0b is the prefix, followed by the binary digits. 1010 = 8 + 0 + 2 + 0 = 10 in
print(b)       # prints 10
```

In base 8 (octal):

```
c = 0o12      # 0o is the prefix; 12 in octal represents 1*8 + 2 = 10
print(c)
```

In base 16 (hexadecimal):

```
d = 0xa       # 0x is the prefix; a represents 10 in hexadecimal
print(d)
```

All four values are equal to 10:

```
print (a == b == c == d)
```

Converting Between Representations

Python also provides functions to display a number as binary, octal, or hexadecimal:

```
print(bin(10))
print(oct(10))
print(hex(10))
```

And to convert a string in a given base back to an integer:

```
print(int('1010', base=2))
print(int('12', base=8))
print(int('a', base=16))
```

If the input string already includes its base prefix (`0b`, `0o`, `0x`), setting `base=0` lets Python detect the base automatically:

```
print(int('0b1010', base=0))
print(int('0o12', base=0))
print(int('0xa', base=0))
```

Showing Numbers in Different Bases

To use the values from the previous lesson inside text, the `format()` function or f-strings can be used.

```
b = format(10, 'b')    # number 10 in binary format
lb = format(10, '#b')  # binary format with the '0b' prefix
print(b)
print(lb)
```

The same can be done with f-strings:

```
b2 = f"{10:b}"
print(b2)
```

In base 8:

```
o = f"{10:o}"
print(o)
```

In base 16, with the literal prefix:

```
x = f"{10:#x}"
print(x)
```

Recap

`format()` and f-strings are not only used to build dynamic strings — they can also perform type and base conversions. Note that the `format()` function and the `.format()` string method are not the same thing: `.format()` is used to fill placeholders in a string, while `format()` is used to convert a value into a specific representation.

```
number = 42
```

```
formatted_message = "The binary representation of {} is {}.".format(number, format(number, 'b'))  
print(f"Using .format() method (with format() function): {formatted_message}")
```

Readable Number Literals

Now that we know numbers can be defined in four bases in Python, let's look a bit deeper.

Consider a large number:

```
a = 12345678
```

At first glance, a number like this can be hard to read. Python allows underscores to be inserted as visual separators:

```
b = 12_345_678  
print(a == b)
```

The underscores do not have to separate every three digits — they can be placed however is most readable:

```
c = 1_2_3_45678  
print(a == c)
```

This works for all numeric literals, including binary:

```
binary = 0b1101100100010001  
print(binary)
```

```
binary2 = 0b_1101_1001_0001_0001  
print(binary == binary2)
```

This is recommended for large numbers, to keep them readable.

Logical Data

Logical Value of Objects in Python

The `bool()` function converts other data types to a boolean value, or checks the truthiness of a value. Almost everything in Python is considered `True`, but some values are `False`:

```
print(bool(1))  
print(bool(0))  
print(bool(0.0))  
print(bool(0j))  
print(bool(None))  
print(bool(''))  
print(bool([]))  
print(bool({}))
```

These all return `False`: `0`, `0.0`, `0j`, `None`, `''` (empty string), `[]` (empty list), and `{}` (empty dictionary).

How Python Determines Truthiness

There are two ways a value's truthiness is determined. Some types define a `__bool__()` dunder (magic) method, which is called directly and returns `True` or `False`. Other types, such as lists, do not define `__bool__()`, but define `__len__()` instead — if `__len__()` returns `0` (meaning the object is empty), the value is considered `False`.

Defining Truthiness for a Custom Class

These dunder methods can also be added to custom classes. For example, for a `Rectangle` class, a rectangle with an area of `0` could be considered `False`, and any other rectangle `True`.

```
class Rect():
    family = "rects"
    def __init__(self, width=1, height=1):
        self.width = width
        self.height = height

    def CalArea(self):
        return self.width * self.height

    def __bool__(self):
        if self.width * self.height == 0:
            return False
        else:
            return True

    def __str__(self):
        return f"This rect has width: {self.width} and height: {self.height}"
```

`__bool__()` is defined here to return `False` if the rectangle's area is `0`, and `True` otherwise. `__str__()` defines a readable string representation of the object, used when the object is printed directly.

Using the Class

```
rect1 = Rect(0, 12)
print(dir(rect1))
print(rect1.CalArea())
print(rect1)
print(bool(rect1))
print(rect1.family)

rect2 = Rect(3, 12)
print(dir(rect2))
print(rect2.CalArea())
```

```
print(rect2)
print(bool(rect2))
print(rect2.family)
```

`rect1` has a width of 0 and a height of 12, so its area is 0. Calling `bool(rect1)` returns `False`, since `__bool__()` evaluates `0 * 12 == 0` as `True`. `rect2` has a width of 3 and a height of 12, giving an area of 36, so `bool(rect2)` returns `True`.

`dir(rect1)` lists all attributes and methods available on the object, including inherited dunder methods and the custom ones defined here. `print(rect1)` uses `__str__()` to display a readable description of the rectangle. `rect1.family` and `rect2.family` both return `"rects"`, since `family` is a class attribute shared by all instances.

and/or Short-Circuit Evaluation in Python

Both `and` and `or` in Python use short-circuit evaluation. With `and`, if the first expression is `False`, Python does not evaluate the second expression at all and immediately returns `False`. With `or`, if the first expression is `True`, Python does not evaluate the second expression and immediately returns `True`. This results in less computation overall.

```
if 5 < 4 and 6 > 2:
    print('this text wont be printed.')
```

The first expression, `5 < 4`, is `False`, so Python does not even evaluate `6 > 2`. The overall condition is `False`, and the `print` statement does not run.

```
if 1 or False:
    print('false wont be checked.')
```

The first expression, `1`, is `truthy`, so Python does not evaluate `False` at all. The overall condition is `True`, and the `print` statement runs.

This behavior is useful when later conditions are complex or resource-intensive. In such cases, simpler conditions should be placed first in `and/or` expressions, so that the more expensive checks can be skipped when possible.

Strings

Classic Ways to Create Strings

This file demonstrates different ways to create and format strings in Python, using a sample dictionary of accounts.

```
data = {
    101: {'name': 'John', 'balance': 1000},
    102: {'name': 'Jane', 'balance': 500},
    103: {'name': 'Joe', 'balance': 2100},
    104: {'name': 'Jim', 'balance': 600},
    105: {'name': 'Jeff', 'balance': 750},
}
```

```
id = 101
item = data[id]
```

`data` is a dictionary where each key is an account ID, and each value is itself a dictionary containing `name` and `balance`. `item` holds the dictionary for account 101.

Method 1: Using `print` with Multiple Arguments

```
print("Method 1: print function")
print(item['name'], 'has', item['balance'], 'units in their account.')
```

`print()` can take multiple arguments and prints them separated by spaces by default.

Method 2: String Concatenation

```
a = item['name'] + ' has ' + str(item['balance']) + ' units in their account.'
print("Method 2: string concatenation", a)
```

Strings are joined together using the `+` operator. Since `item['balance']` is an integer, it must first be converted to a string with `str()` before it can be concatenated with other strings.

Method 3: `str.join()`

```
t = (item['name'], 'has', item['balance'].__str__(), 'units in their account.')
s = '_'.join(t)
print("Method 3: string join", s)
```

`t` is a tuple of values. `item['balance'].__str__()` converts the integer to a string by directly calling its `__str__()` dunder method, which is equivalent to using `str(item['balance'])`. `.join()` is a string method: the string it is called on is used as the separator between each element of the iterable passed to it. Here, `'_'.join(t)` joins all elements of `t` together, separated by underscores.

The `%` Operator for Strings

We have seen three methods to create strings in previous lessons. This method is a very old one, but we should know it exists.

The `%` operator uses placeholders inside a string template:

- `%s` — for strings
- `%d` — for integers
- `%f` — for floating point numbers

Python is smart enough that if you use `%s` and pass it an integer or float, you will not get an error. However, if you use `%d` or `%f` and pass the wrong type, the program will crash.

Method 4 — % with a Tuple

Values are passed as a tuple after the % operator. The order of values in the tuple must match the order of placeholders in the template.

```
data = {
    101: {'name': 'John', 'balance': 1000},
    102: {'name': 'Jane', 'balance': 500},
    103: {'name': 'Joe', 'balance': 2100},
    104: {'name': 'Jim', 'balance': 600},
    105: {'name': 'Jeff', 'balance': 750},
}

id = 101
item = data[id]

s = '%s has %d units in their account.' % (item['name'], item['balance'])
print("Method 4: % operator", s)

s2 = "There are %d units in %s's account." % (item['balance'], item['name'])
print("Method 4", s2)
```

As you can see, if the template changes, you also have to reorder the variables in the tuple. We can make this smarter:

Method 4.5 — % with a Dictionary

Instead of a tuple, we pass a dictionary. Placeholders are written as %(key)s or %(key)d, and Python fills them in by key name. This means the order no longer matters.

```
s3 = '%(name)s has %(balance)d in their account.' % {'name': item['name'], 'balance': item['balance']}
print("Method 4.5", s3)

s4 = 'There are %(balance)d units in %(name)s\'s account.' % {'name': item['name'], 'balance': item['balance']}
print("Method 4.5: % operator with dict", s4)
```

Method 5 — Passing the Dictionary Directly

Since the placeholder keys `name` and `balance` match the keys in `item` exactly, we can pass the `item` dictionary itself directly to the template.

```
s5 = 'There are %(balance)d units in %(name)s\'s account.' % item
print("Method 4.5.2", s5)
```

The .format() Method for Strings

We have seen different methods so far, most of them traditional and a bit hard to write. String types have a method called `.format()`. It works like the % operator but the syntax is easier and smarter.

Method 6 — Positional Placeholders

Placeholders are written as {} and values are passed as arguments in order. It has the same ordering problem as the % operator.

```
data = {
    101: {'name': 'John', 'balance': 1000},
    102: {'name': 'Jane', 'balance': 500},
    103: {'name': 'Joe', 'balance': 2100},
    104: {'name': 'Jim', 'balance': 600},
    105: {'name': 'Jeff', 'balance': 750},
}

id = 101
item = data[id]

s = '{} has {} units in their account.'.format(item['name'], item['balance'])
print(s)
```

Method 7 — Named Placeholders

To fix the ordering problem, we can use named placeholders like {name} and pass keyword arguments to .format(). Now the order of arguments no longer matters.

```
s = '{name} has {balance} units in their account.'.format(name=item['name'], balance=item['balance'])
s2 = 'There are {balance} units in {name}\''s account.'.format(name=item['name'], balance=item['balance'])
print(s)
print(s2)
```

Method 8 — Dictionary Unpacking with **

Passing each key manually is a bit verbose. Since our dictionary keys already match the placeholder names, we can unpack the dictionary directly using **.

Using ** before a dictionary tells Python to unpack it into keyword arguments. So **item is the same as writing name='John', balance=1000.

```
s = '{name} has {balance} units in their account.'.format(**item)
print(s)
```

f-Strings

f-strings are another method for string formatting, with the easiest syntax of all. Simply prefix the string with f and write expressions directly inside {}.

```
data = {
    101: {'name': 'John', 'balance': 1000},
    102: {'name': 'Jane', 'balance': 500},
    103: {'name': 'Joe', 'balance': 2100},
    104: {'name': 'Jim', 'balance': 600},
    105: {'name': 'Jeff', 'balance': 750},
}
```

```
id = 101
item = data[id]

print(f"{item['name'].upper()} has {item['balance']} units in their account.")
```

You can put any valid Python expression inside the braces, including method calls like `.upper()`.

Variables and Self-Documenting f-Strings

```
a = 10
e = 20
print(f"a = {a}, e = {e}")
print(f"{a = }, {e = }")
```

Adding `=` inside the braces is a Python feature called a self-documenting f-string. It prints both the variable name and its value, which is useful for quick debugging.

Number Formatting

You can format numbers into different bases by adding a format specifier after `:` inside the braces.

```
print(f"a = {a}, {a:b}, {a:o}, {a:x}, {a:#b}")
```

- `{a:b}` — binary
- `{a:o}` — octal
- `{a:x}` — hexadecimal
- `{a:#b}` — binary with the `0b` prefix

Inline Expressions

Any Python expression can be evaluated directly inside an f-string.

```
print(f"{a} + {e} = {a+e}")
```

The Template Class

Besides all the previous methods, we can also create dynamic strings using the `Template` class from Python's standard `string` module. Placeholders are written with a `$` sign instead of `{}`.

Method 10 — Template with Keyword Arguments

We create an instance of the `Template` class and call its `.substitute()` method, which works similarly to `.format()`.

```
from string import Template
```

```
data = {
    101: {'name': 'John', 'balance': 1000},
```

```

102: {'name': 'Jane', 'balance': 500},
103: {'name': 'Joe', 'balance': 2100},
104: {'name': 'Jim', 'balance': 600},
105: {'name': 'Jeff', 'balance': 750},
}

id = 101
item = data[id]

template = Template("$name has $balance units in their account.")

s = template.substitute(name=item['name'], balance=item['balance'])
print(s)

```

Method 11 — Template with Dictionary Unpacking

Since `.substitute()` also accepts a dictionary, we can unpack `item` directly with `**`. As a bonus, we can also pass the dictionary without unpacking and it will work the same way — `substitute(item)` and `substitute(**item)` both work here.

```

s2 = template.substitute(**item)
print(s2)

```

The `Template` class can be useful for preventing malformed string operations, since it is more strict about missing or extra keys compared to `.format()`.

ord() and chr()

Sometimes we need to inspect strings and characters as numbers. In the ASCII table, every character has a corresponding number. For example, 'A' is decimal 65 and hex 41.

ord(character) returns the decimal ASCII code of a character. **chr(number)** does the opposite — it converts a decimal number back to its character.

```

print(ord("A"))      # 65
print(ord("\n"))     # 10
print(hex(ord("A"))) # 0x41

print(chr(65))       # A
print(ord(chr(65)))  # 65

print(chr(10))       # newline

```

One practical use case is steganography — hiding text inside an image by replacing pixel values with the ASCII codes of the characters.

Uppercase vs Lowercase

All lowercase letters have an ASCII decimal value exactly 32 higher than their uppercase equivalents.

```
print(ord("A"), ord("a"))    # 65, 97
print(ord("B") - ord("b"))   # -32
print(ord("C") - ord("c"))   # -32
```

Naming Techniques

The Underscore Variable

The underscore `_` is an important character in Python and is used in many different contexts. It can be used as a variable name on its own.

In Interactive Python Shells

In an interactive shell like IPython, `_` automatically holds the value of the last evaluated expression that was **not** assigned to a variable.

```
In [1]: 1 + 2
Out[1]: 3
```

```
In [2]: a = 3 * 4
```

```
In [3]: _
Out[3]: 3
```

Notice that `a = 3 * 4` is skipped because it was assigned. `_` only captures unassigned results.

`__` holds the second-to-last unassigned result, and `___` holds the third:

```
In [4]: 1
Out[4]: 1
```

```
In [5]: 2
Out[5]: 2
```

```
In [6]: 3
Out[6]: 3
```

```
In [7]: _, __, ___
Out[7]: (3, 2, 1)
```

The Out and In Dictionaries

All previous outputs are accessible through `Out`, and all previous inputs through `In`:

```
In [8]: Out
Out[8]: {1: 3, 3: 3, 4: 1, 5: 2, 6: 3, 7: (3, 2, 1)}
```

```
In [9]: In
Out[9]: ['', '1 + 2', 'a = 3 * 4', '_,', '1', '2', '3', '_, __, ___', 'Out', 'In']
```

You can also access a specific output by its line number using `_` followed by the number:

```
In [17]: 2 * 5
Out[17]: 10
```

```
In [18]: 3 ** 4
Out[18]: 81
```

```
In [19]: _17
Out[19]: 10
```

```
In [20]: _18
Out[20]: 81
```

The Evaluation Trap

Since reading `_` is itself an unassigned evaluation, it shifts the values of `__` and `___`. The safe way to capture multiple previous results is to assign them immediately:

```
In [13]: a = _
In [14]: b = __
In [15]: c = ___

In [16]: a, b, c
Out[16]: (3, 2, 1)
```

Using `__` in Scripts

In a regular script, `_` is just a normal variable name. It is commonly used in loops when the loop variable is not needed — it signals to anyone reading the code that the value is intentionally being ignored.

```
for _ in range(10):
    print(_)
    print(_ * ' * ')
```

For nested loops where both variables are unneeded, `__` can be used for the inner one:

```
for _ in range(5):
    for __ in range(8):
        print(__ * ' * ')
```

Using `_` as a variable name in an interactive shell is not recommended, as it overwrites the default behavior described above.

Single Underscore Naming Convention in Python

This example demonstrates two common uses of a single underscore (`_`) in Python:

1. Marking names as internal/private by placing an underscore at the beginning.
2. Avoiding conflicts with Python keywords by placing an underscore at the end.

The Module: `vehicle.py`

The following module defines a class, an internal variable, and an internal function.

```
class Vehicle:
    brand = None
    color = None

    # If a name conflicts with a Python keyword,
    # an underscore can be added at the end.
    type_ = None
    class_ = None

    _engine_type = None

_some_var = 'Nothing Here!'

def _some_func():
    print('Bingo')
```

Using an Underscore at the End

Sometimes a desired name conflicts with a Python keyword such as `class`, `type`, `from`, or `global`.

In these situations, it is common to add a trailing underscore:

```
type_ = None
class_ = None
```

This prevents conflicts with Python syntax while keeping the name readable.

Using an Underscore at the Beginning

Names that start with a single underscore are treated as internal implementation details.

```
_engine_type = None
_some_var = 'Nothing Here!'

def _some_func():
    print('Bingo')
```

Python does not enforce privacy here. These names are still accessible, but the underscore serves as a signal to other developers that they should not be accessed or modified directly.

Importing the Module

The following script demonstrates how wildcard imports interact with underscore-prefixed names.

```
from vehicle import *

v = Vehicle()
print(v.brand)
```

```
v.brand = 'BMW'
v.color = 'RED'
v._engine_type = 'V8'

print(_some_var)
_some_func()
```

What Happens Here?

The wildcard import:

```
from vehicle import *
```

imports public names from the module.

By default, names that start with an underscore are excluded from wildcard imports.

Therefore:

```
Vehicle
```

is imported successfully, but:

```
_some_var
_some_func
```

are not imported.

As a result, the last two lines will raise a `NameError`:

```
print(_some_var)
_some_func()
```

because those names are not available in the current namespace.

Why Does This Happen?

Python treats underscore-prefixed names as non-public.

When using:

```
from module import *
```

Python skips those names unless the module explicitly exports them through `__all__`.

Example:

```
__all__ = ['Vehicle', '_some_var', '_some_func']
```

If `__all__` is defined like this, the wildcard import will include those names as well.

Accessing Internal Names Explicitly

Even though wildcard imports ignore underscore-prefixed names, they can still be imported directly.

```
from vehicle import _some_var
from vehicle import _some_func
```

or:

```
import vehicle

print(vehicle._some_var)
vehicle._some_func()
```

Both approaches work because the underscore convention does not provide true access control.

Key Takeaway

A single leading underscore is a naming convention, not a security mechanism.

It communicates:

- “This attribute is intended for internal use.”
- “Avoid accessing or modifying it directly.”
- “Wildcard imports should ignore this name.”

Python developers are expected to respect this convention even though the language does not enforce it.

Double Underscores at the Start of Variable Names

```
class A():
    name = 'Name'
    _type = 'Unknown'
    __var = 'A'
    __k = 'k'
    __o__ = 'o'

print(A.name)
print(A._type)
# print(A.__var)    # We will get an error. Why? What is the difference between _ and __ at th

print(dir(A))
```

You can see that `__var` also exists, but with `_A` added before its name (`_A__var`). This is done to protect the variable from being accessed accidentally. This mechanism is called **name mangling**.

If a variable name starts with double underscores and does not end with double underscores, name mangling will occur.

```
class B(A):
    __var = 'B'

print(dir(B))
```


Notice that we now have two `__var` attributes: one related to A and the other related to B.

Let's go deeper into the code.

```
print("Go to part two")
```

Name Mangling in Python

Alright, let's make it practical.

```
class A():
    # We have two types of attributes here: public (name) and private (__var)
    name = 'Name'
    __var = 'A'

    def __init__(self, name, var):
        self.name = name
        self.__var = var

class B(A):
    def __init__(self, name, var):
        self.name = name
        self.__var = var

a = A("First A", 1000)
b = B("First B", 500) # It will contain __var related to B and also __var related to A
```

B inherits from A, but it cannot overwrite the `__var` attribute that belongs to A.

These variable names are modified internally to prevent conflicts between parent and child classes.

```
# print(a.__var)
```

The commented-out line above would raise an error because name mangling prevents direct access using the original name.

At first sight, `name` and `__var` may look similar, but Python handles them differently.

When we write:

```
self.name = ...
```

we create a normal instance attribute.

The purpose of using double underscores at the start of a variable name is to make it private-like and prevent subclasses from accidentally overriding it.

```
class Parent:
    def __init__(self, name, var):
        self.name = name
        self.__var = var

class Child(Parent):
    def __init__(self, name, var, child_var):
```

```

super().__init__(name, var)    # We could call Parent.__init__(), but super() is preferred
self.name = name
self.__var = child_var
self.child_var = child_var

```

```

p = Parent("Parent Name", 1000)
c = Child("Child Name", 500, 2000)

```

When a Child object is created, the parent constructor is called first. The `var` argument becomes `_Parent__var`.

Then the child constructor runs, creating `_Child__var`.

Notice that `name` is overwritten, but `__var` is not. This happens because one becomes `_Parent__var` and the other becomes `_Child__var`.

The moment the parent constructor runs, name mangling occurs. The same happens when the child constructor runs. As a result, we end up with two different `__var` attributes: one for the parent and one for the child.

```

print(f"p.name: {p.name}")
# print(f"p.__var: ??? (AttributeError expected)")
print(f"p._Parent__var: {p._Parent__var}")

print("-" * 20)

print(f"c.name: {c.name}")
# print(f"c.__var: ??? (AttributeError expected)")
print(f"c._Child__var: {c._Child__var}")
print(f"c._Parent__var: {c._Parent__var}")
print(f"c.child_var: {c.child_var}")

```

Double Underscores at the Start and End

If a name contains double underscores at both the start and the end, it becomes something special in Python.

```
print(dir(str))
```

There are many properties that start and end with double underscores. These properties are called **magic methods** or **dunder methods**.

For example, `__init__` is a dunder method that gets executed when an instance of a class is created.

We have also seen `__bool__`, `__len__`, and `__str__` in previous lessons. We even defined some of them in our `Rect` class.

These methods are callable, but most of the time they are used by other functions. For example, `__bool__` is used by the `bool()` function, and `__len__` is used by the `len()` function.

```

print((10).__bool__())
print([1, 2, 3, 4].__len__())

```

Magic methods are defined in classes and are available to all instances of those classes.

```
class Rect():
    family = "rects"

    def __init__(self, width=1, height=1):
        self.width = width
        self.height = height

    def CalArea(self):
        return self.width * self.height

    def __bool__(self):
        if self.width * self.height == 0:
            return False
        else:
            return True

    def __str__(self):    # This will be used whenever we need the object as a string.
        return f"This rect has width: {self.width} and height: {self.height}"
        # If you do not define __str__, printing a Rect object will show its memory address and

    def __repr__(self):
        return str(self)    # This prevents the default object representation, although changing
        # for example in this code, representation could be:
        # return f"Rect({self.width}, {self.height})"

rect1 = Rect(3, 12)

print(dir(rect1))
print(rect1.CalArea())
print(rect1)
print(str(rect1))    # Same as above.
print(f"{rect1}")    # Also the same as above.
print(bool(rect1))
print(rect1.family)
```

Logging in Python

Imagine we have a program that performs some operations.

Whether the program is large or small, it is always useful to leave logs during runtime to inspect errors, unexpected behavior, or even successful operations.

In simple programs, we often use `print()`, but printed output is not saved anywhere. It is usually better to keep this information in a file.

Python provides a standard library for this purpose.

Basic Logging

```
import logging

# #####
import os

# ADDITIONAL:
print(__file__)    # This variable contains the full path of the current script.

path = os.path.dirname(__file__)    # Returns the absolute path of the script's directory.

# If you want it to be more reliable:
path = os.path.dirname(os.path.abspath(__file__))
print(path)
# #####
```

Logs have different severity levels.

- DEBUG (lowest level)
- INFO
- WARNING
- ERROR
- CRITICAL (highest level)

WARNING, ERROR, and CRITICAL are enabled by default. DEBUG and INFO are not shown unless logging is configured to include them.

```
logging.basicConfig(
    level=logging.DEBUG,    # Log all levels starting from DEBUG.
    # Any level equal to or higher than the specified level will be logged.
    # DEBUG is the lowest level (10), so everything will be logged.

    filename=f'{path}/myApp.log',    # Redirect logs from the terminal to a file.

    # Sometimes the log file may be created in an unexpected location.
    # Using the script path ensures that the log file is written beside the script.

    filemode='w',
)

logging.debug(f'This is a DEBUG ({logging.DEBUG}) level log.')
logging.info(f'This is an INFO ({logging.INFO}) level log.')    # Logging levels can be shown
logging.warning(f'This is a WARNING ({logging.WARNING}) level log.')
logging.error(f'This is an ERROR ({logging.ERROR}) level log.')
logging.critical(f'This is a CRITICAL ({logging.CRITICAL}) level log.')
```

Without configuration, DEBUG and INFO messages are not displayed.

If the program is executed multiple times, logs are appended to the file by default. To overwrite the file each time, use:

```
filemode='w'
```

The default mode is:

```
filemode='a'
```

which appends new logs to the existing file.

You can also make logs more useful by including timestamps and additional metadata.

Improving the Logging Process

The previous example writes logs to a file. We can improve the output format using additional configuration options.

```
import logging
import os

path = os.path.dirname(os.path.abspath(__file__))

logging.basicConfig(
    level=logging.DEBUG,
    filename=f'{path}/myApp.log',
    filemode='w',

    # The format parameter controls how log entries are displayed.
    format='%(levelname)s -- %(asctime)s -- %(filename)s:%(lineno)s -- %(message)s',

    # Customize the date and time format.
    datefmt='%Y-%m-%d => %H:%M:%S'
)
```

We have a parameter called `format`, and its available values can be found in the Python documentation.

Search for Python logging and visit the documentation. In the LogRecord attributes section, you can find useful fields such as:

- `filename`
- `funcName`
- `levelno`
- `lineno`
- `message`

and many others.

There are also different formatting styles available, such as:

```
%(filename)s
${filename}
{filename}
```

similar to the string formatting techniques discussed earlier.

This section only renders the message content:

```
logging.debug(f'This is a DEBUG ({logging.DEBUG}) level log.')
logging.info(f'This is an INFO ({logging.INFO}) level log.')
logging.warning(f'This is a WARNING ({logging.WARNING}) level log.')
logging.error(f'This is an ERROR ({logging.ERROR}) level log.')
logging.critical(f'This is a CRITICAL ({logging.CRITICAL}) level log and this is some calculation')
```

After that, the log file is written using the specified format.

Each time a logging function is called, a new log entry is created using the provided message.

We can use logging instead of simple printing, or use both together.

We can even store calculation results and other runtime information inside log files.

Example Output

The following output is generated by the previous script:

```
DEBUG -- 2026-04-04 => 01:48:21 -- improveLoggingProcess.py:22 -- This is a DEBUG (10) level log
INFO -- 2026-04-04 => 01:48:21 -- improveLoggingProcess.py:23 -- This is an INFO (20) level log
WARNING -- 2026-04-04 => 01:48:21 -- improveLoggingProcess.py:24 -- This is a WARNING (30) level log
ERROR -- 2026-04-04 => 01:48:21 -- improveLoggingProcess.py:25 -- This is an ERROR (40) level log
CRITICAL -- 2026-04-04 => 01:48:21 -- improveLoggingProcess.py:26 -- This is a CRITICAL (50) level log
```

Iterables

The `map()` Function

There are some functions that can be used with iterable objects.

Iterable objects include strings, lists, tuples, dictionaries, and more.

```
for _ in "SORA":
    print(_)
```

Now, imagine we have an iterable and want to perform an operation on every member, returning a result for each one.

In JavaScript, `map()` and `filter()` are primarily used with arrays. When you use `map()` on an array, it creates a new array containing the result of applying a function to each element.

In Python, `map()` can be used with almost any iterable, giving us more flexibility in how we process data.

```
A = [1, 2, 3, 4, 5]
```

`map()` takes a function and applies it to every member of an iterable.

```
B = list(map(lambda x: x**2, A))
C = list(map(lambda x: (x, x**2), A))    # What if we want a tuple containing the number and its square?
```

```
print(B)
print(C)
```

Now imagine we have two iterables and want to iterate over them at the same time, combining values from the same index.

```
A = [1, 2, 3, 4]
B = [5, 6, 7, 8]
```

```
joined = list(map(lambda x, y: x + y, A, B))
print(joined)
```

The function passed to `map()` can accept any number of parameters, but you must provide the same number of iterables.

Note

`map()` is a lazy-evaluation function. It does not immediately return all values.

Instead, it returns a map object, and the values are generated when needed.

For example:

```
list(map(...))
```

converts the result into a list and forces evaluation.

The `filter()` Function

The `filter()` function is used similarly to `map()`.

`filter()` takes a callback function and an iterable. It runs the function on every member of the iterable, and if the result is `True`, that member is kept.

```
A = [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
result = list(filter(lambda x: x % 2 == 0, A))    # filter() is also lazy-evaluated.
```

```
# We could also write:
```

```
# lambda x: not x % 2
```

```
# Since x % 2 returns 1 or 0, odd numbers produce 1 (True) and even numbers produce 0 (False).
```

```
# Using not reverses the result, causing only even numbers to pass the filter.
```

```
print(result)
```

If we use the `None` keyword instead of a function, `filter()` evaluates the iterable's values directly and keeps only truthy values.

```
print(list(filter(None, A)))
```

This returns all members because every value in `A` is truthy.

But what happens if we include `0` in the list?

```
print(list(filter(None, [0, 1, 2, 3, 4, 5])))
```

0 does not pass the filter because it is considered a falsy value in Python.

The `reversed()` Function

We can reverse the order of elements in an iterable using slicing in Python.

```
A = [1, 2, 3, 4, 5, 6, 7]
```

```
print(A[2:])      # Simple slicing (from index 2 to end)
print(A[:4])      # Slicing (from start to index 4)
print(A[::-1])    # Reverse the list
```

Using `reversed()`

Python also provides a built-in function called `reversed()`.

```
rev = reversed(A)    # reversed() returns a reverse iterator
print(rev)
print(list(rev))
```

`reversed()` does not return a list directly. Instead, it returns a **reverse iterator**, which can be converted into a list when needed.

Lazy Evaluation

Functions like `map()`, `filter()`, and `reversed()` are lazy. They do not compute all values immediately.

Instead, they generate values only when required, which makes them memory efficient.

Iterating with `reversed()`

```
for _ in reversed(A):
    print(_)
```

This prints elements in reverse order without creating a new reversed list in memory.

The `sorted()` Function

Have you ever wanted to sort data inside an iterable?

```
A = [1, -2, 4, 0, -8, 8, 9, -10, 6, 7, 5, -3]
```

Basic Sorting

```
print(sorted(A))
```

`sorted()` returns a new sorted list from the input iterable.

Reverse Sorting

```
print(sorted(A, reverse=True))
```

This sorts values in descending order.

Sorting with a Key Function

We can control sorting behavior using the `key` parameter.

```
print(sorted(A, key=abs))    # Sort based on absolute value
```

The key function is called on each element before sorting.

We can also use a lambda function:

```
print(sorted(A, key=abs, reverse=True))
print(sorted(A, key=lambda x: x ** 2))
```

More Complex Sorting

```
B = [1, 3, 4, 2, 7, 8, 9, 2, 3, 4, 5, 10]
```

```
print(sorted(B, key=lambda x: x % 2)) # Even numbers come first
```

Here, numbers are grouped based on parity: - 0 $\rightarrow$ even numbers - 1 $\rightarrow$ odd numbers

Sorting Within Groups

If we want even and odd numbers to also be sorted individually:

```
print(sorted(sorted(B), key=lambda x: x % 2))
```

First, the list is sorted normally, then grouped by parity.

Tuple-Based Sorting (Multi-Level Sorting)

```
sorted(B, key=lambda x: (x % 2, x))
```

This is a cleaner way to achieve multi-level sorting.

How Tuple Sorting Works

When the key returns a tuple, Python compares elements in order:

1. First element of the tuple
2. If equal, second element
3. And so on

So in:

```
(x % 2, x)
```

- `x % 2` groups even numbers first (0) and odd numbers second (1)
- `x` ensures values inside each group are sorted numerically

Key Idea

Tuple-based sorting allows multi-level sorting in a single step, making it more efficient and readable than chaining multiple `sorted()` calls.

The `enumerate()` Function

Imagine we have items in a list or a string and we want to iterate over them.

```
A = "Jello World!"
```

Basic Iteration

```
for c in A:
    print(c, c.upper(), c.lower())
```

This loops through each character, but does not give the index.

Manual Indexing (Not Pythonic)

```
for i in range(len(A)):
    c = A[i]
    print(i, c, c.upper(), c.lower())
```

This works, but it is not considered a clean Python style.

Using `enumerate()`

```
for i, c in enumerate(A):
    print(i, c, c.upper(), c.lower())
```

`enumerate()` provides both: - index (`i`) - value (`c`)

Converting to a List

```
print(list(enumerate(A)))
```

This shows that `enumerate()` returns an iterator of (`index`, `value`) pairs.

Key Idea

`enumerate()` is the Pythonic way to access both the index and the value of an iterable at the same time, without manually using `range(len(...))`.

The `zip()` Function

What if we have multiple iterables (like lists) and we want to iterate over them at the same time?

`zip()` lets us combine them element by element.

Manual Way (Not Recommended)

```
a = [1, 2, 3, 4]
b = [10, 20, 30, 40, 50, 60]

for i in range(len(a)):
    print(a[i], b[i])
```

This works, but it is not clean and can cause issues if the lengths are different.

Pythonic Way: Using zip()

```
c = 'Hello'

for i, j, h in zip(a, b, c):
    print(i, j, h)
```

zip() Output

```
print(list(zip(a, b, c)))
```

zip() pairs elements together and stops at the shortest iterable.

Example result: - (a1, b1, c1) - (a2, b2, c2) - ...

Accessing Zipped Data

```
L = list(zip(a, b, c))

print(L[0])
print(L[1])
print(L[2])
print(L[3])
```

Each element of L is itself a tuple.

Unzipping (Reversing zip)

You can “unzip” data using * (argument unpacking / expand operator).

```
print(list(zip(L[0], L[1], L[2], L[3])))
```

A cleaner way:

```
print(list(zip(*L)))
```

This is useful when you want to expand all vaules.

You can also directly unzip a zipped object:

```
print(list(zip(*zip(a, b, c))))
```

Argument Unpacking (* operator)

The * operator expands an iterable into individual values.

```
zip(*L)
```

This is often used to reverse the effect of zip().

Relationship with enumerate()

enumerate() is conceptually similar to zip().

We can manually recreate it using zip():

```
print(list(zip(range(len(b)), b)))
```

However, enumerate() is more flexible.

Why enumerate() is Better

Unlike zip(range(len(...)), ...), enumerate() allows:

```
enumerate(b, start=80)
```

So it can start indexing from any value, which makes it more powerful and readable.

Key Idea

- zip() → combines multiple iterables
- *zip() → unpacks them back
- enumerate() → specialized version of zip(index, value)

iter() and next() Functions

In Python, we have iterables and iterators. Iterables are objects that we can iterate on them, and iterators are objects that we can iterate iterables with them.

iter() function creates an iterator, and we can get iterator values one by one with next() function. Also, zip() function works with next, not “indexes”.

We want to iterate over some iterable normally:

```
s = 'ABCDEFGHIJKLMNOP'
```

```
for i in s:  
    print(i)
```

If we want more control, we can use break and continue in loops.

But what if I want even more control, like iterating myself?

We can create an iterator on the current iterable. It doesn't mean it only holds all values, but it knows how far it's gone and what is the next value, etc.

```
print(list(iter(s)))  
it = iter(s)
```

To get the next values, I use `next()` function. (remember generators in basic lessons?)

```
print(next(it))
print(next(it))
print(next(it))
print(next(it))
print(next(it))
print(next(it))
```

```
print()
print("REMAINING:")
print(it.__length_hint__())    # this dunder tells us how many iterations/members are remaining
print()
```

```
print(it.__next__())    # remember dunder? next() executes __next__()
print(next(it))
print(next(it))
# print(next(it))
```

It goes on and on until the iterator is over and we get an error (`StopIteration`).

We can handle this error in two ways: - with a while loop + try/except and break condition - or we can pass another parameter to `next()` function to return a default value when iterator is over:

```
print(next(it, None))
print(next(it, 'Over'))
print(next(it, 'Over'))
print(next(it, 'Over'))
```

There are also other interesting things `iter()` can do. It can even iterate over a function that generates some value.

`input()` is a function that takes input from user. We can even iterate over its return value:

```
a = list(iter(input, '.'))
print(a)
```

This means iteration continues until we reach `'.'`.

Now let's see a real advanced example:

```
A = list(range(20))
print(A)
```

We want to group these values every 4 members.

We know that we can multiply lists like this:

```
print([1, 2, 3] * 3)
```

And we also know that `iter(A)` creates an iterator and can be stored like:

```
print(iter(A), [iter(A)])
print([iter(A)] * 4)
```

When you multiply a list, its element will be repeated. If the element is a reference object like an iterator, that object itself is repeated (same object).

So `[iter(A)] * 4` creates a list with 4 references to the same object. Imagine:

```
it2 = iter(A)
[it2, it2, it2, it2]
```

These all refer to the same iterator.

And also we know that iterators have memory and they save the ‘state’, so the 4 iterator objects in the list we create will not be exactly the same in behavior because of the shared state of the iterator.

```
print(list(zip(*[iter(A)] * 4)))
```

`zip()` takes iterables and calls them for values, then returns tuples with the same length as given iterables, and continues until values are finished.

In simpler way:

```
it2 = iter(A)
print(list(zip(it2, it2, it2, it2)))
```

NOTE: `zip()` function also has a strict mode to prevent mismatched lengths in this special operation:

```
zip(..., strict=True)
```

range() and slice()

We know some things about the `range()` function. It can also take a start value and a step.

```
print(list(range(10, 100, 10)))
print(list(range(10, 1, -1)))
```

The slice() Function

We also have a function called `slice`. Its parameters are similar to `range`.

Slicing has already been covered in basic Python.

```
s = "Hello World!"
print(s[1:5:2])    # from char 1 to 5 with step: 2
print(s[1::2])
print(s[::2])
```

Using `slice()` Instead of `[]`

Instead of using `[]` for slicing, we can also use the `slice()` function.

```
print(s[slice(3, 7)])  
print(s[slice(1, -1, 2)])
```

Key Idea

The benefit of using `slice()` is that it allows dynamic values to be passed more easily.

Useful DataTypes In Python

`namedtuple` from `collections`

In Python we have different built-in or standard library data types to work with data.

Built-in containers include: - lists - dicts - tuples - sets

Tuples are immutable and cannot be changed after creation.

Using a Dictionary

A dictionary is useful when we want named access to values.

```
home = {  
    'x': 10,  
    'y': 20  
}  
  
print(home['x'])
```

Using Tuples with Indexes

If we want immutability, we can use a tuple, but then we lose named access and must rely on indexes.

```
home = (10, 20)  
  
CHORD_X = 0  
CHORD_Y = 1  
  
print(home[CHORD_Y])
```

This works, but it is not very clean.

Named Tuples

If we want both: - immutability - named fields

we can use `namedtuple` from the `collections` module.

```
from collections import namedtuple
```

`namedtuple` is a class factory (a tool to create classes).

```
Location = namedtuple('Location', 'x y name')
```

The first argument is the class name, and the second argument defines the fields (space-separated).

```
print(Location)
print(type(Location))
```

Now `Location` can be used like a class to create tuple-like objects.

```
a = Location(10, 20, 'home')

b = Location(name='work', y=30, x=20)

print(a)
print(b)
```

Accessing Values

We can access values in multiple ways:

```
print(a.x)
print(b.name)
print(b[1])
```

Immutability

Named tuples are immutable.

For example, this is not allowed:

```
# a.name = "something else"
```

Modifying Data

However, we can create a modified copy using `_replace()`:

```
a = a._replace(name="something else")
print(a.name)
```

`_replace()` does not modify the original object; it returns a new one.

We can also convert it to a dictionary:

```
a = a._asdict()
print(a)
```

Another Example: Point

```
Point = namedtuple('Point', ('x', 'y'))

p1 = Point(10, 20)
p2 = Point(30, 40)
```

Adding Namedtuples

If we try to add them directly:

```
print(p1 + p2)
```

It returns a tuple-like result.

We can define custom behavior using dunder methods.

Example (not always recommended in practice):

```
Point.__add__ = lambda self, p: Point(self.x + p.x, self.y + p.y)
```

Or define a function externally:

```
def point_addition(p1, p2):
    return Point(p1.x + p2.x, p1.y + p2.y)

# Point.__add__ = point_addition
print(p1 + p2) # internally calls p1.__add__(p2)
```

NamedTuple from typing

There is also an improved version of `namedtuple` that gives more control and is generally better.

```
from typing import NamedTuple
```

This version is based on class inheritance.

Defining a NamedTuple Class

```
class Point(NamedTuple):
    x: int    # we only specify field names and types
    y: int
```

We do not need to write `self.x = ....` We only declare fields.

This syntax is called **type hinting**.

Type hinting is not required, but it is recommended after Python 3.5.

It is similar to TypeScript, but Python does not enforce types unless external tools like `mypy` are used.

Type hinting can be used anywhere in Python, not only in `NamedTuple`.

Type Hint Example

```
name: str = 1234
print(name)
```

Creating Instances

```
p1 = Point(10, 20)
p2 = Point(y=30, x=20)

print(p1)
print(p2)
```

Accessing Values

```
print(p1.x)
print(p1[1])
```

Using `ypstruct`

There is a library called `ypstruct` that can be installed via `pip`.

It provides additional data structures, such as objects that allow dot notation access like JSON.

```
from ypstruct import structure
```

This is not a standard library, but it is useful.

Basic Usage

```

p = structure(x=10, y=20)
print(p)

dict(p)
print(p)

print(p['x'])
print(p.y)    # like JSON-style access

```

We can also modify values:

```

p.x = 11
print(p)

```

Type and Behavior

```

print(type(p))

```

Repeating Structures

```

empty_point = structure(x=None, y=None)
points = empty_point.repeat(10)

points[1].x = 15
points[1].y = 25

print(points)

```

Multiplication Shortcut

```

somePoints = empty_point * 5
print(somePoints)

```

This is a simpler way to duplicate structured objects.

Counter from collections

Counter is one of the most useful classes in the `collections` module.

```

from random import randint

```

```

A = [randint(1, 6) for _ in range(10)]

```

Creating a Counter

```
from collections import Counter

c = Counter(A)
```

This counts every element in the list and returns how many times each value appears (sorted from most common to least common).

```
print(c)
```

Accessing Counts

```
print(c[1])
print(c[2])
print(c[3])
```

Most Common Elements

```
print(c.most_common())      # all elements sorted by frequency
print(c.most_common(1))    # most common element
print(c.most_common(2))    # top 2
print(c.most_common(3))    # top 3
print(c.most_common(-1))   # least common
```

Getting Elements Back

```
print(c.elements())        # returns an iterator of elements
```

Updating Counter

```
B = [randint(1, 6) for _ in range(10)]

c.update(B)
```

Existing values are not removed; new counts are added.

```
print(c)
```

Total Count

```
print(c.total())
print(list(c.elements()))
```

Removing Elements

```
c[1] = 0    # keeps key but sets count to 0
del c[2]    # removes key completely

c.clear()   # removes everything
print(c)
```

Creating Counter in Different Ways

```
c1 = Counter({'cat': 3, 'dog': 2, 'mouse': 1})
print(c1)

print(c1.elements())
print(c1.total())

c2 = Counter(cat=5, mouse=3, horse=2)
print(c2)
print(c2.total())
```

Combining Counters

```
print(c1 + c2)    # combines counts like update
```

Set-Like Operations

Intersection (minimum values)

```
print(c1 & c2)
```

Union (maximum values)

```
print(c1 | c2)
```

Difference

```
print(c1 - c2)
print(c2 - c1)
```

defaultdict from collections

Dictionaries do not contain default values.

```
a = dict()
```

```
a['name'] = 'sora'
print(a)
print(a['name'])
```

```
# print(a['fname'])    # KeyError
```

To avoid errors, we often use `.get()`:

```
print(a.get('fname', 'key does not exist'))
```

But this is not always convenient in real code.

Solution: defaultdict

```
from collections import defaultdict
```

`defaultdict` is like a normal dictionary, but with an extra feature:

If a key does not exist, instead of throwing an error, it returns a default value.

Example with Default str

```
d = defaultdict(str)
```

```
d['name'] = 'sora'
```

```
print(d['name'])
print(d['fname'])
```

If the key does not exist: - `str` → returns "" (empty string) - `int` → returns 0 - or any callable function

```
# d = defaultdict(lambda: 'key not found')
```

The argument must be callable.

Using `defaultdict` as a Counter

A `defaultdict` with `int` is basically a simple counter implementation.

Practical Script: Word Counter (Using `defaultdict`)

```
#!/usr/bin/env python3
import re
from collections import defaultdict
```

Reading Input File

```
filePath = input('file to read: ')

with open(filePath, 'r') as f:
    content = f.read()
```

The script reads a file path from the user and loads the entire content into memory.

Preprocessing Text

```
text = content.lower()
```

The text is converted to lowercase to make counting case-insensitive.

Extracting Words

```
words = re.findall(r'\b[a-zA-Z0-9\u0600-\u06FF]+\b', text)
```

This regular expression extracts words from the text, including: - English letters - Numbers - Persian/Arabic Unicode range (`\u0600-\u06FF`)

Counting Words

```
wordCounts = defaultdict(int)

for word in words:
    wordCounts[word] += 1
```

`defaultdict(int)` automatically initializes missing keys with `0`, making it ideal for counting.

Sorting Results

```
sortedWordCont = sorted(
    wordCounts.items(),
    key=lambda item: item[1],
    reverse=True
)
```

Words are sorted based on their frequency (highest first).

Output Top Results

```
print(sortedWordCont[:5])
```

This prints the top 5 most frequent words in the file.

OrderedDict from collections

A normal dictionary does not give control over the order of elements (even if modern Python preserves insertion order, it is still not meant for manual order manipulation).

```
d = dict(cat=1, dog=2, mouse=3, horse=4, lion=5)
```

Using OrderedDict

```
from collections import OrderedDict

od = OrderedDict(cat=1, dog=2, mouse=3, horse=4, lion=5)
print(od)
```

It looks the same as a normal dictionary, but it provides extra control over ordering.

Moving Elements

```
od.move_to_end('cat')
print(od)
```

This moves the key 'cat' to the end of the dictionary.

Removing Last Item


```
litem = od.popitem()
print(litem)
print(od)
```

`popitem()` removes and returns the last key-value pair.

Removing a Specific Key

```
od.pop('mouse')
```

`pop()` removes a specific key and its value from the dictionary.

ChainMap from collections

We have a family with preferred values:

```
family = dict(food='pizza', drink='coca', team='AC', city='london')
```

And a child with their own preferences:

```
son = dict(food='pasta', team='FC', os='Windows', proglang='python')
```

Some values overlap, and some are new.

Problem

How can we combine these dictionaries while keeping the original sources separate?

We want: - If a value exists in `son`, use it - Otherwise, fall back to `family`

Solution: ChainMap

```
from collections import ChainMap
```

```
ch = ChainMap(son, family)
print(ch)
```

`ChainMap` groups multiple dictionaries without merging them.

Accessing Values

```
print(ch['food'])    # taken from son
print(ch['drink'])   # fallback to family
print(ch['os'])      # only in son
```

It searches in order: 1. son 2. family

Parent Maps

```
print(ch.parents)
```

parents returns all maps except the first one (removes the top layer).

Key Idea

ChainMap is useful when you want layered dictionaries where: - the first dictionary has priority - others act as fallback sources

It is especially useful for configuration systems and scoped overrides.

The deque Data Type

deque stands for double-ended queue — a queue that can be modified from both ends.

```
from collections import deque
```

A regular list is similar:

```
A = [1, 3, 4, 2, 6]
A.append(7)
```

This works fine, but in terms of time complexity, appending or removing elements from a list is $O(n)$. With deque, the same operations are $O(1)$, which is much faster.

A Note on Time Complexity

Time complexity is an important concept in computer science — it describes how long an algorithm or operation takes to complete, especially as the amount of data grows.

$O(n)$ means the operation's duration depends directly on the number of elements — more elements means more time. $O(1)$ means the operation's duration does not depend on the number of elements; it takes roughly constant time regardless of size.

Basic Usage

```
dq = deque([1, 3, 2, 6, 7, 3, 4, 5, 9, 1, 2, 1])
print(dq)

dq.append(8)           # add to the right side
dq.appendleft(8)       # add to the left side
print(dq)
```

```
print(dq.pop())      # removes and returns the last element (rightmost)  
print(dq.popleft()) # removes and returns the first element (leftmost)
```

Extending a Deque

To add multiple items at once, use `.extend()` (right side) or `.extendleft()` (left side):

```
dq.extend([5, 6, 7])  
dq.extendleft([5, 6, 7])  
print(dq)      # why are 7, 6, 5 added to the start instead of 5, 6, 7?  
               # because extendleft adds elements one at a time from the input –  
               # 5 is added first (becoming the first element), then 6 is added before it, then 7
```

Inserting and Removing by Value

```
dq.insert(3, 15)  
print(dq)
```

```
dq.remove(15)  
print(dq)
```

Reversing and Rotating

`.reverse()` reverses the deque in place:

```
dq.reverse()  
print(dq)
```

`.rotate()` shifts every element by a given number of positions:

```
dq.rotate(2)      # every member is shifted forward by two positions  
print(dq)
```

`.rotate()` also accepts negative values, rotating in the opposite direction:

```
dq.rotate(-2)  
print(dq)
```

Searching by Index

```
print(dq.index(3))      # index of the first occurrence of 3  
print(dq.index(3, 10)) # index of 3, searching starting from index 10
```

Searching within a specific index range:

```
# print(dq.index(3, 0, 4))    # raises ValueError if 3 is not found in this range
print(dq.index(3, 5, 10))
```

In summary, deque behaves much like a list, but is significantly more powerful and efficient for operations at both ends.

Single-Type Arrays

Python is not very strict about data types within a data structure — this is convenient, but can also cause problems in some cases.

```
A = [1, 2, 3, 4, 5]
#A[3] = "Hello"
print(A)
```

To enforce a single data type, the `array` class from the `array` module (`array.array`) can be used. The `array` module's documentation describes the available type codes — some common ones include `'i'` for signed int, `'f'` for float, `'d'` for double, and others corresponding to C data types.

```
from array import array
B = array('i')
print(B)
print(list(B))
```

Elements can be added with `.append()`:

```
B.append(2)
B.append(5)
B.append(7)
print(B)
print(B[0])
```

```
#B.append(7.0)    # TypeError: 'float' object cannot be interpreted as an integer
```

Values can be changed by index:

```
B[0] = 1
#B[0] = "Hello"    # TypeError: 'str' object cannot be interpreted as an integer
print(B)
```

`.extend()` is also available:

```
B.extend([8, 9, 10])
print(B)
```

An array can be declared with initial values, either from an existing list or directly:

```
C = array('i', A)
print(C)
```

```
D = array('i', [9,8,7,6,5])
print(D)
```

MappingProxyType

`namedtuple` has already been covered, with benefits such as named field access and immutability. However, `namedtuple` does not behave exactly like a dictionary. `MappingProxyType` provides a read-only view of an existing dictionary, while keeping dictionary-like behavior.

```
data = {'cat': 'meow', 'dog': 'woof', 'mouse': 'squeek', 'horse': 'neigh', 'frog': 'ribbit'}
print(data)
```

A normal dictionary can always be accessed and modified freely.

```
from types import MappingProxyType

mpt = MappingProxyType(data)

print(mpt['cat'])
print(mpt['horse'])
```

`mpt` is a read-only proxy wrapping `data`. It behaves exactly like the original dictionary for reading values — `mpt['cat']` and `mpt['horse']` work just as they would on `data` directly.

```
#mpt['cat'] = 'meowwww'
#mpt['raven'] = 'crow'
```

These commented-out lines show that attempting to modify or add an item through the proxy raises a `TypeError`, since `'mappingproxy'` objects do not support item assignment.

```
data['frog'] = 'ghourr'
print(data)
print(mpt)
```

Since `mpt` is only a view of `data` and does not hold its own copy of the data, any change made to `data` directly is also reflected when accessing `mpt`.

The concept can be compared to a proxy server in networking: a proxy server does not generate requests or responses itself, but only routes packets between two other parties. Similarly, `MappingProxyType` does not contain the data itself — it simply provides read-only access to the underlying dictionary.

```
'''
in networks, proxy is the middle server, it does not generate request or response but it only
you can think of it like that. MappingProxyType does not contain data itself, but returns the
'''
```

This triple-quoted string is a multi-line comment summarizing the proxy analogy described above. The same explanation has already been incorporated into the prose, so it is not repeated here as code.

To try this yourself, save the code into a file named `practice.py` and run it with `python3 practice.py`.

Useful Python Functions

`all()` and `any()` — Built-in Boolean Checks

Python provides two built-in functions, `all()` and `any()`, that take a collection of values and evaluate them as booleans.

- `any()` — returns `True` if **at least one** value in the collection is `truthy`.
- `all()` — returns `True` only if **every** value in the collection is `truthy`.

All values are cast to booleans internally before evaluation. The logic maps directly to:

```
any([b1, b2, b3, ..., bn]) => b1 or b2 or b3 or ... bn
```

```
all([b1, b2, b3, ..., bn]) => b1 and b2 and b3 and ... bn
```

Save the following as `allAndAny.py` and run it with `python3 allAndAny.py`.

```
A = [1, 0, 0, 0, 1]
B = [1, 1, 1, 1, 1]
C = [0, 0, 0, 0, 0]
```

```
print(any(A))
print(any(B))
print(any(C))
```

```
print(all(A))
print(all(B))
print(all(C))
```

```
# Practical use case: checking if any dice roll in a list equals 6.
```

```
# The list comprehension builds a list of booleans, then any() checks if at least one is True.
```

```
dices = [1,2,4,3,2,4,5,6,2,4,4,2,1,1]
```

```
if any([x == 6 for x in dices]):
    print("we had at least a winner!")
```

```
# This version passes a generator expression instead of a list.
```

```
# It is more memory-efficient because values are produced one at a time rather than building a list.
```

```
print(any(x == 6 for x in dices))
```

`min()` and `max()` — Finding Extremes in a Collection

`min()` and `max()` are built-in Python functions that return the smallest and largest values in a collection respectively.

Save the following as `minAndMax.py` and run it with `python3 minAndMax.py`.

```
A = [1, -1, -2, 3, 4, -5, 7, 9, -10]
```

```
print(min(A))
print(max(A))
```

Boolean Collections

These functions also work on boolean values. When applied to a boolean collection, `max()` behaves like `any()` and `min()` behaves like `all()` — making `all()` and `any()` essentially specialised versions of `min()` and `max()` for boolean data.

```
C = [True, False, False, True, True]
```

```
print(min(C), all(C))
print(max(C), any(C))
```

The key Parameter

Both functions accept a **key** argument — a function applied to each element before comparison. The element with the minimum or maximum *transformed* value is returned, not the transformed value itself.

```
# Returns the element whose square (x ** 2) is smallest or largest.
```

```
print(min(A, key=lambda x: x ** 2))
print(max(A, key=lambda x: x ** 2))
```

```
# Returns the element whose remainder when divided by 7 is smallest or largest.
```

```
print(min(A, key=lambda x: x % 7))
print(max(A, key=lambda x: x % 7))
```

Applying key to Dictionaries

The key parameter is particularly useful when working with collections of dictionaries. For simple lookups, a lambda works fine. For more complex calculations, defining a named function improves readability.

```
data = [
    {'Name': 'A', 'Age': 10, 'Revenue': 100},
    {'Name': 'B', 'Age': 8, 'Revenue': 90},
    {'Name': 'C', 'Age': 15, 'Revenue': 120},
    {'Name': 'D', 'Age': 5, 'Revenue': 60},
    {'Name': 'E', 'Age': 20, 'Revenue': 95},
]
```

```
def get_age(item):
    return item['Age']
```

```
# The oldest entry.
```

```
print('oldest:')
print(max(data, key=get_age))
```

```
# The highest revenue entry.
```

```
print('richest:')
print(max(data, key=lambda x: x['Revenue']))
```

```
# The most efficient entry by revenue-to-age ratio.
```

```
print('efficient')
```

```
print(max(data, key=lambda x: x['Revenue'] / x['Age']))
```

sum() and len() — Totalling and Counting

`sum()` returns the total of all elements in a collection. `len()` returns the number of elements in an iterable.

```
A = [1, 2, 4, 6, 7]
print(sum(A))
print(len(A))
```

Combining them gives you the average of a list:

```
print(sum(A) / len(A))
```

The start Parameter

`sum()` accepts an optional `start` parameter, which sets an initial value that the sum is added to. This is useful when you need to offset the result.

```
print(sum(A, start=5))
```

Summing a Range

`range()` is iterable, so it can be passed directly to `sum()` without converting it to a list first.

```
print(list(range(1, 101)))
print(sum(range(1, 101)))
```

Floating-Point Precision with `math.fsum()`

For calculations where floating-point accuracy matters, `math.fsum()` is more reliable than `sum()`. It uses a more precise algorithm to reduce rounding errors that can accumulate when summing many floats.

```
import math
print(math.fsum([1, 2, 3, 4]))
```

Hashing in Python

A hash function maps a value to a fixed integer. The mapping is one-way — you cannot recover the original value from its hash. This is why passwords are almost universally stored as hashes rather than plaintext.

The authentication flow works like this:

- At registration, the password `P` is hashed to produce `h`, which is stored.
- At login, the submitted password `P'` is hashed to produce `h'`.
- If `h` and `h'` match, the login succeeds — and with high confidence, `P` and `P'` are the same value.


```
print(hash('SoraVoid'))
print(hash('SoraVoid1'), 'difference between this line and line above is a "1"')

print(hash('SoraVoid') == hash("SoraVoid"))
```

Even a single character difference produces a completely different hash value. Two identical strings always produce the same hash.

Password Comparison with Hashes

```
passwd = input("Enter password: ")
if hash(passwd) == hash('root'):
    print('Hello Boss!')
else:
    print('Who are you?')
```

It is also common to store and display hashes in hexadecimal format:

```
print(hex(hash('SoraVoid')))
```

Hashing Tuples

Hashes are useful for comparing compound data. Tuples are hashable because they are immutable:

```
data = ('Sora', 24, 185, 'A+')
data2 = ('Sora', 24, 185, 'A+')
data3 = ('Dora', 26, 175, 'B+')

print(hash(data) == hash(data2))
print(hash(data) == hash(data3))
```

Lists are **not** hashable because they are mutable. To hash a list, convert it to a tuple first.

Custom `__hash__()` in Classes

You can define `__hash__()` on a class to control what Python uses as the basis for comparison. In the example below, two objects are considered equal by hash if they share the same name, regardless of other attributes:

```
class ObjectCreator():
    def __init__(self, name, age, color):
        self.name = name
        self.age = age
        self.color = color

    def __hash__(self):
        return hash(self.name)

obj1 = ObjectCreator('sora', 24, 'white')
```

```
obj2 = ObjectCreator('sora', 36, 'black')
obj3 = ObjectCreator('anna', 24, 'white')

print(hash(obj1) == hash(obj2))
print(hash(obj1) == hash(obj3))
```

obj1 and obj2 share the name 'sora', so their hashes match. obj3 has a different name, so its hash differs.

Copying Variables — Value Types vs Reference Types

Primitive (Value) Types

With primitive types such as numbers, strings, and `None`, copying a variable produces a fully independent value. Modifying the original has no effect on the copy.

```
a = 1
b = a

a += 1

print(a)
print(b)
```

Reference Types

Lists, dictionaries, and other compound objects are reference types — a variable holds a pointer to a location in memory, not the data itself. Assigning one variable to another copies the reference, not the contents. Both names then point to the same object.

```
a = [1, 2, 3, 4, 5]
b = a
a[2] = 10

print(a)
print(b)
```

b reflects the change because it points to the same list in memory.

Shallow Copy

The `copy` module solves the top-level aliasing problem. `copy.copy()` creates a new object, so replacing an element in one list does not affect the other.

```
import copy

a = [1, 2, 3, 4, 5]
b = copy.copy(a)
a[2] = 10
print('using copy module:')
```

```
print(a)
print(b)
```

However, `copy.copy()` is a **shallow copy** — it only duplicates the outer container. Nested objects inside it are still shared by reference. Modifying a nested element affects both copies.

```
A = [[1, 2], [3, 4, 5], [6, 7, 8, 9]]
B = copy.copy(A)

A[1][2] = 100
print(A)
print(B)
```

Deep Copy

`copy.deepcopy()` recursively duplicates every nested object. The two collections become completely independent, and changes to one do not affect the other.

```
A = [[1, 2], [3, 4, 5], [6, 7, 8, 9]]
B = copy.deepcopy(A)

A[1][2] = 100
print('Using deep copy:')
print(A)
print(B)
```

For nested structures, `deepcopy()` is the correct choice.

== vs is in Python

Python's `==` compares two objects **element by element** (value equality). The `is` keyword checks whether two names point to the **exact same object in memory** (identity).

This differs from JavaScript, where `==` applies type coercion before comparing, and `===` enforces strict type and value equality — both operators check reference equality when applied to objects, meaning two separate arrays with identical contents are not considered equal.

```
a = [1, 2, 3, 4, 5]
b = [1, 2, 3, 4, 5]

print(a == b)
print(a is b)
```

`a == b` returns `True` because the contents match. `a is b` returns `False` because they are two distinct objects in memory.

Accurate Calculations with decimal

Standard Python floats use binary floating-point representation (IEEE 754), which cannot represent many decimal fractions exactly. This causes subtle rounding errors.

```
print(0.1 + 0.2 == 0.3)
print(0.1 + 0.2)
```

`0.1 + 0.2` does not equal `0.3` — the result is `0.30000000000000004`. Binary floats also have a fixed number of significant digits, which limits precision in division:

```
print(1/7)
```

The `Decimal` Type

The `decimal` module provides a `Decimal` type that performs arithmetic in base 10, eliminating the rounding errors introduced by binary float representation.

```
from decimal import Decimal
```

```
print(Decimal('0.1') + Decimal('0.2') == Decimal('0.3'))
print(Decimal('1') / Decimal('7'))
```

Always pass values as strings when constructing `Decimal` objects. If you pass a float literal directly, the binary rounding error is captured before `Decimal` can do anything about it:

```
print(Decimal(0.1) + Decimal(0.2))
print(Decimal(0.1))
```

Configuring Global Precision with `getcontext()`

`getcontext()` returns the active decimal context, which controls settings such as precision (the number of significant digits).

```
from decimal import getcontext

print(getcontext())
print(getcontext().prec)

getcontext().prec = 50
print(Decimal('1') / Decimal('7'))
```

Changing `getcontext().prec` affects all subsequent `Decimal` operations in the module.

Temporary Precision with `localcontext()`

When you only need a different precision for a specific block of code, use `localcontext()` as a context manager. Changes made inside the `with` block are scoped locally and do not affect the global context.

```
from decimal import localcontext

print(Decimal('1') / Decimal('7'))

with localcontext() as c:
    c.prec = 100
    print(Decimal('1') / Decimal('7'))
```

```
print(Decimal('1') / Decimal('7'))
```

The first and last `print` calls use the global precision. Only the call inside the `with` block uses a precision of 100.

Comprehensions

List, Set, and Dictionary Comprehensions

Comprehension is a concise Python syntax for building collections where each member is the result of an expression applied to items in an iterable.

List Comprehension

```
A = [x**2 for x in range(1, 10)]
B = [x**2 for x in range(1, 10) if x % 2 == 1]
print(A)
print(B)
```

A contains the square of every number from 1 to 9. B filters the range first, producing squares of odd numbers only. A condition can be added after the `for` clause to filter which elements are included.

The expression can be any valid Python code, including tuple construction:

```
print([(x-1, x, x+1) for x in range(1, 10)])
```

Set Comprehension

Wrapping the comprehension in curly braces `{}` produces a set instead of a list. Because sets do not allow duplicates, repeated values are automatically collapsed.

```
C = [1, 2, -1, 3, -2, 4, 5, 3]
D = {x**2 for x in C}
print(D)
```

The following is an equivalent alternative using a list comprehension wrapped in `set()`:

```
# set([x**3 for x in C])
```

Dictionary Comprehension

A dictionary comprehension uses the `key: value` syntax inside curly braces. Each iteration produces one key-value pair.

```
E = {x: (x**2, x**3) for x in C}
print(E)
```

Each element of C becomes a key, mapped to a tuple of its square and cube. Note that because dictionaries cannot have duplicate keys, repeated values in C (such as 3) will result in only one entry per unique key.

Generator Comprehensions

You are already familiar with list, set, and dictionary comprehensions. Using parentheses instead of square brackets does not create a tuple — Python has no tuple comprehension syntax. Instead, it produces a **generator**.

```
A = [x**2 for x in range(1, 10)]
print(A)
```

```
B = (x**2 for x in range(1, 10))
print(B)
```

A is a list with all nine values computed and stored in memory immediately. B is a generator object — it computes and yields one value at a time, only when asked.

Consuming a Generator with `next()`

When a generator is used in a `for` loop, Python calls `next()` on it automatically at each iteration. You can also call `next()` manually. The optional second argument is a default value returned when the generator is exhausted, instead of raising `StopIteration`.

```
print(next(B, 'Over'))
print(next(B, 'Over'))
print(next(B, 'Over'))
print(next(B, 'Over'))
print(next(B, 'Over'))
print(next(B, 'Over'))
print(next(B, 'Over'))
print(next(B, 'Over'))
print(next(B, 'Over'))
print(next(B, 'Over'))
print(next(B, 'Over'))
```

The generator holds nine values (1 through 9 squared), so the tenth and eleventh calls return 'Over'.

Memory Efficiency

Because generators produce values on demand rather than building a full collection in memory, they can be passed directly to any function that accepts an iterator — such as `max()` or `sum()`.

```
print(max([x**2 for x in [1, -3, 5, 2, 6]])) # list built in memory first
print(max((x**2 for x in [1, -3, 5, 2, 6]))) # generator passed directly
print(max(x**2 for x in [1, -3, 5, 2, 6]))  # parentheses optional when passed as sole argu
print(sum(x**2 for x in [1, -3, 5, 2, 6]))
```

The core of a generator expression is the expression and `for` clause — the surrounding parentheses are only required for grouping. When the generator is the sole argument to a function, the function's own parentheses are sufficient.

Generator comprehensions are one type of generator. Python also supports generator functions using the `yield` keyword, which will be covered separately.

Itertools

Infinite Iterators — itertools

The `itertools` module provides a set of tools for working with iterables. Among them are three **infinite iterators** — iterators that never stop producing values on their own and must be controlled externally with a `break` or by consuming them with `next()`.

The three infinite iterators are:

- `count` — counts upward from a starting value, with an optional step, indefinitely.
- `cycle` — repeatedly loops through the items of an iterable indefinitely.
- `repeat` — yields a single value either indefinitely or a specified number of times.

`count`

```
from itertools import count, cycle, repeat
```

```
for i in count(5, 2):
    if i > 15:
        break
    print(i)
```

`count(5, 2)` starts at 5 and increments by 2 on each step. Without the `break`, this loop would run forever.

`cycle`

`cycle` takes any iterable and loops through its members indefinitely, restarting from the beginning each time it reaches the end.

```
basicCounter = 0
for c in cycle('ABCDEFGF'):
    print(c)
    basicCounter += 1
    if basicCounter == 20:
        break
```

You can also consume a `cycle` manually with `next()`:

```
c = cycle("ABCDEFGF")
print(next(c))
print(next(c))
print(next(c))
print(next(c))
print(next(c))
print(next(c))
print(next(c))
print(next(c))
print(next(c))
print(next(c))
print(next(c))
```

```
print(next(c))
print(next(c))
```

A practical use of `cycle` is pairing it with `zip()`. Because `zip()` stops at the shortest iterable, an infinite cycle can be used to assign repeating labels or styles to a finite list:

```
s = cycle(['_', ':', '.-'])
A = ['f1', 'f2', 'f3', 'g1', 'g2', 'g3', 'h1', 'h2', 'h3']
print(list(zip(A, s, c)))
```

Each element of `A` is paired with the next value from `s` and the next value from `c`. Since both `s` and `c` are infinite, `zip()` stops when `A` is exhausted after 9 elements.

repeat

`repeat` yields the same value every time it is called. Providing a second argument limits the number of repetitions. Without it, `repeat` runs indefinitely.

```
for i in repeat(10, 5):
    print(i)
```

```
r = repeat('Hello!')
print(next(r))
print(next(r))
print(next(r))
print(next(r))
print(next(r))
print(next(r))
print(next(r))
print(next(r))
```

zip_longest — Zipping by the Longest Iterable

The built-in `zip()` function combines two or more iterables element by element, producing tuples. It stops as soon as the shortest iterable is exhausted.

```
A = range(10)
B = 'ABCDE'

print(list(zip(A, B)))
```

The output contains only 5 tuples because `B` has 5 characters and `zip()` stops there, discarding the remaining values from `A`.

zip_longest

`itertools.zip_longest()` works the same way but continues until the **longest** iterable is exhausted. Shorter iterables are padded with `None` for the missing positions.

```
from itertools import zip_longest

print(list(zip_longest(A, B)))
```


This produces 10 tuples. Positions 5 through 9 have `None` in place of the missing `B` values. The same applies when combining three or more iterables of different lengths:

```
C = [10, 20]
print(list(zip_longest(A, B, C)))
```

Here, `A` is the longest with 10 elements. `B` runs out at position 5 and `C` runs out at position 2, with `None` filling the gaps for both.

accumulate — Running Aggregations

`itertools.accumulate()` produces a running aggregation over an iterable. For each element, it applies an operation using the current element and the accumulated result from all previous elements, yielding the result at each step.

Default Behaviour — Running Sum

By default, `accumulate()` performs addition, producing a running total:

```
from itertools import accumulate

A = [1, 2, 3, 4, 5]

C = accumulate(A)
print(C)
print(list(C))
```

The output is `[1, 3, 6, 10, 15]` — each value is the sum of all elements up to and including that position.

The `initial` Parameter

An `initial` value can be provided. It is prepended to the output before any elements from the iterable are processed, making the output one element longer than the input:

```
print(list(accumulate(A, initial=10)))
```

Custom Operations

The second argument accepts any two-parameter function, replacing the default addition. The function receives the stored accumulated value and the current element, and its return value becomes the new stored value.

For a running product, a named function or a lambda both work:

```
def prod(x, y):
    return x * y

print(list(accumulate(A, prod)))
print(list(accumulate(A, lambda x, y: x * y)))
```

The function always takes exactly two parameters because `accumulate` maintains a single stored value internally. The first element is taken as the initial stored value with no calculation applied. The first actual computation happens at the second element: $2 * 1 = 2$. That result is stored, and at the next step $3 * 2 = 6$ becomes the new stored value, then $4 * 6 = 24$, and so on.

Chaining Iterables with `itertools.chain`

We can chain some iterables together manually:

```
A = [10, 20, 30]
B = [40, 50, 60]
C = []
for i in A:
    C.append(i)
for i in B:
    C.append(i)
print(C)
```

A better approach is to use `itertools.chain`:

```
from itertools import chain

D = []
for i in chain(A, B):
    D.append(i)
print(D)

print(chain(A, B)) # chain returns an iterator that iterates over A and then B
print(list(chain(A, B)))
```

Another example using `repeat`:

```
from itertools import repeat

for i in chain(repeat(10, 3), repeat(20, 2), repeat(30, 5), repeat(40, 3)):
    print(i)
```

Flattening Lists of Lists

```
A = [
    [1, 2, 3, 4],
    [5, 6, 7, 8, 9, 10, 11],
    [1, 2, 3],
    [3, 4, 5, 6, 7, 8, 9]
]

print(A)
print(list(chain(A[0], A[1], A[2], A[3])))
print(list(chain(*A)))           # using unpacking
print(list(chain(*A[1:])))       # using unpacking with slicing
```

This is a common way to flatten an array of arrays into a single flat iterable.

Filtering Iterables with `filter`, `filterfalse`, `compress`, `dropwhile`, and `takewhile`

The built-in `filter()` function applies a predicate to every element of an iterable and keeps only those that return `True`.

```
A = [1, 2, 3, 4, 5, 6, 7, 8, 9, 2, 3, 4, 5]
B = filter(lambda x: x % 3 == 0, A)
print(list(B)) # returns numbers divisible by 3
```

To keep elements that do **not** satisfy the condition, you can negate the predicate:

```
C = filter(lambda x: not x % 3 == 0, A)
print(list(C))
```

`itertools.filterfalse`

`itertools.filterfalse` does the opposite of `filter` — it keeps elements for which the predicate returns `False`.

```
from itertools import filterfalse

D = filterfalse(lambda x: x % 3 == 0, A)
print('using filterfalse')
print(list(D))
```

`itertools.compress`

`compress` filters elements from the data iterable using a selector iterable of boolean values (or truthy/falsy values). Both iterables must be of the same length.

```
A = [1, 2, 3, 4, 5, 6, 7, 8, 9, 2, 3, 4, 5]
E = [1, 1, 1, 1, 0, 0, 0, 0, 0, 1, 1, 1, 0] # flags: keep if 1 (truthy)

from itertools import compress
F = compress(A, E)
print(list(F))
```

You can also generate the selector list randomly:

```
from random import randint

E = [randint(0, 1) for _ in range(len(A))]
F = compress(A, E)
print(list(F))
```

Because the number of `True`/`False` values varies, the output size will differ on each run.

dropwhile and takewhile

`dropwhile` drops elements from the start of the iterable while the predicate is `True`. Once the predicate becomes `False`, it yields all remaining elements.

```
from itertools import dropwhile, takewhile

A = [1, 2, 3, 4, 5, 6, 7, 8, 9, 2, 3, 4, 5]
DW = dropwhile(lambda x: x < 5, A)
print(list(DW))
```

`takewhile` does the opposite: it yields elements as long as the predicate is `True`. As soon as the predicate becomes `False`, it stops.

```
TW = takewhile(lambda x: x < 6, A)
print(list(TW))
```

Working with Consecutive Pairs Using `itertools.pairwise`

Sometimes we need to iterate over an iterable while accessing both the current and the previous (or next) element at the same time. This is especially useful when working with time-series or sequential data.

```
A = [3, 2, 5, 6, 3, 6, 7, 2, 1, 6, 7, 8, 4, 9]
```

```
from itertools import pairwise
```

```
for p in pairwise(A):
    print(p) # p is a tuple containing a pair of consecutive elements
```

We can also unpack the tuple directly:

```
for a, b in pairwise(A):
    print(f'{a = }, {b = }, {b - a}')
```

Since `pairwise` returns an iterator, we can convert it to a list:

```
print(list(pairwise(A)))
```

Alternative Implementation with `zip`

The same result can be achieved using the built-in `zip` function:

```
print(A)
print(A[:-1])
print(A[1:])

B = zip(A[:-1], A[1:])
print(list(B))
```

Or more concisely:

```
C = zip(A, A[1:])
print(list(C))
```

Both approaches produce identical results because `zip` stops at the shortest iterable.

Sliding Windows of Three Elements

You can create overlapping triples by nesting `pairwise` or using `zip` with multiple slices:

```
# Using nested pairwise
print(list(pairwise(pairwise(A))))

# Better approach with zip
D = zip(A[:-1], A[1:], A[2:])
print(list(D))
```

Applying Functions to Argument Tuples with `itertools.starmap`

The built-in `pow()` function raises a number to a given power:

```
print(pow(4, 2))
```

When we have a list of tuples containing argument pairs, we can apply the function using a list comprehension:

```
A = [(2, 5), (3, 3), (4, 2), (5, 3), (7, 2)]

B = [pow(x, y) for x, y in A]
print(B)

C = [pow(p[0], p[1]) for p in A]
print(C)

D = [pow(*p) for p in A]
print(D)
```

`itertools.starmap` provides a clean way to achieve the same result by unpacking each tuple as arguments to the function:

```
from itertools import starmap

sm = starmap(pow, A)
print(list(sm))
```

`starmap` is equivalent to calling `pow(2,5)`, `pow(3,3)`, etc., for each tuple in the input iterable.

Cartesian Product with `itertools.product`

The `itertools.product` function computes the Cartesian product of input iterables — essentially generating all possible combinations of elements.

```
from itertools import product
```

```
A = [1, 2, 3]
```

```
B = [4, 5]
```

```
print(list(product(A, B)))
```

This produces 6 tuples: all possible pairs where the first element comes from A and the second from B.

You can also compute the product of an iterable with itself:

```
print(list(product(A, A)))
```

The repeat parameter allows repeating the same iterable multiple times:

```
print(list(product(A, B, repeat=2)))
```

```
print(list(product(A, repeat=2)))
```

```
print(list(product(A, repeat=3)))
```

Practical Example: Generating Combinations

```
# Traditional nested loops
```

```
for i in 'ABC':
```

```
    for j in [1, 2, 3]:
```

```
        print(i, j)
```

```
# Cleaner version using product
```

```
for i, j in product('ABC', [1, 2, 3]):
```

```
    print(i, j)
```

product is very useful for generating test cases, coordinate grids, or exhaustive combinations in algorithms.

Permutations and Combinations with itertools

Permutations

Permutations represent all possible orderings of elements.

```
A = [1, 2, 3, 4]
```

```
from itertools import permutations
```

```
print(list(permutations([1, 2])))
```

```
print(list(permutations([1, 2, 3], 2))) # specify output length
```

```
print(list(permutations(A)))
```

Combinations

Combinations are selections of elements where order does not matter.

```
from itertools import combinations

print(list(combinations([1, 2, 3, 4], 2)))
```

Combinations with Replacement

This allows elements to be repeated (like drawing with replacement).

```
from itertools import combinations_with_replacement as cwr

print(list(cwr([1, 2, 3], 2)))

# Example: all possible DNA sequences of length 6
print(list(cwr('ATCG', 6)))
```

Functions In Python

Functions in Python: Definition and Properties

Functions are an important part of every good program. In Python, functions are first-class objects.

```
def func(x):
    ...

    this is a document string about func:
    this function takes a number and returns number + 1
    ...

    return x + 1

print(func(3))
print(dir(func))
print(callable(func)) # checks if the object has a __call__() method

print(func.__call__(2))
print(func)
print(func.__name__)
```

We can access the docstring using the `__doc__` attribute or the `help()` function:

```
print(func.__doc__)
help(func)
```

Functions can be assigned to variables:

```
a = func
print(a)
print(a.__name__)

del func
print(a.__name__)
```

Using `map` with Functions

The `map()` function applies a function to every item of an iterable. It is lazy, so it does not execute until the result is consumed.

```
mapObj = map(print, "sora")
list(mapObj)
```

Applying Multiple Functions to Data

```
A = [3, 2, 4, 5, 6, 1, 6, 8, 3, 4, 9]

def mean(x):
    return sum(x) / len(x)

funcs = [min, max, mean, len, sum]
print(list(map(lambda f: f(A), funcs)))
```

Passing Functions as Arguments

In Python, functions are first-class objects, which means they can be passed as arguments to other functions.

```
def value_at_zero(f):
    return f(0)

def func1(x):
    return x + 1

def func2(x):
    return x - 1

def func3(x):
    return x * 1

print(value_at_zero(func1))
print(value_at_zero(func2))
print(value_at_zero(func3))
```

A more practical version that includes logging:

```
def call_with_prompt(f, x):
    print(f'Calculating {f.__name__}(x)')
    print(f'Result: {f(x)}')
    print()
    return f(x)

call_with_prompt(func1, 5)
```


Functions Returning Functions and Closures

Functions in Python can be defined inside other functions (nested functions) and can also return other functions.

```
def func(x):
    def f1(t):
        return x + t

    def f2(t):
        return x - t
    return f1(2), f2(2)
```

```
print(func(10))
```

Returning Functions (Factory Pattern)

```
def adder(n):
    def f(x):
        return x + n
    return f

a = adder(1)  # returns a function that adds 1
print(a(5))

c = adder(2)
print(c(5))

print(adder(2)(5))  # direct call
```

Checking Callability

```
print(callable(adder))          # True
print(callable(adder(6)))       # True
print(callable(adder(5)(6)))    # False
```

Making Any Object Callable

```
def make_callable(x):
    if callable(x):
        return x
    else:
        def f(*args, **kwargs):
            return x
        return f

a = abs
b = 10
callable_a = make_callable(a)
```

```
callable_b = make_callable(b)

print(callable_a(3))
print(callable_b(5, "hello"))
```

Simple Function Wrapper (Introduction to Decorators)

```
def someFunc(f):
    print(f"Calling function {f.__name__}:")
    def internal(*args, **kwargs):
        return f(*args, **kwargs)
    return internal

new_abs = someFunc(abs)    # new_abs is internal function which returns the abs function output
print(new_abs(-10))
```

This pattern is the foundation of Python decorators.

Callable Classes Using `__call__`

We can achieve similar behavior to the `adder` factory function (from a previous lessons in this topic) by implementing a class with the `__call__` special method. This makes instances of the class callable like functions.

```
class Adder():
    def __init__(self, n):
        self.n = n

    def __call__(self, x):
        return x + self.n

a = Adder(1)
print(a.n)
print(a(6))    # calls the __call__ method

a.n = 2
print(a.n)
print(a(6))
```

You can also use the class directly without storing the instance:

```
print(Adder(3).n)
print(Adder(3)(6))
```

This approach is useful when you need stateful callable objects or want to leverage object-oriented features alongside function-like behavior.

Variable Positional Arguments (*args)

Python allows functions to accept an arbitrary number of positional arguments using `*args`.

```
def func(*args): # args can be any valid variable name
    print('Parameters:')
    for arg in args:
        print(arg)

print("With no params:")
func()
print("With one argument:")
func(1)
print("With two arguments:")
func(1, 2)
print("With three arguments:")
func(1, 2, 3)
```

Another way to inspect the arguments:

```
def func(*args):
    print(f"{args = }")

print("With no params:")
func()
print("With one argument:")
func(1)
print("With two arguments:")
func(1, 2)
print("With three arguments:")
func(1, 2, 3)
```

You can unpack a list or other iterable when calling the function:

```
print("With a list:")
L = [1, 2, 3, 4, 5, 'a']
func(*L) # unpacking the list
```

This unpacking behavior with `*` is the same mechanism used internally for `*args`. We also know that dicts can be unpacked with `**dictname`. so let's talk about `**kwargs`!

Variable Keyword Arguments (**kwargs)

We talked about `*args` in the previous lesson. What if we want to take an unknown amount of keyword arguments or named arguments? We should use `**kwargs`.

```
def func(**kwargs):
    print(f"{kwargs = }")

print("With no params:")
func()
```

```

print("With one argument:")
func(first=1)
print("With two arguments:")
func(first=1,second=2)
print("With three arguments:")
func(first=1,second=2,third=3)

```

```

print("With a dictionary:")
D = {
    'a': 1,
    'b': 2,
    'c': 3,
    'd': 4,
    'e': 5,
}
func(**D) # unpacking dictionary and it looks exactly like **kwargs

```

We can combine `*args` with `**kwargs`:

```

def fun(*args, **kwargs):
    print(f'{args = }')
    print(f'{kwargs = }')

```

```

fun(1, 2, 3, a=4, b=5, msg="hello")

```

fun(1, 2, 3, a=4, 5, b=6) # throws an error. why? because positional args must be written before

We can also have normal parameters at the start:

```

def fun(x, y, *args, **kwargs):
    print(f'{x = }')
    print(f'{y = }')
    print(f'{args = }')
    print(f'{kwargs = }')

```

```

fun(1, 2, 3, a=4, b=5, msg="hello")

```

Separating Positional and Keyword Arguments

Imagine we have a simple function:

```

def func(x, y):
    print(f'{x = }')
    print(f'{y = }')

```

This function takes positional arguments but...

We can call the function using parameter names. This allows changing the order and makes the code clearer in some cases.

```

func(10, 20)
func(y=10, x=20)

```

If we do not want one of these calling styles for any reason, we can enforce restrictions

using / and * in the function definition.

Positional-Only Arguments (/)

Anything before the / can only be passed as positional arguments (keyword arguments are not allowed).

```
def func1(x, y, /):
    print(f'{x = }')
    print(f'{y = }')

func1(10, 20)
# func1(y=10, x=20) # TypeError: func1() got some positional-only arguments passed as keyword
```

Keyword-Only Arguments (*)

Everything after the * can only be passed as keyword arguments.

```
def func2(*, x, y):
    print(f'{x = }')
    print(f'{y = }')

func2(x=10, y=20)
# func2(10, 20) # TypeError: func2() takes 0 positional arguments but 2 were given
```

In a function definition we have three fields in this order:

POSITIONAL ONLY, /, **POSITIONAL OR KEYWORD**, *, **KEYWORD ONLY**

```
def fun(x, /, y, *, z):
    print(f'{x = }')
    print(f'{y = }')
    print(f'{z = }')

fun(1, 2, z=3)
fun(1, y=2, z=3)
# fun(1, z=3, 2) # SyntaxError: positional argument follows keyword argument
fun(1, z=3, y=2)
```

Generators In Python

Generator Functions with yield

Generators are one of Python's coolest features. The values are generated on the fly when we call next() on the generator object, unlike a normal list that stores all data in memory.

```
for i in [1,2,3,4]: # this is a list
    # more memory usage / more speed
    print(i)

for i in range(1, 5): # range is a kind of generator
```

```
# less memory usage / less speed
print(i)
```

We can create generators using comprehensions, functions, or classes. Let's look at generator functions.

```
def repeat():
    return 1
```

```
print('****')
print(repeat())
print(repeat())
print(repeat())
print(repeat())
print(repeat())
print('****')
```

A regular function like this cannot be iterated over directly.

```
# for i in repeat():
#     print(i) # TypeError: 'int' object is not iterable
```

Using yield instead of return turns the function into a generator:

```
def repeat2():
    yield 1 # with yield instead of return, this function becomes iterable
```

```
for i in repeat2():
    print(i) # only yields one '1'
```

```
# Infinite generator example
```

```
def repeat3():
    while True:
        yield 1
for i in repeat3():
    print(i)
```

We can also consume generators manually with next():

```
print(next(repeat2(), "Over"))
print(next(repeat2(), "Over"))
print(next(repeat2(), "Over"))
print(next(repeat2(), "Over"))
print(next(repeat2(), "Over")) # does not end because every call yields the static value
```

Real-World Generator Example (Similar to range)

```
def counter(start=0, end=10, step=1):
    i = start
    while True:
        yield i
        i += step
```

```

        if i >= end:
            break

for i in counter(0, 20, 2):
    print(i)

```

```

c = counter(1, 4, 1)
print(next(c, 'Over'))
print(next(c, 'Over'))
print(next(c, 'Over'))
print(next(c, 'Over'))
print(next(c, 'Over'))
print(next(c, 'Over'))

```

Fibonacci Sequence Generator

```

def fibonacci():
    a, b = 1, 1
    while True:
        yield a
        a, b = b, a + b # left side of assignment is future, right side is past

for i, j in enumerate(fibonacci()):
    print(f'{i} => {j}')
    if i >= 12:
        break

```

Generator Expressions (Comprehensions for Generators)

We can create generators using generator expressions, which are similar to list comprehensions but use parentheses instead of square brackets.

```

x = (x**2 for x in range(10))
print(x)

for i in (1 for _ in range(10)):
    print(i)

```

Generator expressions can be used to create infinite generators as well (when combined with infinite iterables).

```

A = range(10)
B = (x**2 for x in A)
C = (x**2 - x for x in A)
D = ((x, y) for x, y in zip(B, C))

```

*# All of these are generator expressions. They use very little memory until consumed.
 # When we convert them to a list or iterate over them, the values are evaluated.*

```
print(list(D))
```

Class-Based Generators

Now that we know about generator functions and generator expressions, we can also implement generators as classes.

```
class Counter():
    def __init__(self, start=0, end=10, step=1):
        self.start = start
        self.end = end
        self.step = step
        self.value = self.start

    # To make this class work as a generator (iterator), we need to implement __iter__ and __next__
    def __iter__(self):
        return self # for __iter__ we must return an object that has a __next__ method

    def __next__(self):
        old_value = self.value
        self.value += self.step
        # to raise an error in a specific condition, use the raise keyword followed by the error name
        if old_value >= self.end:
            raise StopIteration()
        return old_value

for _ in Counter(0, 20, 2): # running loop on a nameless instance of Counter class
    print(_)
```

Memory Usage and Performance: Lists vs Generators

Generators use less memory but are generally slower than lists. Let's see this in action:

```
A = [x**2 for x in range(1000)] # list
B = (x**2 for x in range(1000)) # generator

from sys import getsizeof # to get the size in memory

print(getsizeof(A)) # ~8.8 kB
print(getsizeof(B)) # ~200 B
print(getsizeof(A)/getsizeof(B)) # ~44 times bigger
```

In larger datasets, lists and other static data structures consume much more memory. However, using generators comes at the cost of performance — there is no free lunch.

```
sum([x**2 for x in range(10000)])
sum(x**2 for x in range(10000))
```

We can use `cProfile` to compare execution time and function calls:


```
import cProfile

# pass the code as a string:
print(cProfile.run('sum([x**2 for x in range(10000)])')) # faster, fewer calls
print(cProfile.run('sum(x**2 for x in range(10000))')) # slower, many more calls
```

Decorators In Python

Decorators Introduction

Functions in Python are also objects and there are many things that we can do with them. There is a feature in Python to change these functions' behavior when needed. These things are called **decorators**.

```
def func(): # i want something to be printed before and after this function.
    print('Hello!')
```

```
def addMessage():
    print("before calling funciton...")
    func()
    print("after calling function...")
```

```
addMessage()
```

Now imagine we have many more functions like `func`. Do we have to write a controller function like `addMessage` for all of them? Of course not!

```
def addMessage2(f):
    def wrapper():
        print('Before calling...')
        res = f() # wrapper knows that f is the original function. it remembers that and saves
        # about closure: when we define a function inside another, inner function has access to
        # this is called closure. original func is entered addMessage2 as a parameter. wrapper s
        # python automatically locks this permission, even when addmessage2 is done, wrapper s
        # will be assigned to new func. when new func is called, actually wrapper gets called
        print('After calling...')
        return res

    return wrapper # from previous lessons we know that output of addMessage2 is a function na

# at this point, addMessage2 takes a function as input, and calls it.
func2 = addMessage2(func)
print(func2)
func2()
```

This is what really happens in the background:

```
# in one line:
func = addMessage2(func)
func()
```

A decorator is a callable that takes a function and returns a function.

We can also use syntactic sugar (@) for better appearance:

```
@addMessage2
def sayBye():
    print('Bye!')

sayBye()
```

Decorators for Functions with Arguments

We had a simple decorator in the previous part. What if our functions have parameters and inputs?

```
import functools
```

```
def myDec(f):
    @functools.wraps(f) # also we will talk about decorators with parameters later in this section
    def wrapper(*args, **kwargs):
        print(f'Calling function: {f.__name__}...')
        print(f'{args = }')
        print(f'{kwargs = }')
        res = f(*args, **kwargs)
        return res

    return wrapper
```

```
@myDec
def greet(name=None):
    """
    a simple greeting function.
    """
    if name:
        return f"Hello {name}!"
    else:
        return f"Hello!"

print(greet('sora'))
print(greet())

# in this example, if you dont use @functools.wraps(f) for function, there will be no name and doc
print(greet.__name__)
print(greet.__doc__)
```

We can fix the issue of losing `__name__` and `__doc__` by using `functools.wraps` (we will talk about the module later but for now it is fixed).

Multiple Decorators

It is not always about adding something before or after a function. We can also modify the output itself using decorators. For example, we can change “Hello” to “Hi” or convert the result to uppercase.

```
import functools

def convert_to_hi(f):
    @functools.wraps(f)
    def wrapper(*args, **kwargs):
        result = f(*args, **kwargs)
        result = result.replace('Hello', 'Hi')
        return result
    return wrapper

def upper_case(f):
    @functools.wraps(f)
    def wrapper(*args, **kwargs):
        result = f(*args, **kwargs)
        result = result.upper()
        return result
    return wrapper
```

We can stack multiple decorators. The decorator closest to the function definition is applied first (they are executed from bottom to top).

```
@upper_case
@convert_to_hi
def greet(name=None):
    """
    a simple greeting function.
    """
    if name:
        return f"Hello {name}!"
    else:
        return f"Hello!"

print(greet('sora'))
print(greet())
```

This pattern can also be useful when working with servers and HTML rendering.

```
def strong(fun):
    @functools.wraps(fun)
    def wrapper(*args, **kwargs):
        res = fun(*args, **kwargs)
        return f"<strong>{res}</strong>"
    return wrapper

def italic(fun):
    @functools.wraps(fun)
```

```

def wrapper(*args, **kwargs):
    res = fun(*args, **kwargs)
    return f"<i>{res}</i>"
return wrapper

@italic
@strong
def greet2(name=None):
    """
    simple function
    """
    if name:
        return f"Hello {name}!"
    else:
        return f"Hello!"

print(greet2("iggy"))

```

Decorators with Parameters

In previous examples, we defined similar decorators like `italic` and `strong`. We can make the code cleaner by creating one decorator factory that accepts parameters.

```

import functools

def add_tag(tag): # decorator constructor / factory
    def decorator(fun): # actual decorator
        @functools.wraps(fun)
        def wrapper(*args, **kwargs): # wrapper
            res = fun(*args, **kwargs)
            return f"<{tag}>{res}</{tag}>"
        return wrapper
    return decorator

@add_tag('i')
@add_tag('b')
def greet(name=None):
    """
    simple function
    """
    if name:
        return f"Hello {name}!"
    else:
        return f"Hello!"

print(greet("iggy"))

```

This works because: - `add_tag('i')` returns a decorator. - That decorator is then applied to `greet`. - The final result is the wrapped function with the tags applied from bottom to top.

Using the **decorator** Module

In the previous file there was a lot of repeated code. The **decorator** module helps prevent this repetition.

```
from decorator import decorator
```

```
@decorator # instead of inner function and returning it, just keep the main part (wrapper) w
def addTag(fun, tag='tag', *args, **kwargs): # instead of giving function to parent, bring it
    res = fun(*args, **kwargs)
    return f"<{tag}>{res}</{tag}>"
```

```
@addTag(tag='strong')
@addTag(tag='i')
@addTag(tag='span')
def greet(name=None):
    """
    simple function
    """
    if name:
        return f"Hello {name}!"
    else:
        return f"Hello!"
```

```
print(greet("iggy"))
print(greet.__name__)
print(greet.__doc__)
```

Functools

Caching Function Outputs with **functools.cache**

The **functools** module is part of Python's standard library and provides useful features for working with functions.

Imagine we have a recursive function that reuses results from previous executions, such as factorial or Fibonacci.

Note: A recursive function calls itself. In this example, if *n* is not 0 or 1, the function calls itself until it reaches the base case (0 or 1) and then uses those values to calculate the next ones.

```
from functools import cache
```

```
@cache
def fibonacci(n):
    if n == 0:
        return 0
    if n == 1:
        return 1
    else:
```

```

        return fibonacci(n-1) + fibonacci(n-2)

for i in range(50): # if you change the number to a higher value like 50 without caching, it
    print(f'F[{i}] = {fibonacci(i)}')
```

Without caching, recursive functions like this recalculate the same values many times, which makes them very slow for larger inputs. The `@cache` decorator solves this by storing (memoizing) previous results.

LRU Cache with `functools.lru_cache`

There is another version of caching called **LRU Cache** (Least Recently Used). It keeps only the most recently used results and removes older ones when the cache is full.

`lru_cache` requires a `maxsize` parameter to determine how many results to keep.

```

from functools import lru_cache

# if we give it no value for maxsize, it is considered as None and works exactly like cache
@lru_cache(maxsize=10) # in our case we need two last values, so it will work great with maxs
def fibonacci(n):
    if n == 0:
        return 0
    if n == 1:
        return 1
    else:
        return fibonacci(n-1) + fibonacci(n-2)

for i in range(50):
    print(f'F[{i}] = {fibonacci(i)}')
```

Partial Functions with `functools.partial`

We can define partial functions using `functools`. A partial function is a new function created from an existing one where some parameters are pre-fixed.

```

print(pow(5, 3))
print(pow(4, 3))
print(pow(3, 3))
print(pow(2, 3))
```

For example, if we always want to raise a number to the power of 3 (cube), we can create a specialized function:

```

def cube(x):
    return pow(x, 3) # the exponent 3 is fixed

print(cube(5))
```

We can achieve the same result more cleanly using `functools.partial`:

```
from functools import partial
```

```
cube2 = partial(pow, exp=3) # we should use the actual parameter name from the target function  
print(cube2(4))
```

Another Example

```
A = [2, -3, 5, -7, 3, 2, -2]
```

```
print(max(A))  
print(max(A, key=lambda x: abs(x)))
```

Fix the key parameter

```
maxabs = partial(max, key=lambda x: abs(x))  
print(maxabs(A))
```

If we frequently use a function with the same fixed parameters, it is a good sign that we should use `partial`.

We can also pass the fixed parameters using a dictionary:

```
params = {'key': lambda x: abs(x)}  
maxabs2 = partial(max, **params) # unpack the dictionary  
print(maxabs2(A))
```

Reducing Iterables with `functools.reduce`

The `reduce()` function takes an iterable, such as a list or tuple, and repeatedly applies an operation to its elements until only a single value remains.

The process works as follows:

1. The element at index 0 is combined with the element at index 1.
2. The result becomes the new accumulated value.
3. That accumulated value is then combined with the next element.
4. The process continues until all elements have been processed.

Example

```
A = [1,2,3,4,5]
```

```
from functools import reduce
```

reduce takes a function that should always have two parameters.

```
print(reduce(lambda x, y: x+y, A))  
print(reduce(lambda x, y: x*y, A))  
print(reduce(lambda x, y: max(x, y), A))
```

The examples above demonstrate how `reduce()` can be used to:

- Calculate the sum of all elements.

- Calculate the product of all elements.
- Find the maximum value in the iterable.

Combining `reduce()` with `partial()`

The following example combines multiplication with concepts learned previously, using `partial()` to create a reusable product function.

```
from functools import partial

prod = partial(reduce, lambda x, y: x*y)
print(prod(A))
print(prod([1,3,2,5,4,6]))
```

Here, `partial()` preconfigures `reduce()` with a multiplication function, allowing `prod()` to be called with only the iterable argument.

Enum Classes

Enum Classes Definition

Sometimes we need a new data type that does not provide special behavior and is only used to store predefined values and constants.

Instead of using raw numeric values directly in our code, we can assign meaningful names to those values. This makes code easier to read and maintain.

Common examples include:

- Days of the week
- Keyboard and mouse events
- Configuration parameters
- Status codes
- Fixed sets of options

Python provides the `Enum` class for this purpose. To create an enumeration, inherit from `enum.Enum`.

Defining an Enumeration

```
from enum import Enum

class WeekDay(Enum):
    Saturday = 1
    Sunday = 2
    Monday = 3
    Tuesday = 4
    Wednesday = 5
    Thursday = 6
    Friday = 7
```



```

weekday = WeekDay.Friday
print(f'{weekday = }')
print(type(weekday))
print(dir(weekday))
print(weekday.name) # Friday
print(weekday.value) # 7

```

Accessing Enum Members

Each enum member contains two important attributes:

- `name` — the symbolic name of the member.
- `value` — the value assigned to the member.

The `type()` function shows that `weekday` is an instance of the `WeekDay` enumeration, and `dir()` can be used to inspect the available attributes and methods.

Adding Operators to an Enum Class

Now let's add some operators, for example `__add__()` itself.

```

from enum import Enum

```

```

class WeekDay(Enum):
    Saturday = 1
    Sunday = 2
    Monday = 3
    Tuesday = 4
    Wednesday = 5
    Thursday = 6
    Friday = 7

```

Add the `__add__()` operator:

```

def __add__(self, w):
    a = self.value

    if isinstance(w, WeekDay):
        b = w.value
    else:
        b = int(w)

    c = a + b
    c %= 7

    if c == 0:
        c = 7

    return WeekDay(c) # a new member of WeekDay() is returned.

```

- `a` stores the value of the current instance.

- If `w` is also a `WeekDay` member, such as `WeekDay.Thursday`, its value is assigned to `b`.
- `c` will always be a value between 0 and 6.
- Since `7 % 7` returns 0, we convert 0 back to 7.
- A new `WeekDay` member is returned.

```
weekday = WeekDay.Friday

print(WeekDay.Monday.value)
print(f'{weekday = }')

print(weekday + WeekDay.Monday) # 7 + 3 = 10 --> 10 % 7 = 3 --> WeekDay(3) --> Monday
print(weekday + 1)
print(weekday + 145)
```

Enums are different from normal classes in Python. This is not just a property assignment:

```
weekday = WeekDay.Friday
```

`weekday` is an instance of the `WeekDay` class.

Every enum member, such as `WeekDay.Monday`, is also an instance of the enum class itself.

```
for i in range(10):
    w = weekday + i
    print(f"Day #{i + 1}: {w.name}")
```

Using `auto()` in Enum Classes

The values of enum members can be generated automatically instead of assigning them one by one. For this, we can use the `auto()` function from the `enum` module.

```
from enum import Enum, auto

class WeekDay(Enum):
    Monday = auto()      # starts from 1
    Tuesday = auto()     # next number after 1 (2)
    Wednesday = auto()  # ...
    Thursday = auto()
    Friday = auto()
    Saturday = auto()
    Sunday = auto()

weekday = WeekDay.Friday
print(WeekDay.Monday.value)
print(f'{weekday = }')
```

Unique Enum Values

The following example contains a logical error (without the unique decorator):

```
from enum import Enum, unique
```

```
@unique
```

```
class WeekDay(Enum):
```

```
    Monday = 1
```

```
    Tuesday = 2
```

```
    Wednesday = 3
```

```
    Thursday = 3    # imagine there is something wrong here. I copied and pasted the code and
```

```
    Friday = 5
```

```
    Saturday = 6
```

```
    Sunday = 7
```

```
weekday = WeekDay.Thursday
```

```
print(f'{weekday = }')
```

This type of error can be difficult to find.

In the `enum` module, we have a decorator named `unique`. It ensures that there are no duplicate values in the enum.

If duplicate values are found, Python raises an error similar to:

```
ValueError: duplicate values found in <enum 'WeekDay': Thursday -> Wednesday
```

Object Oriented Programming

Instance Methods

In classes, different behaviors can be added by defining methods.

```
class Point():
```

```
    def __init__(self, x=0, y=0):
```

```
        self.x = x
```

```
        self.y = y
```

```
    def radius(self):
```

```
        r = (self.x ** 2 + self.y ** 2) ** 0.5
```

```
        return r
```

```
    def __str__(self):
```

```
        return f"Point x= {self.x} and y= {self.y}"
```

```
    # reuse __str__ instead of defining __repr__ separately:
```

```
    __repr__ = __str__
```

```
p = Point(3, 4)
```

```
print(p)
```

```
print(f'{p = }')
```

```
print(f'r = {p.radius()}')
```

`radius` and `__str__` are methods defined at the object layer — they belong to `p`, which is an instance of `Point`. Methods like these are called **instance methods**.

```

print(p.radius())
print(Point.radius(p))    # an instance must be passed explicitly here to avoid errors
print(type(p))
print(type(Point))
print(type(type))

```

Methods can also be defined at the class layer and above — outside the regular instance-level method definitions.

Class Methods and Static Methods

The methods `__init__`, `radius`, and `__str__` from the previous file are all instance methods. Methods can also be defined at the class layer using the `@classmethod` decorator.

```

class Point():
    def __init__(self, x=0, y=0):
        self.x = x
        self.y = y
    def radius(self):
        # r = (self.x ** 2 + self.y ** 2) ** 0.5
        r = self.distance(self.x, self.y, 0, 0)
        return r
    def __str__(self):
        return f"Point x= {self.x} and y= {self.y}"
    __repr__ = __str__

```

Class Methods

```

@classmethod
def origin(cls):    # instance methods receive 'self' (the instance); class methods receive 'cls'
    return Point(0, 0)

```

Static Methods

There is also another concept: the **static method**. It is similar to a class method, but takes no automatic first parameter at all — neither `self` nor `cls`.

```

@staticmethod
def distance(x1, y1, x2=0, y2=0):
    dx = x2 - x1
    dy = y2 - y1
    return (dx**2 + dy**2)**(1/2)

```

Static methods are most useful inside classes when the logic does not depend on any instance or class data — essentially like a regular function defined outside the class, but kept accessible everywhere inside the class and reusable multiple times.

Usage

```
print(Point.origin())
```

Recall the concept of scope: instances have access to the parent class's properties and methods, so a class method can also be called through an instance:

```
p = Point(3, 4)
print(p.origin())
print(p)
print(f"{p = }")
print(f"r = {p.radius()}")
print(Point.distance(0, 0, 3, 4))
```

Abstract Classes

```
from abc import ABC, abstractmethod
```

A safer alternative is to import `ABCMeta` instead of `ABC`, and define the class as `class Base(metaclass=ABCMeta)`. A metaclass is not simple inheritance — it operates at a higher level.

```
class Base(ABC):    # an abstract class should inherit from ABC
    @abstractmethod
    def the_method(self):    # this method is abstract, so it must be decorated with @abstractmethod
        pass

class MyClass(Base):
    # if the_method were not implemented here, instantiating MyClass would raise:
    # TypeError: Can't instantiate abstract class MyClass without an implementation for abstractmethod the_method
    def the_method(self):
        print('the method is defined in concrete class')

obj = MyClass()
```

`Base` defines a method that must be filled in later — like a required field on a form. Every class that inherits from `Base` is required to implement `the_method` itself; once it does, the child class is free to be used normally.

`Base` is an **abstract class**, and `MyClass` is a **concrete class**. This requirement is enforced using the `abc` module.

```
obj.the_method()
```

Properties with the @property Decorator

```
class Point():
    def __init__(self, x=0, y=0):
```

```

        self._x = x
        self._y = y
    def __repr__(self):
        return f"Point (x={self._x}, y={self._y})"

```

`x` and `y` are properties of this class's objects, and by default they can be changed to anything — there is no way to enforce, for example, that they must be positive numbers or fall within a specific range. Other languages like C++ have built-in features to control property access.

In Python, this is done by first making the properties private (by convention, prefixing with `_`), then using the `@property` decorator.

```

@property
def x(self):
    return self._x

```

Whenever `x` is accessed, `_x` is returned. By default, this acts as a read-only **getter**.

```

@property
def y(self):
    return self._y

```

A property can also be given write permission using a **setter**:

```

@y.setter
def y(self, value):
    if type(value) not in [int, float]:
        raise ValueError("Value of y must be numerical.")
    self._y = value

```

There is also a corresponding `@<prop_name>.deleter` decorator, useful when a resource needs to be freed:

```

@y.deleter
def y(self):
    del self._y

```

Properties can also perform more complex computations:

```

@property
def r(self):
    return (self._x**2 + self._y**2)**(1/2)

```

Usage

```

p = Point(3, 4)
print(f"{p = }")
print(p._x)
print(p.x)

# p.x = 10      # AttributeError: property 'x' of 'Point' object has no setter

```

```
p._x = 10      # raises no error - _x has no property protection
print(p.x)    # represents p._x
print('r', p.r)

p.y = 12
# p.y = "sora"    # ValueError: Value of y must be numerical.
print(p.y)
print(p._y)

del p.y
# print(p._y)    # AttributeError: 'Point' object has no attribute '_y'. Did you mean: '_x'?
# print(p.y)     # AttributeError: 'Point' object has no attribute '_y'. Did you mean: '_x'?
```

Python Tools

At this stage, you are already familiar with Python syntax and core programming concepts.

Now the focus shifts toward building practical tools using Python modules.

For each module used in this section, you are expected to do a bit of independent research, understand its types, functions, and behavior, and then apply it efficiently.

Since the fundamentals are already covered, the emphasis here is on implementation, experimentation, and building real-world tools.

Real World Projects: Freecollapse — Payload Generator

You have probably heard of `recollapse` which is famous payload generator, But it works on URLs and not EVERY text, Also users are not allowed to fuzz characters at ANY index. That why i wrote this script and named it `Freecollapse`

Freecollapse inserts bytes into a string at specific positions to generate mutated payloads. It is used to fuzz input validation and test filter bypass techniques — for example, finding XSS bypasses by injecting special bytes at different positions inside a known payload.

Save it as `freecollapse.py` and run it with `python3 freecollapse.py [options]`.

Imports

- `argparse` — a standard library used to handle command-line arguments
- `sys` — writes output directly to stdout as raw bytes

Functions

- `parse_hex_list(s)` — parses a byte range or list like `0x00-0x3C` or `0x5B,0x5C` into a list of integers
- `parse_index_list(s)` — parses an index range or list like `1-3,7` into a sorted list of positions
- `generate_payloads(text, bytes_list, per_index, indices, encode)` — inserts each byte at each position and yields a mutated string per insertion; optionally percent-encodes the inserted byte as `%HH`
- `main()` — parses arguments, reads input, and writes payloads to stdout and optionally to a file

Script

```
import argparse
import sys
```

```
def parse_hex_list(s): # 0x00-0x3C,0x5A --> [0, 1, 2, ..., 60, 90]
    parts = [p.strip() for p in s.split(',') if p.strip()] # "0x00-0x3C,0x5A" --> ["0x00-0x3C", "0x5A"]
    vals = []
    seen = set()

    for p in parts:
        if '-' in p: # if 0x10-0x20
            a, b = p.split('-') # 0x10-0x20 --> 0x10, 0x20
            a_v, b_v = int(a, base=0), int(b, base=0) # note that you can pass base also as a string
            # validate that both ends of the range are valid bytes (0x00-0xFF)
```

```

        if not (0 <= a_v <= 0xFF and 0 <= b_v <= 0xFF and a_v <= b_v):
            raise argparse.ArgumentTypeError(f"Invalid range: '{p}'")
        for x in range(a_v, b_v + 1):
            if x not in seen: # duplicate removal
                seen.add(x)
                vals.append(x)
    else:
        v = int(p, 0)
        if not 0 <= v <= 0xFF:
            raise argparse.ArgumentTypeError(f"Invalid byte: '{p}'")
        if v not in seen:
            seen.add(v)
            vals.append(v)

    if not vals:
        raise argparse.ArgumentTypeError("Empty range")
    return vals

def parse_index_list(s):
    parts = [p.strip() for p in s.split(',') if p.strip()] # 1,3,5-7 --> [1, 3, 5, 6, 7]
    idxs = set()

    for p in parts:
        if '-' in p: # if 2-4 for example
            a, b = p.split('-') # 2-4 --> 2, 4
            a_i, b_i = int(a, 0), int(b, 0)
            if a_i < 0 or b_i < 0 or a_i > b_i:
                raise argparse.ArgumentTypeError(f"Invalid range: '{p}'")
            for x in range(a_i, b_i + 1):
                idxs.add(x)
        else:
            v = int(p, 0)
            if v < 0:
                raise argparse.ArgumentTypeError(f"Invalid index: '{p}'")
            idxs.add(v)

    return sorted(idxs)

def generate_payloads(text, bytes_list, per_index=None, indices=None, encode='none'):
    L = len(text)
    # use all positions if no indices specified (if user entered index, use it. else use all p
    positions = indices if indices else range(0, L + 1)
    # only keep indices that are within bounds (filter invalid indexes)
    positions = [i for i in positions if 0 <= i <= L]

    for idx in positions: # loop on indexes
        count = 0

```

```

    for b in bytes_list: # loop on bytes lists (like 88, 89, 90)
        # insert as %HH if url encoding is on, otherwise insert raw character
        insert = ('%%%02X' % b) if encode == 'url' else chr(b)
        # in this part we build the 'thing' we want to insert to specified index
        # about this part: ('%%%02X' % b) -> its just a string formatting trick for decimal
        # %X means convert to hex. (remember string creation ways and operators?)
        # also 0 means if its less than one digit, add a 0 prefix and 2 means at least two
        # for example output of %02X for a decimal like 65 would be '41'
        # we also need the '%' itself for url encoding and we have to use a second % to escape
        # so the final format is: '%%%02X'
        # '%%%02X' % 65 --> %41
        # we can also write this with fstrings: f"%{b:02X}" (b is our decimal in this example)

    yield text[:idx] + insert + text[idx:]
    count += 1
    # stop at this position if per_index limit is reached
    if per_index is not None and count >= per_index:
        break

def main():
    p = argparse.ArgumentParser(description='Freecollapse: byte-insertion payload generator')
    group = p.add_mutually_exclusive_group(required=True) # (only one of members per this group)
    group.add_argument('-t', '--text', help='Input text inline') # for example -t and -f cannot be used together
    group.add_argument('-f', '--file', help='Input file (one string per line, UTF-8)')
    p.add_argument('-r', '--range', required=True, type=parse_hex_list,
        help='Byte range or list. Example: 0x00-0x3C or 0x5B,0x5C') # argparse gives parse_hex_list
    p.add_argument('-i', '--indices', type=parse_index_list, default=None,
        help='Insertion positions. Example: 1,3,5 or 1-3. Use 0 for before first character')
    p.add_argument('-p', '--per-index', type=int, default=None,
        help='Max payloads per position') # limits payload insertion per index
    p.add_argument('-o', '--output', help='Output file (also prints to terminal)')
    p.add_argument('--encode', choices=['none', 'url'], default='none',
        help='Percent-encode inserted bytes as %HH') # encode types: url encoded or none

    args = p.parse_args()
    # after this line (based on the names we just set) we will have:
    # args.text
    # args.range
    # args.indices
    # args.per_index
    # args.output
    # args.encode

    out_fp = open(args.output, "wb") if args.output else None
    # why write binary? this is going too far for a beginner but python terminal has two layers
    # print --> string
    # buffer --> bytes
    # we could handle all this with print but since our payloads may contain non-printable bytes

```

```

def write(payload): # payload is generated string
    # encode payload as UTF-8 bytes and write to stdout
    data = payload.encode('utf-8') + b'\n' # "A" --> b"A" --> b"A"\n
    sys.stdout.buffer.write(data)
    sys.stdout.buffer.flush() # send everything in buffer to stdout right now.
    if out_fp:
        out_fp.write(data) # write in file
        out_fp.flush() # save immediately

if args.file:
    # process the input file line by line
    with open(args.file, 'r', encoding='utf-8', errors='replace') as fh:
        for line in fh:
            line = line.rstrip('\n')
            for pl in generate_payloads(line, args.range, args.per_index, args.indices, args.
                write(pl)
else:
    for pl in generate_payloads(args.text, args.range, args.per_index, args.indices, args.
        write(pl)

if out_fp:
    out_fp.close()
    print(f'\nWrote payloads to: "{args.output}"', file=sys.stderr)

if __name__ == '__main__':
    main()

```

How to Use

Insert bytes 0x5B-0x60 at position 25, URL-encoded, and save to a file:

```
python3 freecollapse.py -t "<img src='x' onerror=alert(origin)>" -r 0x5B-0x60 -i 25 --encode url --output file
```

Insert non-printable bytes across all positions and view results page by page:

```
python3 freecollapse.py -t "hello" -r 0x00-0x1F --encode url | less
```

Limit to 5 payloads per position:

```
python3 freecollapse.py -t "test" -r 0x20-0x7E -p 5
```

Process a file of strings and save all payloads:

```
python3 freecollapse.py -f targets.txt -r 0x00-0xFF --encode url -o out.txt
```

Real World Projects: hiKrawler — Passive URL Collector

hiKrawler collects URLs for a target domain passively using `waybackurls` and `gau`, filters out static assets, deduplicates results, and saves them to a `.passive` file.

Save it as `hiKrawler.py` and run it with `echo domain.tld | python3 hiKrawler.py`.

Imports

- `sys` — reads input from `stdin` and handles `exit`
 - `os` — checks file existence and removes temp files
 - `tempfile` — creates a temporary file to store intermediate results
 - `subprocess` — runs external tools like `waybackurls` and `gau`
 - `urllib.parse` — parses URLs to extract hostname and path
-

Functions

- `run_command_in_bash(command)` — runs a shell command and returns its `stdout`, or `False` on error
 - `get_hostname(url)` — extracts the hostname from a URL or returns the input as-is if it is already a domain
 - `good_url(url)` — returns `False` if the URL points to a static asset (image, font, archive, etc.), otherwise `True`
 - `finalize(file_path, domain)` — deduplicates and filters URLs from a temp file, then writes clean results to `domain.passive`
 - `is_file(file_path)` — returns `True` if the given path is an existing file
 - `generate_temp_file()` — creates and returns the path of an empty temporary file
 - `run_nice_passive(domain)` — orchestrates passive collection: runs all commands, finalizes results, and prints a summary
 - `get_input()` — reads input from `stdin` if piped, otherwise from the first CLI argument
-

Script

```
#!/usr/bin/env python3
import sys
import os
import tempfile
import subprocess
from urllib.parse import urlparse, urlsplit
```

```

class colors:
    GRAY = "\033[90m"
    RESET = "\033[0m"

def run_command_in_bash(command): # running system commands
    try:
        result = subprocess.run(["bash", "-c", command], capture_output=True, text=True, check=True)
        if result.returncode != 0:
            print("Error occurred:", result.stderr)
            return False
        return result.stdout.strip() # return results, stripped.
    except subprocess.CalledProcessError as exc:
        print("Status : FAIL", exc.returncode, getattr(exc, "stdout", ""), getattr(exc, "stderr", ""))
        return False
    except Exception as e:
        print("Unexpected error:", str(e))
        return False

def get_hostname(url): # extracting hostname
    if url.startswith('http'):
        return urlsplit(url).netloc # urlsplit breaks url to 5 parts: scheme://netloc/path?querystring#fragment
    return url.strip() # if not url, return itself. for example: example.com --> example.com

def good_url(url):
    extensions = [
        '.json', '.js', '.fnt', '.ogg', '.css', '.jpg', '.jpeg', '.png', '.svg',
        '.img', '.gif', '.exe', '.mp4', '.flv', '.pdf', '.doc', '.ogv', '.webm',
        '.wmv', '.webp', '.mov', '.mp3', '.m4a', '.ppt', '.pptx', '.scss',
        '.tif', '.tiff', '.ttf', '.otf', '.woff', '.woff2', '.bmp', '.ico', '.eot',
        '.htc', '.swf', '.rtf', '.image', '.rf', '.txt', '.xml', '.zip'
    ]
    try:
        path = urlparse(url).path or ""
        return not any(path.endswith(ext) for ext in extensions)
    except Exception as e:
        print(f"Error: {str(e)}")
        return None

def finalize(file_path, domain):
    unique_lines = set()
    with open(file_path, 'r') as file:
        for line in file:
            line = line.strip()
            if line and good_url(line):

```

```

        unique_lines.add(line)

    if not unique_lines:
        return set()

    with open(f"{domain}.passive", 'w') as file:
        for element in sorted(unique_lines):
            file.write(element + '\n')

    return unique_lines

def is_file(file_path):
    return os.path.isfile(file_path)

def generate_temp_file():
    tmp = tempfile.NamedTemporaryFile(mode='w', delete=False) # create a temporary file which
    tmp.close()
    return tmp.name

def run_nice_passive(domain):
    temp_file = generate_temp_file()
    print(f"{colors.GRAY}gathering URLs passively for {domain}{colors.RESET}")

    commands = [
        f"echo https://{domain}/ | tee {temp_file}",
        f"echo {domain} | waybackurls | sort -u | tee -a {temp_file}",
        f"gau {domain} --threads 2 --subs | sort -u | tee -a {temp_file}"
    ]

    for command in commands:
        print(f"{colors.GRAY}Executing: {command}{colors.RESET}")
        run_command_in_bash(command)

    print(f"{colors.GRAY}merging results for {domain}{colors.RESET}")
    res = finalize(temp_file, domain)

    try:
        os.remove(temp_file)
    except OSError:
        pass

    print(f"{colors.GRAY}done for {domain}, results: {len(res) if res else 0}{colors.RESET}")

def get_input():
    if not sys.stdin.isatty(): # if data is being received from pipeline...

```

```

        return sys.stdin.readline().strip()
    elif len(sys.argv) > 1:
        return sys.argv[1]
    return None

if __name__ == "__main__":
    input_data = get_input()
    if input_data is None:
        print("Usage: echo domain.tld | hiKrawler.py")
        print("Usage: cat domains.txt | hiKrawler.py")
        print("Usage: hiKrawler.py https://domain.tld")
        print("Usage: hiKrawler.py domains.txt")
        sys.exit()

    if is_file(input_data):
        with open(input_data, 'r') as file:
            for line in file:
                run_nice_passive(get_hostname(line))
    else:
        run_nice_passive(get_hostname(input_data))

```

How to Use

Collect URLs for a single domain:

```
echo target.com | python3 hiKrawler.py
```

Process a list of domains from a file:

```
python3 hiKrawler.py domains.txt
```

Results are saved to `target.com.passive` in the current directory.

Real World Projects: fav — Favicon Hash Generator

Shodan indexes favicon hashes under `http.favicon.hash`. This script fetches a favicon, computes its MurmurHash3, and prints a ready-to-use Shodan search link — useful for finding other hosts running the same application.

Save it as `fav.py` and run it with `python3 fav.py <favicon_url>`.

Imports

- `mmh3` — computes MurmurHash3, the same algorithm Shodan uses for favicon indexing
- `requests` — fetches the favicon over HTTP/HTTPS
- `codecs` — base64-encodes the favicon bytes before hashing

- `sys` — reads the URL from CLI arguments
-

Functions

- `main()` — fetches the favicon, base64-encodes it, computes the MurmurHash3, and prints the hash and Shodan search link
-

Script

```
#!/usr/bin/env python3
import mmh3
import requests
import codecs
import sys
from requests.packages.urllib3.exceptions import InsecureRequestWarning

requests.packages.urllib3.disable_warnings(InsecureRequestWarning) # trun off ssl warnings

if len(sys.argv) < 2:
    print("[!] Error!")
    print(f"[i] Use: python3 {sys.argv[0]} http://example.com/favicon.ico")
    print("[i] Get all hosts with the same favicon!")
    sys.exit()

def main():
    response = requests.get(sys.argv[1], verify=False) # verify=False --> dont check SSL
    favicon = codecs.encode(response.content, "base64") # response.contents : raw binary byte
    hash_favicon = mmh3.hash(favicon)
    print("[i] http.favicon.hash:" + str(hash_favicon))
    print("[*] View Results:\n> https://www.shodan.io/search?query=http.favicon.hash%3A" + str(hash_favicon))

if __name__ == "__main__":
    main()
```

How to Use

Fetch a favicon and get its Shodan search link:

```
python3 fav.py http://example.com/favicon.ico
```

The script prints the hash and a direct Shodan URL to find other hosts with the same favicon.

JavaScript Fundamentals

Most modern applications today are delivered over the web.

Almost every company runs a website or web-based service, often backed by complex server infrastructures.

In web security and application analysis, it is essential to be able to read and understand JavaScript code, identify logic flows, and recognize potentially vulnerable parts of an application.

JavaScript and the DOM — Script Placement

The browser, like most scripting language interpreters, reads code from top to bottom. Because one of JavaScript's most important jobs is manipulating the HTML, the HTML must be loaded before JavaScript can interact with it. This is why the `<script>` tag is placed at the end of `<body>` — after all other HTML elements have been loaded.

JavaScript Basics — Variables

It is good practice to leave comments in code to let others know what is going on.

```
<script>  
    // if script is here it will be loaded sooner  
</script>
```

There are six main concepts in JavaScript, and variables are one of them.

In JavaScript, every statement ends with a semicolon (;) — this is required syntax.

Every user input on the web, in Python scripts, or anywhere else is passed to the program as a string. To hold a value like a name, an address, or alphanumeric text, we use strings, written between `' '` or `" "`.

```
var fname = 'sora';  
var lname = 'hikari';  
// var stands for variable – we use var to declare our variables  
// after declaring a variable, we can access it and manipulate its value  
lname = 'voidSeraphim';  
  
// variables can hold different data types: strings, numbers, and more, covered in later lessons  
var age = 23;  
  
// in python we print to console using print(); in javascript we use console.log()  
console.log(fname + ' ' + lname + ' is ' + age + ' years old.')
```

JavaScript Basics — More About Variables

Unlike Python, in JavaScript you can declare a variable without giving it a value immediately. The value can be calculated later, taken from user input, or simply assigned afterward.

```
var name;    // declaration  
name = 'sora';
```

Like Python, it is good practice to write variable names in `camelCase`. The reader is expected to already be familiar with the different naming case conventions.

var, let, and const

We have talked about `var`. There are also two other variable declaration keywords introduced in ES6: `let` and `const`. In Python, there is no equivalent — a name is simply assigned a value and that's it.

```
// var name = 'sora';  
// var age = 23;
```

```
let name = 'sora';  
const age = 23;
```

```
console.log(name)  
console.log(age)
```

Why Not Just Use `var`?

```
var myFav = 'dora';  
console.log(myFav)  
var myFav = 'iggy';  
console.log(myFav)
```

With `var`, a variable can be declared multiple times with no error and no control. In a large script, declaring two variables with the same name can become a real nightmare to debug.

Python and JavaScript are both dynamically typed — the value can change from a string to a number freely. However, JavaScript prevents redeclaring the same variable with `let` or `const`:

```
// let name = 'dora';    // Uncaught SyntaxError: Identifier 'name' has already been declared  
  
// but the value can still be changed at any time:  
name = 'dora';  
console.log(name);
```

`const`

What if a value should never change later in the code? Use `const` instead of `let` or `var`.

```
age = 24;  
console.log(age)  
// This line throws: Uncaught TypeError: Assignment to constant variable.  
// because age was declared with const, its value cannot be reassigned.
```

JavaScript Basics — Booleans

In JavaScript, as in every other language, there is a concept called **boolean**. Conditions are based on these values: `true` or `false`, yes or no, 1 or 0. A boolean can also represent the simple answer to a question. For example:

```
console.log(5 > 2)    // returns true  
console.log(2 > 5)    // returns false
```

Conditions such as `if/else` statements and `while` loops are based on booleans — if a condition is true, do something; while a condition is true, keep doing something.

Boolean values can also be set manually:

```
let condition = true;

if(condition == true){
    console.log('thats true');
}else{
    console.log('not true')
}
```

JavaScript can also convert values between types. For example:

```
let x = 10/'hello';    // NaN means "Not a Number"
console.log(x)
console.log(Boolean(x))    // in python we use bool() to convert to a bool, along with str(),
```

JavaScript Basics — Arrays

Array is another data type in JavaScript, with syntax similar to lists in Python. What if we want to store more than one single value in a variable? There are several data types for this concept, and array is one of them.

```
let students = ['sora', 'dora', 'iggy', 'ghedas', 23, true];
```

Elements are accessed by index, starting from 0:

```
console.log(students[0]);
console.log(students[1]);
console.log(students[10]);    // out of range returns undefined, which is fine – in python th
```

Values can be modified by accessing their index:

```
students[3] = 'kitkat';
console.log(students[3]);
```

In arrays, elements can have different data types — an array can hold strings, numbers, booleans, other arrays, or objects all at the same time:

```
console.log(students[4])
console.log(students[5])
```

Unlike Python, JavaScript does not support direct negative indexing like `students[-1]`. Instead, use the `.at()` method:

```
console.log(students.at(-2))
```

JavaScript Basics — Objects

As we saw in Python, almost everything is an object — a string is an instance of the string class. JavaScript is also based on objects, but the structure is a bit different. JavaScript also has a data type called `object`, and sometimes we need to create our own custom type.

The block below is called an **object**, or more generally, we can relate them to **JSON** (JavaScript Object Notation) in future:

```
let student1 = {
  'name' : 'sora',
  'phone' : '123123',
  'email' : 'sora@mail.com',
  'isAdmin' : true,
  'address' : {
    'country' : 'iran',
    'city' : 'tehran'
  }
};

let student2 = {
  'name' : 'dora',
  'phone' : '123456',
  'email' : 'dora@mail.com'
};

let student3 = {
  'name' : 'iggy',
  'phone' : '876123',
  'email' : 'iggy@mail.com'
};
```

This is a custom type, and it has several benefits we will explore further later on.

To access values inside a JSON-like JS object, use `.keyname` (in general, `object.property`). Nested properties are accessed the same way, chained together. In Python, when JSON data is loaded, it gets translated into a dictionary.

```
console.log(student1.name)
console.log(student2.phone)
console.log(student3.email)

if(student1.isAdmin == true){
  console.log(student1.name + ' is an admin')
  console.log(student1.address.city)
}
```

A brief note on JSON: it is a standard format used to transfer data between applications.

Browser Console Basics

This page demonstrates how the browser console works and how it can be used for debugging JavaScript code.

If a `<script>` tag is placed inside the `<head>` section, it will be loaded and executed before the rest of the page content, since the head is parsed first.

The following script runs in the page and produces different types of console output:

```
<script>
  console.error('HELLO THIS IS AN ERROR')
  console.info('and this is info')
  console.log('hello this is some output')
</script>
```

`console.log` is used to print anything we want to the console, similar to `print` in Python. The console is mainly used for debugging and for checking the output of code at intermediate steps.

To try this yourself, save the code into a file named `practice.html` and open it in a browser. Then open the developer tools (usually with F12 or right-click → Inspect) and switch to the Console tab to see the output.

JavaScript Functions

Functions are blocks of code that can be executed multiple times, or only when needed. They are callable code blocks, and each time they are called, the code inside is executed.

In JavaScript, a function definition consists of the `function` keyword, parentheses (which may or may not take parameters depending on the operation), and curly brackets containing the code to execute.

Defining and Calling a Function

```
let myName = 'sora';

function greetings(name) {
  console.log('hello ' + name);
  alert('hello ' + name);
};
```

Functions are called by adding `()` after their name. In the example above, the function takes one parameter, which is a variable:

```
greetings(myName)
```

This function can be triggered from the page using a button:

```
<button onclick="greetings(myName)">greetings</button>
```

Working with Form Input

The following function reads two values from input fields, converts them to numbers, and returns their sum:

```
function getSum(){
  let a = Number(document.getElementById('num1').value);
  let b = Number(document.getElementById('num2').value);
  console.log('given numbers are: ' + a + ' and ' + b + ' and sum is : ' + a + b);
  return a + b;
};
```

Note that the `console.log` line in this function is left as is intentionally. Since `a + b` is being concatenated into a string with the `+` operator there, the output will show string concatenation rather than numeric addition, illustrating that input values are strings by default and must be converted using `Number()` before being used in arithmetic.

The function is triggered by a button inside a form:

```
<form>
  <input type="text" name="num1" id="num1">
  <input type="text" name="num2" id="num2">
  <button onclick="alert(getSum())">getsum</button>
</form>
```

Template Literals

```
function fullName(fname, lname) {
  let name = `your name is ${fname} ${lname}`
  return name
}

console.log(fullName('sora', 'void'))
```

This is similar to f-strings in Python. In JavaScript, template literals use backticks and the `${}` syntax for embedding variables, for example: ``hello my name is ${name}``.

Returning Values from Functions

```
function getSum2(a, b) {
  return a + b;
}

console.log(getSum2(2, 3));
```

`console.log` and `alert` are useful for debugging, but when a function's output needs to be used in actual operations, the function must return a value so it can be used elsewhere in the code. For example, `getSum2(2, 3)` is both a function call and a value at the same time, and it can be used directly in code, such as in a condition like "if `getSum2(2, 3)` is greater than 4, do something, otherwise do something else."

If a function does not return anything, it is referred to as `void`.

To try this yourself, save the code into a file named `practice.html` and open it in a browser. Open the developer tools console to see the logged output, and interact with the buttons and input fields to test the functions.

More Ways to Define Functions

The standard function definition was covered in the previous section, but JavaScript provides additional ways to define functions. One of these is to start with a variable declaration instead of the `function` keyword.

Function Expressions

```
let Sum = function(a, b) {  
    return (a+b)  
}
```

```
console.log(Sum(1, 3))
```

Here, an anonymous function is assigned to the variable `Sum`. The function can then be called using `Sum(1, 3)`, just like a normally defined function.

To try this yourself, save the code into a file named `practice.html` and open it in a browser, then check the developer tools console for the output.

Arrow Functions

In addition to standard function definitions, JavaScript also has a syntax called arrow functions.

Basic Arrow Function Syntax

```
let SumOfTwo = (a, b) => {return a + b}
```

```
console.log(SumOfTwo(3, 4))
```

This is an arrow function. The general syntax is: `let functionName = (parameters) => { print or return something, do some operation }`.

Short Arrow Function Syntax

Arrow functions also have a shorter syntax, which is similar to lambda functions in Python:

```
let greetings = (name) => `hello dear ${name}`
```

```
console.log(greetings('sora'))
```

When curly brackets are not used, the `return` keyword is not needed, and the expression's result is returned automatically.

To try this yourself, save the code into a file named `practice.html` and open it in a browser, then check the developer tools console for the output.

Dynamic Strings with Template Literals

Template literals can be used to generate dynamic strings, similar to f-strings in Python.

```
const price = 9000;
```

```
console.log(`the product price is ${price} \nand this is a new line`)
```

The `\n` inside the template literal creates a new line in the console output.

Updating Page Content Dynamically

The following function reads a value from an input field and displays it on the page using a template literal:

```
function showInput() {  
    let inputval = document.getElementById('inputval').value  
    document.getElementById('priceID').innerText = `you have entered: ${inputval}`  
}
```

```
<h1 id="priceID"></h1>  
<input type="text" name="price" id="inputval">  
<button onclick="showInput()">click</button>
```

To try this yourself, save the code into a file named `practice.html` and open it in a browser. Type a value into the input field, click the button, and the entered value will be displayed inside the `<h1>` element.

Conditional Statements

Conditions are one of the most important concepts in every programming language. They make a program more responsive to events that occur during execution.

Almost every program has at least one `if` (conditional) statement, which is based on boolean values (true or false).

```
const myMoney = 3000;  
const coolLCD = 2000;  
const coolKeyboard = 5000;  
  
if(myMoney > coolKeyboard){  
    console.log('i can buy lcd or keyboard')  
}else if(coolKeyboard > myMoney && myMoney > coolLCD){  
    console.log('i can only buy lcd')  
}else{  
    console.log('i can buy nothing')  
}
```

In this example, `myMoney` (3000) is compared against `coolKeyboard` (5000) and `coolLCD` (2000). Since `myMoney` is greater than `coolLCD` but less than `coolKeyboard`, the second condition is true, and the output will be `i can only buy lcd`.

To try this yourself, save the code into a file named `practice.html` and open it in a browser, then check the developer tools console for the output.

Comparison Operators

```
const x = 5;  
  
if (x == 8 || x === 5) {  
    console.log('true');  
}
```

```
}else{
    console.log('false');
};
```

Since `x === 5` is true, the output will be true.

Comparison Operator Reference

The following operators are used for comparisons in JavaScript:

```
// ==    equal to
// ===   equal to (plus type comparison)
// !=    not equal to
// !==   not equal to (plus type)
// >     greater than
// <     lower than
// >=    greater than equal to
// <=    lower than equal to
```

To check multiple conditions at once, the following logical operators are used:

```
// &&    and
// ||    or
```

Ternary Operator

When there is less code to deal with, a shorthand `if` statement can be used:

```
condition ? do this if true : do this if false
```

This is called the ternary operator. Example:

```
x < 4 || x === 5 ? console.log('yes') : console.log('no');
```

Checking Existence in Conditions

```
let age = 23;

if(age){
    alert(1)
}else{
    alert(0)
}
```

This checks whether `age` exists and evaluates to a truthy value. If `age` is `0`, it is treated as `false`; otherwise it is treated as `true`. The same applies to strings: an empty string `''` is `false`, while any non-empty string is `true`. In general, if a variable exists and is not `0`, an empty string, an empty array, or similar empty/falsy values, it is treated as `true`.

To try this yourself, save the code into a file named `practice.html` and open it in a browser, then check the developer tools console for the output.

A Deeper Look into `let` and `var`

```
if(1){
  let x = 5;
  console.log(x)
}else{
  let x = 10;
}
```

This logs 5, since 1 is truthy and the first block executes.

Variables declared with `let` or `const` are limited to the block of code in which they are declared. A block of code is everything between `{` and `}` — `for` loops, `if` statements, functions, and so on all have their own blocks.

```
// console.log(x)
```

This line is commented out because `x` was declared with `let` inside the `if/else` block above, so it does not exist outside that block. Attempting to log it here would result in a `ReferenceError`, not `undefined`.

Comparison with Python

In Python, variables declared inside `for` or `if` blocks remain accessible in the enclosing scope, and they can also be reassigned or have new variables declared from there:

```
In [1]: x = 4
In [2]: if x < 5:
...:     y = 3
...:
In [3]: y
Out[3]: 3
```

In JavaScript, using `var` instead of `let` behaves similarly to Python, in the sense that the declaration becomes accessible outside the block (function-scoped or global, depending on where it is declared).

Global Variables and Blocks

```
let y = 6;

if(y > 4){
  y += 1;
}

console.log(y)
```

Here, `y` is declared outside the `if` block, so it is already accessible (global to the script), and the block can modify it without any issue. The output will be 7.

To try this yourself, save the code into a file named `practice.html` and open it in a browser, then check the developer tools console for the output.

Introduction to the Document Object

So far, we have been working with the console, but JavaScript is meant to make changes to the HTML page itself.

```
<h1>Javascript</h1>
<h2 class="head">loading...</h2>
<button onclick="chContent()">change content</button>
```

The document object, along with its methods and properties, allows us to work with the HTML document. For example, typing `document.images` in the browser console will list all images on the page.

Accessing Elements in the HTML Document

```
document.getElementsByClassName('')    // name attr
document.getElementsByTagName('')       // tag name like a, img, h1 and ...
document.getElementsByClassName('')    // class name
document.getElementById('')           // id attr
```

These methods allow access to elements by their name attribute, tag name (such as `a`, `img`, `h1`, etc.), class name, or id attribute, respectively.

```
document.querySelector('')
```

`querySelector` is more powerful and can handle all of the above accessing methods. For example, `'a'` returns all `a` tags, `'classname'` returns elements with that class, and `'#id'` returns the element with that id.

Changing Content Safely with a Function

```
const el = document.querySelector('.head');
el.textContent = 'hello im sora voidSeraphim';
```

These two lines change the text content of the `h2` element. However, if this code runs immediately and the script is placed before the `h2` tag in the page, the element will not exist yet when the script executes, and this will throw an error.

To avoid this problem, the code is placed inside a function and linked to a button instead:

```
function chContent() {
    const el = document.querySelector('.head');
    el.textContent = 'hello im sora voidSeraphim';
}
```

This way, the code only runs when the button is clicked, after the page has fully loaded.

We can manipulate the entire document: select elements, change their content, make new elements, change attributes, and much more.

If there is more than one element with the class `head`, or if elements are accessed by tag name, `querySelector` and similar methods return only the first matching element, and any change will only affect that first occurrence. To access all matching elements, they must be accessed as a list (array), for example using `querySelectorAll`.

Changing Element Styles

```
<h1 class="head"> This is my heading </h1>
<h2 class="sora"> sora voidSeraphim </h2>
<button onclick="chStyle()">chStyle</button>
```

```
function chStyle() {
  const el = document.querySelector('.head');
  el.style = 'background-color: red; color: blue;';

  const sora = document.querySelector('.sora');
  sora.style = `color: ${el.style.backgroundColor}`;

  sora.style.setProperty('background-color', 'yellow', 'important')
};
```

There are multiple ways to change an element's style with JavaScript.

```
// el.style.backgroundColor = 'blue';
```

This commented-out line shows an alternative approach: setting an individual style property directly using `el.style.backgroundColor`. The line below it, `el.style = 'background-color: red; color: blue;'`, sets multiple style properties at once using a CSS string, which is the approach used here.

After the `head` element's background color is set to red, the `sora` element's text color is set to that same background color using `el.style.backgroundColor` inside a template literal.

Another way to set a style property is using `setProperty`:

```
sora.style.setProperty('background-color', 'yellow', 'important')
```

The third argument, `'important'`, is optional and is used when a value needs to override existing styles (equivalent to `!important` in CSS).

Changing Classes Dynamically

```
<style>
  .myClass{
    background-color: red;
    color:whitesmoke;
  }
</style>

<h1 class="head">This is my heading</h1>
<h2 class="sora">sora voidSeraphim</h2>
<h4>this is some other text</h4>
<button onclick="chStyle()">chStyle</button>
```

When working with CSS frameworks like Tailwind or Bootstrap, it is often necessary to change class names dynamically rather than setting individual style properties.

```
const h1Elements = document.querySelectorAll('h1, h2, h3, h4, h5, h6');
h1Elements.forEach(heading => {
  heading.addEventListener('mouseover', () => {
    heading.classList.add('myClass');
  });

  heading.addEventListener('mouseout', () => {
    heading.classList.remove('myClass');
  });
});
```

Explanation

`querySelector` returns only the first matching element, but `querySelectorAll` takes a selector and returns a node list of all matching elements. A single selector can include multiple selectors separated by commas (,). A node list is similar to an array — individual elements can be accessed using `nodeList[0]` and iterated with `forEach` — but it is not a complete array.

`forEach` is a method available on node lists, arrays, and maps. It takes a function and executes that function on every member of the iterable. In the example above, `heading` is the function parameter, and adding event listeners is the operation performed on each element.

`forEach` is similar to `map` in Python, but the difference is that `map` returns a value, while `forEach` only performs an operation without returning anything.

When adding a class with `classList.add`, the class name is written without a leading dot (.), unlike in CSS selectors.

Adding Event Listeners

```
<style>
  .myClass{
    background-color: red;
    color:whitesmoke;
  }
</style>

<h1 class="head">This is my heading</h1>
<h2 class="sora">sora voidSeraphim</h2>
<h4>this is some other text</h4>
<button id="Btn">chStyle</button>
```

There are two ways of adding event listeners to an element. One way is adding an attribute such as `onclick`, but not all elements support this attribute. The other way is using the `.addEventListener()` method in JavaScript code.

```
const element = document.querySelector('.sora');
```

```
const btn = document.querySelector('#Btn');
btn.addEventListener('click', (e) => {
  element.style.color = 'red';
  console.log(e.target);
});

window.addEventListener('keydown', (e) => {
  console.log(e.key);
  if (e.code === 'Space') {
    console.log('space pressed!');
  }
});
```

Anonymous functions can also be used with `addEventListener`:

```
// btn.addEventListener('click', function(e){
//     block of code;
// });
```

This commented-out example shows the alternative syntax using a standard anonymous function instead of an arrow function — both forms work the same way as the second argument to `addEventListener`.

More Events and the Event Object

Other common events include `click`, `mouseover`, `mouseout`, `keydown`, `keyup`, `submit`, `load`, `scroll`, `animationstart`, `animationiteration`, and more.

The second argument to `addEventListener` is a function — either a standard function or an arrow function. These functions are called callback functions, which will be explained in more detail later.

The callback function automatically receives a parameter, conventionally named `e` or `event`, which is the event object. This object stores data about the event, such as mouse coordinates, the pressed key, and the target element. The data it holds depends on the type of event — for example, if the event is a key press, `e.key` returns the key that was pressed.

To try this yourself, save the code into a file named `practice.html` and open it in a browser. Click the button to change the color of the `h2.sora` element to red, and press any key on the keyboard to see it logged in the console (pressing the Space bar will additionally log “space pressed!”).

Practical Project: Word Counter

This project builds a simple word and character counter, which also tracks remaining characters allowed for Twitter and Facebook-style limits.

HTML Structure


```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>wordCounter</title>
  <link rel="stylesheet" href="styles.css">
  <script src="script.js" defer></script>
</head>
<body>
  <div class="background"></div>
  <h1 class="first-heading">
    Word <span class="first-heading--special">Analytics</span>
  </h1>
  <h2>sora voidSeraphim</h2>
  <div class="container">
    <textarea class="textarea" placeholder="enter your text"></textarea>
    <section class="states">
      <section class="state">
        <span class="state__number state__number--words">0</span>
        <h2 class="second-heading">words</h2>
      </section>
      <section class="state">
        <span class="state__number state__number--characters">0</span>
        <h2 class="second-heading">characters</h2>
      </section>
      <section class="state">
        <span class="state__number state__number--twitter">14</span>
        <h2 class="second-heading">twitter</h2>
      </section>
      <section class="state">
        <span class="state__number state__number--facebook">22</span>
        <h2 class="second-heading">facebook</h2>
      </section>
    </section>
  </div>
  <footer class="footer">
    <small class="copyright">
      &copy;copyright by soraVoidSeraphim
    </small>
    <small class="data">
      last checked 1 day ago
    </small>
  </footer>
</body>
</html>

```

The `defer` keyword on the `<script>` tag delays the execution of `script.js` until after the HTML document has been parsed, so the script can safely access elements defined below

it in the page.

JavaScript Logic

```
const textAreaEl = document.querySelector('.textarea');
let characters = document.querySelector('.state__number--characters');
let words = document.querySelector('.state__number--words');
let twitter = document.querySelector('.state__number--twitter');
let facebook = document.querySelector('.state__number--facebook');

let charsLen = textAreaEl.value.length;
let twitterInNumber = Number(twitter.textContent);
let facebookInNumber = Number(facebook.textContent);
```

The script selects the textarea and the elements that display word count, character count, and the remaining character limits for Twitter and Facebook. The initial values for twitterInNumber (14) and facebookInNumber (22) are read once from the page at load time and represent the starting character limits.

Handling Input Events

```
textAreaEl.addEventListener('input', function(){
    charsLen = textAreaEl.value.length;

    let currentTxt = textAreaEl.value;
    let wordsArray = currentTxt.split(/[s\n.,!?:;]+/);
    let filteredWords = wordsArray.filter(word => word.length > 0);
    let wordCount = filteredWords.length;

    words.textContent = String(wordCount);
    characters.textContent = String(charsLen);
    twitter.textContent = String(twitterInNumber - charsLen);
    facebook.textContent = String(facebookInNumber - charsLen);

    if(Number(twitter.textContent) < 0){
        twitter.classList.add('redTxt')
    }else{
        twitter.classList.remove('redTxt')
    }

    if(Number(facebook.textContent) < 0){
        facebook.classList.add('redTxt')
    }else{
        facebook.classList.remove('redTxt')
    }
});
```

Every time the user types in the textarea, the input event fires. The number of characters is recalculated, and the text is split into words using a regular expression.

Splitting Text into Words

`.split()` is a method that splits a string into smaller parts based on the specified separator. In this example, the separator is the regular expression `/[\s\n.,!?:;]+/`, which matches one or more occurrences of whitespace, newline, period, comma, exclamation mark, question mark, semicolon, or colon. In JavaScript, a regular expression is written between two forward slashes (`/.../`), whereas in Python it would be written as a raw string `r'...'`. To split a string with a regex in Python, `re.split(r'[\s\n.,!?:;]+', string)` is used; for splitting a normal string without regex, `csv_data.split(',')` is used instead. `.split()` returns an array of the separated items.

`.filter()` is similar to `filter` in Python: it runs a function on every element of an iterable, and keeps only the elements for which the function returns `true`. In this example, `.filter()` checks whether each element's length is greater than 0; if so, it is considered a word. This removes empty strings produced by `.split()` (for example, from consecutive separators or trailing punctuation).

Design Note

The values `twitterInNumber` and `facebookInNumber` are read only once, when the page loads. As the user types, `twitter.textContent` and `facebook.textContent` are overwritten with the remaining character count on every input event, but `twitterInNumber` and `facebookInNumber` themselves are never re-read from the page. This works correctly because these two variables are captured before any input event can occur, so they always hold the original limits (14 and 22) used as the basis for the subtraction.

String Methods in Depth

This section covers more advanced string methods in JavaScript.

```
const text = 'Sora voidSeraphim';
```

Length

```
console.log(text.length);
```

`.length` returns the number of characters in a string, since strings are iterable, similar to how `len('sora')` would count the characters `'s'`, `'o'`, `'r'`, `'a'` in Python.

Includes

To check whether certain characters exist within a string:

```
console.log(text.includes('ora'));
```

`.includes()` is case-sensitive, so the first call returns `true` and the second returns `false`.

Case Changing

```
console.log(text.toUpperCase());  
console.log(text.toLowerCase());
```

`.toUpperCase()` transforms the text to uppercase, and `.toLowerCase()` transforms it to lowercase.

Trim

```
console.log(text);  
console.log(text.trim());
```

The original string may contain trailing whitespace or newline characters. To get the pure text — for example, when handling input similar to reading from `stdin` — the string needs to be trimmed using `.trim()`. This is also useful when receiving user input for operations such as comparing credentials during a login process.

Substring

Sometimes only part of a string needs to be extracted:

```
console.log(text.substring(1,10));  
console.log(text.substring(10));  
console.log(text.substring(1,));
```

`.substring()` works with index positions. The ending index is optional — `.substring(1,)` returns the text from index 1 to the end of the string.

Split

To separate a string into parts using a separator:

```
console.log(text.split(' '));  
console.log(text.split('S'));
```

`.split()` returns an array of substrings, divided at each occurrence of the separator.

Converting to Strings

Other data types can be converted to strings using `String(...)` or the `.toString()` method.

Numbers in Depth

```
let x = 3;  
let y = 3.1415;  
let z = 3.1415e10;  
  
console.log(x);  
console.log(y);  
console.log(z);
```

JavaScript numbers can be integers, floating-point values, or written in exponential notation (e.g. `3.1415e10`).

Checking Types

```
console.log(typeof(x));  
console.log(typeof(y));  
console.log(typeof(z));
```

`typeof()` is used to get the type of a value, similar to `type()` in Python. All three variables above are of type `number`.

Number Quirks

```
num1 = '2';  
num2 = 3;  
console.log(num1 - num2);  
console.log(num1 + num2);
```

The first operation returns `-1`, since the `-` operator converts the string `'2'` to a number before subtracting. The second operation returns the string `"23"`, since the `+` operator concatenates a string and a number instead of adding them.

toFixed

```
console.log(y.toFixed());  
console.log(y.toFixed(2));
```

`.toFixed()` is used to remove decimal points from a number, or to fix the number to a specified number of decimal places.

Operator Priority

Mathematical priorities in JavaScript follow standard math rules: parentheses first, then powers or roots, then multiplication or division, then addition or subtraction.

Converting to Numbers

```
console.log(Number(num1));  
console.log(Number(true));  
console.log(Number(false));  
console.log(Number(12,34));  
console.log(Number( 123  ));  
console.log(Number(" 123  "));  
console.log(Number("sora"));
```

`Number(...)` converts a value to a numeric type. `Number(true)` returns `1` and `Number(false)` returns `0`. `Number(12,34)` returns `12`, since `Number()` only accepts a single argument — any extra arguments are ignored. Both `Number(123)` and `Number("123")` trim surrounding whitespace and return `123`. `Number("sora")` returns `NaN` (“Not a Number”), since the string cannot be converted to a numeric value.

Unlike Python, which has separate `int` and `float` types, JavaScript has a single `number` type that handles integers, floating-point values, and exponential notation all at once.

Arrays in Depth

```
let numbers = [23, 26, 30];

console.log(numbers);
console.log(numbers.length);
```

When inspecting an array in the browser console, additional properties such as `length` can also be seen alongside the array's contents.

Push

```
numbers.push(46);
console.log(numbers);
```

`.push()` adds a new value to the end of an array.

Includes

```
console.log(numbers.includes(23))
```

`.includes()` checks whether a given value exists in the array, returning `true` or `false`.

forEach

```
let doubled = [];

numbers.forEach((number) => {
    doubled.push(number*2);
});

console.log(doubled);

doubled.forEach(function(number){
    console.log(number);
});
```

`forEach` is a loop that works on iterables. It is a method available on node lists, arrays, and other iterables, and it takes a function that is executed on every member of the iterable. In the example above, `number` is the function parameter, and the operation doubles each value and pushes it into another array.

`forEach` is similar to `map` in Python, but with one key difference: `map` returns a new value (or iterable), while `forEach` only performs an operation without returning anything.

Booleans in Depth

```
const text = 'sora voidSeraphim';
```

```

if(!text.includes('voidSeraphim')){
    console.log('not part of the clan');
}else{
    console.log('is a part of group');
};

```

This checks whether 'voidSeraphim' is part of the text string. ! means “not” — if something is true, then !something is false. The === operator can also be used for the same purpose, by writing something === false instead of !something. Since text.includes('voidSeraphim') is true, !text.includes('voidSeraphim') is false, so the output is 'is a part of group'.

Checking Conditions

Checking a condition’s boolean value can be useful when working with a debugger during security testing:

```

console.log(Boolean(text));
console.log(13 > 10);

```

Both of these log true.

Converting Other Types to Boolean

```

console.log(Boolean(124));

```

Boolean(...) converts other types to a boolean value. Any non-zero number converts to true.

```

const devide = 10/'sora';
console.log(Boolean(devide));

```

Dividing 10 by the string 'sora' results in NaN (“Not a Number”), and Boolean(NaN) evaluates to false.

Arrays of Objects

```

let dudes = [
    {
        'name': 'sora',
        'age': 23,
        'job': 'pentester'
    },
    {
        'name': 'dora',
        'age': 26,
        'job': 'animator'
    },
    {
        'name': 'iggy',

```

```

    'age': 4,
    'job': 'theDog'
  }
];

```

In JavaScript, object keys can be written with or without quotation marks. Unlike dictionaries in Python, where keys must be inside quotation marks, JavaScript's behavior does not change whether keys are quoted or not, and either way, the value can be accessed using `.key` (dot notation).

There is also another way to access object values besides dot notation: bracket notation, similar to accessing dictionary values in Python, using `objName['key']`.

Accessing Values

```

console.log(dudes[2].name);
console.log(dudes[0]['job']);

```

The first line accesses the `name` property of the third object in the array using dot notation, returning `'iggy'`. The second line accesses the `job` property of the first object using bracket notation, returning `'pentester'`.

Adding a New Object

```

dudes.push({
  'name': 'ghedas',
  'age': 1000,
  'job': 'noOneKnows'
});

console.log(dudes);
console.log(dudes.length)

```

`.push()` adds a new object to the end of the array.

JSON Compared to JavaScript Objects

JSON has stricter rules than JavaScript objects: both keys and values in JSON must be inside quotation marks (with the exception of numbers, booleans, and `null`, which are not quoted).

Nested Objects

Objects can be nested, meaning they can contain other objects (and arrays) inside them.

```

let user = {
  name: 'sora',
  age: 23,
  phones: ['1234', '2345'],
  address: {

```



```

        country: "Iran",
        city: "Tehran",
        street: "137",
        number: "65"
    }
};

console.log(user);
console.log(user.phones);
console.log(user['phones'][0]);
console.log(user.address.street);
console.log(user['address']['city']);

```

`user.phones` returns the array `['1234', '2345']`, and `user['phones'][0]` accesses the first element of that array, `'1234'`. `user.address.street` and `user['address']['city']` access values inside the nested `address` object using dot notation and bracket notation respectively.

Objects like this are particularly useful when working with registration and login forms, where related data (such as a user's name, contact details, and address) needs to be grouped together.

Simple Math Operators

```

let x = 9;

x = x + 3;
console.log(x);

```

A variable's value can be changed by reassigning it, as shown above.

Shorthand Operators

```

x += 3;
console.log(x);

```

`+=` is shorthand for `x = x + 3`.

```

x++;
console.log(x);

```

`++` means "add 1" (increment).

Besides `+`, all standard math operators can be used in JavaScript, including `-` (minus), `*` (multiply), and `**` (power).

```

x--;
console.log(x);

```

`--` means "subtract 1" (decrement).

Refactoring Functions

In real projects, it is common for functions to call themselves or call each other.

```
function greetings(name) {
  console.log(`Hello ${name}`)
}

function someName() {
  let name = 'sora';
  console.log(name)
  greetings(name)
}

someName()
```

`someName` calls `greetings`, which is a simple example of one function calling another.

What Is Refactoring?

Refactoring is an important concept in programming. Imagine building one function to greet admins and another to greet users:

```
function samePartOfFunctions() {
  console.log('WELCOME!');
  console.log('how was your day?');
};

function admin() {
  samePartOfFunctions();
  console.log('dear admin.');
```

```
};

function user() {
  samePartOfFunctions();
  console.log('dear user.');
```

```
};

// console.log('WELCOME!')
// console.log('how was your day?')
```

These commented-out lines (appearing inside both `admin` and `user`) represent the original duplicated code that printed the welcome message directly in each function. They have been replaced by a call to `samePartOfFunctions()`, which contains that shared logic in one place.

If an experienced programmer reviews this code and suggests refactoring it, it means consolidating the duplicated code that is used in multiple places into a single function, and calling that function wherever it is needed. This is called refactoring.

`samePartOfFunctions` is a simple function with no return value (void). A shared function like this could be more complex, such as an age validation system:

```
// const ageValidator = (age) => {
//     if(age < 18){
//         console.log('sorry little one');
//         // stop the operation
//         return;
//     }else{
//         enterCasino();
//     };
// };
```

This commented-out example shows a function that checks whether a person's age is below 18. If so, it logs a message and returns early to stop the operation; otherwise, it proceeds by calling another function, `enterCasino()`.

A Simpler Example Of `return` KeyWord

```
const checkVal = (number) => {
    if(number < 50){
        return;
    };
    console.log('Approved.');
```

```
};
checkVal(12);
checkVal(100);
```

The `return` keyword exits the function immediately. `checkVal(12)` returns early without logging anything, while `checkVal(100)` passes the condition and logs 'Approved.'.

Adding HTML to Elements

So far, we have been working with the console, but JavaScript is meant to make changes to the HTML page itself. This is how single-page applications (SPAs) work.

```
<h1>Javascript</h1>
<div class="head">loading...</div>
<button onclick="chContent()">change content</button>
```

Sometimes, instead of inserting plain text, an HTML tag itself needs to be inserted.

```
function chContent() {
    const el = document.querySelector('.head');
    el.insertAdjacentHTML('afterend', '<h2>sora</h2><h1>voidSeraphim</h1>')
};
```

```
// el.innerHTML = '<h2>sora</h2><h1>voidSeraphim</h1>'
```

This commented-out line shows an alternative approach: setting `innerHTML` directly replaces the element's content. Instead, `insertAdjacentHTML('afterend', ...)` is used, which inserts new content after the element without replacing its existing content. The position argument can also be 'beforebegin', 'afterbegin', or 'beforeend'.

Methods for Inserting HTML

There are several methods for inserting HTML content into the DOM:

```
.appendChild()    // add to end of the content of an element
parent.appendChild(newElement)
document.body.appendChild(document.createElement('p'));
```

`.appendChild()` adds a new element to the end of the content of a parent element.

```
.prepend()        // add to the start of an element
parent.prepend(newElement)
document.body.prepend(document.createElement('p'));
```

`.prepend()` adds a new element to the start of a parent element's content.

```
.append()         // like .appendChild but can also take text strings
parent.append(newEl, document.createTextNode('extra text'));    // append two things at the sa
```

`.append()` works like `.appendChild()`, but it can also accept plain text strings in addition to DOM nodes.

```
.before() and .after()    // directly pointing to target element
target.before(newEl);     // before target element
target.after(newEl);      // after target element
```

`.before()` and `.after()` insert a new element directly before or after a target element, respectively.

```
.replaceWith()         // replace an entire element with another
target.replaceWith(newEl);
```

`.replaceWith()` replaces an entire element with another element.

```
.insertAdjacentHTML()    // insert pointing to different places, takes only strings like: '<p>
// takes four values: beforebegin, afterbegin, beforeend, afterend
// beforebegin: before the element itself
// afterbegin: inside element, and at the start of it
// beforeend: inside element, at the end of it
// afterend: after element itself
```

`.insertAdjacentHTML()` inserts HTML content relative to a target element, and only accepts strings (such as `<p>something</p>`). It takes one of four position values: `'beforebegin'` (before the element itself), `'afterbegin'` (inside the element, at the start), `'beforeend'` (inside the element, at the end), or `'afterend'` (after the element itself).

```
.insertAdjacentElement()    // like insertAdjacentHTML but takes DOM nodes instead of HTML str
```

`.insertAdjacentElement()` works like `.insertAdjacentHTML()`, but it takes DOM nodes instead of strings, and uses the same four position values.

Hoisting

```
x = 10;
console.log(x);
```

```
var x;
```

At first glance, this might be expected to throw an error, since `x` appears to be used before it is declared. However, it works correctly and logs `10`.

This is because of a behavior introduced by ECMAScript called hoisting. Hoisting means moving something “up”. JavaScript automatically moves variable declarations that have no initial value to the top of the script (or function/block) before execution.

```
y = 8;  
console.log(y);
```

This also works and logs `8`, even though `y` is never explicitly declared with `var`, `let`, or `const`. In this case, JavaScript automatically creates `y` as a global variable when it is assigned.

```
sayHi();  
  
function sayHi() {  
    console.log('Hello there')  
}
```

Function declarations are also hoisted to the top of the script, so `sayHi()` can be called before its definition appears in the code, and it logs `'Hello there'`.

Hoisting in this form only applies to variables declared with `var`. Variables declared with `let` or `const` are also hoisted, but they remain in an uninitialized state and cannot be accessed before their declaration (this is known as the “temporal dead zone”).

Timers

```
console.log('hello this is some normal command.');
```

Sometimes code should not execute immediately, but after a delay.

setTimeout

```
setTimeout(() => {  
    console.log('this line has a 5 seconds delay.')  
}, 5000);
```

`setTimeout()` is a function that executes some code after a specified delay. The first parameter is a handler — a callback function that performs the desired action — and the second parameter is the delay, given in milliseconds.

A predefined function can also be called with `setTimeout`. If the function needs to be called with parameters, it must be wrapped in an anonymous function inside `setTimeout`:

```
let name = 'sora';  
  
function greetings(n) {  
    alert(`hello my dear ${n}`)  
};
```

```
setTimeout(() => {  
    greetings(name)  
}, 6000);
```

setInterval

```
setInterval(() => {  
    console.log('this codes executes every 6 seconds.')  
}, 6000);
```

setTimeout() only executes the given code once, after the specified delay. setInterval() is different: it executes the given code repeatedly, every n milliseconds, until it is stopped.

Loops

Like every other programming language, JavaScript has loops for performing operations on iterables. forEach is one nice loop already covered, but JavaScript has several others.

Sample Data

```
let customers = [  
    {  
        'name': 'sora',  
        'age': 23,  
        'mobiles': ['0123', '1234'],  
        'job': 'pentester'  
    },  
    {  
        'name': 'dora',  
        'age': 26,  
        'mobiles': ['2345', '3456'],  
        'job': 'animator'  
    },  
    {  
        'name': 'iggy',  
        'age': 4,  
        'mobiles': ['4567', '5678'],  
        'job': 'theDog'  
    }  
];
```

Normal for Loop

A for loop runs for a specified number of iterations, or on arrays using indexes. It has three parts separated by semicolons: the first is a variable holding the index, the second is the condition that must be met to continue, and the third defines what happens to the index variable after each iteration.

```
for (let i = 0; i < customers.length; i++) {
    console.log(customers[i].mobiles[0]);
};
```

```
// console.log(customers[i]['mobiles'][0]);
```

This commented-out line shows the same access written with bracket notation instead of dot notation; both produce the same result.

for...of Loop

```
for (let customer of customers) {
    console.log(customer.mobiles[1]);
};
```

for...of is a loop for working with arrays or other iterables. It works similarly to forEach and does not require indexes — the loop variable becomes each element itself on every iteration, similar to a for...in loop in Python. It is recommended to declare the loop variable with let.

for...in Loop

```
let user = {
    'name': 'sora',
    'age': 23,
    'mobiles': ['0123', '1234'],
    'job': 'pentester'
};

for (let customer in customers) {
    console.log(customer);
    console.log('name of customer: ');
    console.log(customers[customer].name)
};

for (let key in user) {
    console.log(`key: ${key}, value: ${user[key]}`);
};
```

for...in is best suited for iterating over the keys of an object, and is not recommended for arrays. When used on an array, it returns the index of each element (as a string), similar to a normal for loop. When used on an object, it returns each key, allowing both the key and its corresponding value to be accessed.

Nested Loops

```
for (let i in customers) {
    console.log(`customer: ${customers[i].name}`);
    for (let key in customers[i]) {
```

```

        console.log(`key: ${key}, value: ${customers[i][key]}`)
    };
};

```

Loops can be nested. Here, the outer loop iterates over each customer, and the inner loop iterates over the keys of each customer object.

while Loop

```

let count = 0;
while (count < 10) {
    console.log(count);
    count++;
};

```

In a `while` loop, the number of iterations is not known in advance — the loop keeps running as long as its condition remains true. It is important to update the relevant state on every iteration to avoid an infinite loop.

do...while Loop

```

let i = 5;
do {
    console.log('try number', i);
    i++;
} while (i < 3);

```

A `do...while` loop is similar to a `while` loop, but the code inside it executes at least once, even if the condition is false from the start. In this example, the condition `i < 3` is false from the beginning, but the code still runs once before the condition is checked.

Fetching Data from a Server

This section covers how to read data from a server in JavaScript, using both the modern `fetch` API and the older `XMLHttpRequest` (XHR) approach. Both examples work with the same sample JSON data.

Sample Data

```

[
  {"userId": 1, "id": 1, "title": "First test post", "body": "This is a simple sample text t"},
  {"userId": 1, "id": 2, "title": "Second user's post", "body": "In this post we check how t"},
  {"userId": 2, "id": 3, "title": "Sample post for second user", "body": "The second user ha"},
  {"userId": 3, "id": 4, "title": "Fourth post of third user", "body": "With these data entr"}
]

```

This data is saved as `jsonDataOutput.json` and served locally, for example by running a simple HTTP server in the same directory.

Using `fetch`

In JavaScript, there are two main ways to send HTTP requests: XHR (the older way, with more complicated syntax) and `fetch` (the modern, recommended way).

```
fetch(url, options)
  .then(response => {
    // operation with response, example:
    if(response.ok){
      return response.json();
    }
  })
  .then(data => {
    console.log(data)
  })
  .catch(error => {
    console.log('there was a problem...', error)
  });
```

This block shows the general structure of a `fetch` request. The request is sent to the given URL; the first `.then(response => ...)` block executes when the server sends a response; the second `.then(data => ...)` block executes once `response.json()` resolves successfully, with `data` holding the parsed result from the previous block; and `.catch(error => ...)` executes if an error occurs at any point.

Sending a POST Request

```
fetch(url, {
  method: 'POST',    // GET, PUT, PATCH, DELETE, OPTIONS, HEAD
  headers: {
    'Content-Type': 'application/json',
    'Authorization': 'Bearer SOME_TOKEN',
    'X-Custom-Header': 'MyValue'
  },
  body: JSON.stringify({
    name: 'John Doe',
    email: 'john.doe@example.com'
  })
})
  .then(response => {
    // operation with response, example:
    if(response.ok){
      console.log(response.text()); // we also have .json(), .blob(), .formData(), .arrayBuffer()
      for (let [key, value] of response.headers) {
        console.log(`header: ${key}, value: ${value}`); // to read response headers
      }
      return response.json();
    }
  })
  .then(data => {
    console.log(data)
```

```

    })
    .catch(error => {
      console.log('there was a problem...', error)
    });

```

This block shows a more detailed example of sending a POST request. It includes a method, headers (such as Content-Type, Authorization, and a custom header), and a body containing JSON data created with `JSON.stringify()`. The `method` field can also be set to GET, PUT, PATCH, DELETE, OPTIONS, or HEAD. Inside the response handler, `response.text()` reads the response as plain text — other available methods include `.json()`, `.blob()`, `.formData()`, and `.arrayBuffer()`, depending on the expected response type. The `for (let [key, value] of response.headers)` loop reads each response header.

Sending the Actual GET Request

```

const url = 'http://localhost:8000/jsonDataOutput.json';

fetch(url, {
  method: 'GET'
})
  .then(res => {
    console.log(res.status);
    if (res.ok){
      console.log(res);
      return res.json();
    }
  })
  .then(data => {
    console.log(data)
    for (index in data) {
      console.log(data[index]);
      console.log(data[index].title);
      console.log(data[index]['title']);
    }
  })
  .catch(error => {
    console.log('an error occurred:', error)
  });

```

This sends a GET request to the given URL and logs `res.status`, the HTTP status code of the response. If `res.ok` is true, the response object itself is logged, and `res.json()` is returned. Note that this `return` inside `.then()` does not return a value out of the whole `fetch` chain — it passes the resolved value to the next `.then()` block, which receives it as `data`.

```
// console.log(res.json());
```

This commented-out line shows a common mistake: calling `res.json()` without `return` and without awaiting it would log a pending Promise rather than the actual data. We will talk much more about promises later.

```
for (let i = 0; i < data.length; i++) {
  console.log(data[i]);    // iterate on the json that we have to show all the childs.
  console.log(data[i].title); // iterate on the json that we have to show all the childs
  console.log(data[i]['title']); // we can access it also with bracket notation.
} // or we can use for in loop.
```

This block shows an alternative way to iterate over the array using a normal `for` loop with an index, accessing each item's `title` property using both dot notation and bracket notation. The actual code below it uses a `for...in` loop instead, achieving the same result.

Using XMLHttpRequest (XHR)

`XMLHttpRequest` is the older way to send HTTP requests in JavaScript. `fetch` is used nowadays, but it is still useful to know the XHR syntax.

```
var xhr = new XMLHttpRequest;

var url = 'http://localhost:8000/jsonDataOutput.json';

xhr.onreadystatechange = function () {
  console.log(xhr.readyState);
  console.log(xhr.status);
  console.log(xhr.responseText);
  if (xhr.readyState === 4 && xhr.status === 200) {
    var objects = JSON.parse(xhr.responseText);
    console.log(objects)
    for (let i = 0; i < objects.length; i++) {
      console.log(objects[i].title)
    }
  } else if (xhr.readyState === 4) {
    console.log('status was not okay:', xhr.status)
  }
};

xhr.open('GET', url, true);

xhr.send();
```

An instance of `XMLHttpRequest` is created first. A callback function is assigned to `xhr.onreadystatechange`, which runs every time the request's state changes. `xhr.readyState` can have the following values:

```
// 0 : UNSET
// 1 : OPENED
// 2 : HEADERS_RECEIVED
// 3 : LOADING
// 4 : DONE
```

`xhr.status` holds the HTTP status code, and `xhr.responseText` holds the raw response body as text. When `xhr.readyState === 4` (request finished) and `xhr.status === 200`

(success), the response text is parsed as JSON using `JSON.parse()` and iterated with a `for` loop to log each item's `title`. If the request finished but the status was not `200`, an error message is logged instead.

`xhr.open('GET', url, true)` configures the request method and URL; the third argument, `true`, makes the request asynchronous. `xhr.send()` sends the request to the server — if the method were `POST` or `PUT`, the data to send would be passed as an argument to `.send()`.

Sending a POST Request with XHR

```
var xhr = new XMLHttpRequest;
var url = 'https://api.example.com/users';
var data = JSON.stringify({
  name: 'Jane Doe',
  email: 'jane.doe@example.com'
});

xhr.onreadystatechange = function () {
  if (xhr.readyState === 4 && xhr.status >= 200 && xhr.status < 300) {
    console.log('Success:', xhr.responseText);
  } else if (xhr.readyState === 4) {
    console.error('Error: status ', xhr.status);
  }
};

// configuration
xhr.open('POST', url, true);

// send the post request containing data in json form:
xhr.send(data);
```

This block shows how to send a `POST` request using XHR: a new `XMLHttpRequest` instance is created, the data to send is converted to a JSON string with `JSON.stringify()`, the request is configured with `xhr.open('POST', url, true)`, and `xhr.send(data)` sends the request with the JSON data as its body. The status check `xhr.status >= 200 && xhr.status < 300` covers the full range of successful HTTP responses, not just `200`.

To try this yourself, save each file (`TakingInfoFromServer-Fetch.html` and `TakingInfoFromServer-XHR.html`) along with `jsonDataOutput.json` in the same location, start a local HTTP server (for example with `python3 -m http.server 8000`), and open each HTML file through `http://localhost:8000/` in a browser. Check the developer tools console for the output.

Showing API Data on the Page

```
<h1>Working with APIs</h1>
<hr>
<h1>Posts:</h1>
<ul>
```

```
    <!-- ive created a ul to add li one by one. -->
</ul>
```

A `<ul>` element is created to hold the list items that will be added one by one.

So far, data has been retrieved from an API using `fetch` and `XHR`, but it has only been logged to the console. This section shows how to display that data on the HTML page.

```
let url = 'http://localhost:8000/jsonDataOutput.json';
```

`fetch` is based on promises, which will be covered in more detail later. This means the operation does not complete immediately — the result becomes available later, which is why `.then()` is used to access the resolved value of the promise. Because of this, the response content cannot be accessed and processed immediately after calling `fetch`.

```
fetch(url, {
  method: 'GET'
})
  .then(res => {
    let resText = res.text();
    return resText;
  })
  .then(resText => {
    data = JSON.parse(resText)
    return data;
  })
  .then(data => {
    console.log(data);
    data.forEach(post => {
      const El = document.createElement('li');
      El.textContent = post.title
      document.querySelector('ul').insertAdjacentElement('beforeend', El)
    });
  })
```

```
// .then(res => {
//   // console.log(res);
//   return res.json();
// })
```

This commented-out block shows a shorter alternative: `res.json()` parses the response body as JSON directly, combining the steps of reading the text and parsing it. The longer approach using `res.text()` followed by `JSON.parse()` is used here instead to demonstrate `JSON.parse()` explicitly within a fetch chain.

In the final `.then(data => ...)` block, the parsed data (an array of post objects) is logged to the console, and then iterated with `.forEach()`. For each post, a new `<li>` element is created with `document.createElement('li')`, its text content is set to the post's title, and it is inserted at the end of the `<ul>` element using `insertAdjacentElement('beforeend', El)`.

To try this yourself, save the code into a file named `practice.html` alongside `jsonDataOutput.json`, start a local HTTP server (for example with `python3 -m http.server 8000`),

and open the page through `http://localhost:8000/`. The list of post titles will appear on the page under “Posts”.

Working with Nested API Data

Sample Data

```
{
  "page": 1,
  "per_page": 3,
  "total": 12,
  "total_pages": 4,
  "data": [
    {
      "id": 1,
      "email": "george.bluth@reqres.in",
      "first_name": "George",
      "last_name": "Bluth",
      "avatar": "https://reqres.in/img/faces/2-image.jpg"
    },
    {
      "id": 2,
      "email": "janet.weaver@reqres.in",
      "first_name": "Janet",
      "last_name": "Weaver",
      "avatar": "https://reqres.in/img/faces/2-image.jpg"
    },
    {
      "id": 3,
      "email": "emma.wong@reqres.in",
      "first_name": "Emma",
      "last_name": "Wong",
      "avatar": "https://reqres.in/img/faces/3-image.jpg"
    }
  ],
  "support": {
    "url": "https://reqres.in/#support-heading",
    "text": "To keep ReqRes free, contributions towards server costs are appreciated!"
  }
}
```

This data is saved as `users.json` and served locally.

HTML Structure

```
<h2>User emails:</h2>
<hr>
<ul>
  <!-- parent tag -->
```

```
</ul>
```

The `<ul>` element acts as the parent container into which the list of user emails will be inserted.

HTTP Methods for CRUD Operations

There are four main operations for working with a table of data: create, read, update, and delete (CRUD). In HTTP, reading data is done with a `GET` request, creating data is done with `POST`, updating data is done with `PUT`, and deleting data is done with `DELETE`.

Fetching and Displaying the Data

```
let url = 'http://localhost:8000/users.json';
```

This JSON structure mirrors the real ReqRes API, which supports pagination through a query parameter such as `?page=2`. Since the original API could not be reached, this data was saved as a local file instead, containing only a single page.

```
fetch(url, {
  method: 'GET'
})
.then(res => {
  console.log(res);
  return res.json();
})
.then(data => {
  console.log(data);
  console.log(data.data);
  data.data.forEach(user => {
    const el = document.createElement('li')
    el.textContent = user.email
    document.querySelector('ul').insertAdjacentElement('beforeend', el)
  })
})
```

Unlike the simpler JSON structures worked with previously, this one is nested: the response object contains a `data` property, which itself holds an array of user objects. A property of a JSON object can be accessed using dot or bracket notation, even if that property is itself an object or array — here, `data.data` accesses the array of users inside the outer `data` object.

`.forEach()` iterates over `data.data`, and for each user, a new `<li>` element is created, its text content is set to the user's `email`, and it is inserted at the end of the `<ul>` element using `insertAdjacentElement('beforeend', el)`.

To try this yourself, save the code into a file named `practice.html` alongside `users.json`, start a local HTTP server (for example with `python3 -m http.server 8000`), and open the page through `http://localhost:8000/`. The list of user emails will appear on the page under “User emails:”.

Introduction to the DOM

HTML by itself is not a programming language; JavaScript is what makes a page dynamic. When a page loads, JavaScript reads the entire HTML document and builds a tree-like structure called the Document Object Model (DOM).

The DOM is a tree representation in which all tags and attributes are translated into a form that can be accessed and manipulated programmatically. The browser provides a `window` object representing the browser window, and inside `window`, the `document` is loaded. The document has a `head` and a `body`, each containing their own children, and those children can themselves be nested and contain further children.

In JavaScript, `document` refers to the current document, and every element and attribute inside it can be accessed and modified.

HTML Structure

```
<h1>Voidseraphim</h1>
<h3>this is sora</h3>
<hr>
<h2 class="heading">hello everyone!</h2>
<button class="btn">click me</button>
<input type="text" class="input">
```

Accessing and Modifying Elements

```
const buttonEl = document.querySelector('.btn');
buttonEl.style = 'background-color: yellow; color: blue; padding: 10px; font-size: 20px;';
buttonEl.style.borderRadius = '10px';

const inputEl = document.querySelector('.input');
inputEl.value = 'hello!';

const headingEl = document.querySelector('.heading');
headingEl.innerText = 'Hi everyone!';

// buttonEl.disabled = true;
```

This commented-out line shows how to disable a button by setting its `disabled` attribute to `true`.

```
// buttonEl.focus();
```

This commented-out line shows how to give the button keyboard focus every time the page is refreshed, using `.focus()`.

The button's style is set using a CSS string for multiple properties at once, followed by setting `borderRadius` individually. The input field's value is set to `'hello!'`, and the heading's text is changed to `'Hi everyone!'` using `innerText`.

Generating Page Content with JavaScript

It is common to create HTML content dynamically with JavaScript, as is done in single-page applications.

```
<h2 class="heading">
  Hello everyone!
</h2>

const headingEl = document.querySelector('.heading');
headingEl.insertAdjacentHTML('beforeend', '<a href="http://gerdoo.me">GERD00 SE</a>');

const user = {
  name: 'Sora Voidseraphim',
  phone: '01234'
}

headingEl.innerHTML = `
hello <i>${user.name}</i> with phone number:
<b>${user.phone}</b>
`;

// headingEl.innerHTML += 'hi! <a href="http://gerdoo.me">GERD00 SE</a>';
```

This commented-out line shows an alternative approach: using `+=` with `innerHTML` appends the new content to the existing content of the element. However, a better way is used instead — `insertAdjacentHTML('beforeend', ...)`, which inserts new content at the end of the element without re-parsing and replacing the entire existing content.

After the link is inserted, `headingEl.innerHTML` is overwritten entirely with a template literal that embeds the `user` object's `name` and `phone` properties inside `<i>` and `<b>` tags. This overwrite replaces the link added in the previous step, since `innerHTML` (with `=`, not `+=`) replaces all existing content.

Working with Events: The Target Property

An event is something that happens as a result of a user interaction with the page.

```
<h1>VoidSeraphim</h1>
<h3>this is Sora</h3>
<hr>
<div class="container">
  <h1 class="heading">hello users</h1>
  <button class="btn">Click Me</button>
</div>
<ul class="list">
  <li class="item">1</li>
  <li class="item">2</li>
  <li class="item">3</li>
  <li class="item">4</li>
  <li class="item">5</li>
</ul>
```

Listening for Clicks on a Container

```
const containerEl = document.querySelector('.container');
containerEl.addEventListener('click', (e) => {
  console.log(e);
  console.log(e.target);
})
```

An event listener has been added to the entire `.container` div. Since the div contains multiple child elements, the `target` property of the event object is used to determine exactly which element was clicked. `console.log(e)` prints the full event object, which contains additional details such as the exact coordinates of the click.

Working with the Target Property

```
const listEl = document.querySelector('.list');
listEl.addEventListener('click', (e) => {
  console.log(e.target.textContent);
  console.log(e.target.tagName);
  if (e.target.tagName === 'LI') {
    console.log('li clicked!')
  }
})
```

This listener is attached to the `.list` element. When any item inside it is clicked, `e.target.textContent` logs the text content of the clicked element, and `e.target.tagName` logs its tag name in uppercase (e.g. `'LI'`). The condition checks whether the clicked element is specifically an `<li>` element, and if so, logs a confirmation message.

Practical Project: Feedback Board (CorpComment)

This is a small front-end project that lets users submit public feedback about companies, tag it with a hashtag, upvote feedback items, filter by hashtag, and expand long feedback text. Data is fetched from and posted to a backend API.

For this project, assume there is an API running at some site that returns and accepts JSON data in the following shape:

```
{
  "public": true,
  "sorted": true,
  "feedbacks": [
    {
      "company": "ByteGrad",
      "badgeLetter": "B",
      "upvoteCount": 593,
      "daysAgo": 4,
      "text": "Hi #ByteGrad, I really like you!"
    },
    {
```

```

    "company": "voidseraphim",
    "badgeLetter": "V",
    "upvoteCount": 0,
    "daysAgo": 0,
    "text": "hello im from #voidseraphim , my name is sora and salary is not enough"
  }
]
}

```

A GET request to this endpoint returns the full list of feedback items. A POST request with a JSON body containing text, upvoteCount, badgeLetter, daysAgo, and company appends a new feedback item to the stored data.

HTML Structure

The relevant elements referenced by the JavaScript:

```

<form action="#" class="form">
  <textarea class="form__textarea" id="textarea" spellcheck="false" maxlength="150" type="text"
    placeholder="Enter feedback"></textarea>
  <label for="textarea" class="form__label">Enter your feedback here, remember to <span
    class="u-medium">#hashtag</span> the company <i
    class="fa-solid fa-pen form__icon"></i></label>
  <div class="form__bottom">
    <p class="counter u-italic">150</p>
    <button class="submit-btn">
      <span class="submit-btn__text">Submit</span>
    </button>
  </div>
</form>

<ol class="feedbacks">
  <div class="spinner"></div>
</ol>

<ul class="hashtags">
  <li class="hashtags__item">
    <button class="hashtag hashtag--all">#All</button>
  </li>
  <li class="hashtags__item">
    <button class="hashtag">#JavaScript</button>
  </li>
  <li class="hashtags__item">
    <button class="hashtag">#Metalab</button>
  </li>
</ul>

```

JavaScript

```
// when starting a project, where exactly should we start?
// a project can be divided into smaller parts, handled one by one. these small parts are called components
// components can be related and affect each other, or be independent.

// it is better to declare global variables at the top of the script.
const serverAPI = 'http://localhost:8000/sample.json'; // assume the API is on local host
const textAreaEl = document.querySelector('.form__textarea');
const formEl = document.querySelector('.form');
const maxCharsEl = document.querySelector('.counter');
const maxChars = Number(maxCharsEl.textContent); // 150
const feedbacksEl = document.querySelector('.feedbacks');
const spinnerEl = document.querySelector('.spinner');
const submitEl = document.querySelector('.submit-btn');
const hashtagListEl = document.querySelector('.hashtags');

function inputHandler() {
  // alert('works');
  const charsLen = textAreaEl.value.length;

  // console.log(charsLen);
  // console.log(maxChars);
  maxCharsEl.textContent = String(maxChars - charsLen);
};

function clickHandler(e) {
  const clickedEl = e.target;
  const upvoteEl = clickedEl.className.includes('upvote');

  if (upvoteEl) {
    // if upvote clicked...
    console.log(upvoteEl);
    const upvoteBtnEl = clickedEl.closest('.upvote');
    upvoteBtnEl.disabled = true;

    let upvoteCountEl = upvoteBtnEl.querySelector('.upvote__count')
    let upvoteCount = Number(upvoteCountEl.textContent);
    console.log(upvoteCount)
    upvoteCountEl.textContent = String(upvoteCount + 1);
  } else {
    // we could write all this logic manually, or just use the toggle method
    if (!clickedEl.className.includes('feedback--expand')) {
      clickedEl.closest('.feedback').classList.add('feedback--expand');
    } else {
      clickedEl.closest('.feedback').classList.remove('feedback--expand');
    }
    // clickedEl.classList.toggle('feedback--expand');
  }
}
```

```

    }
};

function hashtagClickHandler (e) {
    const clickedEl = e.target;

    if (clickedEl.className === 'hashtags') {
        return
    };

    const companyNameFromHashtag = clickedEl.textContent.substring(1).trim();
    console.log(companyNameFromHashtag);

    feedbacksEl.childNodes.forEach(childNode => {
        if (childNode.nodeType === 3) return;
        // const childCompanyName = childNode.querySelector('.feedback__content').querySelector('p');
        const childCompanyName = childNode.querySelector('.feedback__company').textContent.toLowerCase().trim();
        console.log(childCompanyName)

        if (companyNameFromHashtag.toLowerCase().trim() === 'all') {
            childNode.style.display = '';
        } else {
            if (childCompanyName !== companyNameFromHashtag.toLowerCase().trim()) {
                childNode.style.display = 'none';
            } else {
                childNode.style.display = ''; // empty string returns the value to its default
            };
        };
    });
};

function submitHandler(e) {
    e.preventDefault();
    const text = textAreaEl.value;

    console.log(text);
    // validate that the textarea is not empty and contains a hashtag
    if (text.includes('#') && text.length > 5){
        changeClass('form--valid');
        // if the condition is met, the value should be appended to the list of comments.
        // this could be done here, but for readability it is handled below, with the else branch
    }else{
        changeClass('form--invalid');
        textAreaEl.focus();
        return;
    };
    // alert('we have reached here means condition is met');
    // now lets work on extracting parts and manipulating text.

```

```

// one of the functions that can break a string apart is split
const hashtag = text.split(' ').find( (word) => {
  if (word.includes('#')) {
    return word;
  }
});
// the company name is inside the hashtag, but how do we extract it?
// substring helps us get the desired part of a string.
const company = hashtag.substring(1);

// slice is used here, which is similar to substring but has some differences
const badgeLetter = company.slice(0,1).toUpperCase();
let dayAgo = 0;
let upVoteCount = 0;

// this part used to render the new <li> element inline, but after refactoring it was moved
// const feedItem = `
// <li class="feedback">
//   <button class="upvote">
//     <i class="fa-solid fa-caret-up upvote__icon">U</i>
//     <span class="upvote__count">${upVoteCount}</span>
//   </button>
//   <section class="feedback__badge">
//     <p class="feedback__letter">${badgeLetter}</p>
//   </section>
//   <div class="feedback__content">
//     <p class="feedback__company">${company}</p>
//     <p class="feedback__text">${text}</p>
//   </div>
//   <p class="feedback__date">${dayAgo < 1 ? "NEW" : `${dayAgo}d`}</p>
// </li>
// `
// feedbacksEl.insertAdjacentHTML('beforeend', feedItem);

// instead, the feedback is packed into an object and passed to renderFeedbackItem
const feedback = {
  text: text,
  upvoteCount: upVoteCount,
  badgeLetter: badgeLetter,
  daysAgo: dayAgo,
  company: company
};
renderFeedbackItem(feedback);

fetch(serverAPI, {
  method: 'POST',
  body: JSON.stringify(feedback),
  headers: {
    'Content-Type': 'application/json'
  }
});

```

```

    }
  }).then(res => {
    if (!res.ok) {
      console.log('I Cant Post Your Data');
    }else{
      console.log('Done!');
    }
  }).catch(error => {
    console.log('Error occoured:', error);
  });

  textAreaEl.value = '';
  submitEl.blur();
  inputHandler();

  // check if the spinner exists, then remove it
  if (spinnerEl) {
    // .remove() removes the element from the DOM
    spinnerEl.remove()
  }

  console.log(hashtag)
  console.log(company);
  console.log(badgeLetter);
};

function renderFeedbackItem(feedback) {
  const feedItem = `
<li class="feedback">
  <button class="upvote">
    <i class="fa-solid fa-caret-up upvote__icon">U</i>
    <span class="upvote__count">${feedback.upvoteCount}</span>
  </button>
  <section class="feedback__badge">
    <p class="feedback__letter">${feedback.badgeLetter}</p>
  </section>
  <div class="feedback__content">
    <p class="feedback__company">${feedback.company}</p>
    <p class="feedback__text">${feedback.text}</p>
  </div>
  <p class="feedback__date">${feedback.daysAgo < 1 ? "NEW" : `${feedback.daysAgo}d`} </p>
</li>
`;
  feedbacksEl.insertAdjacentHTML('beforeend', feedItem);
}

function changeClass (className) {
  formEl.classList.add(className);
  setTimeout(() => {

```

```

        formEl.classList.remove(className);
    }, 2000);
};

// handle character count as the user types
// the 'input' event fires when the value of an input, textarea, or select changes – typing,
textareaEl.addEventListener('input', inputHandler);

// handle form submission
formEl.addEventListener('submit', submitHandler);

// handle clicks inside the feedback list – expanding items and upvoting
feedbacksEl.addEventListener('click', clickHandler);

// handle clicks on hashtag filter buttons
hashtagListEl.addEventListener('click', hashtagClickHandler);

// fetch existing feedback data from the API on page load
fetch(serverAPI, {
    method: 'GET'
}).then(res => {
    return res.json();
}).then(data => {
    console.log(data.feedbacks);
    for (let i in data.feedbacks) { // forEach would also work here

        // the inline rendering below was replaced by renderFeedbackItem during refactoring
        // const feedItem = `
        // <li class="feedback">
        //     <button class="upvote">
        //         <i class="fa-solid fa-caret-up upvote__icon">U</i>
        //         <span class="upvote__count">${data.feedbacks[i].upvoteCount}</span>
        //     </button>
        //     <section class="feedback__badge">
        //         <p class="feedback__letter">${data.feedbacks[i].badgeLetter}</p>
        //     </section>
        //     <div class="feedback__content">
        //         <p class="feedback__company">${data.feedbacks[i].company}</p>
        //         <p class="feedback__text">${data.feedbacks[i].text}</p>
        //     </div>
        //     <p class="feedback__date">${data.feedbacks[i].daysAgo < 1 ? "NEW" : ` ${data.feedbacks[i].daysAgo} days ago`}</p>
        // </li>
        // `
        // feedbacksEl.insertAdjacentHTML('beforeend', feedItem);

        const feedback = {
            text: data.feedbacks[i].text,
            upvoteCount: data.feedbacks[i].upvoteCount,
            badgeLetter: data.feedbacks[i].badgeLetter,

```



```

    daysAgo: data.feedbacks[i].daysAgo,
    company: data.feedbacks[i].company
  }
  renderFeedbackItem(feedback)

  if (spinnerEl) {
    spinnerEl.remove()
  }
};
}).catch(error => {
  feedbacksEl.textContent = `Failed to fetch feedback items. error: ${error}`
});

```

Flow Summary

On page load, the script grabs references to all the relevant DOM elements (form, textarea, feedback list, hashtag list, spinner, character counter). It then sends a `GET` request to the API, receives the stored feedback items, and renders each one into the `<ol class="feedbacks">` list using `renderFeedbackItem`. Once items are rendered, the loading spinner is removed.

Four event listeners drive the interactivity:

- Typing in the textarea triggers `inputHandler`, which updates the remaining character counter.
- Submitting the form triggers `submitHandler`, which validates that the text contains a hashtag and is longer than 5 characters. If valid, it extracts the hashtag and company name, builds a feedback object, immediately renders it on the page, and sends it to the API via `POST`. If invalid, it shows an error state on the form.
- Clicking inside the feedback list triggers `clickHandler`, which either increments the upvote count (if an upvote button was clicked) or toggles the expanded view of a feedback item (if the text itself was clicked).
- Clicking a hashtag button triggers `hashtagClickHandler`, which filters the visible feedback items by company name, or shows all of them if `#All` is selected.

Methods Used

`.find()` is called on an array and takes a callback function. It searches for the first element for which the callback returns a truthy value, and returns that element. In this project, it is used to locate the word containing `#` inside the split feedback text.

`.substring(start, end)` extracts part of a string between two indices. The end index is optional, and the character at the end index is not included. It is used here to strip the `#` from the hashtag.

`.slice()` works similarly to `.substring()`, but also supports negative indices, which count from the end of the string. It is used to get the first letter of the company name for the badge.

`.substr(start, length)` is also similar to `.substring()`, but instead of a start and end index, it takes a start index and a length. It is mentioned here for reference but not used directly in this script.

`.closest(selector)` is called on a DOM element and returns the nearest ancestor (or the element itself) that matches the given selector — a class, id, or tag name. It is used to find the parent `.upvote` button or `.feedback` item from a clicked child element.

`.classList.toggle()` adds a class if it is not present, or removes it if it is. It is mentioned as a simpler alternative to the manual add/remove logic used in `clickHandler`.

`.trim()` removes leading and trailing whitespace from a string. It is used when comparing company names and hashtag text.

`.childNodes` is a property available on DOM elements that returns all child nodes, including both elements and text nodes. Every node has a `nodeType`: type 1 is an element node (an HTML tag), and type 3 is a text node (text content or whitespace). This is used in `hashtagClickHandler` to skip over whitespace text nodes while iterating through the feedback list.

Arrays in More Depth

Variables are important in every programming language, and can be stored for anywhere from minutes to years, depending on how they are used. Arrays have already been introduced, but this section explores them in more depth.

```
let numbers = [1, 2, 3, 4, 5, 6];
console.log(numbers);
```

push

```
numbers.push(7);
numbers.push(8, 9, 10);
numbers.push([8, 9, 10]);
console.log(numbers);
```

`.push()` adds one or more values to the end of an array, similar to `append()` in Python. Multiple values can be pushed at once, and an entire array can also be pushed as a single nested element.

forEach

```
numbers.forEach(num => {
  console.log(`This is: ${num}`);
});
```

`forEach` is a flexible loop, used not only on arrays but on iterables in JavaScript in general.

pop

```
let last = numbers.pop();
console.log(numbers);
console.log(last);
```

`.pop()` removes the last element of an array and returns the removed element. By default it removes the last index, though it can be combined with other methods to target different positions.

unshift

```
numbers.unshift(0);
console.log(numbers);
```

`.unshift()` adds one or more elements to the start of an array — the opposite of `.push()`.

shift

```
let first = numbers.shift();
console.log(numbers);
console.log(first);
```

`.shift()` removes the first element of an array and returns it — the opposite of `.pop()`.

splice

```
numbers.splice(0, 1, 55, 56);
console.log(numbers);
```

`.splice()` adds, removes, or replaces elements at a specific index. It works similarly to `insert()` in Python, but is more complete. Its syntax is: `.splice(startIndex, deleteCount, newElement1, newElement2, ...)`, where `startIndex` is the index to begin changes, `deleteCount` (optional) is how many elements to remove from that index, and any further arguments are new elements to insert at that position.

slice

```
let numbersPart = numbers.slice(2, 5);
console.log(numbersPart);
```

`.slice()` returns a copy of a portion of the original array, without modifying it.

concat

```
let a = [1, 2];
let b = [3, 4];

let merged = a.concat(b);
console.log(merged);
```

`.concat()` combines multiple arrays into a new array.

Negative Indexes

Negative indexes can be accessed by appending `.at(-1)` to an array, which returns the last element. `.splice()` and `.slice()` also accept negative indexes directly.

Reference Behavior of Arrays

```
let newNumbers = numbers;
numbers.push(77)
console.log(numbers);
console.log('this is newNumbers: ');
console.log(newNumbers);
```

At this point, `newNumbers` is not a copy of `numbers` — it refers to the same array in memory. Any change made to `numbers` is also reflected in `newNumbers`, since both variables point to the same array object.

map

```
// let newNumbersX2 = numbers.map(num => {
//     return num*2;
// });
```

This commented-out block shows the standard way to use `.map()`: it returns a new array directly. `.map()` performs an operation on every element of an array and returns a new array containing the results. Unlike Python's `map()`, which works on any type of iterable, `.map()` in JavaScript is an array method and only works on arrays. When `.map()` is called, it creates an empty array to hold the results, executes the callback function on every element, places each returned value into that new array, and finally returns the new array.

```
let newNumbersX2 = [];
numbers.map(num => {
    let newNum = num*2;
    newNumbersX2.push(newNum);
});

console.log(numbers);
console.log('this is newNumbersX2');
console.log(newNumbersX2);
```

The version above achieves the same result, but builds the result array manually using `.push()` inside the callback instead of relying on `.map()`'s return value. This form may be more familiar to programmers coming from an imperative programming background, though it does not take advantage of `.map()`'s built-in return behavior.

join

```
let joinRes = newNumbersX2.join('|');
console.log(joinRes);
```

`.join()` runs on an array and converts its elements into a single string, using the separator passed as an argument between each element.

some

```
let checkSome = newNumbersX2.some(num => {
  return num > 20;
});
console.log(checkSome);
```

`.some()` iterates over an array and checks whether at least one element meets the given condition, returning `true` or `false`.

every

```
let checkEvery = newNumbersX2.every(num => {
  return num > 20;
});
console.log(checkEvery);
```

If checking whether *every* element of an array meets a condition is needed, rather than just one, `.every()` is used instead of `.some()`.

JavaScript Arrays — `find()` and `filter()`

```
const numbers = [1, 2, 3, 4, 5, 6, 7, 8]

// .find() iterates an array and returns the first element that meets the given condition
const result = numbers.find( num => {
  return num > 3;
});

console.log(result);
```

`.find()` only returns the first match. To get all matching elements, you could manually filter using a loop:

```
const newNums = [];
numbers.forEach(num => {
  if (num > 3) {
    newNums.push(num);
  }
});
console.log(newNums);
```

The same result can be achieved more concisely with `.filter()`:

```
const numbers2 = [9, 8, 7, 6, 5, 4];

let newNums2 = numbers2.filter(num => {
  return num > 5;
});

console.log('newNums2:');
console.log(newNums2)
```

JavaScript Objects In More Depth

The object data type provides custom types in JavaScript.

```
const user = {
  name: 'sora',
  family: 'voidSeraphim',
  age: 23
};
```

`user` is declared with `const`, which means `user` itself cannot be reassigned — = cannot be used on it again. However, the values of its keys can still be changed:

```
user.name = 'dora';

console.log(user);
console.log(user.name);
console.log(user['age']);
```

JavaScript — Callback Functions and Methods

Callback Functions

Functions come in different forms — callback functions, methods, and more.

A **callback function** is a function defined to be called later when needed. It is not called directly — instead, it is passed as an **argument** to another function. Syntactically, callback functions are just like normal functions, and can also be written as arrow functions.

`.map()`, `.filter()`, and `.forEach()` all take a function to call as needed — that function is the callback.

- `.map()` takes a callback and runs it on every element of an array.
- `.filter()` takes a callback and runs it on every element to validate values against it.
- `.forEach()` takes a callback and performs the desired operation on each element during iteration.

```
let nums = [1,2,3,4,5];
```

```
function thisIsACallbackFuntions(num) {
    console.log(num)
};

nums.forEach(thisIsACallbackFuntions)
```

Methods

In Python and JavaScript, almost everything is an object. These objects are created from built-in classes — for example, when text is stored in a variable, that variable becomes an instance of the `String` class.

These classes define functions that operate on their instances, executed using dot notation — for example, `someArray.push()` or `someText.toUpperCase()`. These functions are called **methods**.

```
let someText = 'hello this is sora from voidseraphim';

console.log(someText.toUpperCase());
```

JavaScript — Default Parameter Values

```
const discount_price = (parameter=0) => {
    return parameter*0.8
}

console.log(discount_price(100));
```

This function works correctly when an argument is passed. But what happens if it is called with no arguments at all?

```
console.log(discount_price());
```

Without a default value, `parameter` would be `undefined`, and `undefined * 0.8` evaluates to `NaN` (Not a Number). To avoid this, the parameter is given a default value directly in the function definition — `parameter=0`. This way, whenever the function is called without an argument, `parameter` falls back to `0`, and the function returns `0` instead of `NaN`.

JavaScript — The `this` Keyword in Objects

`this` is a keyword that always refers to something called the **context** — the object that the current method or property belongs to. When using `this`, it effectively means “I am working with the object that this method or property is part of.” It allows access to the object’s data from inside its own methods.

```
const obj = {
    name: 'sora voidSeraphim',
    hobbies: [
```

```

        'coding',
        'dayDreaming',
        'gaming',
        'hacking'
    ],
    calculateAge: function () {
        return 23 + this.hobbies.length;    // this = obj
    }
};

console.log(obj.name);

```

Why can `hobbies` not be accessed directly inside `calculateAge`, even though it is defined in the same object? To access a property of an object from within itself, the `this` keyword must be used — hence `this.hobbies.length`. Note that `this` cannot be used inside arrow functions in the same way.

```
console.log(obj.calculateAge());
```

In the example above, `calculateAge` is a **method** — a function defined as a property of the object.

this in JavaScript vs self in Python

`this` is similar to `self` in Python — both refer to the current object. However, `self` always refers to the instance that called a method or property, while in JavaScript, when a method is called like `someObject.method()`, `this` refers to that specific object — its value depends on how the function is called, not where it is defined.

In Python, methods are defined inside classes, and objects are created from those classes. In JavaScript, the object concept is a bit different — methods can be defined directly as object properties. When a function is defined as a property of an object, it is called a **method**.

The behavior of `this` becomes more involved when working with classes, where it works similarly to `self` in Python but in a more dynamic way.

JavaScript — Destructuring

```

const user = {
    name: 'sora',
    age: 23,
    hobbies: [
        'hacking',
        'coding',
        'sleeping',
    ],
    city: 'Tehran'
}

```



```
console.log(user);  
console.log(user.name);
```

Sometimes a value needs to be extracted from an object and used as its own variable:

```
// const name = user.name;  
// console.log(name);
```

Multiple values — like name, age, and city — can be extracted at once using **destructuring**:

```
const { name, age, city } = user;  
  
console.log(name, age, city);
```

Each extracted value becomes its own variable, accessible directly by name.

Destructured variables can also be given aliases and default values:

```
const { name: username, city: hometown, age: years, country = 'some island' } = user;  
console.log('with aliases and default values:');  
console.log(username, hometown, years, country);
```

This is especially useful in functions that take an object as an argument:

```
// function printUser(user) {  
//   console.log(`Name: ${user.name}, Age: ${user.age}, City: ${user.city}`);  
// };  
  
// function printUser({ name, age, city }) {  
//   console.log(`Name: ${name}, Age: ${age}, City: ${city}`);  
// };
```

Destructuring also works with arrays:

```
const numbers = [5, 10, 15, 20, 25]  
  
const [ a, b ] = numbers;  
  
console.log(a, b)
```

JavaScript — Spread and Rest Operators

Spread Operator (...)

```
const number1 = [5, 10, 15];  
const number2 = [30, 40, 60];
```

These three dots are called the **spread operator**. ... means “open it up” — it spreads the elements or properties of an array or object into a new structure:

```
const newNumbers = [...number1, ...number2];  
console.log(newNumbers);
```

New members can also be added at any position:

```
const hobbies = ['hacking', 'coding'];
const updatedHobbies = ['sleeping', ...hobbies, 'smoking']; // thats a bad hobby btw
console.log(updatedHobbies);
```

Common uses include quickly copying arrays or objects, and merging arrays or objects together.

The same ... syntax can also act like `*args` in Python — but when used in a function parameter, it is called the **rest operator** instead of the spread operator:

```
function sUm (...nums) {
  let total = 0;
  for (let num of nums) {
    total += num;
  };

  return total;
};

console.log(sUm(1,2,3,4,5,6));
```

Primitives and Reference Values

Primitives are basic values, such as 'hello im here' or 23. Their actual value is stored directly, not a reference to it. The primitive data types in JavaScript are: strings (e.g. "hello im sora"), numbers (e.g. 23), booleans (true/false), null, undefined, symbol, and bigint.

```
let a = 10;
let b = 10;
console.log( a === b );
```

This returns true, since a and b are primitive values and === compares their actual values directly.

Reference Values

Reference values are data that point to an address in memory, rather than holding the value itself. Objects, arrays, and functions are all reference values.

```
console.log( [1,2,3] == [1,2,3] );
console.log( [1,2,3] === [1,2,3] );
```

Both of these return false. When an object, array, or function is created, JavaScript stores it somewhere in memory, and the variable holding it is just a reference to that memory address. Two arrays can look exactly the same in terms of content, but since they occupy different locations in memory, they are not considered equal. Both == and === compare the memory address for reference values, not just their contents.

undefined vs null

This section covers the difference between something that is not defined and something that is defined but empty.

```
let number;  
console.log(number);
```

`number` is declared but not given a value, so logging it returns `undefined`. `undefined` means a variable exists but has no value assigned to it.

```
const data = {  
  name: 'sora',  
  city: 'tehran',  
  age: 23,  
  family: null  
};
```

```
console.log(data.fam);
```

`data.fam` does not exist as a property on `data` (the property is named `family`, not `fam`), so this also returns `undefined`.

```
console.log(data.family);
```

This returns `null`. Sometimes a property needs to represent something that is intentionally empty, but still defined and present on the object — this is what `null` is used for.

Practical Use Case

```
let jobApplicationForm = {  
  id: 12,  
  name: null,  
  lastName: null,  
  age: null  
};
```

```
console.log(jobApplicationForm)
```

This pattern is useful for representing a form where all expected fields are defined upfront with `null` as a placeholder, indicating that the fields exist but have not yet been filled in by the user.

Short-Circuiting with `&&` and `||`

JavaScript's `&&` (and) and `||` (or) operators are already familiar from conditional statements.

```
const price = 1000;  
  
if (price > 500 && price < 2000) {
```

```
    console.log('yes!')
}
```

With `&&`, every condition must be `true` for the overall expression to be `true`.

Short-Circuit Evaluation

```
price > 500 && console.log('hello');
```

This syntax is similar to short-circuiting in Bash. With `&&`, if the first part is `true`, the second part is executed. Here, `price > 500` is `true`, so `console.log('hello')` runs.

```
price > 2000 || console.log('hiiii, idk whats going on') || price < 2000 || console.log('where
```

With `||`, as soon as one part evaluates to `true`, the rest of the expression is not evaluated. Tracing through this line: `price > 2000` is `false` (1000 is not greater than 2000), so evaluation continues to `console.log('hiiii, idk whats going on')`, which runs and prints the message. However, `console.log()` always returns `undefined`, which is `falsy`, so evaluation continues to `price < 2000`, which is `true` (1000 is less than 2000). Since this is `true`, the `||` chain stops here, and the final `console.log('where am i?')` never executes.

Asynchronous JavaScript

What Does “Asynchronous” Mean?

JavaScript is an interpreted language, meaning code is read and executed line by line. If a block of code takes a long time to complete, it can delay the rest of the program’s execution.

For example, imagine downloading an image that takes 3 seconds. If JavaScript were not asynchronous, the browser would be locked for those 3 seconds, and nothing else could happen during that time.

“Asynchronous” means that operations which take a long time will not block the execution of other code. There are several ways to implement asynchronous behavior in JavaScript: callback functions, promises, and `async/await`.

A Real-World Analogy

In normal (synchronous) execution, commands run one after another, and order matters:

1. Say hello
2. Go to the store, search for something, and buy it
3. Make a coffee
4. Get some rest

In an asynchronous approach, some of these tasks can happen at the same time. For example, with durations of 1 minute, 20 minutes, 5 minutes, and 1 hour respectively, running them one after another would take 86 minutes in total.

Now imagine a group of three or four people doing this together:

- Everyone says hi

- One person goes to the store (async)
- Another person makes coffee (async)
- Everyone rests together afterward (async)

Two tasks started at the same time — this is asynchronous behavior. However, making coffee takes less time than shopping, so the coffee will be ready before the person returns from the store. Should everyone have coffee and start resting without that person and the items they were buying?

This is where **await** becomes useful. **await** means waiting for the result of an asynchronous task before continuing. If there is a function that coordinates all of these operations, **await** tells it: before this function is considered done, wait for the results of the asynchronous tasks it started.

Why Is This Useful?

Consider a form submission where an email and an SMS need to be sent to the user, and the user should see a success message without waiting for both to complete. Sending the email and the SMS can happen asynchronously in the background, while the user immediately sees a “successfully sent” message, with the actual sending completed on the backend afterward.

Working with Promises

```
<p>
  <button class="btn">GetUsers</button>
</p>

const btnEl = document.querySelector('.btn');
const clickHandler = () => {
  fetch('http://localhost:8000/users.json', {
    method: 'GET'
  }).then(res => {
    console.log(res)
    return res.json()
  }).then(data => {
    console.log(data.data)
  })

  console.log(1234);
}
btnEl.addEventListener('click', clickHandler)
```

```
// console.log(fetch('http://localhost:8000/users.json'));
```

This commented-out line shows that logging the result of **fetch(...)** directly returns a Promise in a pending state, rather than the actual data.

Why Does `fetch()` Return a Promise?

When a request is sent with `fetch()`, it may take time to receive a response. During this time, the program should not be locked and must be able to perform other tasks.

A **Promise** is an object that JavaScript provides to represent this situation. It essentially says: “I cannot give you the final result right now, but I promise to give it to you once it’s ready.”

When `fetch('someAddress')` is logged immediately, it returns **Promise {<pending>}**, meaning the request has been sent to the server and is waiting for a response.

`.then()` specifies a function to run once the promise returned by `fetch()` is resolved. Inside the first `.then()`, logging `res` shows that it is no longer pending — its state has changed to **fulfilled**, and it contains the response information. `res.json()` is then called and returned.

`res.json()` itself returns another promise, since parsing the response body as JSON may also take time. The second `.then()` runs once this second promise (`res.json()`) resolves, and at that point, `data` holds the final parsed result.

Execution Order

```
console.log(1234);
```

This line actually executes before the contents of `.then()`. Once `fetch()` is called, it immediately returns a pending promise, and the rest of the synchronous code continues running right away. The code inside `.then()` only runs later, once the promise’s pending state resolves.

Async/Await Syntax

```
<button class="btn">klik me</button>
```

```
const btnEl = document.querySelector('.btn')
```

```
const clickHandler = async () => {
  try {
    const response = fetch('http://localhost:8000/users.json');
    console.log(response);

    const res = await fetch('http://localhost:8000/users.json');
    console.log(res);

    if (!res.ok) {
      console.log('there was a problem with fetching data.');
      return;
    }

    const data = await res.json();
    console.log(data.data);
  }
}
```

```

    } catch (error) {
      console.log('you had an error:', error)
    };
  }

  btnEl.addEventListener('click', clickHandler)

```

How to Use `async` and `await`

To use `await`, the `async` keyword must first be placed before the function definition, so that JavaScript understands the function is meant to perform asynchronous operations.

```

const response = fetch('http://localhost:8000/users.json');
console.log(response);

```

As seen previously, logging the result of `fetch()` directly returns a pending promise. If the actual result is needed for further operations, the `await` keyword must be used before it.

```

const res = await fetch('http://localhost:8000/users.json');
console.log(res);

```

`await` only works inside `async` functions. When `await` is placed before a promise, it pauses the function's execution until the promise resolves, and then assigns the resolved value to the variable. In other words, `await` means “wait for this to return something.”

```

if (!res.ok) {
  console.log('there was a problem with fetching data.');
```

```

  return;
}

const data = await res.json();
console.log(data.data);

```

After confirming the response was successful (`res.ok`), `res.json()` is also awaited, since it returns its own promise.

Error Handling

```

try {
  // ...
} catch (error) {
  console.log('you had an error:', error)
};

```

Errors in `async/await` syntax are handled using `try/catch`, similar to `try/except` in Python. Any error thrown inside the `try` block — including a failed `fetch()` call — is caught in the `catch` block.

It is also worth noting that when sending data with `fetch()` (e.g. a POST request) and the response itself is not needed, awaiting the fetch is not strictly required. However, it is still common practice to check whether the request succeeded.

To try this yourself, save the code into a file named `practice.html` alongside `users.json`, start a local HTTP server (for example with `python3 -m http.server 8000`), and open the page through `http://localhost:8000/`. Click the button and check the developer tools console for the output.

Practical Project: Async/Await with UI Feedback

This example builds on the previous syntax by adding error handling with `try/catch/finally`, and updating the page to show loading and result states to the user.

```
<button class="btn">Get Users Data</button>
<p id="userInfo"></p>
<p id="loadingMessage" style="display: none;">Loading...</p>

const btnEl = document.querySelector('.btn');
const userInfoEl = document.getElementById('userInfo');
const loadingMessageEl = document.getElementById('loadingMessage');

const clickHandler = async () => {
  loadingMessageEl.style.display = 'block';
  userInfoEl.textContent = '';

  try {
    const response = await fetch('http://localhost:8000/users.json');
    console.log('Fetch response object:', response);

    if (!response.ok) {
      throw new Error(`HTTP error! status: ${response.status}`);
    }

    const data = await response.json();
    console.log('Parsed JSON data:', data);

    if (data && data.data) {
      userInfoEl.textContent = JSON.stringify(data.data, null, 2);
    } else {
      userInfoEl.textContent = 'No user data found in the response.';
    }
  } catch (error) {
    console.error('An error occurred:', error);
    userInfoEl.textContent = `Failed to load data. Error: ${error.message}`;
  } finally {
    loadingMessageEl.style.display = 'none';
  }
}
```



```
};  
  
btnEl.addEventListener('click', clickHandler);
```

When the button is clicked, the loading message is shown immediately, and any previous result text is cleared. Inside the `try` block, the request is awaited, and `response.ok` is checked. If the response indicates an error status, an `Error` object is explicitly thrown using `throw new Error(...)`, with the HTTP status code included in the message. This throw is caught by the `catch` block below, just like any other error.

If the request succeeds, the JSON body is parsed and awaited, and the result is checked for a `data` property before being displayed on the page using `JSON.stringify(data.data, null, 2)`, which formats the JSON with two-space indentation for readability.

The `catch` block logs the error and displays a failure message to the user, including the error's message text.

The `finally` block always runs, regardless of whether the `try` block succeeded or the `catch` block was triggered — this is used here to hide the loading message in either case.

Review: `for...of` and `while` Loops

This part reviews the `for...of` loop and the `while` loop, both of which were introduced previously.

```
const numbers = [1,2,3,4,5,6,7,8];  
  
for (let num of numbers) {  
  console.log(num);  
};
```

`for...of` works much like `forEach` — it iterates directly over the values of an array.

```
for (let num = 0; num < numbers.length; num++) {  
  console.log(numbers[num] + 1)  
};
```

A normal `for` loop is more manual, requiring an index variable, a condition, and an increment step.

`while` Loop

```
let counter = 0;  
while (counter < 10) {  
  console.log('hello world!');  
  counter++  
};
```

A `while` loop continues running as long as its condition remains true.

Iterating Over a String

```
const text = 'hello im sora voidSeraphim';

// text.forEach(char => {
//     console.log(char)
// });
```

This commented-out code would throw an error, since `.forEach()` only works on arrays, maps, and sets — not on strings. Maps and sets will be covered in a later lesson.

```
for (char of text) {
    console.log(char);
}
```

`for...of` works on strings as well, since strings are iterable. This loop iterates over each character in `text` and logs it individually.

Sets and Maps

JavaScript has two other useful data types: `Set` and `Map`.

Set

A `Set` is similar to a set in Python — it accepts only unique values, and duplicates are automatically removed.

```
const mySet = new Set();
console.log(mySet);
```

An empty set is created with `new Set()`.

```
const initialArray = [1, 2, 2, 3, 4, 4, 4, 5];
const someSet = new Set(initialArray);
console.log(someSet);
```

A set can also be created from an array. Duplicate values in the array are automatically removed.

```
mySet.add(6);
mySet.add(1);
console.log(mySet);
```

`.add()` adds a value to a set.

```
console.log(mySet.has(6));
```

`.has()` checks whether a value exists in the set.

```
mySet.delete(1);
console.log(mySet);
```

`.delete()` removes a value from the set.

```
someSet.forEach(val => {  
    console.log(val);  
})
```

`forEach` works on sets, iterating over each value.

```
mySet.clear();  
console.log(mySet);
```

`.clear()` removes all values from the set.

Map

A **Map** is similar to an object in JavaScript, but with a key difference: keys in a **Map** can be almost any data type — a string, a number, a boolean, an object, or even a function. A **Map** stores key-value pairs, mapping each key to a value.

```
const myMap = new Map();  
console.log(myMap);
```

An empty map is created with `new Map()`.

```
const sora = [  
    ['name', 'sora'],  
    ['age', 23],  
    ['job', 'pentester']  
];
```

```
const newMap = new Map(sora);  
console.log(newMap);
```

A map can be created from an array of key-value pairs, where each pair is itself an array of two elements.

```
newMap.set('city', 'tehran');  
newMap.set('job', 'hacker');  
console.log(newMap);
```

`.set()` adds a new key-value pair to the map, or updates the value if the key already exists.

```
console.log(newMap.get('city'));
```

`.get()` retrieves the value associated with a given key.

```
console.log(newMap.has('job'));
```

`.has()` checks whether a key exists in the map.

Iterating Over a Map

```
for (let [key, value] of newMap) {  
    console.log(`key: ${key}, value: ${value}`)  
}
```

A `for...of` loop with array destructuring (`[key, value]`) can be used to iterate over each key-value pair in the map directly.

```
for (let key of newMap.keys()) {
  console.log(key, newMap.get(key))
}
```

Alternatively, `.keys()` returns an iterable of all keys, and `.get(key)` retrieves the corresponding value for each one.

```
console.log('with forEach:');

newMap.forEach((value, key) => {
  console.log(key, value)
});
```

`forEach` also works on maps. Note the parameter order: the callback receives (`value`, `key`), with the value first and the key second — the opposite of what might be intuitively expected.

```
newMap.clear();
console.log(newMap);
```

`.clear()` removes all key-value pairs from the map.

Practical Project: JavaScript Modules

Large projects are usually not written in a single file. Code can be organized into separate files (modules), where functions and variables defined in one file can be exported and then imported into another. This is also how external libraries from the open-source community are typically used.

HTML Setup

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Practice Page</title>
  <style>
  </style>
  <script>
    // if script is here it will be loaded sooner
  </script>
</head>
<body>
  <script src="Importing.js" type="module"></script>
</body>
</html>
```

The `type="module"` attribute on the `<script>` tag is required to use `import/export` syntax in the browser.

Exporting.js

```
export const convertCurrency = (usdAmount) => {
  return usdAmount*0.9; // to euro
};
function sayHello (name) {
  console.log(`Hello ${name}`);
};
function secret () {
  console.log('this is a secret that should not be accessible from outside.');
```

};

```
export const name = 'soraVoidSeraphim';
const value = 12345;

export default {
  greet: sayHello,
  variable: value
};
```

The `export` keyword is placed before any function or variable that should be accessible from other files. In this file, `convertCurrency` and `name` are exported individually, while `sayHello`, `secret`, and `value` are not directly exported and remain private to this module — unless they are included in the default export.

Default Export

```
export default {
  greet: sayHello,
  variable: value
};
```

A module can also have a `default` export, which is what gets imported when the entire module is imported as a single value, without naming individual exports. A module can have only one default export. This is also useful for grouping project-wide variables or functions into a single file and exporting them together, making the main code more readable.

Importing.js

```
import {
  convertCurrency,
  name
} from './Exporting.js'

import * as myExports from './Exporting.js';
```

```
import defaults from './Exporting.js'

const newVal = convertCurrency(100);
console.log(newVal);
console.log(myExports.name)
alert(name);
console.log(defaults.variable)
defaults.greet('dora')
```

There are several ways to import from a module:

The first form, `import { convertCurrency, name } from './Exporting.js'`, imports specific named exports directly, making them available by their original names.

The second form, `import * as myExports from './Exporting.js'`, imports the entire module as a single object, and individual exports are accessed as properties of that object — similar to importing a module with `import module` in Python and accessing its members with dot notation. Importing everything this way is useful when a file exports many functions or variables and only some of them are needed, similar to “bringing the whole fridge over just to grab some water from it.”

The third form, `import defaults from './Exporting.js'`, imports the module’s default export. `defaults` here is simply the name chosen for it in this file — it could be any valid identifier.

In the code, `convertCurrency(100)` converts 100 USD to euros. `myExports.name` and the directly imported `name` both refer to the same exported value. `defaults.variable` accesses the `value` property from the default export object, and `defaults.greet('dora')` calls the `sayHello` function (renamed to `greet` in the default export) with the argument `'dora'`.

To try this yourself, save `Importing.html`, `Importing.js`, and `Exporting.js` in the same location, start a local HTTP server (for example with `python3 -m http.server 8000`), and open `Importing.html` through `http://localhost:8000/`. Note that opening the HTML file directly from the filesystem (without a server) will cause module imports to fail due to browser security restrictions on the `file://` protocol.

Switch/Case Statements

```
let age = 23;

if (age == 18) {
  console.log('youre not ready')
} else if (age == 20) {
  console.log('not yet')
} else if (age == 23) {
  console.log('you still have to try harder')
} else {
  console.log('do your best!')
}
```

This `if/else if/else` structure was already covered in the conditions lesson. Since `age` is

23, the third condition matches and logs 'you still have to try harder'.

This kind of structure, where a single variable is compared against multiple fixed values, can also be written using a different syntax called `switch/case`.

```
let browser = 'Chrome';
console.log(navigator.userAgent)
```

`navigator.userAgent` returns the user agent string of the browser running the script, which can be useful for fingerprinting or identifying the client.

```
switch(browser){
  case 'Firefox': {
    console.log('this is firefox');
    break;
  }
  case 'Chrome': {
    console.log('this is chrome')
    break;
  }
  case 'Edge': {
    console.log('this is edge')
    break;
  }
  default: {
    console.log('you are using a browser')
  }
}
```

`switch` evaluates the value of `browser` and compares it against each `case`. When a match is found, the code inside that block runs. `break` stops execution from continuing into the next case (without it, execution would “fall through” to the following case regardless of whether it matches). `default` runs if no case matches. Since `browser` is 'Chrome', the output is 'this is chrome'.

The Window Object

One of the most important objects in JavaScript is `window`. It is the global, top-level object — every other object can be reached as a property of `window`. `alert()`, for example, is technically `window.alert()`, but since `window` is the global object, it can be called as just `alert()`.

In other words, every function or variable defined in the global scope becomes a property of the `window` object. Since `window` is the global object, code running in the global scope does not need to prefix anything with `window..`

```
console.log(window);
console.log(window.document);
console.log(window.console);
```

`document` and `console` are both children of the global `window` object.

URL-Related Properties

```
console.log(window.location);  
console.log(window.location.protocol);  
console.log(window.location.pathname);  
console.log(window.location.href);  
console.log(window.location.search);
```

`window.location` provides information about the current page's URL. `window.location.protocol` returns the scheme used (`http:`, `https:`, `file:`, `ftp:`, etc.). `window.location.pathname` returns the path portion of the URL. `window.location.href` returns the full URL. `window.location.search` returns the query string portion of the URL (everything after `?`).

Navigating with `location.href`

```
window.location.href = 'javascript:alert(123)'
```

Setting `window.location.href` navigates the browser to the given value. Here, the value is a `javascript:` URI, which causes the browser to execute the given JavaScript code (`alert(123)`) immediately, rather than performing a normal navigation.

This is relevant from a security perspective: `javascript:` URIs are a classic vector for cross-site scripting (XSS) when user-controllable input is assigned to `location.href`, `<a href>`, or similar navigation-related properties without proper sanitization.

This line also affects the rest of the script: because the `javascript:` URI executes and effectively replaces the current page content, any code placed after this line — such as the `URLSearchParams` example below — will not run in practice, since execution does not continue normally past this point.

Reading Query String Parameters

```
const params = new URLSearchParams(window.location.search);  
console.log(params.get('name'))
```

`URLSearchParams` converts the query string into an object that can be queried for individual parameters. `params.get('name')` retrieves the value of the `name` parameter from the URL's query string, if present.

To try this yourself, save the code into a file named `practice.html`, open it in a browser with a query string such as `practice.html?name=sora`, and check the developer tools console for the output. Be aware that the `window.location.href` line will trigger an `alert(123)` popup and interrupt further execution, as noted above.

Math Functions

JavaScript provides several built-in functions for working with numbers, available through the `Math` object.

Rounding Down

```
console.log(Math.floor(5.6));  
console.log(Math.floor(5.2));
```

`Math.floor()` rounds a number down to the nearest integer, regardless of the decimal value.

Rounding Up

```
console.log(Math.ceil(5.1))
```

`Math.ceil()` rounds a number up to the nearest integer.

Rounding to the Nearest Integer

```
console.log(Math.round(5.7));
```

`Math.round()` rounds a number to the nearest integer, following standard rounding rules.

Power

```
console.log(Math.pow(4,2));
```

`Math.pow(base, exponent)` raises a number to a given power. Here, 4 is raised to the power of 2.

Square Root

```
console.log(Math.sqrt(36));
```

`Math.sqrt()` returns the square root of a number.

Absolute Value

```
console.log(Math.abs(-11));
```

`Math.abs()` returns the absolute value of a number, removing its sign.

Random Numbers

```
console.log(Math.random());  
console.log(Math.random()*10);
```

`Math.random()` returns a random floating-point number between 0 (inclusive) and 1 (exclusive). Multiplying the result by 10 produces a random number between 0 (inclusive) and 10 (exclusive).

Minimum and Maximum

```
console.log(Math.min(4, 9, 1, 7));  
console.log(Math.max(4, 9, 1, 7));
```

`Math.min()` returns the smallest of the given numbers, and `Math.max()` returns the largest. Both can take any number of arguments.

Date and Time

JavaScript is an object-oriented language — like Python, almost everything in JavaScript is an object, and these objects are created from classes. An object created from a class is called an instance of that class.

JavaScript has a built-in `Date` class, used to create date objects.

```
const date = new Date(2050,10,15,10,10,10,500);  
console.log(date);
```

The `new` keyword creates a new instance of a class. `Date` takes up to seven numeric arguments: `Date(year, monthIndex, day, hours, minutes, seconds, milliseconds)`. Note that `monthIndex` is zero-based, so `10` represents November. This creates a date object representing November 15, 2050, at 10:10:10.500.

Getting Date Components

```
console.log(date.getFullYear());  
console.log(date.getMonth());  
console.log(date.getDay());  
console.log(date.getHours());  
console.log(date.getMinutes());  
console.log(date.getSeconds());  
  
console.log(date.getTime());
```

`.getFullYear()` returns the year, `.getMonth()` returns the zero-based month index (so `10` for November), and `.getDay()` returns the day of the week (`0` for Sunday through `6` for Saturday) — not the day of the month. `.getHours()`, `.getMinutes()`, and `.getSeconds()` return the corresponding time components. `.getTime()` returns the date as a timestamp in milliseconds since January 1, 1970.

Setting Date Components

```
console.log(date.setFullYear(2026));  
console.log(date.toLocaleDateString());
```

`.setFullYear()` changes the year of the date object and returns the new timestamp in milliseconds. `.toLocaleDateString()` returns the date formatted according to the local time zone and locale settings.

Getting the Current Date and Time

```
function get_time_now() {
  const now = new Date();
  const year = now.getFullYear();
  const month = now.getMonth() + 1; // month starts from 0
  const day = now.getDate();
  const dayOfWeek = now.getDay(); // 0 for Sunday, 1 for Monday, etc.
  const hours = now.getHours();
  const minutes = now.getMinutes();
  const seconds = now.getSeconds();

  console.log(`${hours}:${minutes}:${seconds}`);
};

setInterval(() => {
  get_time_now();
}, 1000);

// console.log(now.toLocaleDateString());
```

This commented-out line shows an alternative way to log the current date in a locale-formatted string, instead of building it manually from individual components.

`new Date()` with no arguments creates a date object representing the current date and time. Since `.getMonth()` is zero-based, 1 is added to get the conventional month number. `.getDate()` returns the day of the month, while `.getDay()` (used above) returns the day of the week — these are easy to confuse. `setInterval()` calls `get_time_now()` every second, logging the current time in HH:MM:SS format.

Object-Oriented Programming: From Objects to Classes

The Problem: Repeating Objects

Almost everything in JavaScript is an object, and these objects belong to different types. These objects are created from classes — for example, a `Date` object (covered previously) is an instance of the `Date` class, strings are instances of the `String` class, numbers are instances of the `Number` class, and so on.

Plain JavaScript objects, as covered earlier, can already group properties and methods together:

```
const apartment1 = {
  Meters: 50,
  numberOfBedrooms: 3,
  isBig: function () {
    if (this.Meters > 80) {
      return true;
    } else {
      return false;
    }
  }
};
```

```

    };
  },
  calPrice: function () {
    return this.Meters * this.numberOfBedrooms
  }
};

const apartment2 = {
  Meters: 90,
  numberOfBedrooms: 5,
  isBig: function () {
    if (this.Meters > 80) {
      return true;
    } else {
      return false;
    }
  },
  calPrice: function () {
    return this.Meters * this.numberOfBedrooms
  }
};

console.log(apartment1);
console.log(apartment1.Meters);
console.log(apartment1.isBig());
console.log(apartment1.calPrice());

console.log(apartment2);
console.log(apartment2.Meters);
console.log(apartment2.isBig());
console.log(apartment2.calPrice());

```

These two apartments share the same properties and methods, only with different values. Creating many more apartments this way — say, 100 of them — would require copying and pasting the same structure repeatedly, producing a large amount of duplicated code and reducing readability. The solution to this is working with classes.

Classes and Inheritance

Classes solve this problem: instead of manually building each object, a class can be defined once, describing the shared properties and methods, and individual objects (instances) can then be created directly from it — similar to how a `Date` object is created by passing values like year and month, and the `Date` class already knows what to do with them.

Defining a Class

```

class Apartment {
  Type = 'Apartment';

```

```

    constructor(meters, rooms) {
        if (meters < 10) {
            alert('too small to be an apartment');
            return
        }
        this.Meters = meters;
        this.numberOfBedrooms = rooms;
    };

    isBig () {
        if (this.Meters > 80) {
            return true;
        } else {
            return false;
        }
    };

    calPrice () {
        return this.Meters * this.numberOfBedrooms
    };
};

```

A property declared outside the constructor, such as `Type = 'Apartment'`, is given to every instance of the class automatically. The `constructor` is a special function that runs the moment a new instance is created, similar to `__init__` in Python. Inside it, `this` refers to the specific instance being created — not all instances of the class collectively. Methods such as `isBig()` and `calPrice()` are defined without the `function` keyword.

Creating Instances

```

const apartment1 = new Apartment(50,2);
const apartment2 = new Apartment(100,3);
const apartment3 = new Apartment(5,3);

console.log(apartment1);
console.log(apartment1.Meters);
console.log(apartment1.isBig());
console.log(apartment1.calPrice());

console.log(apartment2);
console.log(apartment2.Meters);
console.log(apartment2.isBig());
console.log(apartment2.calPrice());

console.log(apartment3);
console.log(apartment3.Meters);
console.log(apartment3.isBig());
console.log(apartment3.calPrice());

```

The `new` keyword creates a new instance of a class, passing the constructor's arguments. `apartment1` is created with 50 square meters and 2 bedrooms, `apartment2` with 100 square meters and 3 bedrooms, and `apartment3` with 5 square meters and 3 bedrooms — the last of which triggers the `alert('too small to be an apartment')` check inside the constructor, since 5 is less than 10.

Inheritance

A new class, `PentHouse`, can be created that inherits all of `Apartment`'s properties and methods, while adding its own. Class inheritance in JavaScript is similar to Python, but with different syntax — the `extends` keyword is used.

```
class PentHouse extends Apartment {
  constructor (meters, rooms, hasBalkoni) {
    super(meters, rooms);
    this.hasBalkoni = hasBalkoni;
    this.Type = 'PentHouse';
  };

  displayBalconyInfo () {
    if (this.hasBalkoni == true) {
      console.log('this pent has blakoni');
    } else {
      console.log('this pent does not have blakoni');
    }
  };

  isBig () {
    return super.isBig();
  };

  calPrice () {
    if (this.hasBalkoni) {
      return super.calPrice() * 2;
    } else {
      return super.calPrice();
    }
  };
};
```

Instead of repeating the parent class's constructor logic, `super(meters, rooms)` calls the constructor of `Apartment` directly. After that, `PentHouse` adds its own `hasBalkoni` property and overwrites `Type` to `'PentHouse'`.

`displayBalconyInfo()` is a new method specific to `PentHouse`. The `isBig()` method is redefined to simply call the parent class's version using `super.isBig()`. `calPrice()` is also redefined: if the penthouse has a balcony, the price calculated by the parent class's `calPrice()` (via `super.calPrice()`) is doubled; otherwise, the parent's result is returned unchanged.

Using the Subclass

```
const pent1 = new PentHouse(120, 4, true);
console.log(pent1);
console.log(pent1.Meters);
pent1.displayBalconyInfo()
console.log(pent1.isBig());
console.log(pent1.calPrice());
```

pent1 is created with 120 square meters, 4 bedrooms, and hasBalkoni set to true. displayBalconyInfo() logs that the penthouse has a balcony. isBig() returns true, since 120 is greater than 80. calPrice() returns $120 * 4 * 2 = 960$, since the balcony doubles the base price calculated by the parent class.

Prototypes

Classes and constructors have already been covered as ways to create objects. There is another underlying mechanism in JavaScript for creating objects: prototypes.

Every function in JavaScript has a hidden property called **prototype**, which is itself an object. When a function is used as a constructor (whether a class constructor or a regular function), JavaScript automatically creates a prototype object for it. Any methods added to `constructorFunction.prototype` — such as `isBig` and `calPrice` in the example below — become accessible to every instance created from that constructor.

Before ES6 introduced the `class` syntax, regular functions were used as constructors:

```
function Apartment (Meters, numberOfBedrooms) {
  this.Meters = Meters;
  this.numberOfBedrooms = numberOfBedrooms;
};
Apartment.prototype.isBig = function () {
  return this.Meters > 80 ? true : false;
};
Apartment.prototype.calPrice = function () {
  return this.Meters * this.numberOfBedrooms;
}
```

When this function is called with `new`, JavaScript creates a new empty object. Inside the function, `this` refers to that new object, and the code adds the `Meters` and `numberOfBedrooms` properties to it. The resulting object is then assigned to a variable — this is exactly what `class` constructors do under the hood.

In other words, a constructor is like a company that builds products (objects), and the prototype is like a shared toolbox that every product built by that company can use.

Using a Prototype-Based Constructor

```
const apartment1 = new Apartment(50,2);
console.log(apartment1);
```

```
console.log(apartment1.isBig());  
console.log(apartment1.calPrice());
```

Usage is much like classes. When `apartment1.isBig()` is called, JavaScript first looks for an `isBig` method on the `apartment1` object itself. If it is not found there, it searches the object's prototype — in this case, `Apartment.prototype`, since `apartment1` was created from `Apartment`.

Built-In Prototypes

`Number`, `String`, `Object`, `Boolean`, and `Function` are all built-in classes with their own constructors, and creating an array, a string, or any other built-in type uses these constructors behind the scenes. Each of these types has its own prototype object — for example, `Array.prototype` contains methods such as `push`, `pop`, and `slice`, which are accessible on every array instance through the prototype chain.

Promises

Introduction to Promises

Promises are one of the more complex concepts in JavaScript. Some operations take time to complete — the `fetch` API is one example, but this can apply to any function that performs a time-consuming task.

Now that `async/await` has been covered, promises can be understood as exactly what the name suggests: JavaScript gives a promise that says “I will perform this operation and return the result once it's done, while other code keeps running in the meantime without waiting.”

There are two ways to work with promises: using one that already exists, such as `fetch` combined with `.then()`, or creating a new promise directly with `new Promise()`.

```
const p = new Promise((resolve, reject) => {  
  const ticketPrice = 200;  
  const budget = 30;  
  if (ticketPrice <= budget) {  
    resolve('Success');  
  } else {  
    reject('Not enough promotion');  
  }  
});  
  
console.log(p);
```

A promise takes a callback function as its argument, and this callback receives two parameters, which are themselves functions: `resolve` and `reject`. Calling one of these tells JavaScript whether the promise completed successfully or failed, which determines how the result is handled afterward. The names `resolve` and `reject` are fixed and cannot be changed.

Note: in this example, `ticketPrice` is 200 and `budget` is 30, so the condition `ticketPrice <= budget` (`200 <= 30`) is `false`. This means the promise will actually call `reject('Not`

enough promotion'), not resolve('Success').

```
p.then(result => {
  console.log(result);
}).catch(reason => {
  console.log(reason);
});
```

`.then()` handles the case where the promise resolves successfully, and `.catch()` handles the case where it is rejected.

Promises with Async/Await

```
const p = new Promise((resolve, reject) => {
  const ticketPrice = 20;
  const budget = 30;
  if (ticketPrice <= budget) {
    resolve('Success');
  } else {
    reject('Not enough promotion');
  }
});
```

```
const getResult = async () => {
  try {
    const res = await p;
    console.log(res);
  } catch (reason) {
    console.log(reason);
  }
};
```

```
getResult();
```

To use `await` on a promise, the code must be inside an `async` function. Here, `ticketPrice` (20) is less than or equal to `budget` (30), so the promise resolves with 'Success', `await p` returns 'Success', and it is logged inside the `try` block. If the promise had been rejected instead, the rejection reason would be caught and logged in the `catch` block.

Why and Where to Use Async Code

```
const name = 'sora VoidSeraphim';
const a = 123;
const isactive = true;
```

A program normally executes line by line, in order. If some operation takes a long time, this would freeze the rest of the program. JavaScript is synchronous by default, but constructs like promises, `async/await`, and `setTimeout` allow asynchronous behavior.

```
console.log(fetch('http://localhost:8000/users.json'));
```

`fetch` is an example of a promise — it does not return its result immediately, but only once the request completes. If fetching data from a server takes around 30 seconds depending on connection speed, the application should not be frozen for that entire time. This is exactly where asynchronous code is needed.

`fetch` is itself a promise and does not strictly require `async/await`, but using it is recommended for readability.

```
const getUsers = async () => {
  try {
    const res = await fetch('http://localhost:8000/users.jsoni');
    console.log(res);

    if (!res.ok) {
      Promise.reject('Failed to fetch.');
```

```
      return;
    }

    const data = await res.json();
    console.log(data.data);
  } catch (error) {
    console.log('You had an error:', error);
  }
};

getUsers();
```

```
// console.log('failed to fetch');
```

This commented-out line shows a simpler alternative for handling a failed request: just logging a message directly, instead of explicitly rejecting a promise with `Promise.reject(...)`.

Note: the URL in this example contains a typo — `'users.jsoni'` instead of `'users.json'`. This would cause the request to fail (likely a 404 response), making `res.ok` `false` and triggering the `Promise.reject('Failed to fetch.')` branch rather than successfully parsing data.

Promise.all and Promise.any

Sometimes there is more than one promise, and the goal is to wait until all of them are complete.

```
const myFetchapi1 = fetch('https://somewhere.com').then(...)
const myFetchapi2 = fetch('https://somewhereElse.com').then(...)
```

This block represents two example promises (each from a `fetch` call followed by `.then()`) that the rest of the code refers to.

```
Promise.all([
  myFetchapi1,
  myFetchapi2,
]).then(values => {
```

```
    // render, reject, anything.
  });
```

`Promise.all()` takes an array of promises and resolves only once all of them have resolved, with values containing the results of each in order.

```
Promise.any([
  myFetchapi,
  myFetchapi2,
]).then(value => {
  // some operation.
});
```

`Promise.any()` resolves as soon as at least one of the given promises resolves, rather than waiting for all of them.

Note: `myFetchapi` and `myFetchapi2` are only defined inside comments in this file and are never actually declared as real variables. As written, this code would throw a `ReferenceError` if executed, since both names are undefined at the point where `Promise.all()` and `Promise.any()` are called. This file is intended to illustrate the syntax and concept, not to run as-is.

A Practical Example: Promise.all with Async Functions

Imagine sora, dora, and iggy meet up. To get things done faster: sora goes to buy bread, dora makes coffee, and iggy waits until everything is ready.

```
async function buyBread() {
  console.log('sora went for bread...');
  await new Promise(resolve => setTimeout(resolve, 10000));
  console.log('bread is ready!');
  return 'bread';
};
```

When a function is declared as `async`, JavaScript automatically wraps its return value in a promise, and the result can be accessed later through that promise. Inside `buyBread`, a new promise is created that calls `resolve` after 10 seconds (using `setTimeout`), and `await` pauses execution until that promise resolves before moving to the next line.

If an `async` function returns a value, that value becomes the resolved result of the promise it implicitly returns. If it throws an error instead, that becomes a rejection of the promise, and the caller can handle either outcome.

```
async function makeCoffe() {
  console.log('dora went for coffe...');
  await new Promise(resolve => setTimeout(resolve, 5000));
  console.log('coffe is ready!');
  return 'coffe';
};
```

`makeCoffe` works the same way, but with a 5-second delay instead of 10.

```

async function handler() {
  const BreadPromise = buyBread();
  const CoffePromise = makeCoffe();

  const [bread, coffe] = await Promise.all([
    BreadPromise,
    CoffePromise
  ]);

  console.log(`${bread} and ${coffe} are ready! lets eat together!`)
};

handler();

```

Calling `buyBread()` and `makeCoffe()` starts both operations immediately, and each returns a promise right away (without waiting for either to finish). `Promise.all()` waits for both promises to resolve, and array destructuring (`[bread, coffe]`) assigns their resolved values — 'bread' and 'coffe' — to two separate variables. Since both functions start at roughly the same time and run concurrently, the total wait time is approximately 10 seconds (the longer of the two), rather than 15 seconds (the sum of both).

The Call Stack and Event Loop

JavaScript has a concept called the event loop. The order in which operations execute is not always the same as the order they appear in the code, since asynchronous operations are managed using a queue.

```

function sayHello () {
  console.log('hello');
  setTimeout(() => {
    soraMancer();
  }, 1);
  done();
  console.log('bye!')
};

function done () {
  console.log('im Done!')
};

function soraMancer () {
  console.log('sora is here!');
};

sayHello();

```

What Happens Step by Step

There is a structure called the call stack. The call stack is a data structure used by JavaScript (and many other languages) to manage which functions are currently running. When a function is called, information about it — such as its name, local variables, and where to resume execution — is pushed onto the top of the call stack. The call stack only handles synchronous functions; asynchronous functions such as `setTimeout`, `async/await`, or promises are managed separately by the event loop.

Tracing through the synchronous part first:

`sayHello()` is called and pushed onto the call stack. `'hello'` is printed to the console. The code then reaches `setTimeout`, an asynchronous function — instead of running immediately, it is handed off to the browser's Web API, which manages timing separately from the call stack. Execution continues without waiting. Next, `done()` is called and pushed onto the call stack, `'im Done!'` is printed, and `done()` finishes and is removed from the call stack. Then `'bye!'` is printed, and `sayHello()` itself finishes and is removed from the call stack, leaving it empty.

The Role of the Web API, Queue, and Event Loop

The Web API handles timers and similar asynchronous tasks in the background. Once the specified time has passed (or the underlying operation completes), the corresponding callback function is placed into a queue, waiting to be executed.

The event loop continuously checks two things: whether the call stack is empty, and whether there is anything waiting in the queue. If the call stack is empty and the queue has an item, the event loop pushes that item onto the call stack to be executed. If the timer has finished but the call stack is not yet empty, the callback waits in the queue until the call stack becomes empty. If the call stack is empty but the timer has not yet finished (for example, a 20-second timer), the event loop remains idle, repeatedly checking both the call stack and the queue until the timer completes.

In this example, after the synchronous code finishes (`'hello'`, `'im Done!'`, and `'bye!'` have all been printed and the call stack is empty), the 1-millisecond timer from `setTimeout` completes. Its callback — which calls `soraMancer()` — is placed into the queue. The event loop sees that the call stack is empty and the queue has an item, so it pushes this callback onto the call stack. `soraMancer()` runs, prints `'sora is here!'`, and is then removed from the call stack, leaving everything empty again.

Expected Output Order

Based on this trace, the console output, in order, is:

```
hello
im Done!
bye!
sora is here!
```

Even though `setTimeout` was given a delay of only 1 millisecond — effectively immediate — its callback still runs only after all the synchronous code (`done()` and `console.log('bye!')`) has completed, because asynchronous callbacks can only run once the call stack is empty.

Regular Expressions: Validation and Matching

Regular expressions (regex) are an important part of every programming language. There are two ways to create a regex in JavaScript: a literal pattern or the `RegExp` constructor.

```
const pattern = /abc/;
```

```
// const pattern = new RegExp("abc");
```

This commented-out line shows the alternative way to create the same pattern using the `RegExp` constructor instead of the literal `/abc/` syntax.

Flags

Several flags can be added after the closing slash of a regex literal: `i` (ignore case), `g` (global search, matching all occurrences instead of just the first), `m` (multi-line mode), `u` (Unicode support), and others.

Testing for a Match

```
console.log(pattern.test('hello im from abc street'));
```

`.test()` checks whether a regex matches anywhere in a string, returning `true` or `false`.

Extracting the First Match

```
const res = /a(bc)/.exec("zabcx");  
console.log(res);
```

`.exec()` returns details of the first match found, or `null` if there is no match. In this example, `res[0]` is `"abc"` (the full match), and `res[1]` is `"bc"` (the first capturing group, defined by the parentheses in the pattern).

Matching All Occurrences

```
console.log("hello hello".match(/hello/g));
```

`.match()` with the `g` flag returns an array of all matches found in the string — here, `["hello", "hello"]`.

```
for(const m of "a1 b2".matchAll(/\w(\d)/g)) {  
  console.log(m);  
}
```

`.matchAll()` returns an iterator with detailed information about every match, including capturing groups, and also requires the `g` flag.

Useful Regex Patterns

The following symbols control how many times something is matched:

```
// .    everything except newLine
// *    zero or more
// +    1 or more
// ?    0 or 1
// {n}   exactly n times
// {n,}  n times at least
// {n,m} between n and m times.
```

Character classes match one character from a defined set:

```
// [abc]   a or b or c
// [0-9]   a digit
// [a-zA-Z] a letter
// [^abc]  everything but a or b or c
```

Shortcuts provide common character classes without writing them manually:

```
// \d    digit
// \w    alphanumeric
// \s    whitespaces
// \D, \W, \S opposite of shortcuts above.
```

Anchors match a position rather than a character:

```
// ^    starts with
// $    ends with
```

Grouping with `|` allows matching one of several alternatives:

```
// (a|b|c)    one of these
```

Optional Chaining

Optional chaining is used for checking conditional statements. The question mark says: if something is not null, return the wanted.

```
const user = {
  id: 1,
  name: "Alice",
  // address: null
};

const street = user.address?.street;
const name = user?.name;

console.log(street); // Output: undefined (because user.address is null in this case)
console.log(name);
```

The commented-out `address: null` line shows what the property would look like if it were explicitly set to null — a realistic scenario when some users have no address on record.

If `user.address` is null or undefined, optional chaining returns `undefined` instead of throwing an error.

Throwing Errors

We have talked about OOP and classes. JavaScript has a built-in class called `Error`, which allows us to create error objects. We also know about `try` and `catch` — using the `throw` keyword, we can throw an error object and catch it with `catch(error)`.

Instead of throwing a plain object with properties like `name` and `message`, we can throw a proper `Error` object:

```
try {
  if (1 < 2) {
    // throw {
    //   name : "error",
    //   message : "you had an error in code."
    // }

    throw new Error('Response not exists.');
```

The commented-out block shows the plain object approach — it works, but using `new Error()` is the preferred way because it gives you access to a proper stack trace and other built-in properties.

The `Error` constructor accepts the following arguments: `Error(message, options)` — only the message is required, all other options are optional.

The Sort Method

`.sort()` is a method that runs on arrays. By default, it sorts elements based on their Unicode values. It takes a callback function as an argument called the compare function, which takes two arguments — `a` and `b` — representing two members of the array.

```
const numbers = [1, 5, 10, 30, 6, 40, 13];
console.log(numbers);
numbers.sort();
console.log(numbers);

var names = ['sora', 'dora', 'iggy', 'ghedas'];
names.sort();
console.log(names);
```

The compare function returns a number, which determines the sort order:

```
numbers.sort((a, b) => {
  return a - b;
});
console.log(numbers);
```

`a` and `b` are two members of the array being sorted. In the block, `a - b` means `a` and `b` are two numbers — they can be the members themselves, or any value we want to sort

the array by. For example, if the array holds person objects and we want to sort by age, we would return `a.age - b.age`.

The rules for the return value are:

- If it returns a **negative** value, member **a** is sorted before member **b**
- If it returns a **positive** value, member **a** is sorted after member **b**
- If it returns `0`, member **a** and member **b** are considered equal in order

State in JavaScript

Sometimes we need to track an item in our program — the item could be anything — and we want to be able to change it from other components. This is when we need state.

State in JavaScript frameworks is handled easily, but in pure JavaScript it is more manual. We need to create a public file that stores global variables, and inside that file we declare and export the state. Inside that variable we define some keys (as many as the items we want to track) with empty placeholders. To update the state from anywhere in the app, we use:

```
state.thatKeyName = value;
```

Practical Project: Flashlight State

This project demonstrates state management in pure JavaScript using a simple flashlight example. It is split across three files: a state file, a controls file, and an HTML entry point.

The State File

The state file holds a single object that represents whether the flashlight is on or off. It is exported so other parts of the application can import and modify it directly.

```
const flashlightState = {
  isOn: false
};

export default flashlightState;
```

The Controls File

This file imports the state and defines functions that update it. `turnOn()` and `turnOff()` simulate button presses by setting `flashlightState.isOn` to `true` or `false`. After each update, `updateUI()` is called to reflect the change — in a real application, this would update a DOM element, toggle a CSS class, or swap an image.

```
import flashlightState from './flashlightState.js';

function turnOn() {
```

```

    flashlightState.isOn = true;
    console.log(`Flashlight is now: ${flashlightState.isOn ? 'ON' : 'OFF'}`);
    updateUI();
}

function turnOff() {
    flashlightState.isOn = false;
    console.log(`Flashlight is now: ${flashlightState.isOn ? 'ON' : 'OFF'}`);
    updateUI();
}

function updateUI() {
    console.log(`--- UI Update: Flashlight is ${flashlightState.isOn ? 'ON' : 'OFF'} ---`);
    // In a real app, you'd change a class on an element, show/hide an image, etc.
}

console.log("--- Initial State ---");
console.log(`Flashlight is initially: ${flashlightState.isOn ? 'ON' : 'OFF'}`);
updateUI();

setTimeout(() => {
    console.log("\nSimulating 'ON' button press...");
    turnOn();
}, 2000);

setTimeout(() => {
    console.log("\nSimulating 'OFF' button press...");
    turnOff();
}, 4000);

setTimeout(() => {
    console.log("\nSimulating 'ON' button press again...");
    turnOn();
}, 6000);

```

The HTML Entry Point

The HTML file loads `controls.js` as a module, which in turn imports the state. Open the browser console to see the state changes as the timeouts fire.

```

<h1>Flashlight Control</h1>
<p>Open your browser's console to see the state changes and UI updates.</p>
<div class="status">Check the console!</div>

<script type="module" src="controls.js"></script>

```

Vanilla JavaScript Router

In frameworks like React and Angular, we have an important component called a router. This component lets our single-page application move between “pages” without refreshing the browser. Since the page is not refreshed, the state remains the same. In React it works like this:

```
<Route path="/about" element={<AboutPage />} />
```

It takes a path and a component to render when the URL changes to that path.

In vanilla JavaScript, implementing a router is a bit more complicated, but possible. We have to interact with browser APIs: `window.location` and `window.history`.

Practical Project: Vanilla JS Router

Save the following as `index.html` and open it directly in your browser. Note that direct URL changes (typing a path and pressing Enter) will not work unless a web server is configured — more on this at the end.

Navigation

`data-link` is a custom HTML attribute. Browsers ignore it, but we can use it in JavaScript to select elements with `a[data-link]`. Links without `data-link` (like the non-existent page below) will behave as normal links and trigger a real browser request.

```
<nav>
  <a href="/" data-link>Main page</a>
  <a href="/about" data-link>about us</a>
  <a href="/contact" data-link>contact us</a>
  <a href="/non-existent-page">non existent page</a>
</nav>

<div id="app">
  loading...
</div>
```

The Routes Object

Each key in `routes` is a URL path, and its value is a function that renders the corresponding content.

```
const routes = {
  '/': () => {
    displayContent(
      'main Page',
      '<h3>welcome to <strong>main Page</strong>, this is a simple program to show routes</h3>'
    );
  },
  '/about': () => {
```

```

        displayContent(
            'about Page',
            '<h3>welcome to <strong>about Page</strong>, this is a simple program to show route'
        );
    },
    '/contact': () => {
        displayContent(
            'contact Us Page',
            '<h3>welcome to <strong>contact Page</strong>, this is a simple program to show route'
        );
    }
};

```

Rendering Functions

`displayContent()` injects a title and HTML content into the `#app` div. `displayNotFound()` is called when no matching route is found.

```

function displayContent(title, contentHtml) {
    const appDiv = document.getElementById('app');
    if (appDiv) {
        appDiv.innerHTML = `<h1 class="page-title">${title}</h1>${contentHtml}`;
    } else {
        console.error("Element with id 'app' not found!");
    }
}

function displayNotFound() {
    displayContent(
        '404',
        '<p class="not-found">page not found!!</p><p>please enter a valid path</p>'
    );
}

```

The Router Function

The router reads the current path from `window.location.pathname`, looks it up in the `routes` object, and calls the matching function. If no match is found, it calls `displayNotFound()`.

```

function router() {
    const currentPath = window.location.pathname;
    const handler = routes[currentPath];

    if (handler) {
        handler();
    } else {

```

```
        displayNotFound();
    }
}
```

Event Listeners

The click listener intercepts any click on an `a[data-link]` element. It prevents the default browser navigation, reads the `href` attribute, and uses `history.pushState()` to update the address bar without reloading the page. It then calls `router()` to render the matching content.

The `popstate` event fires when the user presses the browser's back or forward button, so we call `router()` again to re-render the correct page.

`DOMContentLoaded` runs the router once on page load to render the initial content.

```
document.body.addEventListener('click', (event) => {
    const { target } = event;
    if (target.matches('a[data-link]')) {
        event.preventDefault();
        const url = target.getAttribute('href');
        history.pushState(null, null, url);
        router();
    }
});

window.addEventListener('popstate', () => {
    router();
});

document.addEventListener('DOMContentLoaded', router);
```

Web Server Requirement

When you open the page normally and click the nav links, everything works because the router handles navigation in JavaScript. However, if you directly type a path like `/about` in the address bar and press Enter, the browser sends a real request to the web server, which looks for a file at that path and returns a 404 because no such file exists.

The solution is to configure the web server so that every request — except for real files like `.js` and `.css` — is redirected back to `index.html`. This is standard configuration for single-page applications.

Local Storage

Local storage is a very useful browser feature that lets us store data in key-value pairs on the user's device. Unlike session storage, local storage is permanent — it stays on the

device until the user deletes it. It is client-side data storage, has a space limit, and is only accessible to the domain that stored it.

If you try to set an item with a key that already exists, local storage will not add a duplicate — it will overwrite the existing value.

Storing Data

Use the `setItem()` method to store data. Since local storage only stores strings, non-string values like numbers or objects must be serialized with `JSON.stringify()` before storing.

```
localStorage.setItem('username', 'SoraVoidSeraphim');
localStorage.setItem('userAge', JSON.stringify(30));

const userSettings = { theme: 'dark', notifications: true };
localStorage.setItem('settings', JSON.stringify(userSettings));
```

Reading Data

Use `getItem()` to read stored data. If the original value was serialized with `JSON.stringify()`, use `JSON.parse()` to convert it back to its original type.

```
const username = localStorage.getItem('username');
const ageString = localStorage.getItem('userAge');
const userAge = JSON.parse(ageString);
console.log(userAge);

const settingsString = localStorage.getItem('settings');
const userSettingsGet = JSON.parse(settingsString);
console.log(userSettingsGet.theme);
```

Removing Data

Use `removeItem()` to delete a specific key-value pair, or `clear()` to remove all stored data for that domain.

```
localStorage.removeItem('username');
console.log(localStorage.getItem('username')); // null

localStorage.clear();
```

Counting Stored Items

Use the `length` property to get the number of key-value pairs currently stored in local storage for that domain.

```
console.log(localStorage.length);
```

postMessage API

Imagine you have a website (`yourSite.com`) that contains an `iframe` linked to another website (`thirdparty.com`). By default, these two pages cannot access each other or transfer

data because of the Same-Origin Policy (SOP). `postMessage` is a browser API, like `fetch`, that can make a secure connection between two windows or sites with different domains.

How It Works

`postMessage` has two main parts:

- **Sending:** `postMessage()` is called on the target window or `iframe` object that we want to send a message to.
- **Receiving:** In the receiver window or frame, we listen for the `message` event using `addEventListener`.

`postMessage()` takes the following arguments:

- `message` — the data we want to send
 - `targetOrigin` — the origin of the target, used to make sure the message is only delivered to the intended recipient. It can be `'*'`, which means any origin
 - `transfer` (*optional*) — not important for most use cases
-

Practical Project: `postMessage` in Action

This project has three parts: a parent page that sends a message, a child page that receives it, and an attacker page that demonstrates what can go wrong when `postMessage` is implemented without proper validation.

Parent Page

The parent page opens a child window and sends it a message containing a URL. `window.open()` opens `child.html` as a popup and stores a reference to it in `child`. When the user clicks “Send Message”, `postMessage()` is called on that reference with a message object and `'*'` as the target origin.

```
<input type="button" value="Open Child" id="btnopen" onclick="openChild()">
<input type="button" value="Send Message" id="btnSendMessage" onclick="sendMessage()">

var child;
function openChild() { child = window.open('child.html', 'popup', 'height=300px, width=500px') }
function sendMessage() {
  let msg = { url: 'normal-valid.html' }
  child.postMessage(msg, '*');
  child.focus();
}
```

Child Page

The child page listens for the `message` event. When a message arrives, `event.data` holds the received message object. The child reads `event.data.url` and sets it as the `href` of the “Go Back” anchor element, so clicking it redirects the user to whatever URL was sent.

```

<h1>Recipient of postMessage</h1>
<a href="" id="redirection">Go Back</a>
<input id="btnCloseMe" type="button" value="Close Me" onclick="closeMe()">

window.addEventListener('message', (event) => {
  console.log(event);
  console.log(event.data);
  console.log(event.data.url);
  document.getElementById('redirection').href = `${event.data.url}`;
});

function closeMe() {
  try {
    window.close();
  } catch (e) {
    console.log(e);
  }
}

```

The Vulnerability

The child page blindly trusts whatever URL it receives and sets it as the `href` of an anchor element — without validating the origin of the message or sanitizing the data. This is the attack surface.

The attacker page loads `child.html` inside an `iframe` and immediately sends it a malicious message using `postMessage()`. The payload sets the URL to a `javascript:` URI. When the user clicks “Go Back” in the child frame, the JavaScript executes in the context of the child page.

```

<iframe id="frame" src="child.html" height="300px" width="500px"></iframe>

let hackerMessage = { url: "javascript:alert(origin)" };
var iFrame = document.getElementById('frame');
iFrame.contentWindow.postMessage(hackerMessage, '*');

```

This works because:

1. `postMessage` with `'*'` as the target origin accepts messages from **any** sender
2. The child never checks `event.origin` to verify who sent the message
3. The child uses the received data directly in a sensitive sink (`href`), allowing `javascript:` URI injection

The fix is to always validate `event.origin` against a whitelist of trusted origins before processing any received message, and to never use untrusted data in sensitive sinks without sanitization.

Recap: Some Real World Used JavaScript Built-in Methods and Functions

Array Methods

Array.forEach(callback) Runs a function on each item in an array. Does not return anything.

Array.map(callback) Runs a function on each item and returns a new array with the results.

Array.filter(callback) Returns a new array containing only the items that pass the condition.

Array.find(callback) Returns the first item that matches the condition, or `undefined` if nothing is found.

Array.some(callback) Returns `true` if at least one item in the array matches the condition.

Array.sort(compareFunction) Sorts an array in place. The compare function receives two items `a` and `b` — return a negative number to put `a` first, positive to put `b` first, or `0` if they are equal.

Array.slice(start, end) Returns a portion of an array without modifying the original. `end` is not included.

Array.push(item) Adds an item to the end of an array and returns the new length.

Array.join(separator) Joins all array items into a single string, separated by the given separator.

Array.includes(value) Returns `true` if the array contains the given value.

String Methods

String.includes(substring) Returns `true` if the string contains the given substring.

String.toLowerCase() Returns the string converted to lowercase.

String.substring(start) Returns the part of the string starting from the given index.

RegExp.test(string) Tests whether a string matches a regular expression. Returns `true` or `false`.

DOM Methods

document.querySelector(selector) Returns the first element that matches the given CSS selector.

document.querySelectorAll(selector) Returns all matching elements as a `NodeList`.

element.classList.add / remove / toggle Adds, removes, or toggles a CSS class on an element.

element.getAttribute(attr) Returns the value of a given attribute on an element.

element.closest(selector) Walks up the DOM tree and returns the nearest ancestor that matches the selector.

element.insertAdjacentHTML('beforeend', html) Appends HTML to the end of an element without clearing its existing content.

element.blur() Removes focus from an element.

event.preventDefault() Stops the browser's default behavior, such as form submission reloading the page.

Async and Browser APIs

fetch(url, options) Sends an HTTP request and returns a Promise that resolves with the server response.

response.json() Parses the response body as JSON and returns a Promise with the result.

setTimeout(fn, ms) Runs a function after a delay in milliseconds.

history.pushState(state, title, url) Updates the URL in the address bar without reloading the page.

localStorage.setItem(key, value) Stores a string value in local storage under the given key.

localStorage.getItem(key) Reads a value from local storage by key.

JSON.stringify(value) Converts a JavaScript value to a JSON string.

JSON.parse(string) Parses a JSON string and returns the corresponding JavaScript value.

Practical Script: Flask Job API Server

As I promised in Python section, Im going to wite a simple API witch Flask. This can be a useful intro and guide to write Backends

This is a Flask REST API server that reads job data from `data.json` and serves it over HTTP. It exposes two endpoints — one for listing and filtering jobs, and one for fetching a single job by ID.

To run it, save it as `ApiFlaskServer.py` alongside `data.json` and run:

```
python3 ApiFlaskServer.py
```

The server starts on port 8000. Make sure `flask` and `flask-cors` are installed:

```
pip install flask flask-cors
```

CORS is enabled so the frontend, served from a different origin, can make requests to it.

Endpoint 1 — List Jobs: GET /data.json

Returns all jobs. Accepts two optional query parameters:

- ?search=<term> — filters jobs whose title or qualifications contain the search term
- ?new=true — sorts results by `daysAgo` ascending (most recent first)

```
@app.get('/data.json')
def list_jobs():
    data_container = load_data()
    all_jobs = data_container.get('jobItems', [])

    job_list = list(all_jobs)

    search_term = request.args.get("search")
    if search_term:
        s = search_term.lower()
        job_list = [
            j for j in job_list if s in j.get('title', {}).lower() or any(s in q.lower() for q in j.get('qualifications', {}))
        ]

    should_sort_by_date = request.args.get('new', 'false').lower() == 'true'
    if should_sort_by_date:
        def sort_key(item):
            v = item.get('daysAgo')
            try:
                return int(v)
            except:
                return float('inf')

        job_list = sorted(job_list, key=sort_key)

    return jsonify({
        "public": "true",
        "sorted": should_sort_by_date,
        "jobItems": job_list
    })
```

The search filter uses a list comprehension. `any()` checks whether the search term appears in at least one of the job's qualifications by iterating over the list and testing each one.

If `daysAgo` cannot be converted to an integer, `float('inf')` is used as a fallback so those items are sorted to the end.

Endpoint 2 — Single Job: GET /data.json/<job_id>

Returns a single job by ID, or a 404 if not found.

```
@app.get('/data.json/<job_id>')
def get_job(job_id):
    data_container = load_data()
```

```

all_jobs = data_container.get("jobItems", [])

try:
    job_id_int = int(job_id)
    found = next((j for j in all_jobs if j.get("id") == job_id_int), None)
except ValueError:
    found = next((j for j in all_jobs if str(j.get("id")) == job_id), None)

if not found:
    return jsonify({"error": f"Job with id: {job_id} not found"}), 404

return jsonify({
    "public": "true",
    "jobItem": found
})

```

Instead of a traditional for loop with a break, next() is used with a generator expression. It returns the first item that matches the condition, or None if nothing is found. This avoids iterating over the entire list unnecessarily.

The try/except ValueError handles cases where the ID in the URL cannot be converted to an integer — in that case it falls back to a string comparison.

Full Script

```

#!/usr/bin/python3
from flask import Flask, jsonify, request
from flask_cors import CORS
import json
import os

app = Flask(__name__)
CORS(app)

DATA_FILE = 'data.json'

def load_data():
    if not os.path.exists(DATA_FILE):
        return {}
    try:
        with open(DATA_FILE, 'r') as f:
            container = json.load(f)
        return container
    except json.JSONDecodeError:
        return {"Error": "Invalid Json Format"}
    except Exception as e:
        return {"Error": str(e)}

```

```

@app.get('/data.json')
def list_jobs():
    data_container = load_data()
    all_jobs = data_container.get('jobItems', [])
    job_list = list(all_jobs)

    search_term = request.args.get("search")
    if search_term:
        s = search_term.lower()
        job_list = [
            j for j in job_list if s in j.get('title', {}).lower() or any(s in q.lower() for q in j.get('tags', {}))
        ]

    should_sort_by_date = request.args.get('new', 'false').lower() == 'true'
    if should_sort_by_date:
        def sort_key(item):
            v = item.get('daysAgo')
            try:
                return int(v)
            except:
                return float('inf')
        job_list = sorted(job_list, key=sort_key)

    return jsonify({
        "public": "true",
        "sorted": should_sort_by_date,
        "jobItems": job_list
    })

@app.get('/data.json/<job_id>')
def get_job(job_id):
    data_container = load_data()
    all_jobs = data_container.get("jobItems", [])

    try:
        job_id_int = int(job_id)
        found = next((j for j in all_jobs if j.get("id") == job_id_int), None)
    except ValueError:
        found = next((j for j in all_jobs if str(j.get("id")) == job_id), None)

    if not found:
        return jsonify({"error": f"Job with id: {job_id} not found"}), 404

    return jsonify({
        "public": "true",
        "jobItem": found
    })

if __name__ == "__main__":

```

```
app.run(port=8000, debug=True)
```

data.json (Sample)

```
{
  "public": true,
  "sorted": true,
  "jobItems": [
    {
      "id": 435243523542435,
      "title": "Full-Stack Developer",
      "company": "LakeOperations",
      "badgeLetters": "L0",
      "description": "As a full-stack developer, you will be responsible for designing,
      "qualifications": ["JavaScript", "CSS", "React", "HTML", "Node.js", "MongoDB/NoSQL"],
      "reviews": ["Best place I've ever worked.", "I liked that we could work remotely."],
      "duration": "Full-Time",
      "salary": "$80,000+",
      "location": "Global",
      "relevanceScore": 34,
      "daysAgo": 1,
      "coverImgURL": "https://images.unsplash.com/photo-1522202176988-66273c2fd55f",
      "companyURL": "https://fictionallakeoperationswebsite.com"
    }
  ]
}
```

Recap: Python Built-ins and Patterns Used

os.path.exists(path) Checks whether a file or directory exists at the given path. Returns True or False.

json.load(file) Reads a file object and parses its contents as JSON, returning a Python dictionary or list.

dict.get(key, default) Returns the value for a key if it exists, or the default value if it does not. Avoids `KeyError`.

list(iterable) Creates a new list from an iterable. Used here to make a copy of the jobs array so the original is not modified.

str.lower() Returns the string converted to lowercase. Used for case-insensitive search comparisons.

any(iterable) Returns True if at least one item in the iterable evaluates to True. Used to check if the search term appears in any qualification.

next(generator, default) Returns the first item from a generator, or the default value if the generator is empty. Used instead of a `for` loop with `break` to find the first matching

job.

int(value) Converts a value to an integer. Wrapped in `try/except ValueError` to handle cases where the ID is not a valid number.

float('inf') Represents positive infinity. Used as a sort fallback so items with invalid `daysAgo` values are placed at the end.

sorted(iterable, key=fn) Returns a new sorted list based on the value returned by the `key` function. Does not modify the original list.

str(value) Converts a value to a string. Used as a fallback comparison when the job ID cannot be parsed as an integer.

jsonify(dict) A Flask function that converts a Python dictionary to a JSON HTTP response with the correct `Content-Type` header.

request.args.get(key, default) A Flask method that reads a query parameter from the URL. Returns the default if the parameter is not present.

List comprehension [x for x in iterable if condition] A compact way to filter or transform a list in one line. Equivalent to a `for` loop that appends to a new list.

Generator expression (x for x in iterable if condition) Same as a list comprehension but does not build the full list in memory. Used inside `next()` to stop as soon as a match is found.

JavaScript Additional Modules

In this section, several additional modules and concepts are introduced.

These are not core fundamentals, but they are commonly used in real-world web applications.

You are encouraged to review them carefully and understand how they integrate into larger JavaScript-based systems.

JavaScript — Pop-Up Windows

`prompt` is similar to `alert`, but it can also take a value from the user and store it in a variable.

```
let x = prompt("Enter a number: ")
console.log(x)
```

JavaScript — Type Casting: String to Number

To convert string data types to numbers, several methods can be used: the `Number()` constructor, `parseInt()`, and `eval()`.

```
let a = '123'
console.log(Number(a) + 1)
console.log(a + 1)
console.log(parseInt(a) + 1)
console.log(eval(a) + 1)
```

JavaScript — Events

Events are used to add interactivity and control to web pages. Some of the most common event types include:

```
// load      => when the document object has loaded completely
// focus     => when an input is focused – can be useful for showing user hints
// blur      => when an input loses focus – can be used for validation
// change    => when an input's value is changed
// keydown, keyup  => when a key is pressed or released
// click     => when an element is clicked
// mouseover, mouseout => when the mouse enters or leaves an element
// mouseleave => only when the mouse fully exits an element and all of its children
// dblclick  => double click
// contextmenu => right click
// submit    => submitting a form
// animationstart => when an animation starts
```

JavaScript — Error Handling

```
alertt("Hello World!")
```

This will produce an “uncaught” error — “uncaught” is the past-tense negative of “catch”, meaning there is an error that was not caught. This is why `try` and `catch` are used: to handle errors that could otherwise stop the entire script.

JavaScript — The DOM

The relevant HTML for this example:

```

<input type="button" value="Add Link" onclick="addNewLink()">
<input type="button" value="Del Link" onclick="removeLink()">
<div id="d1">
  <a href="#">Link1</a>
  <a href="#">Link2</a>
</div>

```

```

console.log(document.documentElement)    // returns the <html> tag

```

A tag's `.childNodes` returns a `NodeList` of its child nodes. `.parentNode` on a tag returns its parent node. There are also `.firstChild` and `.lastChild`. Whitespace is also considered a child node (type 3). The next sibling of a node can be accessed with `.nextSibling`.

`.nodeName` refers to the tag's name. `.nodeValue` refers to a node's value — HTML element tags have no node value (`null`).

The type of a node can be checked with `.nodeType`. In the DOM, there are 12 node types:

1. element node
2. attribute node
3. text node
4. cdata node
5. entity reference node
6. entity node
7. processing instruction node
8. comment node
9. document node
10. document type node
11. document fragment node
12. notation node

Adding and Removing Elements

```

function addNewLink(){
  var newLink = document.createElement('a');
  var linkText = document.createTextNode('Link3');
  newLink.appendChild(linkText);
  newLink.setAttribute('href', '#');
  newLink.onclick = function(){this.parentNode.style.backgroundColor = 'red'}
  document.getElementById('d1').appendChild(newLink);
}

function removeLink() {
  var prnt = document.querySelector('#d1')
  var child = document.getElementById('d1').getElementsByTagName('a')[0]

  prnt.removeChild(child)
}

```

`addNewLink()` creates a new `<a>` element, sets its text content and `href` attribute, attaches

a click handler that highlights its parent element in red, and appends it to the #d1 div. `removeLink()` selects the #d1 div and removes its first `<a>` child element.

JavaScript Objects and Prototypes

Some String Methods

```
var txt = 'SoraVoidSeraphim'

alert(txt.charAt(4))    // character at index 4
alert(txt.charCodeAt(4)) // character decimal/ASCII code at index 4

console.log(txt.concat('Here!'))
console.log(txt.indexOf('V'))
console.log(txt.replace('Sora', 'Dora'))
console.log(txt.length)

alert(txt.link('#'))
console.log(txt.constructor) // returns the object's constructor
```

Using Prototypes

We can change a JavaScript object's behavior using **prototype**, for example by adding methods or properties.

Before the `class` syntax was introduced, constructor functions and prototypes were commonly used to create class-like objects in JavaScript.

```
function Employee(name, job, born)
{
    this.name = name
    this.job = job
    this.born = born
}
```

Add a method to the prototype:

```
Employee.prototype.introduce = function(){
    return `${this.name} is a ${this.job} and he/she is ${this.born} years old.`
}
```

Create an object from the constructor:

```
var p1 = new Employee('sora', 'hacker', 24)

console.log(p1.job)
```

Properties and methods can also be added after the constructor has been created:

```
Employee.prototype.salary = null

console.log(p1.salary)
```

```
p1.salary = 2000

console.log(p1.salary)
console.log(p1.introduce())
```

Browser Object Model (BOM)

Window and Screen Information

```
console.log(window.innerWidth)    // window width
console.log(window.innerHeight)    // window height

console.log(screen.availHeight)    // device screen
console.log(screen.availWidth)
```

Browser History

```
alert(window.history)

// window.history.forward()    // one step forward in history
// window.history.back()       // one step backward in history
```

Browser Information

```
console.log(window.navigator.userAgent)
```

Opening and Closing a Window

```
let Win;

function openWin(){
    Win = window.open('https://example.com', '_blank', 'width=500,height=500')
}

function closeWin(){
    Win.close()
}
```

Interval

```
<h3 style="background-color: yellow; color: blue; text-align: center;" id="timer"></h3>

<input type="button" value="Pause" onclick="toggleTimer(this)">
```

```
<button onclick="openWin()">open</button>
<button onclick="closeWin()">close</button>
```

Send the input element itself to JavaScript:

```
<input type="button" value="Pause" onclick="toggleTimer(this)">
```

```
var t = setInterval(function(){timer()}, 1000)
```

```
function timer() {
    var d = new Date()
    document.getElementById('timer').innerHTML = d.toLocaleTimeString()
}
```

Pause and Resume the Timer

```
var timerState = true;
```

```
function toggleTimer(btn) {
    clearInterval(t)

    if(timerState){
        btn.value = "Play"
        timerState = false
    }else{
        t = setInterval(function(){timer()}, 1000)
        timerState = true
    }
}
```

JavaScript Tools

With JavaScript, you can build powerful tools that run directly in the browser or on the server using Node.js.

This allows you to implement dynamic behavior, automation logic, and interactive functionality with real-time response and visual feedback.

In this section, the focus is on building practical tools and understanding how logic translates into real-world web environments.

Real World Projects: G-dorking — Google Dork Generator

A minimal browser tool that generates Google dork queries for a target domain. Enter a domain and click any link to open the search directly in Google.

Save it as `G-dorking.html` and open it in a browser.

Script

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Google Dorker</title>
  <style>
    body { font-family: monospace; background: #111; color: #ccc; padding: 40px; }
    h1 { color: #fff; margin-bottom: 20px; }
    input { background: #222; border: 1px solid #444; color: #fff; font-family: monospace; }
    a { display: block; color: #aaa; text-decoration: none; padding: 4px 0; word-break: break-all; }
    a:hover { color: #fff; }
  </style>
</head>
<body>
  <h1>Google Dorker</h1>
  <input id="domain" oninput="updateSearchQuery()" value="sora.tld">
  <div id="results"></div>
  <script>
    function updateSearchQuery() {
      var domain = document.getElementById('domain').value.trim();
      var dorks = [
        'site:' + domain + ' inurl:&',
        'site:' + domain + ' ext:php | ext:aspx | ext:jsp | ext:html | ext:htm | ext:a',
        'site:' + domain + ' ext:log | ext:txt | ext:conf | ext:cnf | ext:ini | ext:er',
        'site:' + domain + ' inurl:url= | inurl:return= | inurl:next= | inurl:redir= |',
        'site:' + domain + ' inurl:http | inurl:url= | inurl:path= | inurl:dst= | inur',
        'site:' + domain + ' inurl:config | inurl:env | inurl:setting | inurl:backup |',
        'site:' + domain + ' inurl:email= | inurl:phone= | inurl:password= | inurl:sec',
        'site:' + domain + ' inurl:apidocs | inurl:api-docs | inurl:swagger | inurl:ap',
        'site:' + domain + ' inurl:cmd | inurl:exec= | inurl:query= | inurl:code= | in',
        'site:' + domain + ' inurl:(unsubscribe|register|signup|join|contact|profile|u',
        'site:' + domain + ' intitle:"Welcome to Nginx"'
      ];
      var resultDiv = document.getElementById('results');
      resultDiv.innerHTML = '';
      dorks.forEach(function(dork) {
        var a = document.createElement('a');
```

```

        a.href = 'https://google.com/search?q=' + encodeURIComponent(dork); // this t
        a.target = '_blank';
        a.textContent = dork;
        resultDiv.appendChild(a);
    });
}
updateSearchQuery();
</script>
</body>
</html>

```

How to Use

Open the file in a browser, enter a target domain, and click any generated link to run that dork in Google.

Real World Projects: JS Path Extractor — jscrawl

This bookmarklet extracts URL paths from all JavaScript files loaded on the current page, as well as from the raw HTML source. Results are printed to the page after 3 seconds.

Paste it into the browser console or save it as a bookmarklet.

How It Works

1. Finds all `<script>` tags with a `src` attribute and fetches each file
2. Scans fetched JS content and the page's raw HTML for paths matching the pattern `"/some/path"`
3. Collects unique results in a `Set` and writes them to the page after 3 seconds

The regex targets strings wrapped in quotes that start with `/`, covering relative API endpoints, routes, and asset paths embedded in JS.

Script

```

javascript:(function(){
    var scripts = document.getElementsByTagName("script");
    var regex = /(?!<=("|'|`|`))\/[a-zA-Z0-9_?&=\/\-\#\.\.]+(?!<=("|'|`|`))/g;
    const results = new Set;

    // fetch and scan each external JS file
    for (var i = 0; i < scripts.length; i++) {
        var src = scripts[i].src;
        if (src != "") {
            fetch(src)

```



```

        .then(function(r) { return r.text(); })
        .then(function(t) {
            for (let m of t.matchAll(regex)) results.add(m[0]);
        })
        .catch(function(e) { console.log("An error occurred: ", e); });
    }
}

// also scan the raw HTML of the current page
var pageContent = document.documentElement.outerHTML;
for (const match of pageContent.matchAll(regex)) results.add(match[0]);

// print results to page after 3s (to allow async fetches to finish)
setTimeout(function() {
    results.forEach(function(path) {
        document.write(path + "<br>");
    });
}, 3000);
})();

```

How to Use

Paste the minified version into the browser console on any target page:

```
javascript:(function(){var scripts=document.getElementsByTagName("script"),regex=/(?<="(\\'|\\
```

After 3 seconds, all discovered paths will be written to the page.

Bash Fundamentals

This section assumes you already have basic familiarity with Linux systems.

Due to the large scope of command-line concepts, we move directly into syntax and scripting fundamentals.

The goal is to quickly build practical Bash scripting skills for automation and system interaction.

Bash Aliases

If you are tired of typing the same long commands over and over again, aliases are here to help. A bash alias is nothing more than a shortcut.

To list all your current aliases:

```
alias
```

When you run a command like `ls`, it's not the actual `ls` — it's an alias: `alias ls='ls --color=auto'`. If you want to run the real binary, write a backslash before the command, or use `$(which ls)`.

Creating a new alias is pretty straightforward:

```
alias alias_name='command to alias'
```

To make aliases permanent, declare them in `.profile` or `.bashrc`.

To remove an alias:

```
unalias <alias_name>
```

Here are a few practical examples:

```
alias listMyFiles='ls -ltrhF --color=auto'
alias server='ssh -p 2299 user1@1.2.3.4'
alias ports='netstat -tulpn'
```

Shells and Shell Scripting

A shell is an interface between the user and the kernel. It starts when the user logs in and opens the terminal.

To see the current code execution environment:

```
echo $0
```

To see your user's default shell:

```
echo $SHELL
```

You can also find your shell listed in `/etc/passwd`.

Bash (Bourne Again Shell) is the default shell for many distros nowadays, and it is really powerful.

To see all valid login shells:

```
cat /etc/shells
```

To change a user's default shell:

```
chsh -s <path_to_shell>
```

A shell script is basically an executable text file that contains shell commands and other programming structures, like conditions, loops, functions, and more.

Why do we need shell scripting? To avoid running the same commands over and over again, and to build simple tools.

A Basic Shell Script

This is a simple script that creates a directory with a file inside it, then lists the contents.

```
#!/bin/bash

mkdir -p dir1
echo "Hello Bash World!" > dir1/file.txt
tree .
cat dir1/file.txt
```

Adding Scripts to PATH

You may have noticed that you need an absolute or relative path to run your scripts. That is because your scripts folder is not in `PATH`.

Adding your folder to the system's executable path is fairly straightforward:

```
export PATH="$HOME/path/to/directory:$PATH"
```

To make this permanent, add the line above to your `.bashrc` or `.profile`.

You can check your current `PATH` at any time:

```
echo $PATH
```

Shebang

The first line of a script is called a **shebang**, shortened from hash-bang. It tells the file executor which interpreter to use.

When you write `#!/bin/bash`, it means: run this file with the `/bin/bash` interpreter. Without a shebang, you would have to specify the interpreter manually:

```
bash myscript.sh
```

If you do not specify a shebang for an executable file, it will use your default shell, which is probably bash itself. The shebang becomes more important when writing scripts in other languages like Python.

Bash ignores everything after a `#` (hash / number sign). Bash does not officially support multiline comments, but here is a trick to comment out a whole block:

```
: '
this is a multiline comment in bash!
date; ls
'
```

Running a Script

There are multiple ways to run a script in Linux. Besides using the interpreter name directly or relying on the shebang, there is one more way — the `source` built-in command.

Write the following into a file called `runningAScript.sh` and run it with `source runningAScript.sh`:

```
name="sora"
echo "hello world! im $name"
cd /tmp && pwd && cd -
```

When using `source`, the file does not require executable permission.

So what is the difference? When you run a script with `./myScript.sh`, it executes in a new subshell that is started specifically for that script. But when you use `source`, the script executes in the current shell that is running in the terminal.

Also, `source scriptName` is the same as `. scriptName`.

Variables in Bash

Variables are part of every programming language, including bash — which is both a command interpreter and a programming language. A variable is a name for a memory location where a value is stored.

To create a variable, write its name followed by the assignment operator `=` and the value. No spaces are allowed before or after the `=`.

```
os="Linux"
```

Unlike most programming languages where every variable has a strict type, bash is weakly typed. When a numeric value is assigned to a variable, it works as an integer; if the value is text, it is a string.

```
age=23
distro="Ubuntu"
```

To reference a variable, put a `$` sign before its name:

```
my_distro="$os $distro"
echo $my_distro
```

To get a list of all shell variables and functions, run:

```
set
```

There will be a lot of output, so it is better to pipe it to `less`. You can also search for anything specific inside it. If you run `set` directly in your shell, the output will be many lines of code. But inside a script, the environment is more isolated, so you get less output.

To remove a variable, use `unset`:

```
echo age:
echo $age
unset age
```

```
echo age:
echo $age
```

Sometimes we do not want the value of a variable to change — it should be read-only. To declare a constant, use `declare -r`:

```
declare -r LOGDIR="/var/log"    # -r for read-only
```

```
ls $LOGDIR
```

If you try to change the value of a read-only variable, you will get an error:

```
LOGDIR="/etc"
```

You also cannot unset or delete a read-only variable:

```
unset LOGDIR
```

A few naming conventions worth following:

- Environment variables are mostly written in uppercase letters. To avoid conflicts, write your own variable names in `snake_lower_case`.
- Constants should be written in `UPPERCASE`, with parts separated by underscores, and declared at the top of the file.
- A variable name cannot start with a number, and cannot contain spaces or special characters.

Variable Expansion and Quoting

Expansion is a process where bash replaces special characters, variables, and other elements with their actual values. There are several types of expansions in bash.

Variable Expansion

Bash replaces a variable's name with its value. You can use `$...` or `${...}`:

```
name="Alice"
echo "Hi ${name}"
```

Command Substitution

Lets you place a command's output inside another command. You can use `$(...)` or ``...``:

```
echo "Today's date is: $(date +%Y-%m-%d)"
```

Arithmetic Expansion

Used for mathematical operations with `$((...))`:

```
x=5
y=4
myAge=$((x * y + 4))
echo $myAge
```

Brace Expansion

Used to generate partially-similar strings or lists of items. One use case is similar to `range()` in Python. The syntax is `{item..otherItems}`:

```
echo {1..5}
echo {a..e}
echo {a,b,c}
echo file{01..03}
echo {r..7} # won't return anything meaningful, because they are not the same type
```

Parameter Expansion

More advanced ways to work with variables:

Syntax	Behavior
<code>\${variable:-word}</code>	If the variable is empty or undefined, <code>word</code> is used
<code>\${variable:=word}</code>	If the variable is empty or undefined, <code>word</code> is assigned to it
<code>\${variable:?word}</code>	If the variable is empty or undefined, <code>word</code> is shown as an error and the program exits
<code>\${variable:+word}</code>	If the variable is declared, <code>word</code> is used instead

```
echo ${someVar:-"this Is Some Variable"}

echo ${testVar:="this Is TestVar From Now On"}
echo ${testVar}

# echo ${notExistVar:? "notExistVar variable does not exist"}

word="horse"
echo "${word:+ "bird"} is the word"
```

Globbering (Pathname Expansion)

Related to using wildcard characters like `*`, `?`, and `[]` on the command line to match filenames:

```
echo "matching files with pattern: sh<anything>.sh"
ls sh*.sh
```

Variables and Quoting

- **Assigning** a value to a variable means storing something in it.
- **Referencing** a variable means retrieving the value stored in it.

The `$` character introduces parameter expansion, command substitution, or arithmetic expansion:

```
age=23
os="Debian"

echo $age
echo ${os}    # curly braces are the standard form
```

If you want to append a value to a variable, you must use curly braces:

```
echo $os13    # references a variable that does not exist – no error, just a blank value
echo '...'
echo ${os}13
```

Quoting Mechanisms

Quoting is used to remove the special meaning of certain characters or words, such as expansions. Bash has three quoting mechanisms: `'`, `"`, and `\`.

Single quotes print the literal value of an expression — no expansions happen:

```
echo 'im learning ${os} and i am $age years old'
```

Note: using a single quote inside single quotes is not permitted, even with an escape character.

Double quotes print literal values of words, but still allow `$`, backticks, and backslashes. Backslashes only take effect when followed by a special character like a newline:

```
echo -e "im learning ${os} \nand im $age years\" old\n`date`"
```

Backslash is bash's escape character. It preserves the literal value of the character that follows, with the exception of a newline:

```
echo im learning \${os} and im \$age years old
```

For example, `&` is a special character that means “run in background”. Using a backslash removes that meaning:

```
echo me \& you
echo C:\Windows
echo C:\\Windows
echo "C:\\Windows"
```

Environment Variables and Shell-Local Variables

Each time you open a shell, a collection of predefined variables are set with dynamic values. These are used to customize how the system works and behaves. There are two classes of such variables: **environment variables** and **shell-local variables**.

Environment Variables

Environment variables are defined for the current shell and are inherited by any child process. They are used to pass information to processes spawned from the current shell.

To display them, use `env` or `printenv`:

```
env
printenv
```

Running `printenv` without any arguments returns all environment variables. You can also query a specific one:

```
printenv HOME
printenv USER
```

Pipe the output to `less` to make it easier to read.

`PATH` is an environment variable. Most of the time we do not use environment variables directly — they are referenced by applications and services as needed.

For example, your system knows your home directory through this:

```
echo $HOME
```

Shell-Local Variables

Shell-local variables exist exclusively within the shell in which they were set. There are also many predefined shell-local variables. To display them, use `set`:

```
set
set | grep HOST
```

If you declare a shell-local variable, it will not appear in `env` or `printenv` output, but you can find it in the `set` output:

```
abc=123
echo "this is my abc var value: $(set | grep abc)"
```

The `set` command also returns all shell functions, which makes the output overwhelming. You can tell `set` to operate in POSIX mode to suppress function output.

What is POSIX? Portable Operating System Interface — a standard for Unix systems that makes programs more portable across different operating systems like Linux, macOS, or even some Windows versions.

Enable POSIX mode with `set -o posix` and disable it with `set +o posix`.

Scope and Exporting Variables

A shell-local variable is only accessible from the shell it was defined in. If you want child shells to access it too, declare it with the `export` keyword. To make it permanent, declare it in `.bashrc`.

To make a variable system-wide and available to all users, declare it in `/etc/profile` or `/etc/bash.bashrc`. There is also `/etc/environment`, which is used specifically for system-wide environment variables.

Note: Environment variables are sometimes called exported variables.

A local variable is available only to the current shell. A global variable — also known as an exported variable — is inherited by every child shell or process.

Getting User Input

Bash provides a built-in command called `read` to get user input.

When the interpreter reaches a `read` command, execution stops and the shell waits for the user to type a value and press Enter:

```
read name
echo $name
```

To read multiple values at the same time, list multiple variable names after `read`. You can also display a prompt for the user with the `-p` flag:

```
read -p "enter fname, lname, age (space separated) " fname lname age
echo fname is $fname and lname is $lname and age is: $age
```

The user enters three values separated by spaces. The first word is assigned to the first variable, the second to the second, and so on.

If no variable name is supplied, the value is stored in a default variable called `REPLY`:

```
read
echo $REPLY
```

To read sensitive information without displaying it on screen, use the `-s` flag:

```
read -s -p "Enter password: " rootPass
if [[ $rootPass == 'root' ]]
then
    echo correct!
else
    echo you are not admin
fi
```

Practical Script: Packet Dropper

This script drops all incoming packets from a specific IP address, domain, or network address provided by the user. Save the following into a file called `packetDropper.sh` and run it with `sudo bash packetDropper.sh`:

```
#!/bin/bash

read -p "Enter the ip address or domain to block: " ip

iptables -I INPUT -s $ip -j DROP
```

```

if [[ $? -eq 4 ]]
then
    echo "You need to run the script as root"
    exit 1
fi

echo "Done! the address: $ip blocked successfully."

```

Positional Parameters

In many cases, scripts require arguments to be provided as input. These arguments are supplied after the script name.

For example:

```
apt install nginx
```

- `apt` is the command name.
- `install` is the first argument.
- `nginx` is the second argument.

Arguments are separated by spaces.

Parameters referenced by a name are called variables.

Parameters referenced by a number are called positional parameters (or positional arguments).

Parameters referenced by a special symbol are called special parameters.

Common Positional Parameters

Parameter	Description
<code>\$0</code>	The name of the script itself
<code>\$1</code>	The first positional parameter
<code>\$2</code>	The second positional parameter

If the parameter number contains more than one digit, you should wrap it in curly braces:

```
${11}
```

Using Positional Parameters

```

echo "1st arg (\$1): ${1:-"default"}"
echo "2nd arg (\$3): $2"
echo "3rd arg (\$2): $3"

echo "script name is: $0"

```

Default Values

Bash parameter expansion allows you to provide a default value when a positional parameter (or any variable) is not set.

In the following example, a default value is assigned to \$1:

```
${1:-"default"}
```

Practical Script: File Compressor

This script takes a file as an argument, displays its content, and creates a compressed copy of it.

```
#!/bin/bash

echo "Displaying the content of $1"
sleep 2

cat -bs "$1"

echo

echo "Compressing $1"
sleep 2

tar -czvf "$1.tar.gz" $1
```

protectFromHackers.sh

As a system administrator, we need logs and monitoring for everything, especially authentication attempts. But what if our monitoring shows that we are under attack? How do we keep our data safe?

We had written this script before, but using positional arguments makes it better suited for automation.

```
#!/bin/bash

echo "Blocking connections from $1"
sleep 1

iptables -I INPUT -s $1 -j DROP
echo "Done"
```

To use this script:

```
./scriptname <IP_address>
```

Special Parameters

These parameters can only be referenced; assigning to them is not allowed.

```
echo $0      # The path/name of the script being executed
echo $@      # All positional arguments, one by one
echo $*      # All positional arguments as a single string
echo $#      # Number of positional arguments
echo $?      # Exit code of the most recent foreground command
echo $$      # Process ID of the shell
```

More on Exit Codes

Every process returns an exit code after execution. If this exit code is `0`, it means success; any other value is considered a failure. We can use this to control errors in our scripts and write conditional statements based on the outcome.

The Difference Between `$@` and `$*`

They are mostly similar, but the difference appears when they are used inside double quotes.

- `$@` → `"$1 $2 $3 ..."` — **word splitting is performed**
- `"$@"` → `"$1" "$2" "$3" ...` — **word splitting is not performed**

`$*` behaves the same as unquoted `$@`.

`"$*"` places the first character of the `IFS` variable between parameters and returns a single string — **word splitting is not performed**.

```
touch $@      # touch "$1 $2 $3 ..."
touch "$@"     # touch "$1" "$2" "$3" ...

# IFS is whitespace by default.
IFS=,
touch "$*"    # touch "$1,$2,$3..."
```

Intro to Bash Expansions

When you write a command in the terminal and hit Enter, the command is split into tokens — this process is called **tokenization**. The shell must understand what we actually want, so it breaks the command into smaller parts: the command itself, arguments, flags, etc. This is how tokenization works — it divides the command into the smallest parts so the shell can interpret them.

After tokenization, the shell parses and identifies the tokens as simple or compound commands.

Next, the shell performs what is called **expansion**. For example, when the shell sees `$PATH` in a command, it must be translated to its real value for the interpreter to operate on.

I've talked about shell expansions in a previous file: [Variable Expansion And Quoting](#).

Shell expansion is the procedure of getting the value from a referenced entity — for example, expanding a variable to get its value.

After all expansions, **quote removal** is performed. This means the shell removes all quotation marks, but before removing them, it uses them to control behavior.

For example:

```
echo "$HOME" → tokenization: echo & "$HOME"
              → expansion:    echo & "/home/hikari"
              → quote removal: echo & /home/hikari
```

Let's take a deeper look at the different types of expansion and the order in which they are performed. We've seen some basic expansions until now — the \$ character introduces **parameter and variable expansion**.

```
echo $PATH
# will expand the variable PATH to its actual value
```

Brace Expansion

A mechanism to generate arbitrary values, useful in scripts for strings, loops, and more.

Brace expansion types: **String lists** | **Range (sequence) lists**.

```
echo file_{old,new,current,backup}.txt # You are not allowed to put any spaces between or a
```

These expansions can also be nested:

```
echo file_{old,new,{1,2,3}_backup}.file

echo {a..g} # Use two dots for a sequence without spaces.
echo file_{01..10}
echo {001..100..2} # 2 will be the step.
```

We can also reverse the sequence:

```
echo {10..1}
echo {z..a}
```

Brace expansion is performed before variable expansion, which means you can't use variable expansion inside brace expansion. The example below simply won't work:

```
a=1
echo {$a..10}
```

Brace Expansion Project: Generating Log Directories

Imagine a DevOps team wants to install a new application which, in its first phase, should be highly monitored. To keep the application logs separate and make it easier to locate a specific event, the application should write to a separate log file each day. Each day needs a specific directory, and the process will take 3 months to collect logs.

So we have to create these directories and a log file for each day.

```
#!/bin/bash
```

```
# sudo mkdir -p /var/log/myBashScript    # We don't want to create a directory in /var, so th
mkdir -p ./myBraceExpPojectFolder/{January/{01..31},February/{01..28},March/{01..31}}
touch ./myBraceExpPojectFolder/{January/{01..31}/file.log,February/{01..28}/file.log,March/{01..31}/file.log}
```

Tilde Expansion

The tilde (~) expands to several specific pathnames: home directories, the current or previous working directory, and directories from the directory stack.

- ~ → \$HOME of the current user
- ~USER → the specified user's home directory
- ~+ → \$PWD
- ~- → \$OLDPWD

```
echo ~
echo ~hikari
echo ~+
echo ~-
```

Variable Expansion

I've talked about the whole expansions topic before, but this will be a recap with more explanations.

```
# Variable expansions: bash replaces a variable with its value, used as $... or ${...}
name="Alice"
echo "Hi ${name}"
```

The \$ character introduces a parameter or variable expansion and the name or symbol to be expanded. It's a best practice to write it inside curly braces for better readability and to avoid mistakes.

```
name=Sora
echo ${name}
```

There are also some expansion operators which modify the case of the letters in the expanded text:

- ^^ → change all letters to uppercase
- ,, → change all letters to lowercase

```
echo ${name^^}
echo ${name,,}
```

Sometimes, when a variable is not declared, it might cause errors or even break the script. We can handle these cases with parameter expansions — more advanced tools for working with variables:

- \${variable:-word} → if the variable is empty or undefined, word will be used

- `${variable:=word}` → if the variable is empty or undefined, `word` will be assigned to it
- `${variable:?word}` → if the variable is empty or undefined, `word` will be shown as an error and the program exits
- `${variable:+word}` → if the variable is declared, `word` will be used

```
echo ${someVar:="this is some temporary value"}
echo ${somevar}

echo ${testVar:="this value is assigned to testVar from now on"}
echo ${testVar}

# echo ${notExistVar:?notExistVar variables does not exist}

word="horse"
echo "${word:+bird} is the word"
```

Command Substitution

This is probably one of the most useful Bash features. It means running a shell command and storing its output into a variable for later use. There are two syntaxes for this concept: ``command`` or `$(command)`.

```
now="$(date +%Y-%m-%d %H:%M:%S)" # Since the value is a string, it's even better to enclose in quotes
echo $now

users=$(who)
echo $users

output="$(ps -ef | grep bash)"
echo $output
```

It's common to put the date in backup file names.

```
tar -czvf "scripts$(date +%F_%H%M).tar.gz" ~/Scripts/
```

Arithmetic Expansion

We can do mathematical operations using arithmetic expansion. Syntax: `$((operations))`.

Also, in Bash we use `let` to declare integer variables (this is very different from `let` in JS — only the names are the same).

```
let x=$(( 5*2 ))
echo $x

let y=$(( 7*9 ))
echo $y

let y=$(( 2*8 ))
```



```
echo $y
```

```
let y=$(( 2**128 ))    # This is an overflow, because the result is too big to be stored in th
echo $y
```

There is a limitation in Bash: it can only work with integers and cannot handle other number types like floating points. If you want to do decimal operations, you should use another command like `bc` (which stands for “basic calculator”). You can pipe an arithmetic expression as a string to `bc`.

```
res=$(echo "2.5 * 10" | bc)
echo $res
```

```
res2=$(echo "11 / 4" | bc)
echo $res2
```

Why doesn’t this result contain a decimal part? Even though all numbers are represented internally by `bc` as decimals, it won’t display any fractional part by default. To see the fractional part, you have to specify the `scale`, which is the total number of decimal digits after the decimal point.

```
res3=$(echo "scale=2; 11 / 4" | bc)
echo $res3
```

We can also use it like this:

```
echo $(bc <<< "scale=3; 5 / 3")
```

Bash includes the triple-less-than redirection operator, which allows a string to be used as the standard input to a command. This is also called a “**here string**”.

`bc` also has an interactive mode.

Process Substitution

This means referencing the output of a process as a file. You’ve probably heard that in Linux, almost everything is a file, so the output of any process can also be internally represented as a file.

With process substitution, you can feed the output of one or more processes into the `stdin` of another process.

Syntax: `<(command)`

For example, `<(ls)` means `ls` will run and its output will appear as a file. We can then use this output as input for another command.

```
something=`cat <(ls)`
echo $something
echo ...
```

Imagine you want to compare the contents of two directories to see which file names are in one but not the other:

```
diff <(ls dir2) <(ls dir3)
```

Word Splitting

This means splitting the result of unquoted expansions into individual words.

When we write a command in the shell with a string containing several words, without enclosing it in `' '` or `" "`, the shell automatically splits that string into parts (words). This is called word splitting.

Bash uses the `IFS` variable (which by default contains space, tab, and newline) to perform word splitting. This variable tells the shell which characters to use as delimiters. `IFS` stands for “Internal Field Separator”, and we can change the delimiters by changing the `IFS` variable.

```
echo $IFS
echo ${IFS@Q}    # A trick to see the characters.

dirs="d1 d2 d3"
mkdir $dirs
mkdir "$dirs"
```

What is the difference between these two lines?

- The first one interprets `d1`, `d2`, `d3` as three separate values.
- The second one interprets them as a single string.

Bash only performs word splitting on the result of unquoted expansions.

```
IFS=":"
dirs="d1:d2:d3"
mkdir $dirs    # Bash will consider $dirs as three arguments: d1 d2 d3 – but if you use a space
```

Globbering (Filename Expansion)

Globbering is also known as filename expansion. There are three characters that perform globbing: `*`, `?`, and `[]`.

- `*` → matches any string, including an empty one (0 or more, like in regex). Hidden files (starting with a dot) will not be matched by an asterisk.
- `?` → matches any single character.
- `[]` → matches a single character within a range (just like regex).

```
#!/bin/bash
```

```
ls *
echo '***'
```

```
ls brace*.sh
echo '***'
```

```
ls dir[1,2,3]
```

```
echo '***'

ls alias?s.sh
ls pa??.sh
echo '***'
```

Also, when using [], just like regex, we can use the ^ operator to **not match** these characters.

```
ls -l dir[^1,2]
```

Intro to Shell Operation in Depth

In this section, we'll talk about how the shell operates.

Bash reads input from a terminal command or from a file, so when you run a script, Bash will basically read the script line by line, and for each line, it will go through a multi-step process to interpret and execute the command.

TOKENIZATION: The input is broken into arguments and operators, obeying the quoting rules described earlier. Alias expansion is also performed at this stage.

COMMAND IDENTIFICATION: The tokens are then parsed and identified as simple or compound parts.

SHELL EXPANSION: We've talked about this in an entire section, so as a short recap — Bash performs 8 types of expansions, in the following order:

brace expansion → tilde expansion → parameter and variable expansion → command substitution → arithmetic expansion → process substitution → word splitting → filename expansion

QUOTE REMOVAL: After shell expansion, Bash removes quotes.

REDIRECTIONS: Then it performs any necessary redirections.

COMMAND EXECUTION: And finally, it executes the command.

Tokenization

After the shell has read its input from a file or the user's terminal, it will break the input into parts, obeying the quoting rules. This process is called **tokenization**.

A **token** is a sequence of characters identified as a single unit by the shell. Bash uses metacharacters to break the command into tokens (words and operators).

METACHARACTERS: space, tab, newline, |, :, &, (,), <, > (10 in total).

At this point, the shell has tokenized the command line. There are two types of tokens:

- **Words** (don't contain any unquoted metacharacters)
- **Operators** (contain at least one unquoted metacharacter)

There are also two types of operators: **control operators** and **redirection operators** (Bash does not use redirection operators as separators).

A control operator is a token that performs a control function, such as: `newline`, `|`, `||`, `&`, `&&`, `;`, `::`, `;&`, `::&`, `|&`, `()`.

Redirection operators are used to redirect the data streams connected to a command, such as input or output, to a file. Some redirection operators: `>`, `>>`, `>|`, `<>`, `>&`, `&>`, `<`, `<<`, `<<<`, `<&`, `<<-`.

Operators only matter if they are unquoted. Alias expansion is also performed at this step.

```
ls $HOME > ./home.txt
```

In this example, Bash reads its input from a file and goes through a multi-step process to interpret and execute the command. In the first step, it breaks down the input into words and operators, obeying the quoting rules.

The first step of the tokenization process is to identify the metacharacters and use them to break down the command. There is one metacharacter in our example: space. So the tokens will be: `ls`, `$HOME`, `greaterThan`, `./home.txt` (4 tokens).

Next, it will divide the four tokens into words and operators. Words don't contain metacharacters, and operators contain at least one unquoted metacharacter. So we have 3 words and 1 operator.

Since `$` is not a metacharacter, `$HOME` is a word. Also note that `./home.txt` is also part of the command, because `>` is a redirection operator and not a control operator, and the shell uses only control operators to determine a command.

The control operator that terminates the command is the newline (`\n`), which is hidden but exists at the end of the line.

Command Identification

After tokenization, the shell parses and identifies the tokens as simple or compound commands.

A **simple command** is just a sequence of words separated by blanks and terminated by either a newline or one of the shell's control operators. A simple command is the type of command encountered most often.

```
ls dir1 dir2 dir3
```

The first word (`ls`) is the command name, and the others are the command's arguments, separated by spaces and terminated by a newline.

```
ls dir1 dir2 ls
```

The shell tokenizes the command by using unquoted metacharacters to split it into words and operators. In this example, `ls` is the command, and the second `ls` is considered an argument.

That's not what we wanted! We wanted to list two directories and then list all files in the current directory. Let's fix the command by adding a semicolon just before the second `ls`:

```
ls dir1 dir2; ls
```

Simple commands are terminated by either control operators or a newline. Since we have one control operator and one hidden newline, Bash will treat this as two simple commands.

A **compound command** is a command that starts and ends with a Bash reserved word, such as: `if`, `then`, `elif`, `else`, `fi`, `time`, `for`, `in`, `until`, `while`, `do`, `done`, `case`, `esac`, `coproc`, `select`, `function`, `{`, `}`, `[[`, `]]`, `!`.

By the way, we can check if a word is a shell reserved word with `type <word>`:

```
type if
type ls
```

Reserved words start a compound command and represent the Bash programming language syntax. Compound commands are used to create loops, iterations, or run a block of code only if some conditions are met.

```
file="/etc/passwd"
if [[ -f $file ]]
then
    cat $file | grep hikari
fi
```

Compound commands can be written across multiple lines and can contain multiple simple and compound commands inside them.

What happens after Command Identification? Shell Expansion, the procedure to get the value of the referenced entity like expanding a variable to get its value.

Quote Removal

All unquoted occurrences of backslash, single quotes, and double quotes that did not result from an expansion in the previous step are removed.

Quotes normally serve to remove the special meaning of certain characters, and once the shell has used the quotes for this purpose, it removes them.

```
echo $USER
```

Let's say we want the literal word `$USER`:

```
echo \$USER    # Where did the backslash go?
```

It was removed during quote removal. Bash figured out that the `\` was only used to cancel the special meaning of `$`, so it removed it.

```
echo "HELLO"    # Single quotes were removed during quote removal, but the double quotes are
```

```
dir="C:\Windows\System"
```

```
echo $dir    # Bash will not remove the backslashes because they are the result of an expansion
```

Redirections

Every shell command has three data streams connected to it. These data streams are identified by numbers:

- **stdin** — 0, < — data that the program is working with, or the data fed into the program. By default, stdin is the terminal, but it also provides an alternative way of giving input to a command aside from using command-line arguments.
- **stdout** — 1, >, >> — data printed by the program. By default, this is the terminal.
- **stderr** — 2, 2>, 2>> — error messages that the command produces. The default is also the terminal.

Let's instruct the shell to read input from a file instead of the keyboard:

```
tail < /etc/group
cat < /etc/passwd
```

Normally, when we run a command, we want to see the output in the terminal, but sometimes we may wish to save it to a file.

```
cat < ./globbing.sh > stdout.test    # Using stdin and stdout at the same time.
# Use > to overwrite and >> to append
# ip address >> stdout.test
```

We've heard that everything in Linux is a file — I'll show you how even the terminal is treated as a file. To see the file which represents the terminal, run the `tty` command:

```
tty
```

This will return something like `/dev/pts/<number>`. Open another terminal and run `tty` again — the number must be different. In this case, my other terminal is `/dev/pts/1`.

```
ip address > /dev/pts/1
echo hello > /dev/pts/1
```

The result will be printed on the other terminal.

Shell Operations Recap

```
ls $HOME > $(date +%H_%M).txt
```

In this example, the shell takes its input from the user's terminal, and the first step is tokenization. For tokenization, the shell looks for unquoted metacharacters — here we have two: space and the `>` operator.

I've told you that Bash uses metacharacters as delimiters, but there are 2 types of delimiters:

- **Whitespace delimiter:** separates tokens and gets removed. `a b` → `'a'` `'b'`
- **Operator delimiter:** separates tokens and creates a token of itself. `a>b` → `'a'` `'>'` `'b'`

After tokenization comes command identification. Our whole line is a simple command, terminated by a newline.

Then come the shell expansions (in this example, variable expansion and command substitution). The shell will replace `$HOME` with its value, and command substitution with the output of the command. The result will look like this:

```
ls /home/hikari > 10_15.txt
```

After that, all unquoted occurrences of backslashes, single quotes, and double quotes that did not result from an expansion are removed. In our example, there are no backslashes, single quotes, or double quotes, so it moves to the next step.

Redirection: the shell looks for any redirection operator, and there is one — `>` — which performs output redirection. So the shell knows it has to redirect the output to a file instead of stdout (the terminal).

The shell can now execute the command.

Conditions

Like every other programming language, Bash has conditional statements. From now on, we can add logic to our scripts and control the program flow. Let's talk about the `if/else` block first.

```
if [ some condition ]
then
    do something
elif [ some other condition ]
then
    do something else
else
    do this if none of the above was true
fi
```

While writing `if` (and every other syntax in Bash), be careful to obey the syntax rules, or you will get errors. For example, don't forget the spaces in the statement, like in the block above.

`[ condition ]` is just shorthand for the `test` command. Nowadays we use `[[ condition ]]`, which is better for scripting and is like an advanced version of `test` with more operators.

Some operators:

- `==` → equal to (strings)
- `!=` → not equal to (strings)
- `-gt` → greater than
- `-lt` → less than
- `-ge` → greater than or equal to
- `-le` → less than or equal to
- `-eq` → equal to (numbers)
- `-ne` → not equal to (numbers)
- `-f` → check if a file exists / item is a regular file
- `-d` → check if a directory exists / item is a directory
- `-e` → check if the thing exists (it may be a directory)
- `-s` → check if a file exists and its size is more than 0

- `-x` → check if a file exists and is executable
- `-n` → check if a string is not empty
- `-z` → check if a string is empty

Notice that I used quoting for the variable to prevent word splitting. Using double square brackets (the newer syntax) will also prevent word splitting. It also has some other nice features, like regex matching and more operators like logical AND and OR.

```
if [ -f "$1" ]
then
    echo "this is a file"
    sleep 1
    cat "$1"
elif [ -d "$1" ]
then
    echo "this is a directory"
    sleep 1
    ls -l "$1"
else
    echo "file or directory does not exist"
fi
```

Nested Conditions

We can nest our conditional statements to build more complex logic.

Here I'm using the logical `&&` (AND) and `||` (OR) operators. We can also use `-a` for AND and `-o` for OR inside `[...]` (not `[[...]]`).

```
#!/bin/bash

read -p "enter your age: " age
if [[ $# -eq 1 ]]
then
    if [[ age -lt 18 && age -gt 0 ]]
    then
        echo 'you are a minor!'
    elif [[ age -eq 18 ]]
    then
        echo 'you have just became a major'
    elif [[ age -gt 18 && age -lt 100 ]]
    then
        echo 'you are a major'
    else
        echo 'really?'
    fi
else
    echo you can only have one age.
fi
```


String Comparisons

When using the `test` command to check conditions, it's important to obey the syntax rules. If you want to check whether two strings are the same or not, use `=` for `[ ]` and `==` for `[[ ]]`.

- `-n` checks if the string is not empty
- `-z` checks if the string is empty

```
read -p "String1: " str1
read -p "String2: " str2

# In [ ], use quote marks to prevent word splitting.
if [ "$str1" = "$str2" ]      # We could also say: if [[ "$str1" == "$str2" ]]
then
    echo 'the strings are equal'
else
    echo 'not equal'
fi
```

Checking if a String Contains a Substring in Bash

How to check whether a string contains a substring using asterisk pattern matching in Bash.

This example demonstrates using wildcard `*` to match a substring inside a string variable.

```
str1="Nowadays, Linux powers most of the servers on the internet."

if [[ "$str1" == *"Linux"* ]]
then
    echo 'yes it exists.'
else
    echo 'nope.'
fi
```

Checking Network Connectivity Using Ping in Bash

This script checks whether a remote system is reachable by sending ICMP packets using `ping`. The target IP address is provided as the first command-line argument.

It runs a ping test with 3 packets and stores the output in a variable called `output`.

```
#!/bin/bash

# The IP address of the target (remote system) is passed as the first argument.
echo "Please wait while I check the connection..."

output="$(ping -c 3 $1)"

# echo $output
```

```

if [[ $output == *"100% packet loss"* ]]
then
    echo "The network connection to $1 is down"
else
    echo "The network connection to $1 is working"
fi

```

Using the **case** Statement in Bash

The **case** statement allows us to test strings and numbers and is an elegant way to implement logic in our programs.

Remember the switch/case statement in JavaScript? It can be something similar.

With **case**, the shell checks the conditions and controls the flow of the program.

It serves as a shorthand for multiple **if**, **elif**, and **else** statements.

Each **case** statement starts with the **case** keyword, followed by a variable to test and the **in** keyword. The block ends with **esac**.

We can use multiple patterns separated by a vertical bar (**|**). The parentheses operator terminates the pattern list.

A pattern can contain special characters like asterisks and question marks. A pattern and its associated commands are known as a clause and must be terminated with double semicolons (**;;**).

The commands corresponding to the first pattern that matches the variable value are executed.

It is common to use the ***** symbol as the final pattern to define the default case. This pattern will always match.

```

echo -n "Enter your favorite pet: "
read pet

case "$pet" in
    "dog")
        echo "your favorite pet is doggy"
        ;;
    "cat" | "Cat" | "kitty")
        echo "your favorite pet is a cat"
        ;;
    fish | "African trurtle")
        echo "fish or turtles are great..."
        ;;
    *)
        echo "your favorite pet is unknown"
esac

```

Sending Signals to a Process Using Bash

This program takes a kill signal and a PID as arguments, then sends the signal to the specified process.

If exactly two arguments are provided, it uses a **case** statement to decide which signal to send based on the first argument.

```
#!/bin/bash

# this program takes a kill signal and a PID, then sends the signal to process.
if [[ $# -eq 2 ]]
then
    case "$1" in
        1)
            echo "Sending the SIGHUP signal to $2"
            kill -SIGHUP $2
            ;;
        2)
            echo "Sending the SIGINT signal to $2"
            kill -SIGINT $2
            ;;
        15)
            echo "Sending the SIGTERM signal to $2"
            kill -15 $2    # this is the default signal
            ;;
        *)
            echo "Signal number $1 is not valid."
    esac
else
    echo "This script should take exactly 2 arguments, signal and PID."
    echo "Usage: ./scriptName 1 <PID>"
    exit 1
fi
```

Bash Menues and Select Statement

The **select** construct in Bash allows us to easily generate menus.

When the **select** statement is reached, each item in the list is printed on the screen preceded by a number starting from 1.

COUNTRY is a user-defined variable, and the list of options can also come from an array (this will be explained in the future).

When the user selects an item, the variable **COUNTRY** is set to that value, and the variable **REPLY** is set to its associated number.

If the user does not enter anything, the selection prompt will be shown again and again. The **select** loop continues to run until the **break** command is executed.

When running the script, the default prompt is **#?**. We can change it by modifying the predefined variable **PS3**.

```
PS3="Choose your country: "

select COUNTRY in Germany France USA "United Kingdom"
```

```
do
    echo "Your country is: $COUNTRY"
    echo "REPLY is $REPLY"
    break
done
```

Adding Logic to a Bash select Menu

Let's add some logic to our previous select example:

```
PS3="Choose your country: "

select COUNTRY in Germany France USA "United Kingdom" Quit
do
    case $REPLY in
        1)
            echo "you are from Germany($COUNTRY) and you speak duech"
            ;;
        2)
            echo "you speak french"
            ;;
        3)
            echo "you speak american english"
            ;;
        4)
            echo "you speack british english"
            ;;
        5)
            echo "Quiting..."
            sleep 1
            exit 0
            ;;
        *)
            echo "item $REPLY is not in the list"
    esac

    break
done
```

Bash System Administration Menu Script

This script combines previous Bash concepts and builds a simple system administration menu using `select`.

It allows the user to choose between several administrative tasks such as adding a user, listing processes, killing a process, installing a program, or quitting the script.

The menu is controlled using the `PS3` prompt and the `select` construct, while the logic is handled using `if`, `elif`, and case-style numeric checks on `REPLY`.

```
#!/bin/bash
```

```
# lets put together whatever we've learned so far and develop a script that does some common a
```

```
PS3='Your choice: '
```

```
select ITEM in "Add User" "List All Processes" "Kill process" "Install Some Program" "Quit"  
do
```

```
    if [[ $REPLY -eq 1 ]]
```

```
    then
```

```
        read -p "Enter the username: " username
```

```
        output=$(grep -w $username /etc/passwd)
```

```
        if [[ -n "$output" ]]
```

```
        then
```

```
            echo "The user $username already exists."
```

```
        else
```

```
            sudo useradd -m -s /bin/fish "$username"
```

```
            if [[ $? -eq 0 ]]
```

```
            then
```

```
                echo "$username has been added successfully."
```

```
                tail -n 1 /etc/passwd
```

```
            else
```

```
                echo "There was an error adding the user: exit code: $?"
```

```
            fi
```

```
        fi
```

```
        break
```

```
    elif [[ $REPLY -eq 2 ]]
```

```
    then
```

```
        echo "Listing all processes..."
```

```
        sleep 1
```

```
        ps -ef
```

```
        break
```

```
    elif [[ $REPLY -eq 3 ]]
```

```
    then
```

```
        read -p "Enter the process ID to be killed: " process
```

```
        kill $process
```

```
        echo "Done."
```

```
        break
```

```
    elif [[ $REPLY -eq 4 ]]
```

```
    then
```

```
        read -p "Enter the program name to be installed (dont missSpell): " progName
```

```
        sudo apt update && sudo apt install $progName -y
```

```
        if [[ $? -eq 0 ]]
```

```
        then
```

```
            echo "$progName installed successfully"
```

```
        else
```

```

        echo "Could not install $progName\... Did you enter the name correctly?"
    fi
    break

elif [[ $REPLY -eq 5 ]]
then
    echo "Quitting the Admin Automation..."
    sleep 1
    exit 0
    break

else
    echo "Item $REPLY not found... exiting with code 1."
    exit 1
fi
break

done

```

Chaining Commands

Instead of writing each command on its own line, we can chain them together using operators. Bash supports running a list of commands — a sequence of one or more commands written on the same line, separated by one of these operators: `;`, `&`, `&&`, `||`.

Semicolon `;`

The shell waits for each command to finish in turn. The return status is the exit status of the last command executed:

```
mkdir cmdChain; cd cmdChain; touch somefilehere
sleep 5; echo hello
```

This is useful when you want to wait for a specific command to finish before running the next one.

Ampersand `&`

The shell executes the command asynchronously in a subshell — also known as running a command in the background. The shell does not wait for the async command to finish before moving to the next one:

```
sleep 10 & ls
```

Logical AND &&

Runs the second command only if the first one completes successfully:

```
cat /etc/shadow && echo success
```

Logical OR ||

Runs commands from left to right and stops as soon as one succeeds:

```
echo hi || echo this is second
cat echo ls || echo this is second || echo this is third.
```

Practical Script: DoS Attack Without Root Permission — Fork Bomb

A fork bomb (also known as a rabbit virus) is a type of DoS attack that can be executed by an unprivileged user and can bring down an entire Linux system. It works on every Linux distro with just a single line of code.

Warning: Only run this on a VM or a test system you have physical access to. Prepare for a crash and a forced reboot.

```
$0 && $0 &
```

This single line causes resource starvation and crashes the system. `$0` is a special variable that represents the script itself, so the script runs itself recursively twice and then moves to the background for another recursive call — this repeats until the system runs out of resources.

To see the number of available processes for the current user:

```
ulimit -u
```

To see all limits available to the shell and the processes it creates:

```
ulimit -a
```

To prevent fork bombs, lower the maximum number of processes a user can start, so they cannot replicate indefinitely. Edit the limits config:

```
sudo vim /etc/security/limits.conf
```

At the end of the file, add:

```
<userName>      hard    nproc    2000
```

For Loops

Bash supports loops just like any other programming language. The basic syntax of a for loop is:

```
for item in list_of_values
do
    # commands to run
done
```

Here is a simple example iterating over a list of strings:

```
for os in ubuntu "pop os" debian slackWare kali
do
    echo "os is $os"
done
```

Iterating Over Command Output

You can also iterate over the output of a command. The example below loops over all items in the current directory and checks if each one is a regular file:

```
for item in ./*
do
    if [[ -f $item ]]
    then
        echo "$item is a file"
        sleep 0.3
    fi
done
```

Iterating Over a Variable

```
numbers="1 2 3 4 5 6 7 9"

for i in $(echo $numbers)
do
    echo -n "$i - "
done
echo
```

C-Style For Loop

There is also a C-style form of the `for` loop. It is not common in `bash`, but it works:

```
for ((i=0; i<10; i++))
do
    echo $i
done
```

While Loops

While loops are used when you want to loop over something but do not know the exact number of iterations. The loop continues as long as a condition is true. The condition

must be changed at some point, otherwise it becomes an infinite loop and may crash the system.

```
# while [[ condition ]]
# do
#     run some task
#     change the state if needed
# done
```

Here is a basic example:

```
i=0

while [[ $i -lt 10 ]]
do
    echo "i=$i"
    ((i++))
done
```

To create an infinite loop:

```
while [[ 1 ]]
do
    echo "Infinite loop. press CTRL + C to exit"
    sleep 1
done
```

While loops can also be written as a one-liner:

```
cat ./IPS.txt | while read II; do echo $II; sleep 1; done
```

Practical Script: Process Monitor

Imagine you have a server that needs to be continuously monitored. A cron job is not suitable here because its minimum interval is one minute. Instead, this script uses a while loop to check whether a given process is running every 3 seconds.

Save the following into a file called `monitoringWithWhile.sh` and run it with `bash monitoringWithWhile.sh <process_name>`:

```
#!/bin/bash

while :      # the colon is a bash built-in that always returns true – another way to create an
              # you can also use the 'true' keyword instead: while true
do
    # the process to monitor is passed as the first argument ($1)
    # pgrep searches for the process and stores the output in a variable
    output=$(pgrep -l $1)

    # if the process is running, the output will not be empty
    # if it is not running, the output will be an empty string
    if [[ -n $output ]]
    then
```

```
    echo "The process \"$1\" is running"
else
    echo "The process \"$1\" is not running"
fi

sleep 3
done
```

While Loops — Using Commands as Conditions

Instead of a condition that returns true or false, a `while` loop can take any command. As long as the exit code is `0`, it is considered true; any other exit code is considered false.

For example, loop while a host is reachable:

```
# while ping -c 1 example.net
# do
#     echo ping was successful
#     sleep 1
# done
```

The following example reads the output of `ls -ltrh` line by line and prints each line:

```
while read name
do
    echo $name
    sleep 0.1
done <<(ls -ltrh)    # this redirects the output of ls into the while loop as a temporary file
```

File Descriptors

The `read` command reads from a file descriptor, which is `stdin` by default. When a command is executed, the OS assigns it three predefined file descriptors:

FD	Purpose
0	stdin — for receiving input
1	stdout — for sending output
2	stderr — for sending error messages

When you run `read` in a terminal, it waits for input from `stdin`, which by default points to the terminal itself. A file descriptor is essentially a source that the OS can read data from or write data to. The terminal is treated like a file by the OS (recall the `tty` command).

So by default, `read` reads from `stdin`, which is the terminal. In the example above, we are redirecting the output of `ls` into the `while` loop, which in turn passes it as input to `read`. This can also be written as a single-line pipe.

Practical Script: Bulk Packet Dropper

This script reads a list of IP addresses from a file and drops all incoming packets from each one.

Using a For Loop

Save the following into a file called `packetDropperForLoop.sh` and run it with `sudo bash packetDropperForLoop.sh`:

```
#!/bin/bash

for ip in $(cat ./IPS.txt)
do
    echo "Dropping all packets from: $ip"
    sleep 1
    iptables -I INPUT -s $ip -j DROP
done
```

IFS and the Read Command

Before looking at the `while` loop version, it helps to understand `IFS`.

The `IFS` (Internal Field Separator) variable holds the characters that `bash` uses to separate fields in an input line. By default, `bash` uses spaces, tabs, and newlines as separators.

When `IFS` is set to an empty string, `bash` uses only newlines as separators. Since space is no longer a separator, the `read` command reads the entire line without stripping leading or trailing whitespace. If the line contains any `IFS` separators, `read` treats the whole line as a single field and saves it to the variable. This prevents unintended word splitting.

You can test this behavior directly in the terminal (set the `IFS` to "" first -> `IFS=""`):

```
read -r vara varb varc <<< "one two three"
echo $vara
echo $varb
echo $varc
```

The expected output is the entire line assigned to `vara`, with `varb` and `varc` being empty.

Using a While Loop

As long as there are lines left to read, the block between `do` and `done` keeps executing. Save the following into a file called `packetDropperWhileLoop.sh` and run it with `sudo bash packetDropperWhileLoop.sh`:

```
#!/bin/bash

while IFS="" read -r IP;
do
    echo "Dropping all packets from: $IP"
    sleep 1
done
```

```
iptables -I INPUT -s $IP -j DROP
done < <(cat ./IPS.txt)
```

IPS.txt

```
1.1.1.1
8.8.4.4
6.10.100.60
8.8.8.8
10.0.0.10
```

Until Loops

Bash also has an until loop, which executes a block of code until a condition is met — the opposite of a while loop.

```
age=10

until [[ $age -eq 18 ]]
do
    echo "you cant do this"
    echo "you are $age"
    let age=age+1
done

echo "now that youre $age you can do that task"
```

Breaking Loops

Loops run until a condition is met or a list is fully iterated. Sometimes it is necessary to alter the normal flow and terminate early — for example, if you are searching for a value in a list of 10,000 items and find it after just 10 iterations, completing all remaining iterations would be a waste of resources.

Bash provides **break** and **continue** statements to control loop execution.

break exits a for, while, until, or select loop immediately.

```
i=0

while [[ $i -lt 7 ]]
do
    echo "i is $i"
    (( i++ ))
    if [[ $i -eq 3 ]]
    then
        echo "i is $i and im exiting the loop"
        break
    fi
done
```

Practical Example: Ping Monitor with Break

This script continuously pings a host passed as the first argument. If the host stops responding, the loop breaks and a notification is printed. Save the following into a file called `breakingLoops.sh` and run it with `bash breakingLoops.sh <ip_or_domain>`:

```
while true
do
    ping -c 1 $1 &> /dev/null    # &> redirects both stdout and stderr – equivalent to: comman
    if [[ $? -eq 0 ]]
    then
        echo "OK"
    else
        echo "Exiting the while loop"
        break
    fi
    sleep 1
done

echo "Ping to $1 is not working, sending an email to the admin."
```

Continue in Loops

The `continue` statement works similarly to `break`, but instead of exiting the loop entirely, it skips the current iteration and passes control to the next one.

The following example prints only odd numbers between 1 and 15 by skipping even iterations:

```
for i in {1..15}
do
    result=$(( i % 2 ))

    # if result is zero, i is even – skip this iteration
    if [[ $result -eq 0 ]]
    then
        continue
    fi

    echo $i
done
```

Introduction to Bash Arrays

Up until now we have worked only with variables, but a variable can hold only a single value. Bash provides one-dimensional indexed and associative arrays. Think of an array as a variable that stores multiple values. Arrays are one of the most fundamental data structures in any programming language and are useful for storing related data together.

Declaring and Accessing Indexed Arrays

Arrays can be declared in different ways:

```
ages=(20 22 24 38 40)
```

This is a numerically indexed array — every element has a position starting from index 0.

Using `$ages` notation only prints the first element (index 0). To print the entire array:

```
echo $ages           # prints only the first element
echo ${ages[@]}      # prints all elements – @ preserves word splitting
echo ${ages[*]}       # prints all elements – * does not preserve word splitting
```

To print only the indexes of the array:

```
echo ${!ages[@]}
```

To get the length of the array:

```
echo $#ages[@]
```

To access a specific element by index, or using a negative index:

```
echo ${ages[2]}      # third element
echo ${ages[-1]}     # last element
```

Pointing to an index that does not exist will not throw an error — it simply returns an empty value:

```
echo ${ages[9]}
```

Adding and Modifying Elements

An indexed array is created automatically when a value is assigned using this syntax:

```
numbers[0]=100       # creates a new array called numbers with one element at index 0
numbers[1]=200       # adds a second element
echo ${numbers[@]}
echo ${!numbers[@]}
```

To change a value:

```
numbers[1]=400
echo ${numbers[@]}
```

Explicitly Declaring an Array

To explicitly declare an array, use the `declare` built-in:

```
declare -a names
names[0]="Dan"
names[1]="Alina"
names[2]="Diana"
```

Array members do not need to be consecutive. You can add an element at any index, even if there are gaps:

```
names[7]="Maya"
echo ${names[@]}
echo ${!names[@]}
```

Arrays with gaps are called sparse arrays and are common in spreadsheet processing where gaps in data are expected.

Removing Elements

To remove an element, use `unset`:

```
unset names[1]
echo ${names[@]}
echo ${!names[@]}
```

The index itself is also removed. Note that indexed arrays do not reindex automatically after a removal — the gap remains.

Arrays In Depth

Appending to an Array

To add one or more elements to the end of an array, use the `+=` operator:

```
years=(2020 2021 2022 2023)
years+=(2024 2025 2026 2027)
echo ${years[0]}
```

Slicing Arrays

To print all elements starting from a specific index:

```
echo ${years[@]:2}
```

To print a specific number of elements starting at an index, the second number is the length:

```
echo ${years[@]:2:4}
```

Associative Arrays

Associative arrays store key-value pairs, similar to a dictionary, map, or hash in other languages. The main difference from indexed arrays is that instead of integer index positions, associative arrays use string keys to map values.

Unlike indexed arrays, associative arrays must be explicitly declared with `declare -A`:

```
declare -A userdata
userdata[username]="admin"
userdata[password]="root"
userdata[uid]="0110"
```

To query a value by key:

```
echo ${userdata[password]}
echo ${userdata[@]}
echo keys:
echo ${!userdata[@]}
```

Note that the printed values are not guaranteed to be in any particular order.

Adding new elements:

```
userdata[login]=$(date +%H:%M:%S)
```

You can also add multiple elements at once:

```
userdata+=( [shell]="bash" [admin]="True" )
```

Iterating over an associative array:

```
for key in "${!userdata[@]}"
do
    echo key: $key, value: ${userdata[$key]}
done
```

Read-Only Arrays

Just like variables, arrays can be declared as read-only using the `-r` flag:

```
declare -r -A SUPERSTARS=( [Germany]="Boney M" [USA]="Bon Jovi" [England]="The Beatles" )
echo ${SUPERSTARS[@]}
SUPERSTARS[USA]="Sora"    # this will throw an error
```

Removing Elements and Arrays

To remove a specific key from an associative array:

```
unset userdata[password]
echo ${userdata[@]}
```

To remove the entire array:


```
unset userdata
```

Random Numbers in Bash

Bash has a built-in environment variable called `$RANDOM`. Every time it is referenced, it generates a random number between 0 and 32767:

```
echo $RANDOM
```

The range cannot be changed directly, but you can use arithmetic expansion to get a number within a specific range:

```
echo $(( RANDOM % 100 + 1 ))
```

This produces a random number between 1 and 100, because any number modulo 100 always falls between 0 and 99, and adding 1 shifts the range up by one.

Word Splitting and Arrays

Word splitting is an important concept when working with arrays in bash. Consider a list of user accounts where some usernames contain spaces:

users.txt

```
John Doe
root
admin
test user
```

Without Quotations

Save the following into a file called `wordSplitting.sh` and run it with `bash wordSplitting.sh`:

```
#!/bin/bash
```

```
readarray accounts < users.txt
```

```
echo without quotations
```

```
for item in ${accounts[@]}
```

```
do
```

```
    echo "Processing: $item"
```

```
done
```

Without quotes, bash applies word splitting to the array elements. This means entries like John Doe and test user get split into separate words, so they are processed incorrectly as individual items.

With Quotations

```
#!/bin/bash

readarray accounts < users.txt

echo with quotations
for item in "${accounts[@]}"
do
    # use printf instead of echo to avoid printing blank lines
    printf "Processing: %s" "$item"
done
```

Wrapping `"${accounts[@]}"` in double quotes preserves each element as a single unit, including those with spaces. Using `printf` instead of `echo` here also avoids printing unwanted blank lines between entries.

Error Handling in Bash

Just like `try/except` in Python or `try/catch` in JavaScript, bash has its own ways to handle errors.

Method 1: Parameter Expansion with ?

The `${variable:?message}` syntax checks if a variable exists and has a value. If it is empty or undefined, the script exits and displays the message:

```
foo(){
    local val=${1:? "Please enter some valid value..."}
    echo "hello $val"
}

read -p "whats your name? " name
foo $name
```

Method 2: Inspecting the Exit Code

The `$?` environment variable holds the exit code of the last executed command. `0` means success; anything else means failure. You can make decisions based on this to handle errors gracefully:

```
read -t 5 -p "you have 5 seconds to tell me your name: " fname

if [[ $? -eq 142 ]]
then
    echo "your time is up!"
else
```

```
    echo "you entered: $fname"
fi
```

If the user does not enter anything within 5 seconds, the return code will be 142.

Method 3: Set Options for Strict Error Handling

Save the following into a file called `errorHandling.sh` and run it with `bash errorHandling.sh <value>`:

```
#!/bin/bash

# Exit immediately if a command exits with a non-zero status.
set -e

# Treat unset variables as an error and display a standard error message.
set -u

# If any command in a pipeline fails, the entire pipeline returns that failure exit code.
set -o pipefail

# Check if a parameter was provided
input_value=${1}

# Simulate a command that might fail
ls /non_existent_directory
echo "This line will not be printed if ls fails because of set -e"

# Simulate a command that might fail in a pipeline
echo "hello" | grep "world"    # this will fail because "world" is not in "hello"
echo "This line will not be printed if grep fails because of set -o pipefail"

echo "Script finished successfully. Input was: $input_value"
```

Here is a summary of the three `set` options:

Option	Behavior
<code>set -e</code>	Exits immediately if any command returns a non-zero status
<code>set -u</code>	Treats unset variables as an error
<code>set -o pipefail</code>	Makes a pipeline fail if any command within it fails, not just the last one

The readarray Command

`readarray` reads from `stdin` and converts **each line** into an element of an indexed array.

```
readarray months
echo ${months[@]}
```

After each value, press Enter to move to the next line. When done, press **Ctrl+D**. This works, but it is not very practical — most of the time you want to populate arrays from command output or file contents.

To read a file into an array where each line becomes one element, use input redirection with process substitution. The **-t** flag removes trailing newlines from each element:

```
readarray -t days <<(cat daysOfWeek.txt)    # < is input redirection, followed by process sub
echo ${days[@]}
echo ${days[@]@Q}    # shows hidden characters like \n
```

daysOfWeek.txt

```
sat
sun
mon
tue
wed
thu
fri
```

A few more practical examples — reading users from **/etc/passwd** and listing files in **/etc**:

```
readarray users <<(cut -d":" -f1 /etc/passwd)
echo ${users[@]}

readarray -t files <<(ls /etc)
echo ${files[@]}
```

Iterating Over Arrays

This example reads all files in **/etc** into an array and iterates over them. For each entry, it checks whether the item is a regular file and is readable before printing its contents:

```
readarray -t files <<(ls /etc/*)

for file in "${files[@]}"
do
    if [[ -f $file && -r $file ]]
    then
        cat $file
        echo "*****"
    fi
done
```

Practical Script: System Account Creation

This script reads a file containing usernames and group names, then checks whether each group and user already exists. If the group does not exist, it creates it. If the user does not exist, it creates them and adds them to the group.

`readarray` (also known as `mapfile`) is a powerful bash command that reads a file line by line and stores each line as an element in an array. This is especially useful when each line of a file contains structured information and you want to perform operations on each entry.

For example, given a file with this content:

```
apple
banana
cherry
```

You can load it into an array using either of these methods:

```
readarray fruits < <(cat file.txt)    # useful when working with command output
# or
readarray fruits < ./file.txt
```

After that, `fruits[0]` is `apple`, `fruits[1]` is `banana`, and so on.

users.txt

```
test:testgrp
test2:testgrp2
```

systemAccountCreationProject.sh

Save the following into a file called `systemAccountCreationProject.sh` and run it with `sudo bash systemAccountCreationProject.sh`:

```
#!/bin/bash

readarray accounts < <(cat ./users.txt)

for account in "${accounts[@]}"    # using double quotes prevents word splitting
do
    # extract username (everything before the colon)
    user=$(echo $account | cut -d":" -f1)

    # extract group (everything after the colon)
    group=$(echo $account | cut -d":" -f2)

    # check if group already exists in /etc/group
    if [[ -z "$(grep -w $group /etc/group)" ]]    # -w matches only the complete word, so 'adm
    then
        groupadd $group
        echo "Group $group added successfully!"
    else
```

[illegible]

Introduction to Bash Functions

Functions are used to organize code into reusable blocks. They improve readability, modularity, and help keep code DRY — Don't Repeat Yourself.

There are two ways to declare a function in bash:

```
# using the 'function' keyword
function log() {
    echo "Simple logging..."
    return 10
}

# without the 'function' keyword
display() {
    echo "Hey im displaying"
}
```

To call a function, write its name without parentheses — just like running a command:

```
log
echo $?
display
```

Passing Arguments to Functions

In other programming languages, arguments are passed inside the parentheses. In bash, the parentheses are purely decorative — never put anything inside them. Instead, pass data to a function the same way you pass arguments to a script, using `$1`, `$2`, and so on.

The following function creates files and gives them executable permissions. The filenames are passed as arguments:

```

create_files() {
    if [[ $# -eq 0 ]]
    then
        echo "You should pass me at least one argument!"
        exit 2
    fi

    for file in "$@"
    do
        if [[ -e "$file" ]]
        then
            echo "File: \"$file\" exists!"
        fi
        echo Creating "$file"
        touch "$file"
        chmod +x "$file"
        echo "Done!"
    done
}

create_files "$@"

```

Return Values

Unlike most programming languages, bash functions do not return actual values. The `return` keyword only sets the exit status of the function — by convention, 0 means success and any non-zero value means an error occurred. This is the same value stored in `?`.

If you need a function to produce a real return value, use command substitution. The following function returns the number of lines in a file that contain a specific string:

```

function lines_in_file() {
    # first argument is the search string, second argument is the filename
    grep -c "$1" "$2"
}

n=$(lines_in_file "random" "./random.sh")
echo $n

```

Variable Scope in Functions

Scope refers to which parts of a script a variable is visible in. By default, all variables in bash are global — visible everywhere inside the script.

```

var1="AA"
var2="BB"

func1() {

```

```

    echo "Inside func1: var1 is $var1 and var2 is $var2"

    # changing a global variable inside a function affects it everywhere
    var1="XX"

    # to declare a variable that only exists inside the function, use the 'local' keyword
    local var2="YY"
    echo "var2 after making it local: $var2"
    # the global var2 remains unchanged
}

func1
echo "After calling func1: var1 is $var1 and var2 is $var2"

```

It is a best practice to use the `local` keyword inside functions to prevent unintended changes to global variables.

Functions Recap

Functions are one of the most important concepts in any programming language — a block of code that can be reused multiple times with dynamic values.

Defining and Calling Functions

```

Welcome() {
    echo "Hello World!"
}

# call the function without parentheses
Welcome

```

Parameters

In bash, parameters are accessed with `$1`, `$2`, etc. — not inside the parentheses. To declare variables in a local scope, use the `local` keyword:

```

Greet() {
    local name=$1
    # if a function takes more than 9 parameters, use curly braces: ${10}
    echo "Hello $name how are you?"
}

read -p "what is your name? " NAME
Greet $NAME

```

Variable Number of Arguments

In Python we have `*args`, in JavaScript we have rest operators. In bash, functions can accept any number of arguments without special syntax:

Variable	Meaning
\$1, \$2, ...	individual arguments
\$@	all arguments as separate values
\$*	all arguments as a single string
\$#	total number of arguments

The main difference between `$@` and `$*` becomes apparent when they are wrapped in double quotes.

```
foo() {
    echo $@
    echo $*
    echo "Number of args: $#"
```



```
    echo 'with $@ outside quotes:'
    for arg in $@
    do
        echo "arg: $arg"
    done
```



```
    echo 'with $* outside quotes:'
    for arg in $*
    do
        echo "arg: $arg"
    done
```



```
    echo 'with "$@":'
    for arg in "$@"
    do
        echo "arg: $arg"
    done
```



```
    echo 'with "$*":'
    for arg in "$*"
    do
        echo "arg: $arg"
    done
}
```



```
foo 1 2 3 4 5 6 7 "sora"
```

Return Values

Bash functions behave differently from Python or JavaScript. The `return` keyword only sets an exit code, not an actual value:

- `return 0` — success
- `return 1` — failure

However, `echo` inside a function acts like a return value, because `bash` captures the function's stdout output through command substitution:

```
getSum() {
    local total=0
    for x in "$@"
    do
        total=$((total+x))
    done
    echo $total
}

# -a tells read to store input as an array
read -p "Numbers: " -a arr
result=$(getSum "${arr[@]}")
echo "Sum = $result"
```

You can also pass script arguments directly to the function — the script must be called with arguments like `./myscript.bash 1 2 3 4 5`:

```
result=$(getSum "$@")
```

Another example using backtick syntax for command substitution:

```
fun() {
    local num1=10
    local num2=11
    echo $((num1+num2))
}

res=`fun`
echo $res
```

Printing a Function Definition

You can retrieve the literal definition of a function using `declare -f`:

```
getFun(){
    declare -f $@
}

echo $(getFun fun)
# or
getFun fun
```

Bonus: AWK basics

`awk` processes text line by line and applies rules to each line. It is built into every Unix system. `##` `awk` is a small language with its own syntax and cannot fit into about 100 lines, Here are the simple and useful tricks: `>` `awk` variables don't need declaration, just use them and they exist!

```
awk 'pattern { action }' file
```

Fields and Built-in Variables

AWK splits each line into fields by whitespace automatically.

```
$0  whole line      NR  current line number
$1  first field     NF  number of fields
$NF last field
```

```
echo "hello world foo" | awk '{ print $1 }'      # hello
echo "hello world foo" | awk '{ print $NF }'      # foo
awk '{ print NR, $0 }' file.txt                  # print with line numbers
```

-F : Field Separator

```
awk -F: '{ print $1 }' /etc/passwd
echo "alice,30,dev" | awk -F, '{ print $1, $3 }' # alice dev
echo "192.168.1.5" | awk -F. '{ print $4 }'      # 5

# regex separator
echo "a,b;c" | awk -F'[;,]' '{ print $2 }'       # b
```

-v : Pass a Variable

```
threshold=100
awk -v limit="$threshold" '$1 > limit { print }' data.txt # if first number of each line is l
# 39
# 123
# 85
# 908

awk -v a=10 -v b=20 'BEGIN { print a + b }'          # 30 (print a + b at beginning)
```

Patterns and Filtering

```
awk '/error/'      server.log  # lines containing "error"
awk '!/error/'     server.log  # lines NOT containing "error"
awk '$3 > 500'      data.txt    # condition on a field
awk '$1 == "GET"'   access.log
awk '$2 ~ /^admin/' users.txt  # field matches regex
awk 'NR>=5 && NR<=10' file.txt  # line range
awk 'NF > 3'        file.txt    # lines with more than 3 fields
```

BEGIN and END

```
awk 'BEGIN { print "start" } { print } END { print "end" }' file.txt

# count matching lines
awk '/error/ { count++ } END { print count " errors" }' server.log

# sum a column
awk '{ sum += $1 } END { print "Total:", sum }' numbers.txt
```

Useful One-liners

```
# remove duplicate lines
awk '!seen[$0]++' file.txt

# reformat CSV to TSV
awk 'BEGIN{FS=","; OFS="\t"} {$1=$1; print}' file.csv

# count occurrences of each value in field 1
awk '{ count[$1]++ } END { for (k in count) print k, count[k] }' file.txt

# extract unique IPs from access log
awk '{ print $1 }' access.log | sort -u

# sum file sizes from ls -l
ls -l | awk 'NR>1 { sum += $5 } END { print sum, "bytes" }'
```

Bonus: jq Parsing

jq parses and transforms JSON from the command line. Install: `sudo apt install jq`.

```
jq 'filter' file.json
curl -s https://api.example.com | jq '.'
```

Basics

```
echo '{"name":"alice","age":30}' | jq '.'          # pretty-print
echo '{"name":"alice","age":30}' | jq '.name'     # "alice"
echo '{"user":{"city":"london"}}' | jq '.user.city' # "london"
```

`-r` strips quotes from strings.
`-c` gives compact single-line output.

```
jq -r '.name'      # alice (no quotes)
jq -c '.'          # {"name":"alice","age":30}
```

Arrays

```
echo '[1,2,3]' | jq '[0]'      # 1 (first)
echo '[1,2,3]' | jq '[-1]'     # 3 (last)
echo '[1,2,3]' | jq '[]'       # iterate all elements

# extract one field from each object
echo '[{"name":"alice"}, {"name":"bob"}]' | jq '[][.name]'
```

Build New Objects / Arrays

```
# keep only specific fields
echo '{"name":"alice","age":30,"city":"london"}' | jq '{name, age}'

# rename fields
echo '{"name":"alice"}' | jq '{username: .name}'

# collect into array
echo '[{"name":"alice"}, {"name":"bob"}]' | jq '[.[][.name]]'
# ["alice","bob"]
```

map and select

```
# transform every element
echo '[1,2,3]' | jq 'map(. * 2)'      # [2,4,6]

# filter elements by condition
echo '[{"age":30}, {"age":17}]' | jq '[.[] | select(.age >= 18)]'
```

-arg : Pass Shell Variables to jq

```
name="alice"
echo '[{"name":"alice"}, {"name":"bob"}]' \
  | jq --arg n "$name" '[] | select(.name == $n)'

# .[] loops through all array elements and checks a condition
# select acts like an if statement, so the final results will be alice object.
```

Useful One-liners

count items in an array

```
jq '.results | length' data.json
```

all unique values of a field

```
jq -r '[][.status] | unique[]' data.json
```

null-safe access

```
jq '.user?.email? // "no email"' data.json
```

export as CSV

```
jq -r '[] | [.id, .name, .email] | @csv' users.json
```

string interpolation

```
jq -r '"Host: \(.host) - IP: \(.ip)'" data.json
```

pipe IPs to another tool

```
jq -rc '[][.ip]' hosts.json | while read ip; do echo "scanning $ip";
```

Bash Tools

With Bash, you can automate almost any command-line workflow.

This includes system administration tasks, file operations, networking commands, and custom automation pipelines.

The focus of this section is building efficient scripts that simplify and speed up repetitive tasks.

Real World Projects: nice_katana — Active Web Crawler

A Bash wrapper around `katana` that crawls a list of URLs with JS parsing, form filling, and static asset filtering enabled. Results are saved per-host to `<host>.katana`.

Save it as `nice_katana.sh` and run it with `bash nice_katana.sh urls.txt`.

Dependencies

- `katana` — active web crawler
 - `unfurl` — extracts the domain from a URL
-

Script

```
#!/bin/bash

nice_katana() {
    while read line; do
        host=$(echo "$line" | unfurl format %d)
        echo "[*] Crawling $line ..."
        echo "$line" | katana \
            -js-crawl \
            -jsluice \
            -known-files all \
            -automatic-form-fill \
            -silent \
            -crawl-scope "$host" \
            -extension-filter json,js,fnt,ogg,css,jpg,jpeg,png,svg,img,gif,exe,mp4,flv,pdf,doc
        | tee "${host}.katana"
    done
}

if [[ -t 0 && -z "$1" ]]; then # if input is not coming from stdin ...
    echo "Usage:"
    echo "  $0 <file_with_urls>"
    echo "  cat urls.txt | $0"
    echo "  echo https://example.com | $0"
    exit 1
fi

if [[ -n "$1" ]]; then
    nice_katana < "$1" # redirect file to stdin
else
    nice_katana
fi
```

How to Use

Crawl URLs from a file:

```
bash nice_katana.sh urls.txt
```

Pipe URLs directly:

```
cat urls.txt | bash nice_katana.sh
echo https://example.com | bash nice_katana.sh
```

Results are saved to <host>.katana in the current directory.

Real World Projects: param-maker — Query String Builder

Reads a list of parameter names from a file and builds query strings with 25 parameters per line. Useful for mass parameter fuzzing.

Save it as param-maker.sh and run it with `bash param-maker.sh <file> <value>`.

Script

```
#!/bin/bash

echo ""
echo "USAGE: param_maker.sh <input file> <keyword>"
echo "~~~~~"
echo ""

param_maker() {
    filename="$1"
    value="$2"
    counter=0
    query_string=""

    while IFS= read -r keyword; do
        if [ -n "$keyword" ]; then
            counter=$((counter + 1))
            query_string="${query_string}${keyword}=${value}${counter}&"
        fi

        if [ $counter -eq 25 ]; then
            echo "${query_string%?}" # % means delete from end of string and ? means any single character
            query_string=""
            counter=0
        fi
    done
}
```

```
done < "$filename"

if [ $counter -gt 0 ]; then
    echo "${query_string%?}"
fi
}

param_maker "$1" "$2"
```

How to Use

```
bash param-maker.sh params.txt FUZZ
```

Given a file with one parameter name per line, the script outputs query strings of up to 25 parameters each:

```
?id=FUZZ1&name=FUZZ2&email=FUZZ3&...
```

Real World Projects: qs2trash — Query String Stripper

Strips the query string from each URL in a file, keeping only the base path.

Save it as `qs2trash.sh` and run it with `bash qs2trash.sh <file>`.

Script

```
#!/bin/bash
# usage: ./qs2trash.sh <input_file>

while IFS= read -r X; do
    echo "$X" | cut -d"?" -f1
done < "$1"
```

How to Use

```
bash qs2trash.sh urls.txt
```

Input:

```
https://example.com/page?id=1&name=foo
https://example.com/api?token=abc
```

Output:

<https://example.com/page>
<https://example.com/api>

Real World Projects: wlist_maker — Wordlist Builder

Builds a wordlist by inserting a custom value at specific positions among sequential numbers. Useful for testing mass assignment or parameter pollution where position matters.

Save it as `wlist_maker.sh` and run it with `bash wlist_maker.sh <value> [outfile]`.

Note: Don't underestimate how simple this looks — we're going to rely heavily on scripts like this later when we validate fuzzing results and automate verification workflows.

Script

```
#!/usr/bin/env bash
# usage: ./wlist_maker.sh VALUE [OUTFILE]

OUT=${2:-list.tmp}
VAL="$1"

seq 1 100 > "$OUT"
echo "$VAL" >> "$OUT"
seq 101 300 >> "$OUT"
echo "$VAL" >> "$OUT"
seq 301 600 >> "$OUT"
```

How to Use

```
bash wlist_maker.sh admin
bash wlist_maker.sh secret output.txt
```

The output file contains numbers 1–600 with the given value inserted after position 100 and 300.

Real World Projects: dns-brute-full — Full DNS Brute-force Pipeline

A multi-stage subdomain discovery script combining static wordlist brute-force, passive enumeration via `subfinder`, and permutation-based brute-force via `dnsenum`.

Save it as `dns-brute-full` and run it with `bash dns-brute-full <domain>`.

Dependencies

- shuffledns + massdns — fast DNS brute-force and resolution
 - subfinder + dnsx — passive subdomain enumeration
 - dnsngen — generates subdomain permutations
 - anew — appends unique lines to a file
-

Script

```
#!/bin/bash
```

```
# Paths – replace these with your actual paths
```

```
STATIC_WORDLIST="/path/to/subdomains-merged.txt"      # large static subdomain wordlist  
CRUNCH_WORDLIST="/path/to/subsCRUNCH.txt"            # short/4-char subdomain wordlist  
RESOLVERS="/path/to/resolver.txt"                   # DNS resolvers list  
PERMUTATION_WORDLIST="/path/to/words-merged.txt"     # dnsngen permutation words
```

```
echo "cleaning..."
```

```
rm -f "$1.wordlist" "$1.dns_brute" "$1.dns_gen"
```

```
echo "making static wordlist..."
```

```
awk -v domain="$1" '{print $0"."domain}' "$STATIC_WORDLIST" >> "$1.wordlist"
```

```
echo "making 4-char wordlist..."
```

```
awk -v domain="$1" '{print $0"."domain}' "$CRUNCH_WORDLIST" >> "$1.wordlist"
```

```
echo "shuffledns static brute-force..."
```

```
shuffledns -list "$1.wordlist" -d "$1" -r "$RESOLVERS" -m "$(which massdns)" -mode resolve -t 5
```

```
echo "[+] finished, total $(wc -l < "$1.dns_brute") resolved..."
```

```
echo "running subfinder..."
```

```
subfinder -d "$1" -all | dnsx -silent | anew "$1.dns_brute"
```

```
echo "[+] finished, total $(wc -l < "$1.dns_brute") resolved..."
```

```
echo "running dnsngen..."
```

```
dnsngen -w "$PERMUTATION_WORDLIST" "$1.dns_brute" > "$1.dns_gen"
```

```
echo "finished with $(wc -l < "$1.dns_gen") words..."
```

```
echo "shuffledns dynamic brute-force on dnsngen results..."
```

```
shuffledns -list "$1.dns_gen" -d "$1" -r ~/.resolvers -m "$(which massdns)" -mode resolve -t 5
```

```
echo "[+] finished, total $(wc -l < "$1.dns_brute") resolved..."
```

How to Use

```
bash dns-brute-full target.com
```

Before running, set the four path variables at the top of the script to match your environment. Results accumulate in `target.com.dns_brute`.

Real World Projects: get-asn-details — ASN Lookup Tool

Queries RADB for a given ASN and outputs the name, description, contact emails, and IPv4/IPv6 ranges as JSON.

Save it as `get-asn-details` and run it with `bash get-asn-details <ASN>` or pipe an ASN into it.

Script

```
#!/bin/bash

if [ -t 0 ]; then
    asn="$1"
else
    read asn
fi

if [ -z "$asn" ]; then
    echo "Usage: get-asn-details <ASN>"
    exit 1
fi

# query general ASN info (name, description, contacts)
asn_data=$(whois -h whois.radb.net "AS$asn")

# query all route objects originated by this ASN (IP ranges)
route_data=$(whois -h whois.radb.net -i origin "AS$asn")

name=$(echo "$asn_data" | awk -F: '/^as-name/ {print $2; exit}' | xargs)
des=$(echo "$asn_data" | awk -F: '/^descr/ {print $2; exit}' | xargs)

emails=$(echo "$asn_data" | \
    grep -Eoi '[a-z0-9._%+-]+@[a-z0-9.-]+\.[a-z]{2,}' | \
    sort -u)

ipv4_ranges=$(echo "$route_data" | awk '/^route:/ {print $2}' | sort -u)
ipv6_ranges=$(echo "$route_data" | awk '/^route6:/ {print $2}' | sort -u)

# heredoc: prints everything between EOF markers as-is, with variable expansion
# command substitution/ print formatted strings followed by new line character --> remove the
# printf ' "%s",\n' $emails --> command substitution: print each email formatted as "email"
# sed '$ s/,,$//' --> remove the trailing comma from the last line ($ = last line)
```

```

cat <<EOF
{
  "asn": $asn,
  "name": "$name",
  "des": "$des",
  "email": [
$(printf '      "%s",\n' $emails | sed '$ s/,,$//')
  ],
  "ip_ranges": {
    "ipv4": [
$(printf '      "%s",\n' $ipv4_ranges | sed '$ s/,,$//')
    ],
    "ipv6": [
$(printf '      "%s",\n' $ipv6_ranges | sed '$ s/,,$//')
    ]
  }
}
EOF

```

How to Use

```

bash get-asn-details 15169
echo 15169 | bash get-asn-details

```

Real World Projects: get-certificate-improved — TLS Certificate Inspector

An improved version of `get-certificate` that handles both domains (with SNI) and bare IP addresses, and supports processing multiple targets at once via arguments or stdin.

Save it as `get-certificate-improved` and run it with `bash get-certificate-improved <target>` or pipe targets into it.

Script

```

#!/bin/bash

process() {
  input="$1"
  host="${input%:*}" # strip port if present (e.g. example.com:8443 --> example.com)
  # in bash, ${variable%%pattern} is string manipulation and %% means remove the longest match
  # also we have % for shortest match, for example: x="a/b/c" --> echo "${x%/*}" --> a/b (or)
  # IP mode: skip SNI (SNI requires a hostname, not an IP)
  if [[ "$host" =~ ^[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+$ ]]; then # this is how regex works in bash

```

```

openssl s_client -connect "$host:443" </dev/null 2>/dev/null \
    | openssl x509 -noout -text 2>/dev/null
else
    # domain mode: send SNI header so the server returns the correct certificate
    openssl s_client -connect "$host:443" -servername "$host" </dev/null 2>/dev/null \
        | openssl x509 -noout -text 2>/dev/null
fi

echo ""
echo "***"
echo ""
}

if [ ! -t 0 ]; then
    # stdin mode: process one target per line
    while read -r line; do
        [ -n "$line" ] && process "$line"
    done
else
    # argument mode: process each argument
    for arg in "$@"; do
        process "$arg"
    done
fi

```

How to Use

Single target:

```
bash get-certificate-improved example.com
bash get-certificate-improved 93.184.216.34
```

Multiple targets:

```
bash get-certificate-improved example.com sub.example.com
cat targets.txt | bash get-certificate-improved
```

Real World Projects: get-chaos-data — Chaos Dataset Downloader

Downloads and extracts all subdomain datasets from ProjectDiscovery’s public Chaos index.

Save it as `get-chaos-data` and run it with `bash get-chaos-data`.

Script

```
#!/usr/bin/env bash

curl -s https://chaos-data.projectdiscovery.io/index.json \
  | jq -r '.[].URL' \
  | while read -r url; do
    wget -q "$url"
    unzip -q "${basename "$url"}"
  done

rm -f *.zip
```

How to Use

```
bash get-chaos-data
```

Downloads all zip files from the Chaos index, extracts them in the current directory, and removes the zips when done.

Real World Projects: get-ip-asn — IP to ASN Lookup

Queries Team Cymru's whois service to resolve an IP address to its ASN.

Save it as `get-ip-asn` and run it with `bash get-ip-asn <IP>` or pipe an IP into it.

Script

```
#!/bin/bash

if [ -t 0 ]; then
  ip="$1"
else
  read ip
fi

if [ -z "$ip" ]; then
  echo "Usage: get-ip-asn <IP>"
  exit 1
fi

whois -h whois.cymru.com " -v $ip" \
  | tail -n +2 \      # skip the header line
  | awk '{print $1}'  # print the ASN (first field) and default delimiter is space.
```

How to Use

```
bash get-ip-asn 8.8.8.8
echo 8.8.8.8 | bash get-ip-asn
```

Real World Projects: get-ptr — PTR Record Lookup

Reads IP addresses from stdin until `END_OF_INPUT`, then resolves their PTR (reverse DNS) records using `dnsx`.

Save it as `get-ptr` and run it simply with:

```
cat ips.txt | bash get-ptr or
bash get-ptr ips.txt or
echo "8.8.8.8" | bash get-ptr
```

Script

```
#!/usr/bin/env bash
cat "${1:-/dev/stdin}" | dnsx -silent -resp-only -ptr
```

Real World Projects: httpx-full — HTTP Prober

Runs `httpx` with a fixed set of recon flags — title, status code, CDN detection, and tech fingerprinting — against a single target or a list from stdin.

Save it as `httpx-full` and run it with `bash httpx-full <target>` or pipe targets into it.

Dependencies

- `httpx` — fast HTTP probing tool by ProjectDiscovery
-

Script

```
#!/bin/bash

run_httpx() {
    local target="$1"
    httpx \
        -silent \
        -follow-host-redirects \
        -title \
        -status-code \
        -cdn \
        -tech-detect \
```

```

        -threads 1 \
        -H "User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/117.0"
        -H "Referer: https://$target" \
        -u "$target"
    }

    if [ -p /dev/stdin ]; then # this is another form to check if input is coming form pipeline
        while read -r line; do
            [ -n "$line" ] && run_httpx "$line"
        done
    else
        if [ -z "$1" ]; then
            echo "Usage:"
            echo "  httpx-full example.com"
            echo "  echo example.com | httpx-full"
            exit 1
        fi
        run_httpx "$1"
    fi

```

How to Use

```

bash httpx-full example.com
echo example.com | bash httpx-full
cat targets.txt | bash httpx-full

```

Real World Projects: crtsh — Certificate Transparency Subdomain Finder

Queries the crt.sh PostgreSQL database directly to extract subdomains from certificate transparency logs.

Save it as crtsh and run it with `bash crtsh <domain>` or pipe a domain into it.

Dependencies

- psql — PostgreSQL client

Script

```

#!/bin/bash

if [ -t 0 ]; then

```

```

    domain="$1"
else
    read domain
fi

if [ -z "$domain" ]; then
    echo "Usage: crtsh <domain>"
    exit 1
fi

query=$(cat <<-END
SELECT ci.NAME_VALUE
FROM certificate_and_identities ci
WHERE plainto_tsquery('certwatch', '$domain') @@ identities(ci.CERTIFICATE)
END
)

echo "$query" \
| psql -t -h crt.sh -p 5432 -U guest certwatch \
| sed 's/ //g' \
| grep -E ".*\.$domain" \
| sed 's/\*\.//g' \
| tr '[:upper:]' '[:lower:]' \
| sort -u

# second sed behavior: *.example.com → example.com

```

How to Use

```

bash crtsh example.com
echo example.com | bash crtsh

```

Real World Projects: OTX — Passive DNS Lookup via AlienVault

Queries AlienVault OTX for passive DNS records of a domain and prints all discovered hostnames.

Requires an API key from otx.alienvault.com. Set it with `export OTX_API_KEY=YOUR_KEY` before running.

Save it as `OTX` and run it with `bash OTX <domain>` or pipe a domain into it.

Script

```
#!/bin/bash

domain=${1:-$(cat)} # and this is another shorthand to read from argument, if not exists read
api_key=$OTX_API_KEY

if [ -z "$domain" ]; then
    echo "Usage: OTX target.com"
    echo "export OTX_API_KEY=YOUR_KEY"
    exit 1
fi

curl -s \
    -H "X-OTX-API-KEY: $api_key" \
    "https://otx.alienvault.com/api/v1/indicators/domain/$domain/passive_dns" \
    | jq -r '.passive_dns[].hostname' \
    | sort -u
```

How to Use

```
export OTX_API_KEY=YOUR_KEY # (you can hardcode it if you want)
bash OTX example.com
echo example.com | bash OTX
```

Real World Projects: HackerTarget — Subdomain Lookup

Queries the HackerTarget API for subdomains of a given domain.

Save it as HackerTarget and run it with `bash HackerTarget <domain>` or pipe a domain into it.

Script

```
#!/usr/bin/env bash

domain=${1:-$(cat)}
curl -s "https://api.hackertarget.com/hostsearch/?q=${domain}" | cut -d"," -f1
```

Task: run it once without cut to see how can it be useful.

How to Use

```
bash HackerTarget example.com
echo example.com | bash HackerTarget
```

Real World Projects: SecTrails — Subdomain Lookup via SecurityTrails

Queries the SecurityTrails API for subdomains of a given domain. Free plan includes 50 requests per month — get a key at securitytrails.com.

Save it as `SecTrails` and run it with `bash SecTrails <domain>` or pipe a domain into it.

Script

```
#!/usr/bin/env bash

DOMAIN=${1:-$(cat)}
APIKEY="YOUR_API_KEY"

curl -s -H "apikey: $APIKEY" \
  "https://api.securitytrails.com/v1/domain/$DOMAIN/subdomains" \
  | jq -r --arg d "$DOMAIN" '.subdomains[] | "\($d)."+$d'
```

How to Use

```
# you should export or hardcode your API key before using the script
bash SecTrails example.com
echo example.com | bash SecTrails
```

Real World Projects: wayback — Wayback Machine URL Extractor

If a tool like `waybackurls` didn't exist, how would you query and extract data directly from the Wayback Machine?

This script queries the Wayback Machine CDX API for all archived URLs under a domain, filters out static assets, and saves the results to `wayback.output`.

Save it as `wayback.sh` and run it with `bash wayback.sh <domain>` or pipe a domain into it.

Script

```
#!/bin/bash
```

```
DOMAIN=${1:-$(cat)}  
curl -s "https://web.archive.org/cdx/search/cdx?url=*.${DOMAIN}/*&fl=original&collapse=urlkey"  
| grep -vE '\.(png|jpg|jpeg|gif|svg|webp|mp4|webm|woff|woff2|ttf)(\?|$\#)' \  
| sort -u \  
| tee wayback.output
```

How to Use

```
bash wayback.sh example.com  
echo example.com | bash wayback.sh
```

Real World Projects: split-urls — Split URL Lists by Hostname

Takes a file of URLs (e.g. output from waybackurls, gau, or katana) and splits them into per-host files, capped at 1000 URLs each. Save as `split-urls`, make it executable with `chmod +x split-urls`, then pass your URL list as the first argument.

Script

```
#!/usr/bin/env bash  
if [ -z "$1" ]; then  
    echo "Usage: $0 <input_urls_file>"  
    exit 1  
fi  
  
input="$1"  
outdir="by-host"  
max_per_file=1000  
  
mkdir -p "$outdir"  
  
# awk explanation:  
# process every line starting with: http:// or https://  
# whole block inside {} is action  
# set the whole line to host variable  
# remove http:// or https:// from start of host  
# remove everything after '/' in host  
# print host alongside whole url  
# sort the output based on their first column (host name)  
# start the second awk with two variables: outdir and max  
# field one is assigned to host and field two is assigned to url
```

```
# last_host is empty, so the count starts from 0 in part 1 and value of host will be assigned
# if urls count is more than limit, go to next part and set count to 1 again
# append url to outdir{host}.part{part}.urls
```

```
awk '/^https?:\\\/\\\/{
    host = $0
    sub(/^https?:\\\/\\\/, "", host)
    sub(/\\\/.*/, "", host)
    print host "\t" $0
}' "$input" | sort -k1,1 | awk -v outdir="$outdir" -v max="$max_per_file" '
{
    host = $1
    url = $2
    if (host != last_host) { count = 0; part = 1; last_host = host }
    if (++count > max)      { part++; count = 1 }
    print url >> outdir "/" host ".part" part ".urls"
}'
```

How to Use

```
# Basic usage
```

```
./split-urls all-urls.txt
```

```
# Feed output from waybackurls directly
```

```
waybackurls example.com | tee all-urls.txt | ./split-urls /dev/stdin
```

```
# Combine multiple sources then split
```

```
cat wayback.txt gau.txt katana.txt | sort -u > all-urls.txt
```

```
./split-urls all-urls.txt
```

```
# Check output
```

```
ls by-host/
```

```
# example.com.part1.urls
```

```
# api.example.com.part1.urls
```

```
# sub.example.com.part1.urls
```

```
# Pass a host file to x9
```

```
x9 -u by-host/example.com.part1.urls
```

Go Fundamentals

Go is a fast, compiled programming language designed for simplicity and performance.

It is widely used for building high-performance tools, backend services, and command-line applications.

In this section, we cover the basic concepts needed to start writing efficient Go programs and CLI tools.

Introduction to Go

Go was designed at Google to have high performance combined with a straightforward, readable syntax.

What Kind of Language Is Go?

Go is a statically typed, compiled language. It is particularly well-suited for building performant backend services, networking tools, and programs that take advantage of multiprocessing.

Go Use Cases

1. Cloud and network services
2. Command-line interfaces
3. Web development (backend)
4. Automation and DevOps
5. Utilities and standalone tools

Programming Language Fundamentals

There are many programming languages, each with its own features and ideal use cases. Two of the most important characteristics to understand are **type systems** (static vs. dynamic typing) and **execution models** (compiled vs. interpreted).

Interpreters and Compilers

Humans write what is known as **source code**. Before this code can be executed by a computer, it must be converted into a form the machine can understand.

A **compiler** transforms the entire source code into binary (machine code) that can be run directly by the CPU. A **interpreter**, on the other hand, translates and executes source code one line at a time.

- **Interpreted languages:** like Python, PHP, Bash, JavaScript
- **Compiled languages:** for example: C, Go, etc

Using a compiler typically means stricter syntax rules. For example, you are required to explicitly define the types of your variables.

Dynamic vs. Static Typing

Typing refers to when and how the types of variables are determined.

- In a **dynamically typed** language like Python, variable types are decided at runtime — the program figures out the type of a value while it is executing.
- In a **statically typed** language like Go, variable types are decided at compile time — the type of every variable must be known before the program runs.

This distinction matters in practice: static typing catches type-related bugs earlier (at compile time), while dynamic typing offers more flexibility at the cost of potentially surfacing errors only when the code actually runs.

Practical Script: Hello World in Go

Save the following code into a file called `gettingStarted.go` and run it with `go run gettingStarted.go`.

File Structure

Every Go source file must begin by declaring which **package** it belongs to. The `main` package is special — it tells the Go compiler that this file is meant to produce an executable program. If a `main` function exists inside a `main` package, it will be treated as the program's entry point. If the package name is anything other than `main`, the file is treated as a **library** that other programs can import and use.

Importing Packages

After the package declaration, you specify which external packages your file depends on using the `import` keyword. Go enforces a strict rule here: **you cannot import a package and then not use it** — the compiler will refuse to build your program if you do.

In this script, we import `fmt`, which is Go's standard library for input/output operations. Functions like `fmt.Println` belong to this package. It serves the same role as `<stdio.h>` in C — providing basic I/O functionality.

The Main Function

The `main` function is the entry point of any executable Go program. When you run the program, execution begins here.

```
package main

import "fmt"

func main() {
    fmt.Println("Hello, World!")
}
```

Running and Building

- `go run gettingStarted.go` — compiles the code into a temporary file and executes it immediately. Useful during development.
- `go build gettingStarted.go` — compiles the code and produces a standalone binary that you can run directly, without needing the Go toolchain present. On Linux, the output will be an **ELF** (Executable and Linkable Format) binary.

Binary Fundamentals

Before writing Go code that involves variable types and memory sizes, it helps to understand how numbers are represented at the binary level. This section covers number bases, binary arithmetic, and signed vs. unsigned integers — all of which directly affect how you declare and use variables in Go.

Number Bases

In everyday life, we use **base 10** (decimal) numbers. Each digit in a decimal number can be one of ten values: 0 through 9. The value of each digit depends on its position, where the rightmost position is index 0 and each step to the left increases the index by one.

For example, the number 374 breaks down as:

$$\begin{array}{rcl} 4 \times 10^0 & = & 4 \times 1 = 4 \\ 7 \times 10^1 & = & 7 \times 10 = 70 \\ 3 \times 10^2 & = & 3 \times 100 = 300 \\ \text{Total} & = & 374 \end{array}$$

This positional method applies to any number base, not just base 10.

The four bases you will encounter most often are:

Base	Name
2	Binary
8	Octal
10	Decimal
16	Hexadecimal

Binary

In binary (base 2), there are only two possible digit values: **0** and **1**. For comparison:

- Octal (base 8) uses digits 0 through 7
- Hexadecimal (base 16) uses digits 0 through 9 and letters A through F

To convert a binary number to decimal, apply the same positional method used above, but with a base of 2. For example, the binary number **10010**:

$$\begin{array}{rcl} \text{Index:} & 4 & 3 & 2 & 1 & 0 \\ \text{Digit:} & 1 & 0 & 0 & 1 & 0 \end{array}$$

$$\begin{array}{rcl} 0 \times 2^0 & = & 0 \\ 1 \times 2^1 & = & 2 \\ 0 \times 2^2 & = & 0 \\ 0 \times 2^3 & = & 0 \\ 1 \times 2^4 & = & 16 \\ \text{Total} & = & 18 \end{array}$$

So 10010 in binary equals 18 in decimal.

Bits and Bytes

Each individual column (digit position) in a binary number is called a **bit**. Eight bits grouped together form a **byte**.

This matters in Go because variable types are defined by how many bits they use. For example, `int32` is an integer stored in 32 bits.

To understand the range of values a given number of bits can represent, consider a single byte (8 bits):

- **Number of unique values:** $2^8 = 256$
- **Range (unsigned):** 0 to 255 — because counting starts at 0, the maximum is 255, not 256

Signed Integers

So far, all of the above applies to **unsigned** integers — positive values only. To represent negative numbers, binary uses a **sign bit**: an extra bit placed on the left side of the number.

- A sign bit of **0** means positive
- A sign bit of **1** means negative

Because one bit is now used for the sign rather than the value, the maximum positive number you can represent is reduced.

Two's Complement

The standard method for working with signed binary numbers is called **two's complement**. To find the value of a negative signed binary number:

1. Flip all the bits (0 becomes 1, 1 becomes 0)
2. Add 1 to the result
3. Calculate the decimal value
4. Apply a negative sign

For example, the signed byte 11111111:

```
11111111 → flip bits → 00000000
00000000 → add 1     → 00000001 = 1
Result: -1
```

Range of a Signed Byte

With 8 bits and one used as a sign bit:

- **Maximum:** $2^7 - 1 = 127$ (binary: 01111111)
- **Minimum:** $-2^7 = -128$ (binary: 10000000)

This gives a total range of -128 to 127 , still covering 256 unique values — the same count as an unsigned byte, just shifted to include negative numbers # Data Types and Variables in Go

Comments in Go

Go supports two comment styles:

- Single-line comments start with `//`
- Multi-line comments are wrapped between `/*` and `*/`

Core Data Types

Before looking at the code, here is an overview of Go's fundamental data types.

Integers

uint (unsigned integer) stores only non-negative values. By default Go chooses **uint32** or **uint64** depending on the value being stored, but you can specify the size explicitly using **uint8**, **uint16**, **uint32**, or **uint64**. Because there is no sign bit, all bits are used for the value itself:

- Minimum: 0
- Maximum: $2^{32} - 1$ (for **uint32**)
- Total unique values: 2^{32}

int (signed integer) can store both positive and negative values. Like **uint**, the default is **int32** or **int64**, but you can specify the size explicitly. The leftmost bit is reserved as the sign bit:

- Minimum: -2^{31} (for **int32**)
- Maximum: $2^{31} - 1$ (for **int32**)
- Total unique values: 2^{32}

Use **uint** when you know a value will always be positive, and **int** when it may be negative.

Floats

float32 and **float64** are used for floating-point (decimal) numbers. The number indicates how many bits are used to represent the value. **float64** offers greater precision and is the more commonly used type.

Byte

byte is an alias for **uint8**. It is typically used to store a small number or a single ASCII character. It is not a distinct data type — it is simply a more readable way to express **uint8**.

Rune

rune is an alias for **int32**. It is used to represent a single Unicode code point, which is important when working with non-ASCII characters.

Bool

bool holds a boolean value: either **true** or **false**.

String

string holds a sequence of characters enclosed in **double quotes**. Single quotes are not used for strings in Go — they are reserved for individual rune (character) literals. In fact, go **strings** are sequences of bytes and not **chars** like other languages.

Nil

nil is Go's equivalent of `null` in other languages. It represents the absence of a value and can be assigned to pointers, interfaces, maps, slices, channels, and functions.

There are additional types in Go that will be covered later.

Declaring Variables

Go provides two ways to declare variables, similar to `var` and `const` in JavaScript:

- `var` declares a variable whose value can be changed at any time.
- `const` declares a constant whose value cannot be changed after it is set.

Go does not require semicolons at the end of statements.

If you declare a variable without assigning a value, Go assigns it a **zero value** automatically — an empty string `""` for strings, `0` for numeric types, `false` for booleans, and so on.

```
package main

import "fmt"

func main() {
    var x string = "VoidSeraphim"
    // Variables can be declared and assigned a value later.
    fmt.Println(x)

    var y uint32
    y = 12
    y = 15
    fmt.Println(y)

    const z int8 = -27
    // z = -2    // Attempting to reassign a constant causes a compile-time error.
    fmt.Println(z)

    // If you declare a variable without a value and print it,
    // Go will output its default value: "" for strings, 0 for numbers, etc.
}
```

The commented-out line `// z = -2` demonstrates that reassigning a constant is not allowed. It is kept here as a reference to show what would cause a compile-time error.

Implicit Assignment in Go

Explicit vs. Implicit Assignment

When declaring a variable in Go, you have two options.

Explicit assignment means you write out the type yourself using the `var` or `const` keyword:

```
var x uint8 = 2
```

Implicit assignment uses the `:=` operator. Go infers the type automatically based on the value being assigned — still at compile time, but without you having to specify it:

```
y := -123.64
```

A few rules apply to implicit assignment:

- You do not use the `var` keyword with `:=`
- It cannot be used for constants
- Once a variable is declared, its type is fixed — you cannot assign a value of a different type to it later

When to Use Each

Use explicit assignment only when you need to declare a variable without immediately giving it a value. In all other cases, implicit assignment is the preferred approach.

Checking a Variable's Type

You can check the type of any variable at runtime using `fmt.Printf` with the `%T` format modifier:

```
fmt.Printf("%T\n", y)
```

Forcing a Specific Type with Implicit Assignment

If you want a specific type but still prefer the `:=` syntax, you can wrap the value in a type conversion call:

```
z := uint8(43)
fmt.Printf("%T\n", z)
```

This same syntax is used for **type casting** — converting a value from one type to another by passing it as an argument to the target type:

```
a := string(z)
fmt.Printf("%T\n", a)
fmt.Println(a)
```

Note that converting a numeric value to `string` this way interprets the number as a Unicode code point, not as a digit. The number 43 maps to `+` in the ASCII table, so `fmt.Println(a)` will print `+`, not 43.

To convert a number to its string representation as digits, use `fmt.Sprint` instead:

```
b := fmt.Sprint(z)
fmt.Println(b)
fmt.Printf("%T\n", b)
```

```
package main
```

```
import "fmt"
```

```
func main() {
    var x uint8 = 2
    fmt.Println(x)

    y := -123.64
    // y = "sora"    // Reassigning a variable with a different type causes a compile-time error
    fmt.Printf("%T\n", y)

    z := uint8(43)
    fmt.Printf("%T\n", z)

    a := string(z)
    fmt.Printf("%T\n", a)
    fmt.Println(a)

    b := fmt.Sprint(z)
    fmt.Println(b)
    fmt.Printf("%T\n", b)
}
```

The commented-out line `// y = "sora"` is kept as a reference to show that reassigning a variable with a value of a different type is not allowed in Go.

Console Output in Go

The `fmt` Package

`fmt` stands for “format”. It is Go’s standard library for formatting and printing output, as well as constructing formatted strings.

`fmt.Println`

`Println` prints one or more values to `stdout` (the terminal), separated by spaces, and automatically moves to the next line afterward. You can pass values of mixed types in a single call:

```
x := false
fmt.Println("Hello", 2, x)
```

`fmt.Printf`

`Printf` prints a formatted string to `stdout`. You provide a format string containing **format verbs** — placeholders that are replaced by the values you pass as additional arguments:

```
y := "Sora"
fmt.Printf("the variable y has the value: %s and type: %T\n", y, y)
```

The `\n` at the end is required because, unlike `Println`, `Printf` does not add a newline automatically.

The most useful format verbs are:

Verb	Meaning
%T	The type of the value
%v	The value in its default format (same as <code>Println</code>)
%s	String values
%f	Float values
%.<n>f	Float with <n> decimal places of precision
%b	Binary representation of the value
%e	Scientific notation

fmt.Sprintf

`Sprintf` works exactly like `Printf` but does not print anything. Instead, it returns the formatted string so you can store it in a variable:

```
var a int8 = 43
z := fmt.Sprintf("%v", a)
fmt.Println(z)
```

fmt.Sprint

`Sprint` works like `Sprintf` but without format verbs — it simply converts the value to a string and returns it:

```
b := fmt.Sprint(a)
fmt.Println(b)
```

```
package main
```

```
import "fmt"
```

```
func main() {
    x := false
    fmt.Println("Hello", 2, x)

    y := "Sora"
    fmt.Printf("the variable y has the value: %s and type: %T\n", y, y)

    var a int8 = 43
    z := fmt.Sprintf("%v", a)
    fmt.Println(z)

    b := fmt.Sprint(a)
    fmt.Println(b)
}
```

Arithmetic Operators in Go

Importing Multiple Packages

When you need more than one package, Go lets you group them in a single `import` block using parentheses:

```
import (  
    "fmt"  
    "math"  
    "strconv"  
)
```

Arithmetic Operators

Go supports the standard arithmetic operators: `+`, `-`, `*`, `/`, `%`, `++`, `--`. Note that Go does not have a `**` power operator — exponentiation is handled by `math.Pow` instead.

Type Mismatch in Operations

Go does not perform implicit type conversion. If you try to perform an arithmetic operation on two variables of different types, the compiler will throw a type mismatch error:

```
x := uint8(7)  
y := 1000  
// z := x + y    // type mismatch error
```

The fix is to cast the smaller type up to the larger one. Casting in the other direction risks data loss — for example, `y` (1000) cannot fit inside a `uint8`, whose maximum value is 255:

```
// z := x + uint8(y)    // compiles but produces unexpected output due to overflow  
z := int(x) + y  
fmt.Println(z)
```

The first commented-out line is kept as a reference to show what happens when you cast a value that exceeds the target type's range — the result overflows silently rather than producing an error.

Integer Division

When dividing two integers in Go, the result is also an integer — the decimal portion is discarded (equivalent to `math.Floor`):

```
fmt.Println(y / int(x))    // 142
```

To get a float result, cast both operands to `float64` before dividing:

```
fmt.Println(float64(y) / float64(x))  
fmt.Printf("%.2f\n", float64(y) / float64(x))
```

The `math` Package

For operations beyond basic arithmetic, import the `math` package. Note that `math.Min` and `math.Max` each take exactly two arguments:

```
fmt.Println(math.Min(4, 6))
fmt.Println(math.Max(4, 6))
fmt.Println(math.Sqrt(25))
fmt.Println(math.Pow(4, 2))
fmt.Println(math.Round(4.564))
fmt.Println(math.Floor(4.564))
fmt.Println(math.Ceil(4.564))
```

String to Integer Conversion with `strconv`

You cannot cast a string directly to an integer in Go:

```
e := "1234"
// f := int(e)    // compile-time error
```

The `strconv` package provides functions for these conversions. `strconv.Atoi` converts a string to an `int` and returns two values: the result and an error. If no error occurs, the error value will be `nil`:

```
f, err := strconv.Atoi(e)
```

`strconv.ParseInt` does the same but gives you more control. Its arguments are the string, the base of the number represented in the string, and the bit size used to validate that the result fits within the intended range. A bit size of 0 defaults to `int64`:

```
g, err := strconv.ParseInt(e, 10, 32)
```

The base argument must match the format of the string. For example, passing base 2 for the string "1234" would fail because 2, 3, 4 are not valid binary digits.

Similar functions exist for other types: `ParseBool`, `ParseFloat`, and so on. For the reverse direction — converting a number to a string — `strconv` provides `Itoa` and `FormatInt`, though using `fmt.Sprint` or `fmt.Sprintf` is generally simpler.

```
package main
```

```
import (
    "fmt"
    "math"
    "strconv"
)
```

```
func main() {
    x := uint8(7)
    y := 1000
    // z := x + y           // type mismatch error
    // z := x + uint8(y)    // compiles but produces unexpected output due to overflow

    z := int(x) + y
```

```

fmt.Println(z)

fmt.Println(y / int(x))
fmt.Println(float64(y) / float64(x))
fmt.Printf("%.2f\n", float64(y)/float64(x))

fmt.Println(math.Min(4, 6))
fmt.Println(math.Max(4, 6))
fmt.Println(math.Sqrt(25))
fmt.Println(math.Pow(4, 2))
fmt.Println(math.Round(4.564))
fmt.Println(math.Floor(4.564))
fmt.Println(math.Ceil(4.564))

e := "1234"
// f := int(e)    // compile-time error

f, err := strconv.Atoi(e)
g, err := strconv.ParseInt(e, 10, 32)
fmt.Println(f, err, g)
}

```

Conditions in Go

Comparison Operators

Go supports the standard set of comparison operators:

Operator	Meaning
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to
==	Equal to
!=	Not equal to

The result of any comparison is a `bool` — either `true` or `false`.

As with arithmetic operations, Go does not allow comparisons between two different types. You must cast one of them first:

```

x := uint(8)
y := 10
f := y > int(x)
fmt.Println(f)

```

Logical Operators

Logical operators are used to chain multiple conditions together:

Operator	Meaning
<code>\ \ </code>	OR — true if at least one condition is true
<code>&&</code>	AND — true only if both conditions are true
<code>!</code>	NOT — inverts a boolean value

If / Else If / Else

The `if` statement in Go works similarly to other languages, with one important syntax rule: the opening brace `{` must be on the same line as the `if` or `else` keyword. Placing it on a new line causes a compile-time error.

```
name := "dora"
if int(x) < y && name == "sora" {
    fmt.Println("both True")
} else if int(x) > y || name == "dora" {
    fmt.Println("one is True")
} else {
    fmt.Println("both false")
}

package main

import "fmt"

func main() {
    x := uint(8)
    y := 10

    f := y > int(x)
    fmt.Println(f)

    name := "dora"
    if int(x) < y && name == "sora" {
        fmt.Println("both True")
    } else if int(x) > y || name == "dora" {
        fmt.Println("one is True")
    } else {
        fmt.Println("both false")
    }
}
```

Switch Statements in Go

Basic Switch

Go's `switch` statement does not require parentheses around the value being evaluated:

```
a := 2
switch a {
```

```

case 1:
    fmt.Println("One")
    break
case 2:
    fmt.Println("Two")
    break
case 3:
    fmt.Println("Three")
    break
default:
    fmt.Println("else")
}

```

Unlike C or JavaScript, Go **automatically breaks** out of a case once it matches — you do not need to write `break` manually. If you want execution to continue into the next case after a match, use the `fallthrough` keyword explicitly (and next case condition wouldn't be checked).

You can also omit the variable after the `switch` keyword and write full comparison expressions directly in each case, which makes it behave like an `if/else` chain:

```

switch { // if you want to use switch/case statement like an if/else if/else chain, dont specify variable
    case a == 2:
        fmt.Println("Two")
        fallthrough
    case a == 3:
        fmt.Println("Three")
    default:
        fmt.Println("else")
}

```

This commented-out line shows the alternative syntax — you can use any comparison operator in a case when no variable is specified after `switch`.

Matching Multiple Values in One Case

A single case can match multiple values by separating them with commas:

```

b := "hi"
switch b {
case "hi", "hello", "hey":
    fmt.Println("good Day!")
case "bye", "goodbye":
    fmt.Println("bye")
default:
    fmt.Println("wyd?")
}

```

```
package main
```

```
import "fmt"
```

```

func main() {
    a := 2
    switch a {
    case 1:
        fmt.Println("One")
    case 2:
        fmt.Println("Two")
    case 3:
        fmt.Println("Three")
    default:
        fmt.Println("else")
    }

    b := "hi"
    switch b {
    case "hi", "hello", "hey":
        fmt.Println("good Day!")
    case "bye", "goodbye":
        fmt.Println("bye")
    default:
        fmt.Println("wyd?")
    }
}

```

Loops in Go

The for Loop

Go has only one loop construct: the `for` loop. Unlike JavaScript, there are no `for...in`, `for...of`, or `while` variants — but the `for` loop is flexible enough to cover every pattern you need.

The standard form uses an initialiser, a condition, and a post statement:

```

for idx := 0; idx < 10; idx++ {
    fmt.Println(idx)
}

```

While-Style Loop

Since Go has no `while` keyword, you replicate that behaviour by using a `for` loop with only a condition — dropping the initialiser and post statement:

```

a := 1
for a < 10 {
    fmt.Println("Loop")
    a++    // increment to avoid an infinite loop
}

```

A `for` loop with no condition at all runs forever. The line below is kept as a reference to show the syntax — it is not executed here because it would block the program:

```
// for { }    // infinite loop
```

Looping Through a String

How Strings Work in Go

Before looping through a string, it is important to understand what a string actually is in Go. Strings are sequences of **bytes**, not characters. Each byte is of type `uint8`, which is the same as `byte`.

This means that when you access a string by index, you get the raw byte value at that position — not the character itself:

```
str := "Hello World!"
fmt.Println(str[0])           // prints 72 – the decimal ASCII value of 'H'
fmt.Printf("%T\n", str[0])    // prints uint8
```

To get the character instead of its numeric value, cast the byte to `string`:

```
fmt.Println(string(str[0]))    // prints H
```

Modifying a Character in a String

Strings in Go are immutable, so you cannot change a character by index directly. The correct approach is to convert the string to a `[]rune` or `[]byte` (only for ASCII characters) slice, modify the value at the target index, then convert it back to a `string`:

```
cstr := []rune(str)
cstr[0] = 'J'
str = string(cstr)
```

A `rune` is an alias for `int32` and represents a single Unicode code point. You might wonder whether converting to `[]rune` inflates ASCII content — since a `rune` is 32 bits wide and an ASCII character is only 8 bits. The answer is no: each element of the `[]rune` slice holds exactly one character regardless of its byte width, so ASCII content is not expanded to four slots.

ASCII vs UTF-8: Why It Matters for Looping

- ASCII characters occupy **one byte** each.
- UTF-8 characters (such as Arabic, Chinese, or emoji) can occupy **up to four bytes** each.

If your string contains only ASCII, iterating byte by byte with a standard index loop is safe, because every character maps to exactly one byte:

```
for idx := 0; idx < len(str); idx++ {
    fmt.Printf("%c\n", str[idx])
}
```

However, if your string contains multi-byte UTF-8 characters, this approach will read partial bytes mid-character and produce garbage output.

Range Syntax for UTF-8 Strings

The `range` keyword is the correct way to iterate over a string that may contain multi-byte characters. It decodes one rune at a time and automatically advances the index by the correct number of bytes:

```
str2 := "some non انگلیسی"
for i, char := range str2 {
    fmt.Println(i, string(char))
    fmt.Printf("%v, %c\n", i, char)
    fmt.Println()
}
```

`range` yields two values on each iteration: the **byte index** of the current character, and the **rune value** at that position. If you do not need the index, discard it with the blank identifier:

```
for _, char := range str2 { ... }
```

`break` and `continue` work inside `for` loops exactly as they do in other languages.

When iterating over any string, prefer `range` — it handles both ASCII and UTF-8 content correctly.

Save this as `loops.go` and run it with `go run loops.go` to see the results.

```
package main

import "fmt"

func main() {
    for idx := 0; idx < 10; idx++ {
        fmt.Println(idx)
    }

    a := 1
    for a < 10 {
        fmt.Println("Loop")
        a++ // increment to avoid an infinite loop
    }

    // for { } // infinite loop – omitted to avoid blocking execution

    str := "Hello World!"
    fmt.Println(str[0]) // 72 – decimal ASCII value of 'H'
    fmt.Printf("%T\n", str[0]) // uint8
    fmt.Println(string(str[0]))

    cstr := []rune(str)
    cstr[0] = 'J'
    str = string(cstr)
```

```

for idx := 0; idx < len(str); idx++ {
    fmt.Printf("%c\n", str[idx])
}

str2 := "some non انگلیسی"
for i, char := range str2 {
    // if the index is not needed: for _, char := range str2
    fmt.Println(i, string(char))
    fmt.Printf("%v, %c\n", i, char)
    fmt.Println()
}
}

```

Arrays in Go

What Is an Array?

An array is a fixed-size data structure that stores values of the **same type** (unlike JavaScript). When declaring an array in Go, you must specify both the element type and the size. Importantly, the size is part of the type itself — `[2]int` and `[3]int` are two completely different types. Once declared, the size of an array cannot be changed.

```

var arr [2]int
fmt.Println(arr) // prints [0 0] – zero values for int

```

Declaring Arrays

You can also declare and initialise an array in a single line using implicit assignment:

```

arr2 := [3]int{4, 5, 6}
fmt.Println(arr2)

```

If you do not want to count the elements manually, you can use `...` instead of a fixed size. Go will infer the length from the values you provide:

```

arr4 := [...]int{{4, 5}, {6, 7}}
fmt.Println(arr4)

```

Note that `...` is only allowed in the **first dimension**. Inner dimensions must always have an explicit size.

Multi-Dimensional Arrays

To create a matrix or nested array, you stack the dimension declarations:

```

arr3 := [3][3]int{{1, 2, 3}, {4, 5, 6}, {7, 8, 9}}
fmt.Println(arr3)

```

`[3][3]int` means: an array of size 3, where each element is itself an array of size 3 containing integers.

Accessing and Modifying Elements

You access elements by index, and assign to them the same way:

```
fmt.Println(arr2[1]) // prints 5
arr[0] = 9
fmt.Println(arr)     // prints [9 0]
```

When replacing an element inside a multi-dimensional array, you cannot use a bare `{...}` literal — you must include the full type:

```
arr3[2] = [3]int{5, 6, 7}
fmt.Println(arr3)
```

Array Length

Use the built-in `len` function to get the number of elements in an array:

```
fmt.Println(len(arr3)) // prints 3
```

Looping Through Arrays

Use `range` to iterate over an array. For multi-dimensional arrays, nest two `range` loops:

```
for i, nested := range arr3 {
    for _, value := range nested {
        fmt.Println(i, value)
    }
}
```

The outer loop gives you the index and the inner array. The inner loop discards the index with `_` and reads each value directly.

Arrays Are Copied When Passed to Functions

When you pass an array to a function in Go, a full copy of the array is made. Any modifications inside the function do not affect the original:

```
test(arr4)
fmt.Println(arr4) // unchanged – the function received a copy, not the original
```

The `test` function modifies its local copy of the array, but `arr4` in `main` remains the same. This behaviour will be covered in more detail when functions and pointers are discussed.

Arrays in Go are intentionally inflexible — fixed size, fixed type, copied on assignment. For more flexible data structures, Go provides **slices**, which will be covered next.

```
package main

import "fmt"

func main() {
    var arr [2]int
```

```

fmt.Println(arr)

arr2 := [3]int{4, 5, 6}
fmt.Println(arr2)

arr3 := [3][3]int{{1, 2, 3}, {4, 5, 6}, {7, 8, 9}}
fmt.Println(arr3)

arr4 := [...]int{{4, 5}, {6, 7}}
fmt.Println(arr4)

fmt.Println(arr2[1])
arr[0] = 9
fmt.Println(arr)

arr3[2] = [3]int{5, 6, 7}
fmt.Println(arr3)

fmt.Println(len(arr3))

for i, nested := range arr3 {
    for _, value := range nested {
        fmt.Println(i, value)
    }
}

test(arr4)
fmt.Println(arr4) // unchanged – arrays are passed by copy
}

func test(arr [2][2]int) {
    arr[0] = [2]int{100, 200}
}

```

Slices in Go

Slices are a more flexible alternative to arrays and are what you will use in practice almost all the time. Unlike arrays, slices do not have a fixed size — they can grow and shrink dynamically.

What Is a Slice?

A slice is a **view into an underlying array**. It does not store data on its own — it references a portion of an existing array. You can create a slice from an existing array, or declare one directly and let Go create the underlying array for you automatically.

Creating a Slice from an Existing Array

Given an array, you can create a slice using the **slice operator** `[start:end]:`

```
arr := [5]int{1, 2, 3, 4, 5}
sl := arr[:3]
```

`[:3]` means: start from index 0 up to (but not including) index 3. `[:]` with no values means the full array from start to end.

Modifying a slice also modifies the underlying array — a slice is not read-only:

```
sl[1] = 6
fmt.Println(sl) // [1 6 3]
fmt.Println(arr) // [1 6 3 4 5] – the array changed too
```

The Three Properties of a Slice

Every slice has three internal properties:

```
fmt.Println(sl, len(sl), cap(sl))
```

1. **Pointer** — points to the memory address of the slice's first element within the underlying array. For example, if `sl := arr[2:]`, then `sl[0]` is `arr[2]`, and the pointer holds the memory address of that element.
2. **Length** — the number of elements currently visible and accessible in the slice.
3. **Capacity** — how many elements the slice *could* expand to hold without needing a new underlying array. The formula is: the number of elements from the slice's starting position to the end of the underlying array.

Because capacity tracks how much room is left in the underlying array, you can re-slice a slice up to its capacity at any time:

```
// sl = sl[:5]    // valid as long as 5 does not exceed cap(sl)
```

This line is kept as a reference to show that re-slicing is possible — the upper bound can go up to `cap(sl)`, not just the current length.

Declaring a Slice Without an Existing Array

You can declare a slice directly without first creating an array. Go will create the underlying array for you automatically:

```
sl2 := []string{"Hello", "World"}
fmt.Println(sl2)
```

The syntax looks like an array literal but with no size between the brackets. That absence of a size is what distinguishes a slice declaration from an array declaration.

Appending to a Slice

The built-in `append` function adds an element to a slice. It returns a new slice containing the original elements plus the new one, which you assign back to the same variable:

```
for x := 0; x < 10; x++ {
    sl2 = append(sl2, "something")
}
```

```
    fmt.Println(sl2, len(sl2), cap(sl2))
}
```

Internally, `append` checks whether there is remaining capacity in the underlying array. If there is, it adds the element and extends the slice's length. If there is not, Go automatically allocates a **new underlying array with double the capacity**, copies the existing data into it, appends the new element, and updates the slice's pointer. This is why you will see the capacity jump in the output — it doubles each time the underlying array is outgrown.

Creating a Slice with `make`

You can also create a slice with a specified length and capacity using the built-in `make` function:

```
sl3 := make([]int, 10, 20)
fmt.Println(sl3, len(sl3), cap(sl3))
```

The arguments are: element type, initial length, and capacity. If you omit the capacity argument, it defaults to the same value as the length.

Iterating Over a Slice

Iterating over a slice works the same way as iterating over an array, using `range`:

```
for i, v := range sl2 {
    fmt.Println(i, v)
}
```

Slices and Functions

Passing a slice to a function behaves differently from passing an array. Recall that arrays are **value types** — a full copy is sent to the function, so the original is never affected. Slices behave differently:

```
test(sl2)
fmt.Println(sl2) // first element is now "changed"
```

When you pass a slice to a function, the slice struct itself is copied — but the pointer inside it still refers to the **same underlying array**. So any modification the function makes to the slice's elements will affect the original data:

```
func test(sli []string) {
    sli[0] = "changed"
}
```

This is a critical distinction to keep in mind when designing functions that receive slices.

```
package main
```

```
import "fmt"
```

```
func main() {
```

```

arr := [5]int{1, 2, 3, 4, 5}
sl  := arr[:3]

sl[1] = 6
fmt.Println(sl)
fmt.Println(arr)

fmt.Println(sl, len(sl), cap(sl))

// sl = sl[:5]    // re-slicing up to capacity – valid but omitted here

sl2 := []string{"Hello", "World"}
fmt.Println(sl2)

for x := 0; x < 10; x++ {
    sl2 = append(sl2, "something")
    fmt.Println(sl2, len(sl2), cap(sl2))
}

sl3 := make([]int, 10, 20)
fmt.Println(sl3, len(sl3), cap(sl3))

for i, v := range sl2 {
    fmt.Println(i, v)
}

test(sl2)
fmt.Println(sl2)
}

func test(sli []string) {
    sli[0] = "changed"
}

```

Maps in Go

A map is a data structure that stores **key-value pairs**. It is equivalent to objects in JavaScript or dictionaries in Python. When declaring a map in Go, you must specify the type of both the key and the value: `map[keyType]valueType`.

Declaring and Initialising Maps

There are three ways to create a map.

Explicit declaration and initialisation:

```
var mp map[string]int = map[string]int{"a": 1, "b": 2}
```

Implicit assignment:

```
mp2 := map[string]int{"c": 3}
```

Using **make**:

```
mp3 := make(map[string]int)
```

make creates an empty map ready to use. This is the preferred approach when you want to start with an empty map and add entries later.

You can also use more complex value types. For example, a map where each value is a slice of integers:

```
mp4 := map[string][]int{"a": {1, 2, 3}}
```

```
fmt.Println(mp)
fmt.Println(mp2)
fmt.Println(mp3)
fmt.Println(mp4)
```

Adding and Overwriting Entries

To add a new key-value pair, assign a value to a new key. If the key already exists, the value is overwritten:

```
mp4["b"] = []int{4, 5}
fmt.Println(mp4)
```

When the value type is a slice, make sure the value you assign matches that type exactly.

Deleting an Entry

Use the built-in **delete** function to remove a key-value pair. The map is mutated in place — you do not need to reassign it:

```
delete(mp4, "a")
fmt.Println(mp4)
```

Checking if a Key Exists

When you access a key in a map, Go can return two values: the value associated with the key, and a boolean indicating whether the key was found:

```
value, ok := mp4["a"]
fmt.Println(value, ok)
```

If the key does not exist, **value** will be the zero value for the map's value type (e.g. **0** for **int**, **nil** for slices), and **ok** will be **false**. This two-value form is the standard way to safely check for key presence in Go.

Practical Example

The following example counts how many numbers from 1 to 100 are divisible by each divisor from 1 to 5. The map key is the divisor and the value is the count:


```

mp5 := map[uint]uint{}
n := uint(100)

for number := uint(1); number <= n; number++ {
    for d := uint(1); d <= 5; d++ {
        if number%d == 0 {
            mp5[d]++
        }
    }
}
fmt.Println(mp5)

```

For each number, the inner loop checks divisibility by 1 through 5. Every time a number is divisible by `d`, the count stored at `mp5[d]` is incremented. Because accessing a missing key in a map returns the zero value (0 for `uint`), the increment works correctly even on the first hit — no need to initialise each key manually.

```

package main

import "fmt"

func main() {
    var mp map[string]int = map[string]int{"a": 1, "b": 2}
    mp2 := map[string]int{"c": 3}
    mp3 := make(map[string]int)
    mp4 := map[string][]int{"a": {1, 2, 3}}

    fmt.Println(mp)
    fmt.Println(mp2)
    fmt.Println(mp3)
    fmt.Println(mp4)

    mp4["b"] = []int{4, 5}
    fmt.Println(mp4)

    delete(mp4, "a")
    fmt.Println(mp4)

    value, ok := mp4["a"]
    fmt.Println(value, ok)

    mp5 := map[uint]uint{}
    n := uint(100)

    for number := uint(1); number <= n; number++ {
        for d := uint(1); d <= 5; d++ {
            if number%d == 0 {
                mp5[d]++
            }
        }
    }
}

```

```

    }
    fmt.Println(mp5)
}

```

Functions in Go

Basic Function Syntax

A function in Go is declared with the `func` keyword, followed by the function name, its parameters with their types, and the return type. The `main` function is the program's entry point — all other functions must be called from within it (directly or indirectly).

```

func add(num1 int, num2 int) int {
    return num1 + num2
}

```

Multiple Return Values

Go functions can return more than one value. You list all return types in parentheses, and on the calling side you must capture all returned values:

```

func addAndHello(num1 int, num2 int, name string) (int, string) {
    return num1 + num2, name
}

```

Functions as Parameters

Functions are first-class values in Go, meaning you can pass them as arguments to other functions. When declaring such a parameter, you specify the full function signature as its type:

```

func callFunc(callable func(int) int) int {
    return callable(10)
}

func doubleNumber(num int) int {
    return num * 2
}

```

Anonymous Functions

An anonymous function is a function with no name. It can be defined inline and passed directly as an argument, or assigned to a variable:

```

// passed directly as a callback:
value := callFunc(func(x int) int {
    return x + 1
})

// assigned to a variable:
f1 := func() {

```

```

    fmt.Println("Hello")
}
f1()
f1()

```

A function assigned to a variable can be called repeatedly using that variable. Functions that perform an action without returning a value are called void functions — they simply omit the return type.

Returning Functions

Just as you can pass functions as arguments, you can also return them. The return type must include the full signature of the function being returned:

```

func getFunc(str string) func(string) string {
    return func(str2 string) string {
        return str + str2
    }
}

```

The returned anonymous function captures `str` from the outer function's scope. This is known as a **closure** — the inner function retains access to variables from the scope where it was defined, even after that scope has returned.

```

f2 := getFunc("Hello")
res := f2(" World")
fmt.Println(res)           // Hello World
fmt.Printf("%T\n", getFunc) // prints the type signature of getFunc

```

Variadic Functions

A variadic function accepts a variable number of arguments of the same type, using the `...` operator before the type name. Inside the function, the parameter is treated as a slice:

```

func sum(num ...int) int {
    total := 0
    for _, val := range num {
        total += val
    }
    return total
}

```

You can call it with any number of arguments:

```

fmt.Println(sum(1, 2, 3, 4, 5))

```

You can also pass an existing slice to a variadic function using **variadic expansion** — the `...` operator placed after the slice when calling the function. This unpacks the slice into individual arguments. Note that this only works if the function itself uses a variadic parameter, and it does not work directly on an array:

```
fmt.Println(sum([]int{1, 2, 3, 4, 5, 6}...))
```

Variadic expansion is also useful when appending one slice to another using the built-in `append` function:

```
a := []int{1, 2}
b := []int{3, 4}

a = append(a, b...)
```

Named Return Values

Go allows you to name the return values in a function signature. When you do this, Go automatically declares those names as local variables within the function scope, so you can use and return them by name:

```
func minusOne(num int) (result int) {
    result = num - 1
    return result
}
```

This becomes more useful when a function returns multiple values, as it makes the return values self-documenting: `(result int, name string)`.

```
package main

import "fmt"

func add(num1 int, num2 int) int {
    return num1 + num2
}

func addAndHello(num1 int, num2 int, name string) (int, string) {
    return num1 + num2, name
}

func callFunc(callable func(int) int) int {
    return callable(10)
}

func doubleNumber(num int) int {
    return num * 2
}

func getFunc(str string) func(string) string {
    return func(str2 string) string {
        return str + str2
    }
}

func sum(num ...int) int {
```

```

    total := 0
    for _, val := range num {
        total += val
    }
    return total
}

func minusOne(num int) (result int) {
    result = num - 1
    return result
}

func main() {
    fmt.Println(add(3, 4))
    fmt.Println(addAndHello(3, 4, "sora"))
    fmt.Println(callFunc(doubleNumber))

    value := callFunc(func(x int) int {
        return x + 1
    })
    fmt.Println(value)

    f1 := func() {
        fmt.Println("Hello")
    }
    f1()
    f1()

    f2 := getFunc("Hello")
    res := f2(" World")
    fmt.Println(res)
    fmt.Printf("%T\n", getFunc)

    fmt.Println(sum(1, 2, 3, 4, 5))
    fmt.Println(sum([]int{1, 2, 3, 4, 5, 6}...))
    fmt.Println(minusOne(7))
}

```

Structs in Go

A struct is Go's primary tool for defining custom types. It plays a similar role to classes in object-oriented languages — it lets you group related fields together under a single named type.

Defining a Struct

A struct is defined outside of `main` using the `type` keyword, followed by the struct name and the `struct` keyword. Each field is listed with its name and type:

```
type Person struct {
    name string
    age  uint
}
```

Initialising a Struct

There are several ways to create an instance of a struct.

Empty initialisation — all fields are set to their zero values (" for strings, 0 for numbers):

```
p1 := Person{}
fmt.Println(p1)
```

Positional initialisation — values are provided in the same order the fields are declared:

```
p2 := Person{"Sora", 24}
fmt.Println(p2)
```

The explicit form makes the type annotation visible: `var p2 Person = Person{"Sora", 24}` — here `Person` acts as the type.

Labelled initialisation — fields are named explicitly, so order does not matter:

```
p3 := Person{age: 26, name: "Dora"}
fmt.Println(p3)
```

Accessing and Modifying Fields

You can read and write individual fields directly using dot notation:

```
p3.name = "Iggy"
fmt.Println(p3)
```

Passing a Struct to a Function

A struct can be passed to a regular function as a parameter, using its type as the parameter type:

```
func getName(p Person) string {
    return p.name
}
```

```
fmt.Println(getName(p2))
```

Methods on Structs

Go does not have classes, but you can attach functions to a struct type using a **receiver**. A receiver is declared between the `func` keyword and the function name, and it binds the function to that type. The receiver variable (commonly named `self` or anything else) refers to the current instance:

```
func (self Person) tellAge() string {
    return fmt.Sprintf("Hello! I'm %v years old. how about you?", self.age)
}
```

```
fmt.Println(p3.tellAge())
```

When a method is called this way, a **copy** of the struct is passed to the receiver — the original is not modified. The same applies to methods that attempt to change fields:

```
func (self Person) setName(newName string) {
    self.name = newName
    fmt.Println(self) // shows the modified copy
}
```

```
p1.setName("JohnDoe")
fmt.Println(p1) // original is unchanged
```

The modification only affects the local copy inside `setName`. To mutate the original struct from a method, you would use a pointer receiver — this will be covered when pointers are introduced.

Embedded Structs

Go does not have classical inheritance, but it supports **embedded structs** as a form of shallow composition. When you embed one struct inside another, the outer struct gains direct access to the embedded struct's fields and methods without needing to reference it explicitly by name:

```
type Admin struct {
    Person
    role string
}
```

`Admin` embeds `Person`, which means an `Admin` instance can call `tellAge()` directly, as if it were defined on `Admin` itself.

You can initialise an embedded struct in two ways:

```
a1 := Admin{Person: Person{name: "JaneDoe", age: 43}, role: "admin"}
a2 := Admin{Person{"JohnatanDoes", 47}, "Admin"}
fmt.Println(a1)
fmt.Println(a2)

fmt.Println(a2.tellAge())
```

Both forms are valid. The labelled form is more explicit and easier to read, especially when the outer struct has multiple fields.

```
package main

import "fmt"

type Person struct {
    name string
}
```

```

    age uint
}

type Admin struct {
    Person
    role string
}

func (self Person) tellAge() string {
    return fmt.Sprintf("Hello! I'm %v years old. how about you?", self.age)
}

func getName(p Person) string {
    return p.name
}

func (self Person) setName(newName string) {
    self.name = newName
    fmt.Println(self) // shows the modified copy, not the original
}

func main() {
    p1 := Person{}
    fmt.Println(p1)

    p2 := Person{"Sora", 24}
    fmt.Println(p2)

    p3 := Person{age: 26, name: "Dora"}
    fmt.Println(p3)

    p3.name = "Iggy"
    fmt.Println(p3)

    fmt.Println(getName(p2))
    fmt.Println(p3.tellAge())

    p1.setName("JohnDoe")
    fmt.Println(p1) // original unchanged

    a1 := Admin{Person: Person{name: "JaneDoe", age: 43}, role: "admin"}
    a2 := Admin{Person{"JohnatanDoes", 47}, "Admin"}
    fmt.Println(a1)
    fmt.Println(a2)

    fmt.Println(a2.tellAge())
}

```


Modules and Packages in Go

In Go, every directory is a package. The package name is declared at the top of every `.go` file inside that directory. Files in the same directory share the same package and can access each other's identifiers freely.

Project Structure

A typical Go project with a custom package looks like this:

```
Project/
├─ main.go
└─ mypkg/
    └─ exported.go
```

Initialising a Module

Before writing any code that uses imports, you must initialise a Go module at the root of your project:

```
go mod init <folderName> ( anything You Want To Be Module Name )
```

This creates a `go.mod` file that defines the module name. All internal package imports are based on this name. For example, if your module is named `modules`, you import the `mypkg` package as:

```
import "modules/mypkg"
```

Run this command once at the start of the project — imports will not resolve correctly without it.

Exported Names

Go uses a simple rule to control what is accessible outside a package: if an identifier (function, type, struct field, method, interface, variable, constant, or type alias) starts with an **uppercase letter**, it is **exported** and can be used from other packages. If it starts with a **lowercase letter**, it is **unexported** and only accessible from within the same package.

In `mypkg/exported.go`:

```
package mypkg

import "fmt"

type User struct {
    Name string // exported – accessible from other packages
    age  uint   // unexported – only accessible within mypkg
}

func sayHello() {           // unexported
    fmt.Println("Hello")
}
```

```
func SayHello() {           // exported
    fmt.Println("Exported Hello")
}
```

SayHello and User.Name are exported. sayHello and User.age are not — attempting to use them from another package causes a compile-time error.

Using a Package

In main.go, you import the package using the module path and access only its exported identifiers:

```
package main

import (
    "fmt"
    "modules/mypkg"
)

func main() {
    // mypkg.sayHello()    // compile error – unexported function
    mypkg.SayHello()

    u := mypkg.User{}
    u.Name = "Sora"
    // u.age = 24          // compile error – unexported field
    fmt.Println(u)
}
```

The commented-out lines are kept as references to show what causes compile-time errors when the export rule is violated.

To run this project, navigate to the project root (where go.mod is located) and run:

```
go run main.go
```

Or to build and run in one step:

```
go run .
```

Interfaces in Go

An interface defines a **contract** — a set of method signatures that a type must implement. Any type that implements all the methods listed in an interface is automatically considered to satisfy that interface. No explicit declaration of intent is required.

Defining an Interface

An interface is declared with the **type** keyword, followed by the interface name and the **interface** keyword. Inside, you list the method signatures — their name, parameters, and return type. If a method returns nothing, you omit the return type:

```
type Shape interface {
    getPerimeter() uint
}
```

This says: any type that has a `getPerimeter()` method returning `uint` is considered a `Shape`.

Implementing an Interface

A struct satisfies an interface simply by having all the methods the interface defines. There is no `implements` keyword — it is implicit.

```
type Triangle struct {
    a uint
    b uint
    c uint
}

func (self Triangle) getPerimeter() uint {
    return self.a + self.b + self.c
}

func (self Triangle) getSides() []uint {
    return []uint{self.a, self.b, self.c}
}
```

`Triangle` now satisfies the `Shape` interface because it has a `getPerimeter()` method with the correct signature. The `getSides()` method is extra — it belongs to `Triangle` but is not part of the interface.

```
type Square struct {
    width uint
}

func (self Square) getPerimeter() uint {
    return self.width * uint(4)
}
```

`Square` also satisfies `Shape` for the same reason.

Using an Interface as a Type

You can declare a variable using an interface as its type and assign any struct that satisfies that interface to it:

```
var s Shape = Triangle{1, 2, 3}
fmt.Println(s)
fmt.Println(s.getPerimeter())
```

When a struct is stored as an interface type, only the methods defined on that interface are accessible through that variable. Methods that belong to the underlying struct but are not part of the interface are hidden:

```
// fmt.Println(s.getSides())    // compile error – getSides is not part of the Shape interface
```

This line is kept as a reference. It would cause a compile-time error because `getSides` is not declared in `Shape`. This is intentional — interfaces let you control which parts of a type are visible in a given context.

If you need access to struct-specific methods, create an instance of the concrete type directly:

```
tr := Triangle{5, 4, 3}
```

Interfaces Enable Generic Behaviour

One of the most practical uses of interfaces is writing functions that work with multiple struct types without needing a separate version for each:

```
func isEvenPerimeter(shape Shape) bool {
    return shape.getPerimeter()%2 == 0
}
```

This function accepts any value that satisfies `Shape` — whether it is a `Triangle`, a `Square`, or any future type that implements `getPerimeter()`. You do not need to write one version per type.

Slices of Interface Types

You can also declare a slice using an interface type, allowing you to store mixed struct types in a single collection as long as they all satisfy the interface:

```
var shapes []Shape = []Shape{Triangle{2, 3, 4}, Square{4}}
perimeters := uint(0)

for _, shape := range shapes {
    perimeters += shape.getPerimeter()
}

fmt.Println(perimeters)
fmt.Println(isEvenPerimeter(s))
```

This is a powerful pattern — you can iterate over a heterogeneous collection of types and call shared methods on all of them uniformly.

```
package main

import "fmt"

type Shape interface {
    getPerimeter() uint
}

type Triangle struct {
    a uint
    b uint
}
```

```

    c uint
}

func (self Triangle) getPerimeter() uint {
    return self.a + self.b + self.c
}

func (self Triangle) getSides() []uint {
    return []uint{self.a, self.b, self.c}
}

type Square struct {
    width uint
}

func (self Square) getPerimeter() uint {
    return self.width * uint(4)
}

func isEvenPerimeter(shape Shape) bool {
    return shape.getPerimeter()%2 == 0
}

func main() {
    var s Shape = Triangle{1, 2, 3}
    fmt.Println(s)
    fmt.Println(s.getPerimeter())
    // fmt.Println(s.getSides())    // compile error – getSides is not part of the Shape interface

    var shapes []Shape = []Shape{Triangle{2, 3, 4}, Square{4}}
    perimeters := uint(0)

    for _, shape := range shapes {
        perimeters += shape.getPerimeter()
    }

    fmt.Println(perimeters)
    fmt.Println(isEvenPerimeter(s))
}

```

Error Handling in Go

Error handling in Go works differently from languages like JavaScript or Python. Go does not use try/catch blocks. Instead, it has two distinct mechanisms: **returnable errors** for expected failure cases, and **panics** with **recover** for unexpected runtime failures.

Runtime Errors and Panics

Go will not run code that has syntax errors — those must be fixed before compilation. However, runtime errors can still occur. In Go, a runtime error is called a **panic**. A common example is integer division by zero:

```
func devide(a int, b int) int {  
    return a / b  
}
```

```
fmt.Println(devide(1, 0))
```

The original call uses 0 as the divisor, which would cause a panic at runtime.

You can also trigger a panic manually:

```
panic("this operation caused a crash.")    // everything after this line will not execute
```

This line is kept as a reference. Calling `panic` immediately stops normal execution of the current function and begins unwinding the call stack.

The defer Keyword

Before covering how to recover from a panic, it is important to understand `defer`. A deferred statement is guaranteed to run when the surrounding function is about to return — regardless of whether the function exits normally or due to a panic.

For example:

```
func main() {  
    defer fmt.Println("last")  
    fmt.Println("first")  
}
```

Output:

```
first  
last
```

This makes `defer` useful for cleanup tasks, similar to `finally` in JavaScript. A common use case is ensuring a file or network connection is closed even if an error occurs mid-operation. You can also have multiple `defer` statements — they execute in last-in, first-out order.

Recovering from a Panic

The built-in `recover` function stops a panic and retrieves the value passed to it. It must be called inside a deferred function — calling it anywhere else has no effect:

```
func deferredFunc() {  
    fmt.Println("defer")  
    r := recover()  
    if r == nil {  
        fmt.Println("No Error")  
    } else {  
        fmt.Println("Runtime Error occurred: ", r)  
        fmt.Println("some more code to execute... ")  
    }  
}
```

```

    }
}

```

`recover` returns `nil` if no panic occurred, or the panic value if one did. This allows you to inspect the error and continue executing code after what would otherwise have been a crash — similar to how a `catch` block works in JavaScript.

In `main`, the deferred call is registered first, so it runs at the end no matter what:

```

func main() {
    defer deferredFunc()

    fmt.Println(devide(1, 1))
    fmt.Println("run")

    // ...
}

```

Recap: when a panic occurs, execution of the current function stops immediately. If a deferred function exists and calls `recover`, the panic is intercepted, the deferred function runs to completion, and the program can continue from there.

Returning Errors from Functions

For predictable failure cases — such as invalid input — the idiomatic Go approach is to return an error value alongside the normal return value. The `errors` package provides the `errors.New` function to create an error with a message:

```

func devide2(a int, b int) (int, error) {
    if b == 0 {
        return 0, errors.New("division by 0")
    }
    return a / b, nil
}

```

When no error occurs, `nil` is returned in the error position. The caller checks whether the error is `nil` before using the result:

```

val, er := devide2(4, 0)
fmt.Printf("value is: %v and error is: %v", val, er)

```

This pattern — returning `(value, error)` and checking the error before proceeding — is the standard way to handle expected errors in Go. It makes error handling explicit and visible at every call site.

```

package main

import (
    "errors"
    "fmt"
)

func devide(a int, b int) int {

```

```

    return a / b
}

func devide2(a int, b int) (int, error) {
    if b == 0 {
        return 0, errors.New("division by 0")
    }
    return a / b, nil
}

func deferredFunc() {
    fmt.Println("defer")
    r := recover() // recover must be called inside the deferred function itself
    if r == nil {
        fmt.Println("No Error")
    } else {
        fmt.Println("Runtime Error occoured: ", r)
        fmt.Println("some more code to execute... ")
    }
}

func main() {
    defer deferredFunc()

    fmt.Println(devide(1, 1)) // safe – divisor changed to 1 to avoid a panic
    // panic("this operation caused a crash.")    // manually triggers a panic; nothing after

    fmt.Println("run")

    val, er := devide2(4, 0)
    fmt.Printf("value is: %v and error is: %v", val, er)
}

```

Generics in Go

Go is a statically typed language, which means every variable and parameter must have a known type at compile time. Generics solve a specific problem that arises from this: writing a function that works correctly for multiple types without duplicating the code.

The Problem Generics Solve

Consider an `add` function. You might need one version for `int`, another for `float64`, another for `uint`, and so on. Generics let you write a single function that works for all of them.

Defining a Generic Function

To make a function generic, add a set of square brackets after the function name. Inside those brackets, define a **type parameter** — typically a single capital letter like `T` —

followed by a **type constraint** that lists which types are allowed:

```
func add[T int | float64 | uint](x T, y T) T {
    return x + y
}
```

The constraint `int | float64 | uint` means `T` can only be one of those three types. The `x`, `y` parameters and the return value all use `T`, so the function works correctly regardless of which allowed type is passed.

```
fmt.Println(add(8, 9))
fmt.Println(add(8.5, 9.3))
```

Go infers the type from the arguments — you do not need to specify it explicitly at the call site.

Multiple Type Parameters

A generic function can have more than one type parameter. Each parameter can carry its own constraint:

```
func getValues[K comparable, V any](mp map[K]V) []V {
    values := []V{}
    for _, value := range mp {
        values = append(values, value)
    }
    return values
}
```

`K comparable` means the key type must support equality comparison — this is required by Go for all map key types. `V any` means the value type can be anything. The function collects all values from any map into a slice and returns it.

```
mp1 := map[string]int{"a": 8, "b": 9, "c": 10}
fmt.Println(getValues(mp1))
```

Using an Interface as a Type Constraint

Writing type constraints directly inside the square brackets is valid, but the preferred approach is to define the constraint as an interface and reference it by name. In generics, an interface can list concrete types instead of methods:

```
type Number interface {
    int | float64 | uint
}

func add2[T Number](x T, y T) T {
    return x + y
}
```

This is cleaner and reusable. Under the hood, writing the constraint inline is equivalent — Go creates an anonymous interface for it automatically:

```
/*
type _AnonymousConstraint interface {
    int | float64 | uint
}
*/
```

This block is kept to show what Go generates internally when you write type constraints directly in the square brackets.

```
fmt.Println(add2(4, 6))
fmt.Println(add2(4.4, 6.4))
```

This reveals that interfaces in Go serve two distinct roles: **method abstraction** for structs (as covered in the interfaces chapter), and **type constraints** for generics.

The type Keyword

Before looking at generic types, it is important to understand what the `type` keyword does. It creates a **new named type** with an underlying type:

```
// type Age uint8
// var a Age = 20    // underlying type is uint8, but Age and uint8 are not interchangeable
```

This is the same mechanism used when defining structs: `type Person struct { ... }`. The new type inherits all operations that work on the underlying type, but it is treated as a distinct type by the compiler.

Custom Generic Types

You can apply this same `type` keyword to create a **generic type** — a named type that takes a type parameter:

```
type GenericSlice[T any] []T
```

`GenericSlice` is a new type whose underlying type is a slice of `T`, where `T` can be any type. It is not a plain slice — it is a distinct named type that happens to behave like one.

Because it is a named type, you can define methods on it:

```
func (g GenericSlice[T]) Print() {
    for _, v := range g {
        fmt.Println(v)
    }
}
```

The receiver syntax `GenericSlice[T]` is required here — `T` must be carried through so the method knows what type it is working with.

```
gs := GenericSlice[int]{1, 2, 3}
fmt.Printf("gs Type: %T and value: %v\n", gs, gs)
gs.Print()
```

Type Aliases

Go also allows creating **type aliases** using the **type** keyword with an assignment operator:

```
type IntSlice = []int
```

An alias and its original type are completely interchangeable — they are the same type, just under a different name. This is different from a defined type (like `GenericSlice`), where the two types are distinct even though one is built on top of the other.

You cannot define methods on an alias, and you cannot use an alias to extend or modify a built-in type.

```
var x IntSlice = []int{5, 6, 7}
var y []int = IntSlice{5, 6, 7}
fmt.Printf("type for x is: %T and value is: %v\n", x, x)
fmt.Printf("type for y is: %T and value is: %v\n", y, y)
```

```
package main
```

```
import "fmt"
```

```
func add[T int | float64 | uint](x T, y T) T {
    return x + y
}
```

```
func getValues[K comparable, V any](mp map[K]V) []V {
    values := []V{}
    for _, value := range mp {
        values = append(values, value)
    }
    return values
}
```

```
type Number interface {
    int | float64 | uint
}
```

```
func add2[T Number](x T, y T) T {
    return x + y
}
```

```
/*
type _AnonymousConstraint interface {
    int | float64 | uint
}
*/
```

// Go generates an anonymous interface like this internally when type constraints are written

```
type GenericSlice[T any] []T
```

```
func (g GenericSlice[T]) Print() {
```

```

    for _, v := range g {
        fmt.Println(v)
    }
}

type IntSlice = []int

func main() {
    fmt.Println(add(8, 9))
    fmt.Println(add(8.5, 9.3))

    mp1 := map[string]int{"a": 8, "b": 9, "c": 10}
    fmt.Println(getValues(mp1))

    fmt.Println(add2(4, 6))
    fmt.Println(add2(4.4, 6.4))

    gs := GenericSlice[int]{1, 2, 3}
    fmt.Printf("gs Type: %T and value: %v\n", gs, gs)
    gs.Print()

    var x IntSlice = []int{5, 6, 7}
    var y []int = IntSlice{5, 6, 7}
    fmt.Printf("type for x is: %T and value is: %v\n", x, x)
    fmt.Printf("type for y is: %T and value is: %v\n", y, y)
}

```

Pointers and References in Go

A pointer stores the **memory address** of a value rather than the value itself. This lets you read and modify the original data directly, instead of working on a copy. Understanding pointers also explains why slices behave differently from plain variables when passed to functions.

How Variables Are Stored in Memory

When you create a variable, Go allocates a location in memory and stores the value there. When you pass that variable to a function, Go allocates a **new** memory location for the function's parameter and copies the value into it. The original variable is untouched:

```

func change(x int) {
    x = 7
}

a := 10
change(a)
fmt.Println(a) // still 10 – change() received a copy, not the original

```

This is why modifying `x` inside `change` has no effect on `a`. They are two separate locations in memory that happen to hold the same value at the moment of the call.

Why Slices Behave Differently

When you create a slice, Go allocates memory for the underlying array and then creates a separate structure holding the length, capacity, and a **pointer to that array’s address**. When you pass a slice to a function, the structure is copied — but the pointer inside it still points to the same underlying array. So any modification to the elements affects the original data.

This is the same mechanism that makes pointers useful for other types.

The & and * Operators

Go provides two operators for working with pointers:

- `&x` — the **address-of** operator. Returns the memory address where `x` is stored.
- `*x` — the **dereference** operator. Given a pointer, accesses the value stored at the address it holds.

The relationship between a variable and its pointer works like this:

- `a` holds the value
- `&a` is the memory address of `a`
- If `x := &a`, then `*x` gives you the value of `a` directly

```
x := &a // x is a pointer to a
*x = 7  // dereference x and change the value at that address
fmt.Println(x) // prints the memory address
fmt.Println(a) // prints 7 – a was modified through the pointer
```

Passing Pointers to Functions

To allow a function to modify the original variable, pass its address using `&` and declare the parameter as a pointer type using `*`:

```
func change2(x *int) {
    *x = 100
}
```

```
change2(&a)
fmt.Println(a) // prints 100
```

`*int` means “a pointer to an int”. Inside the function, `*x` dereferences the pointer to reach and modify the original value. You can also have pointers to pointers — dereferencing twice gives you the underlying value.

Pointer Receivers on Struct Methods

Recall from the structs chapter that a method with a value receiver receives a copy of the struct, so field modifications do not affect the original. Using a **pointer receiver** solves this:

```
type Book struct {
    id    int
    title string
}
```

```

}

func (b *Book) setTitle(title string) {
    (*b).title = title
    // b.title = title    // also valid – Go dereferences automatically when using dot notation
}

```

The explicit form `(*b).title` dereferences the pointer before accessing the field, because Go evaluates dot notation before the dereference operator. However, Go is smart enough to handle `b.title` on a pointer receiver automatically. Writing it explicitly is considered better practice for readability.

When calling a pointer receiver method, you do not need to pass the address manually — Go does it implicitly:

```

b := Book{23, "Old"}
b.setTitle("New")
fmt.Println(b) // prints {23 New}

/*
b := Book{23, "Old"}

p := &b
p.setTitle("New")

fmt.Println(b)
*/

```

Even though `b` is not declared as a pointer, Go automatically passes `&b` to `setTitle` when the method has a pointer receiver. The method modifies the original struct.

```

package main

import "fmt"

func change(x int) {
    x = 7
}

func change2(x *int) {
    *x = 100
}

type Book struct {
    id    int
    title string
}

func (b *Book) setTitle(title string) {
    (*b).title = title // explicit dereference before dot notation
    // b.title = title    // also valid – Go dereferences automatically on pointer receivers
}

```

```

}

func main() {
    a := 10
    change(a)
    fmt.Println(a) // 10 – unchanged, change() received a copy

    x := &a // x holds the memory address of a
    *x = 7 // dereference x and write 7 to that address
    fmt.Println(x) // memory address
    fmt.Println(a) // 7 – modified through the pointer

    change2(&a)
    fmt.Println(a) // 100

    b := Book{23, "Old"}
    b.setTitle("New")
    fmt.Println(b) // {23 New}
}

```

User Input in Go

Now that pointers have been covered, we can look at how Go reads input — because several input methods require you to pass a pointer to the variable that will receive the data.

Go provides three distinct ways to read user input, each suited to a different situation: `fmt.Scanln` for simple inline input, `bufio.Reader` for full-line input at runtime, and `os.Args` for arguments passed at execution time. There is also a fourth method for reading from a Unix pipeline.

Method 1: `fmt.Scanln`

The simplest approach uses `fmt.Scanln` from the `fmt` package. You declare a variable to hold the input, then pass its address to `Scanln` so the function can write directly to it:

```

var name string
fmt.Println("Enter your name: ")
fmt.Scanln(&name)
fmt.Println("Hello", name)

```

`&name` passes a pointer to `name`. `Scanln` dereferences it internally to store the value. The limitation of this method is that it stops reading at the first whitespace — so it cannot capture a full sentence in one call.

Method 2: `bufio.Reader`

For reading a full line of input including spaces, `bufio.NewReader` is the better option. It wraps `os.Stdin` — the standard input file descriptor — in a buffered reader. Input is first

stored in a buffer, and `ReadString` retrieves it from there up to and including the specified delimiter:

```
reader := bufio.NewReader(os.Stdin)
fmt.Print("Enter text: ")
text, _ := reader.ReadString('\n')
fmt.Println("You wrote:", text)
```

`'\n'` tells `ReadString` to keep reading until it encounters a newline. The second return value is an error, which is discarded here with `_`. The pointer handling is done internally by the `bufio` package.

Method 3: Command-Line Arguments

This method collects input at the moment the program is executed — not while it is running. Arguments are passed directly on the command line and accessed through `os.Args`:

```
args := os.Args
fmt.Println(args)
```

Running the program like this:

```
go run main.go firstArg secondArg
```

will print a slice where `args[0]` is the program name and `args[1]`, `args[2]` are the arguments you passed.

Method 4: Reading from a Unix Pipeline

When your compiled Go program is used as part of a shell pipeline — for example `echo input | ./program` — you need `bufio.NewScanner` to read the piped input line by line. `scanner.Scan()` returns `true` for each line it reads, and `scanner.Text()` returns the content of that line:

```
scanner := bufio.NewScanner(os.Stdin)
for scanner.Scan() {
    line := scanner.Text()
    fmt.Println("got:", line)
}
```

Required packages and sample codes for each method:

```
package main
```

```
import (
    "bufio"
    "fmt"
    "os"
)

func main() {
    var name string
    fmt.Println("Enter your name: ")
```



```

fmt.Scanln(&name)
fmt.Println("Hello", name)

reader := bufio.NewReader(os.Stdin)
fmt.Print("Enter text: ")
text, _ := reader.ReadString('\n')
fmt.Println("You wrote:", text)

args := os.Args
fmt.Println(args)

scanner := bufio.NewScanner(os.Stdin)
for scanner.Scan() { // using for loop as a while loop, continue until there is no new p
    line := scanner.Text()
    fmt.Println("got:", line)
}
}

```

Threading and Concurrency

How a CPU Executes Operations

A CPU contains multiple physical **cores** — typically a number that is a power of two, such as 2, 4, or 8. Each core is an independent processing unit capable of executing operations on its own. You can think of each core as a miniature CPU within the larger chip.

The number of cores determines how many operations the CPU can handle in true **parallel** — meaning at the exact same instant in time. A four-core CPU can execute four independent operations simultaneously.

Each core also has a **clock speed**, measured in GHz (e.g. 4.4 GHz). This value represents how many cycles the core can perform per second and determines the speed of individual operations on that core. A higher clock speed means faster execution per core.

These two properties — core count and clock speed — serve different use cases. A single-core CPU running at 7 GHz is not necessarily better than a four-core CPU running at 2.2 GHz. A high-clock-speed single-core chip is better suited for **single-threaded** tasks, where operations must happen in sequence, while a multi-core chip is better suited for **multi-threaded** tasks, where work can be divided and run in parallel.

Processes and Threads

Every program you run is packaged by the operating system into a **process**. A process includes memory allocations, the runtime or interpreter if one is needed, and various other resources required to execute the program.

A process is made up of one or more **threads**. Up to this point, all the programs in this book have been single-threaded — one process containing one thread, executing one operation at a time.

Threads within the same process share resources like memory, variables, and functions, but they execute **independently**. This is what makes multi-threading useful: you can split a program's workload across multiple threads, and those threads can be distributed across multiple CPU cores to run in parallel.

With one thread, only one CPU core can work on your program at any given moment. With two threads, two cores can work simultaneously — effectively doubling throughput for parallelisable workloads.

Thread Scheduling

In practice, a running system has far more threads than CPU cores. The operating system manages this through a **thread pool** and a scheduler. The scheduler continuously evaluates which threads need to run and assigns them to available cores, rapidly switching between thousands of threads. This switching happens so fast that it creates the illusion of all threads running simultaneously.

This is also what allows your computer to run multiple applications at the same time — the OS is constantly swapping threads in and out across cores.

Concurrency vs Parallelism

These two terms are related but distinct:

Parallelism means multiple operations are executing at the exact same instant in time — this requires multiple CPU cores.

Concurrency means multiple tasks are being progressed at the same time, but not necessarily simultaneously. On a single core, the CPU switches between tasks rapidly — working on one, pausing it, moving to another, and so on. All tasks move forward, but only one is being processed at any given moment.

- Within a single CPU core: **concurrent** execution
- Across multiple CPU cores: **parallel** execution

Modern CPUs also support **hyperthreading** — a hardware technique that allows one physical core to present itself to the operating system as two logical cores, enabling it to keep its execution units more fully utilised per cycle.

You do not need to manually assign threads to specific cores. The OS and the language runtime handle scheduling. Your responsibility as a programmer is to identify which parts of your code can run independently and separate them into different threads so the system can take advantage of parallelism.

When Multi-Threading Matters

Consider an application that needs to do three things on startup:

- Load the user's chat history from a server
- Fetch the user's friends list
- Allow the user to create a post immediately

In a single-threaded program, these tasks run sequentially. If the chat history fetch is slow or fails, the entire program stalls — nothing else can execute because the single thread is blocked waiting for a response. The user cannot interact with anything in the meantime.

By distributing these tasks across multiple threads, all three can progress independently. The two network requests can run in parallel while the UI remains responsive. This is one of the most common and impactful applications of multi-threading in real-world software.

Go Routines

What Are Go Routines?

Go routines are Go's built-in mechanism for concurrency. They are lightweight threads managed by the Go runtime, and they allow different parts of your program to run independently of each other. If you are new to the concept, refer to the [threading and concurrency](#) section before continuing.

To launch a function as a go routine, you simply prefix the call with the `go` keyword.

The `time` Package

The `time` package is imported here to introduce deliberate delays using `time.Sleep`. This is used to simulate work taking different amounts of time across the three functions, and to keep the main go routine alive long enough for the others to finish.

The Worker Functions

Three functions are defined — `run`, `run2`, and `run3` — each sleeping for a different duration before printing output. This makes the concurrency behaviour visible: the functions complete in order of their sleep durations, not in the order they were launched.

```
func run() {
    time.Sleep(2 * time.Second)
    fmt.Println("run")
}
func run2() {
    time.Sleep(4 * time.Second)
    fmt.Println("run2")
}
func run3() {
    time.Sleep(6 * time.Second)
    fmt.Println("run3")
}
```

Launching Go Routines in Main

By default, a Go program has exactly one go routine: the one running `main`. When you prefix a function call with `go`, that function runs in a new, independent go routine and does not block the caller.

This means `main` does not wait for `run`, `run2`, or `run3` to finish. If `main` reaches the end of its own code and exits, the entire program stops — even if other go routines are still running.

To prevent that, `time.Sleep(7 * time.Second)` is called after launching the three go routines. This keeps the main go routine alive long enough for all three functions to

complete (the longest sleeps for 6 seconds).

Note: Using `time.Sleep` to synchronise go routines is not considered clean or reliable code. It is used here only to demonstrate the basic concept. Part 2 covers more robust approaches using channels and `sync.WaitGroup`.

```
package main

import (
    "fmt"
    "time"
)

func run() {
    time.Sleep(2 * time.Second)
    fmt.Println("run")
}

func run2() {
    time.Sleep(4 * time.Second)
    fmt.Println("run2")
}

func run3() {
    time.Sleep(6 * time.Second)
    fmt.Println("run3")
}

func main() {
    go run()
    go run2()
    go run3()
    time.Sleep(7 * time.Second)
    fmt.Println("done")
}
```

Go Routines — Part 2: Channels

The Problem with `time.Sleep`

Using `time.Sleep` to wait for go routines to finish is unreliable. Go provides better mechanisms for synchronisation, and the most fundamental one is the **channel**.

Why You Cannot Use a Return Value from a Go Routine

When you launch a function with `go`, you cannot capture its return value directly (just like Promises in JavaScript, Right?). The following syntax is invalid:

```
x := go add(5, 10)
```

This does not work because the go routine runs independently — there is no moment where the caller can receive a return value the way a normal function call would provide one.

What Is a Channel?

A channel is a typed conduit that allows go routines to pass values to each other. You can send a value into a channel and receive it on the other end. Importantly, receiving from a channel **blocks execution** until a value is available, which gives you a clean way to synchronise go routines without using `time.Sleep`.

You create a channel using `make`, specifying the type of value it will carry:

```
ch := make(chan int)
```

Passing a Channel to a Go Routine

The `add` function accepts two integers and a channel of type `int`. Instead of returning a value, it sends the result onto the channel using the `<-` operator:

```
func add(x int, y int, ch chan int) {  
    ch <- x + y  
}
```

The `<-` operator is used both for sending (as above) and for receiving (shown below). There is no return type on this function because the result is communicated through the channel, not returned to the caller.

Deadlocks

If a go routine is expected to send a value on a channel but never does, the program will throw a fatal runtime error called a **deadlock**. A deadlock occurs when no threads are making progress — one thread is waiting to receive from a channel, but nothing ever sends to it. Go detects this at runtime and terminates the program with an error message.

Receiving from a Channel (Blocking Code)

To receive a value from a channel, you use the `<-` operator on the left-hand side of an assignment:

```
x := <- ch
```

This line is **blocking** — the program will not advance past it until a value arrives on `ch`. This is what makes channels a reliable synchronisation tool: you are guaranteed that by the time execution continues, the go routine has completed its work and delivered its result.

Recap

- You cannot get a return value directly from a go routine.
- Instead, you pass a channel into the go routine and send the result onto it.
- The receiving side blocks until the value arrives, eliminating the need for `time.Sleep`.
- If nothing ever sends on a channel that something is waiting to receive from, Go raises a deadlock error at runtime.

```
package main  
  
import "fmt"
```

```
func add(x int, y int, ch chan int) {
    ch <- x + y
}

func main() {
    ch := make(chan int)
    go add(5, 10, ch)
    x := <-ch
    fmt.Println(x)
}
```

Go Routines — Part 3: Multiple Go Routines on a Single Channel

Receiving from Multiple Go Routines

Building on the previous example, consider running several `add` operations concurrently, all sending their results onto the same channel:

```
ch := make(chan int)
go add(5, 10, ch)
go add(8, 11, ch)
go add(5, 0, ch)
go add(5, -5, ch)
```

All four go routines launch at approximately the same time, each in its own independent thread. Every one of them will print its result and then send a value onto `ch`.

How a Single Receive Works

With only one receive statement:

```
x := <- ch
```

the program waits for the **first** value to arrive on the channel. As soon as any one of the four go routines sends its result, execution unblocks, `x` is assigned that value, and the program prints it. Which go routine wins is not predictable — there is no guaranteed order.

Once `main` prints `x` and reaches the end of its code, the program exits. The remaining three go routines are abandoned, regardless of whether they have finished.

Waiting for All Go Routines

To collect results from all four go routines, you need one receive statement per go routine. The following commented-out lines show the pattern:

```
x := <- ch
// x = <- ch
// x = <- ch
// x = <- ch
```

These three additional receive statements are left commented out here to illustrate the point without changing the program's current behaviour. If you uncomment them, `main` will block until all four go routines have sent their values. Because channels do not guarantee delivery order, `x` will be overwritten on each receive — its final value will be whatever the **last** go routine to send happened to produce.

The `add` Function

The `add` function in this version prints the sum before sending it on the channel, so you can observe each go routine completing even when only one value is being received:

```
func add(x int, y int, ch chan int) {
    fmt.Println(x + y)
    ch <- x + y
}
```

```
package main
```

```
import (
    "fmt"
)
```

```
func add(x int, y int, ch chan int) {
    fmt.Println(x + y)
    ch <- x + y
}
```

```
func main() {
    ch := make(chan int)
    go add(5, 10, ch)
    go add(8, 11, ch)
    go add(5, 0, ch)
    go add(5, -5, ch)
    x := <-ch
    x = <-ch
    x = <-ch
    x = <-ch
    fmt.Println(x)
}
```

Go Routines — Part 4: Enforcing Predictable Order

The Problem with Unpredictable Order

As shown in Part 3, launching all go routines at once and then receiving from the channel gives you no control over which result arrives first. If order matters, a different approach is needed.

Interleaving Launch and Receive

The solution is to launch one go routine, immediately block on a receive, then launch the next, and so on. Each `go` call is followed directly by `x := <-ch` or `x = <-ch`, which forces the program to wait for that specific go routine to finish before the next one is even started:

```
go add(5, 10, ch)
x := <- ch
go add(8, 11, ch)
x = <- ch
go add(5, 0, ch)
x = <- ch
go add(5, -5, ch)
x = <- ch
```

Because each receive blocks until the corresponding go routine sends its value, the operations now execute one at a time in exactly the order they are written. There is no ambiguity about which result is received at each step.

Trade-offs

This approach gives you predictable, sequential execution. However, since each go routine must complete before the next one starts, you lose the performance benefit of true concurrency. For use cases where order is critical and parallelism is not required, this is a clean and readable pattern. When you need both order and concurrency, channels can be combined with `sync.WaitGroup` or buffered channels — these are covered in later sections.

After all four receives complete, `x` holds the result of the last operation ($5 + (-5) = 0$), and that value is printed.

```
package main

import (
    "fmt"
)

func add(x int, y int, ch chan int) {
    fmt.Println(x + y)
    ch <- x + y
}

func main() {
    ch := make(chan int)
    go add(5, 10, ch)
    x := <-ch
    go add(8, 11, ch)
    x = <-ch
    go add(5, 0, ch)
    x = <-ch
    go add(5, -5, ch)
```



```
x = <-ch
fmt.Println(x)
}
```

Go Routines — Part 5: Multiple Channels and the **select** Statement

Using Multiple Channels

Instead of routing all go routine results through a single channel, you can create a separate channel for each go routine. This gives you independent, named conduits for each result:

```
ch := make(chan int)
ch2 := make(chan int)

go add(4, 5, ch)
go add(8, 9, ch2)
```

Both go routines launch concurrently. Because their results travel on separate channels, you can interleave other work between launching them and collecting their output:

```
fmt.Println("Some operation here...")
fmt.Println("Some other operation")
```

The Limitation of Sequential Receives

You could collect both results like this:

```
// x := <- ch
// y := <- ch2
// fmt.Println(x, y)
```

These lines are intentionally left commented out to highlight a problem: this approach waits for `ch` first and then `ch2`, regardless of which go routine finishes first. If `ch2` happens to produce its result before `ch`, the program still blocks on `ch` until it receives — you are imposing an arbitrary order on results that arrive concurrently.

The **select** Statement

The **select** statement solves this by waiting on multiple channels simultaneously. As soon as **any** of the listed channels has a value available, the corresponding **case** executes:

```
select {
    case x := <- ch:
        fmt.Println(x)
    case y := <- ch2:
        fmt.Println(y)
}
```

Whichever channel receives a value first is handled immediately, without waiting for the other. If both channels are ready at the same time, Go picks one of the ready cases at random.

Collecting All Results with a Loop

A single `select` only handles one channel event. To wait for both go routines to deliver their results, wrap the `select` in a `for` loop with its iteration count matching the number of go routines:

```
for i := 0; i < 2; i++ {
    select {
        case x := <- ch:
            fmt.Println(x)
        case y := <- ch2:
            fmt.Println(y)
    }
}
```

On the first iteration, whichever channel is ready first is handled. On the second iteration, the remaining channel is handled. This collects all results in arrival order, without blocking unnecessarily on either channel.

```
package main

import "fmt"

func add(x int, y int, ch chan int) {
    // fmt.Println(x + y)
    ch <- x + y
}

func main() {
    ch := make(chan int)
    ch2 := make(chan int)

    go add(4, 5, ch)
    go add(8, 9, ch2)

    fmt.Println("Some operation here...")
    fmt.Println("Some other operation")

    for i := 0; i < 2; i++ {
        select {
            case x := <-ch:
                fmt.Println(x)
            case y := <-ch2:
                fmt.Println(y)
        }
    }
    fmt.Println("Final operation")
}
```

Go Routines — Part 6: Directional Channels

What Are Directional Channels?

By default, a channel is bidirectional — you can both send to it and receive from it. Go also allows you to restrict a channel to a single direction when passing it to a function. This is a compile-time constraint that makes your intent explicit and prevents misuse.

There are three modes:

- `func transmit(ch chan int)` — bidirectional channel. You can send data with `ch <- data` or receive with `x := <- ch`.
- `func send(ch chan<- int)` — send-only channel. Attempting to receive with `x := <- ch` will cause a compile error.
- `func receive(ch <-chan int)` — receive-only channel. Attempting to send with `ch <- 42` will cause a compile error.

The arrow notation shows the direction of data flow relative to the channel: `chan<-` means data flows into the channel, and `<-chan` means data flows out of it.

Nested Go Routines and Channel Synchronisation

The `add` function uses a general bidirectional channel, which means it is also possible to receive from that channel inside the function. This opens the door to more advanced patterns where go routines spawn their own go routines and pass channels inward to coordinate execution across multiple levels:

```
func add(x int, y int, ch chan int) {  
    ch <- x + y  
}
```

The next section covers **buffered channels**, which allow a channel to hold a fixed number of values without requiring an immediate receiver.

Go Routines — Part 7: Buffered Channels

Unbuffered Channels and Blocking Sends

All channels created with `make(chan type)` are unbuffered by default. With an unbuffered channel, a send operation is **blocking** — the sending go routine pauses at that line and does not continue until another go routine receives the value. Equally, a receive on an empty unbuffered channel blocks until something sends.

This means the following code, without the `r` function and `go r(ch)`, would deadlock immediately:

```
ch <- true
```

There is only one go routine — `main` — and it is blocked waiting for a receiver that never comes. The `r` function exists specifically to act as that receiver on a separate go routine:

```
func r(ch chan bool) {  
    <- ch  
}
```

Launching it with `go r(ch)` before sending on `ch` provides the receiver that unblocks `main`:

```
ch := make(chan bool)
go r(ch)
ch <- true
fmt.Println("DONE")
```

An unbuffered channel handles exactly one send-receive pair at a time. If you send a second value before the first has been received, you deadlock again.

Buffered Channels

A buffered channel is created by passing a capacity as the second argument to `make`:

```
ch2 := make(chan bool, 2)
```

This channel can hold up to two values internally without requiring an immediate receiver. Sends only block when the channel is **full**. Receives only block when the channel is **empty**.

The following sequence demonstrates this:

```
ch2 <- true    // channel holds 1 value, not full – does not block
ch2 <- true    // channel holds 2 values, now full – does not block yet
<- ch2        // removes one value, channel holds 1 value again
ch2 <- true    // channel holds 2 values again – does not block
fmt.Println("DONE AGAIN")
```

After the receive frees up a slot, the third send succeeds without blocking. No additional `go` routine is needed because the channel has internal capacity to absorb values before they must be consumed.

Blocking Rules for Buffered Channels

- **Sending** blocks only when the channel is full.
- **Receiving** blocks only when the channel is empty — attempting to receive from an empty buffered channel causes a deadlock, exactly as with an unbuffered channel.

The next section covers **mutexes** (locks), which provide a different mechanism for coordinating access to shared state across `go` routines.

```
package main

import (
    "fmt"
)

func r(ch chan bool) {
    <-ch
}

func main() {
    ch := make(chan bool)
    go r(ch)
    ch <- true
}
```

```

    fmt.Println("DONE")

    ch2 := make(chan bool, 2)
    ch2 <- true
    ch2 <- true
    <-ch2
    ch2 <- true
    fmt.Println("DONE AGAIN")
}

```

Go Routines — Final Part: Mutexes and Wait Groups

The Problem: Race Conditions

When multiple go routines read and modify a shared variable at the same time, you get a **race condition**. Consider 100 go routines all incrementing a shared counter: two of them might read the same value simultaneously, both increment it to the same result, and one overwrites the other. Some increments are lost and the final count is wrong.

Go’s race detector will catch this. Running your program with `go run -race main.go` will report something like `Found <n> data race(s)` whenever unsynchronised access to shared memory is detected.

The Counter Struct

The shared state is represented as a struct that bundles the integer value together with a `sync.Mutex`:

```

type Counter struct {
    value int
    lock  sync.Mutex
}

```

Keeping the mutex inside the struct that owns the data is a clean pattern — the lock travels with the resource it protects.

Fixing the Race Condition with a Mutex

A mutex (short for mutual exclusion) is a lock that only one go routine can hold at a time. When a go routine calls `Lock()`, every other go routine that tries to call `Lock()` on the same mutex will block until the first one calls `Unlock()`.

In practice this means: “I am working on this value right now — no one else may touch it.” As soon as the lock is released, the next go routine waiting to acquire it gets its turn. Because only one go routine can hold the lock at a time, they effectively take turns modifying the counter, eliminating the race condition entirely.

```

func count(counter *Counter, wg *sync.WaitGroup) {
    counter.lock.Lock()
    defer counter.lock.Unlock()
    (*counter).value++
}

```

```

    fmt.Println(counter.value)
    (*wg).Done()
}

```

`defer counter.lock.Unlock()` ensures the lock is always released when the function returns, even if something goes wrong before it reaches the end. `(*wg).Done()` decrements the wait group counter by one, signalling that this go routine has finished.

Waiting for Go Routines: Three Approaches

The commented-out `time.Sleep` and the commented-out channel loop illustrate two earlier approaches to waiting for go routines:

```

// time.Sleep(2 * time.Second)

/*
for i := 0; i < 100; i++ {
    <- ch
}
*/

```

`time.Sleep` is unreliable — you are guessing how long the work will take. The channel loop is more precise but requires passing a channel into every go routine and sending a value at the end of each one, even when you do not need to return any data.

The recommended approach is a **wait group**.

Wait Groups

A `sync.WaitGroup` is a counter that tracks how many go routines are still running. Three methods are used:

- `wg.Add(n)` — increments the counter by `n`. Call this **before** launching the go routine, not inside it, to avoid a race between the add and the wait.
- `wg.Done()` — decrements the counter by one. Called inside the go routine when its work is complete.
- `wg.Wait()` — blocks until the counter reaches zero, then allows the program to continue.

Rather than adding all 100 at once with `wg.Add(100)`, the counter is incremented inside the loop, one per iteration, immediately before launching each go routine:

```

for i := 0; i < 100; i++ {
    wg.Add(1)
    go count(&counter, &wg)
}

wg.Wait()

```

Both the counter and the wait group are passed by pointer so that all go routines operate on the same shared instances rather than copies.

The `time` import is commented out because `time.Sleep` is no longer needed — the wait group handles synchronisation cleanly without it:

```
// "time"
```

The key rule for shared resources across go routines: always protect them with a mutex so that no two go routines can read and modify the same value at the same time.

Save the code below as `goRoutines-pt8.go` and run it with `go run goRoutines-pt8.go`. To verify that the race condition is fully resolved, you can also run it with `go run -race goRoutines-pt8.go`.

General Explanation:

So in this example, we created a lock for our counter using `sync.Mutex`, and this lock keeps its default value.

Then when we pass our counter to the function, this counter has a value and a lock.

We access its lock and call the `Lock` method on it, and it gets locked.

At the same time, we also create a `WaitGroup` for managing concurrency so it waits until all of them are finished.

Then when our work is done, we call the `Done` method on our `WaitGroup` so the value that we gave to `Add` decreases by one.

And then at the end, using `defer`, we remove the lock from our object.

So `Mutex` and `WaitGroup` do not necessarily have to both exist.

We use `Mutex` in places where we want the data not to get a race condition.

```
package main

import (
    "fmt"
    "sync"
    // "time"
)

type Counter struct {
    value int
    lock  sync.Mutex
}

func count(counter *Counter, wg *sync.WaitGroup) {
    counter.lock.Lock()
    defer counter.lock.Unlock()
    (*counter).value++
    fmt.Println(counter.value)
    (*wg).Done()
}

func main() {
    counter := Counter{value: 0}
```

```

wg := sync.WaitGroup{}

for i := 0; i < 100; i++ {
    wg.Add(1)
    go count(&counter, &wg)
}

// time.Sleep(2 * time.Second)
/*
for i := 0; i < 100; i++ {
    <- ch
}
*/

wg.Wait()
}

```

Go — What's Next?

The core language fundamentals are behind you now. You have covered syntax, types, control flow, functions, pointers, structs, and concurrency with go routines, channels, mutexes, and wait groups.

From here, the best way to move forward is to pick a project and start building. Go has a rich standard library and a straightforward package system — when you need something, look up the relevant package, read its documentation, and apply it. That process of building real tools is where Go starts to feel natural.

Some areas worth exploring next, depending on what you want to build:

- `net/http` — for writing web servers and HTTP clients / web Security tools
- `os` and `io` — for file system interaction and I/O operations
- `encoding/json` — for parsing and producing JSON
- `net` — for lower-level TCP/UDP networking
- `crypto` — for hashing, encryption, and TLS

Go's official documentation at pkg.go.dev covers every standard library package with examples. It is the most reliable reference you will use on a regular basis.

PHP Fundamentals

PHP is considered a traditional backend language, but it is still widely used across many web systems today.

Understanding PHP is important for analyzing legacy systems, identifying potential security issues, and performing small-scale scripting tasks.

This section introduces the core concepts of PHP and provides a basic understanding of server-side rendering and request handling.

What is PHP?

PHP is a server-side scripting language. It runs on servers and is used to build dynamic web pages.

A browser sends an HTTP request to the server. On that server, PHP processes the request and sends HTML back to the browser.

PHP can communicate with databases for many purposes. PHP is most commonly used with relational databases such as MySQL, PostgreSQL, and Oracle.

PHP can be written alongside HTML. The only dependency you really need to write PHP code is PHP itself, which can be installed with:

```
sudo apt install php (on Debian-based systems).
```

As a web server you can use Apache or Nginx.

Getting Started with PHP

Save the following code into a file called `gettingStarted.php`. To run it, you need a PHP interpreter installed. You can serve it with the built-in PHP development server:

```
php -S localhost:8000
```

Then open your browser and navigate to `http://localhost:8000/gettingStarted.php`. Alternatively, you can run it directly in the terminal with `php gettingStarted.php`, though HTML output will appear as raw text rather than rendered markup.

File Structure

Every PHP file begins with the opening tag `<?php` and optionally closes with `?>`. Code outside these tags is treated as raw HTML and is sent directly to the browser.

Comments

PHP uses the same comment syntax as Go and C:

- Single-line comments start with `//`
- Multi-line comments are wrapped between `/*` and `*/`

Output and Semicolons

Every PHP statement must end with a semicolon. The `echo` keyword outputs a string to the browser.

PHP and HTML

PHP is tightly integrated with HTML. You can embed HTML tags directly inside PHP strings — the browser will render them as markup. PHP files can also contain CSS and JavaScript alongside PHP code.

```

<?php
echo "I like pizza <br>";
echo "Its really good";
?>

<!DOCTYPE html>
<html>
<head>
</head>
<body>
    <br />
    <button>Order Pizza</button>
</body>
</html>

```

Variables in PHP

PHP is dynamically typed, so variables do not require a type declaration. The type is inferred from the assigned value. All variable names are prefixed with a \$ sign.

Data Types

PHP variables can hold several types of values:

```

<?php
$uniqueName = "Sora Void";    // string
$food = "pasta";

$age = 24;                    // integer – a whole number
$quantity = 3;

$gpa = 2.5;                   // floating point number
$price = 4.99;

$employed = true;
$online = false;

$total = null;                // null means no value

```

Output and String Interpolation

The echo keyword outputs a value to the browser. PHP supports **string interpolation** — you can embed variables directly inside double-quoted strings using either `{ $variable }` or `$(variable)` syntax. Both forms are valid.

To print a literal \$ sign before an interpolated variable (for example, to display a price), escape it with a backslash: `\$`.

```

echo $uniqueName;
echo "This is a string literal";

```

```

echo "Hello {$uniqueName} <br>";
echo "you like {$food} <br>";
echo "and you are {$age} years old. <br>";
echo "your GPA is {$gpa} <br>";
echo "the price of this product is ${price} <br>";
echo "the price of this product is \${$price} <br>";
echo "Online status: ${online} <br>"; // false outputs nothing
echo "Employment status: ${employed} <br>"; // true outputs 1

echo "your food summary will be ${price} x ${quantity} <br>";
$total = $price * $quantity;
echo "totally: \${$total} <br>";

```

Note that when a boolean variable is interpolated into a string, `true` outputs `1` and `false` outputs nothing.

Type Casting

PHP allows you to explicitly convert a value from one type to another by prefixing it with the target type in parentheses:

Syntax	Converts to
<code>(int) \$x</code>	Integer
<code>(string) \$x</code>	String
<code>(bool) \$x</code>	Boolean
<code>(float) \$x</code>	Float
<code>(array) \$x</code>	Array
<code>(object) \$x</code>	Object

```

$a = "123";
$b = (int)$a;

```

Inspecting Variables with `var_dump`

Unlike `echo`, which only outputs the value, `var_dump` prints the type, value, and — for strings and arrays — the length or structure. It is useful for debugging:

```

var_dump($b);
$x = ["a", "b"];
var_dump($x);
?>

```

Arithmetic Operators in PHP

Variables in PHP

In PHP, all variables are prefixed with a `$` sign. PHP is dynamically typed, so you do not need to declare a type — it is inferred from the assigned value. `null` is a valid initial value representing the absence of a value.

```
<?php
$x = 10;
$y = 2;
$z = null;
```

Arithmetic Operators

PHP supports the following arithmetic operators:

Operator	Operation
+	Addition
-	Subtraction
*	Multiplication
/	Division
**	Exponentiation
%	Modulo (remainder)

Unlike Go, PHP has a built-in `**` operator for exponentiation — no external package is needed.

```
$z = $x + $y;
echo $z;
echo "<br>";

$z = $x - $y;
echo $z;
echo "<br>";

$z = $x * $y;
echo $z;
echo "<br>";

$z = $x / $y;
echo $z;
echo "<br>";

$z = $x ** $y;
echo $z;
echo "<br>";

$z = $x % $y;
echo $z;
echo "<br>";
```

Increment and Decrement

PHP provides several ways to increase or decrease a numeric variable by 1:

```

$counter = 0;

$counter = $counter + 1;    // standard addition
$counter += 1;             // compound assignment operator
$counter++;                // increment operator

echo $counter;
echo "<br>";

$counter--;
echo $counter;
echo "<br>";

```

All four are valid. The ++ and -- operators are the most concise.

Operator Precedence

When multiple operators appear in a single expression, PHP evaluates them in this order:

1. () — parentheses first
2. ** — exponentiation
3. *, /, % — multiplication, division, modulo
4. +, - — addition and subtraction

Use parentheses to explicitly control evaluation order when needed.

GET and POST in PHP

Superglobal Arrays

PHP provides two built-in superglobal arrays for reading data sent via HTTP requests: \$_GET and \$_POST. These are available anywhere in your script without needing to be declared.

GET Requests

With the GET method, data is appended to the URL as query parameters:

`http://example.com/page.php?id=123`

You read that value in PHP with `$_GET['id']`. GET has the following characteristics:

- Data is visible in the URL — not suitable for sensitive information
- Subject to URL length limits
- Responses can be cached by browsers and proxies
- Best suited for search pages or any request that retrieves data without side effects

POST Requests

With the POST method, data is sent inside the HTTP request body and is not visible in the URL:

```
curl -X POST -d "username=soroush" https://example.com/login.php
```

You read that value in PHP with `$_POST['username']`. POST has the following characteristics:

- Data is not exposed in the URL — more appropriate for sensitive information
- No practical data size limit
- Responses are not cached
- Best suited for submitting credentials or any request that modifies data

The HTML Form

The form below sends a POST request to the same file when submitted. The `action` attribute specifies the destination and `method` specifies how the data is sent. Each input element's `name` attribute becomes the key used to access its value via `$_POST`:

```
<html>
<head>
</head>
<body>
<form action="/GETandPOST.php" method="post">
  <label>username</label><br>
  <input type="text" name="username" /><br>

  <label>password</label><br>
  <input type="password" name="password" /><br>

  <input type="submit" value="Log In" /><br>
</form>
</body>
</html>
```

Reading Request Data

The PHP block below reads from both `$_GET` and `$_POST`. The hidden parameter is read from the URL query string — you can test it by appending `?hidden=somevalue` to the URL manually.

String concatenation in PHP is done with a dot (`.`), as shown in the `$_POST['username']` line. You can also store superglobal values into local variables for reuse.

```
<?php
echo "{$_GET['hidden']}" <br>;
```

Note what happens if you discover a hidden GET parameter through fuzzing and inject a value like `<script>alert(123)</script>` — the browser will execute it. This is a reflected **Cross-Site Scripting (XSS)** vulnerability, one of the most common issues in PHP applications that do not sanitize user input before outputting it.

```
echo $_POST['username'] . "<br>";

$password = "{$_POST['password']}" <br>;
```

```
echo $password;  
?>
```

Math Functions in PHP

Unlike Python and Go, PHP includes a wide set of math functions in its standard library — no additional package imports are needed.

Rounding and Absolute Value

```
<?php  
$x = 16.3;  
$total = null;  
  
$total = abs($x);  
echo "${total} <br>";  
  
$total = round($x);  
echo "${total} <br>";  
  
$total = floor($x);  
echo "${total} <br>";  
  
$total = ceil($x);  
echo "${total} <br>";
```

- `abs` returns the absolute value — the non-negative version of a number
- `round` rounds to the nearest integer
- `floor` rounds down to the nearest integer
- `ceil` rounds up to the nearest integer

Power and Square Root

```
$y = 2;  
  
$total = pow($x, $y);  
echo "${total} <br>";  
  
$total = sqrt($x);  
echo "${total} <br>";
```

- `pow($base, $exp)` raises `$base` to the power of `$exp` — equivalent to the `**` operator
- `sqrt` returns the square root of a number

Min, Max, and Pi

```
$z = 15;
```



```

$total = max($x, $y, $z);
echo "{$total} <br>";

$total = min($x, $y, $z);
echo "{$total} <br>";

$total = pi();
echo "{$total} <br>";

```

Unlike Go's `math.Min` and `math.Max`, PHP's `min` and `max` accept any number of arguments — not just two.

`pi()` returns the value of `π` with no arguments required.

Random Numbers

```

$total = rand(1, 6);
echo "{$total} <br>";
?>

```

`rand($min, $max)` returns a random integer between the given minimum and maximum values, inclusive. The example above simulates a dice roll.

If Statements in PHP

Basic If / Elseif / Else

An `if` statement evaluates a condition and executes the corresponding block if it is true. PHP uses `elseif` (one word) for additional conditions, followed by an optional `else` block as a fallback:

```

<?php
$page = -1;

if ($age >= 18){
    echo "you can enter this site.";
}
elseif($age <= 0){
    echo "thats not a valid age.";
}
else{
    echo "you must be 18+ to enter this site.";
}
echo "<br>";

```

Note that the order of conditions matters. PHP evaluates them top to bottom and executes the first block whose condition is true. In this example, `elseif($age <= 0)` is checked only if the first condition fails.

Boolean Conditions

If statements also work directly with boolean variables. A variable holding `true` satisfies the condition on its own — no comparison operator is needed:

```
$adult = true;
if ($adult){
    echo "you're an adult.";
}else{
    echo "you're not an adult.";
}
?>
```

Logical Operators in PHP

Save the following code into a file called `logicalOperators.php` and run it with `php logicalOperators.php`, or serve it with `php -S localhost:8000` and open it in a browser.

Logical operators are used to chain multiple conditions together inside an `if` statement:

Operator	Meaning
<code>&&</code>	AND — true only if both conditions are true
<code>\ \ </code>	OR — true if at least one condition is true
<code>!</code>	NOT — inverts a boolean value

AND

The block executes only if all chained conditions are true:

```
<?php
$temp = 25;

if ($temp >= 0 && $temp <= 30){
    echo "the weather is good.";
}else{
    echo "the weather is bad.";
}
echo "<br>";
```

OR

The block executes if at least one of the conditions is true:

```
if ($temp < 0 || $temp > 30){
    echo "weather is not good.";
}
else {
    echo "weather is not bad.";
}
```

NOT

The `!` operator inverts a boolean value. Here, the block only executes if `$cloudy` is `false`:

```
$cloudy = true;
if (!$cloudy){
    echo "its not cloudy";
}
?>
```

Switch Statements in PHP

A switch statement is a cleaner alternative to a long chain of `if/elseif/else` blocks when you are comparing a single variable against a set of known values. Unlike Go, PHP does **not** break out of a case automatically — you must write `break` explicitly at the end of each case, otherwise execution will fall through into the next one.

Basic Switch

```
<?php
$grade = "B";

switch($grade){
    case "A":
        echo "Great!";
        break;
    case "B":
        echo "Good";
        break;
    case "C":
        echo "Acceptable";
        break;
    case "D":
        echo "Not good";
        break;
    case "F":
        echo "Bad";
        break;
    default:
        echo "${grade} is not a valid Grade";
}
```

The `default` case runs if none of the defined cases match — equivalent to the final `else` in an `if/elseif/else` chain.

Using the `date` Function

PHP's built-in `date` function returns the current date or time formatted according to the string you pass it. The format character `"l"` (lowercase L) returns the full name of the current day of the week:

```

$date = date("l");
switch ($date){
    case "Monday":
        echo "i dont like mondays";
        break;
    case "Tuesday":
        echo "it is tuesday";
        break;
    case "Wednesday":
        echo "its a good day";
        break;
    case "Thursday":
        echo "i like thursdays";
        break;
    case "Friday":
        echo "i love fridays";
        break;
    case "Saturday":
        echo "SAT";
        break;
    case "Sunday":
        echo "SUN";
        break;
    default:
        echo "NOT a valid day.";
}
?>

```

For Loops in PHP

A `for` loop executes a block of code a fixed number of times. The loop header has three parts: an initializer, a condition, and an increment — separated by semicolons:

```

<?php
for ($i = 0; $i < 5; $i++){
    echo "HELLO <br>";
}

```

- `$i = 0` — sets the starting value
- `$i < 5` — the loop runs as long as this condition is true
- `$i++` — increments the counter after each iteration

Iterating Over a String

Individual characters in a string can be accessed by index using bracket notation, the same way you would access an element in an array. PHP's `strlen` function returns the length of a string, which makes it useful as the loop's upper bound:

```

$str = "soroush";
for ($i = 0; $i < strlen($str); $i++){

```

```
    echo $str[$i] . "<br>";  
}  
?>
```

This prints each character of the string on a separate line.

While Loops in PHP

A while loop executes a block of code repeatedly for as long as a given condition remains true. Unlike a for loop, where the iteration logic is contained in the loop header, a while loop is more appropriate when the number of iterations is not known in advance.

```
<?php  
$counter = 0;  
while($counter <= 20){  
    echo $counter . "<br>";  
    $counter++;  
}  
?>
```

The counter is declared before the loop, the condition is checked at the start of each iteration, and the increment is handled manually inside the loop body. If you forget to increment the counter, the condition never becomes false and the loop runs indefinitely.

Arrays in PHP

An array stores multiple values in a single variable. In PHP, arrays are declared using the `array()` constructor. You cannot print an array directly with `echo` — you must access its elements by index, starting from 0:

```
<?php  
$foods = array("apple", "orange", "banana", "pasta");  
  
echo $foods[0] . "<br>";  
echo $foods[1] . "<br>";
```

Modifying Elements

You can overwrite an element by assigning a new value to its index:

```
$foods[1] = "pizza";
```

Array Functions

PHP provides several built-in functions for manipulating arrays:

```
array_push($foods, "tunaFish", "pineapple");
```

`array_push` appends one or more values to the end of an array.

```
array_pop($foods);
```

`unset` removes a specific element from array.

```
unset($foods[2]);
```

Note: after removing an element, values will not be indexed automatically and if you want to re-index your array:

`array_values` returns the elements of an array into a new array (also used to re-index elements)

```
$foods = array_values($foods)
```

`array_pop` removes the last element.

```
array_shift($foods);
```

`array_shift` removes the first element and re-indexes the array.

```
$reverseArray = array_reverse($foods);
```

`array_reverse` does not modify the original array — it returns a new reversed array that you assign to a variable.

Iterating with `foreach`

The `foreach` loop is designed specifically for iterating over arrays. The `as` keyword assigns each element to a temporary variable for use inside the loop body:

```
foreach($foods as $food){
    echo $food . "<br>";
}
foreach($reverseArray as $food){
    echo $food . "<br>";
}
```

Counting Elements

```
echo count($foods);
?>
```

`count` returns the number of elements currently in the array.

Associative Arrays in PHP

An associative array stores key-value pairs instead of numerically indexed elements. Each key is mapped to a value using the `=>` operator. This can be equivalent to the `Map` data type in JavaScript:

```
<?php
$capitals = array(
    "USA" => "Washington D.C",
    "Japan" => "Kyoto",
    "India" => "New Dehli"
);
```

Elements are accessed by their key instead of a numeric index:

```
echo $capitals["USA"];
```

Iterating with foreach

When iterating over an associative array, the `foreach` loop exposes both the key and value using the `as $key => $value` syntax:

```
foreach ($capitals as $key => $value){  
    echo "{$key} = {$value} <br>";  
}
```

Modifying and Adding Entries

To update an existing entry, assign a new value to its key. To add a new key-value pair, simply assign to a key that does not yet exist:

```
$capitals["India"] = "Mumbai";  
  
$capitals["China"] = "Beijing";  
echo $capitals["China"];
```

Removing Elements

```
array_pop($capitals);
```

`array_pop` removes the last key-value pair.

```
array_shift($capitals);
```

`array_shift` removes the first key-value pair.

```
unset($capitals["China"])
```

`unset` removes a specified key-value pair.

Extracting Keys and Values

`array_keys` returns a new indexed array containing only the keys:

```
$arrayKeys = array_keys($capitals);  
foreach ($arrayKeys as $arrKey){  
    echo "{$arrKey} <br>";  
}
```

`array_values` returns a new indexed array containing only the values:

```
$arrayValues = array_values($capitals);  
foreach ($arrayValues as $arrVal){  
    echo "{$arrVal} <br>";  
}
```

Flipping, Reversing, and Counting

`array_flip` swaps keys and values — each value becomes a key and each key becomes a value:

```
$capitals = array_flip($capitals);
```

`array_reverse` returns a new array with the order of elements reversed:

```
$capitals = array_reverse($capitals);  
foreach ($capitals as $key => $value){  
    echo "{$key} {$value} <br>";  
}  
?>
```

To count the number of key-value pairs in an associative array, use `count($capitals)`, the same function used for indexed arrays.

isset and empty in PHP

PHP provides two functions for checking the state of a variable before using it:

- `isset()` returns `true` if a variable has been declared and its value is not `null`
- `empty()` returns `true` if a variable is not declared, or its value is `false`, `null`, or an empty string `""`

These are particularly useful when handling user input, where a variable may or may not be present in the request.

Basic Usage

```
<?php  
$username = "VoidSeraphim";  
echo isset($username) . "<br>";  
  
$username = null;  
echo empty($username) . "<br>";  
  
if(isset($username)){  
    echo "this variable is set <br>";  
}  
else{  
    echo "this variable is not set <br>";  
}
```

After setting `$username` to `null`, `isset` returns `false` and `empty` returns `true`.

Checking GET Parameters Before Use

A common and important use case is checking whether a GET parameter exists before reading it. Accessing `$_GET['hidden']` without checking first will produce a warning if the parameter is absent:


```

if(isset($_GET["hidden"])){
    echo "{$_GET['hidden']} <br>";
    echo "<script>alert('secret...')</script>";
}
?>

```

Note that the value from `$_GET['hidden']` is echoed directly into the page without any sanitization. If an attacker supplies a JavaScript payload as the parameter value — for example, `?hidden=<script>alert(1)</script>` — the browser will execute it. The `<script>alert('secret...')</script>` line demonstrates this: using `isset` to guard a GET parameter does not protect against **reflected XSS**. Input must be sanitized before being output to the page.

Radio Buttons in PHP

The HTML Form

Radio buttons are used when the user must select exactly one option from a group. All radio buttons in the same group share the same `name` attribute — this ensures only one can be selected at a time. The `value` attribute defines what gets sent to the server when that option is chosen:

```

<html>
<head>
</head>
<body>
<form action="./radioButtons.php" method="post">
    <input type="radio" name="payment" value="visa" />
    visa<br>

    <input type="radio" name="payment" value="mastercard" />
    master<br>

    <input type="radio" name="payment" value="express" />
    american express<br>

    <input type="submit" name="confirm" value="confirm" />
</form>
</body>
</html>

```

Handling the Submission

The PHP block runs only after the form is submitted. The outer `isset($_POST['confirm'])` check ensures the code only executes when the submit button has been clicked, not on the initial page load.

The inner check handles the case where the user clicks submit without selecting any option — radio buttons are not submitted at all if none is selected, so `$_POST['payment']` will be absent from the request entirely:

```

<?php
if (isset($_POST['confirm'])){

    if (isset($_POST['payment'])){
        $payment = $_POST['payment'];
    }
    else {
        $payment = "not set.";
    }

    echo $payment;
}
?>

```

Checkboxes in PHP

The HTML Form

Unlike radio buttons, checkboxes allow the user to select multiple options at once. To send multiple selected values under a single key, the `name` attribute uses array notation: `foods[]`. PHP will collect all checked values into an array automatically:

```

<html>
<head>
</head>
<body>
<form action="checkboxes.php" method="post">
    <input type="checkbox" name="foods[]" value="pizza">pizza <br>
    <input type="checkbox" name="foods[]" value="hamburger">hamburger <br>
    <input type="checkbox" name="foods[]" value="hotdog">hotdog <br>
    <input type="submit" name="submit">
</form>
</body>
</html>

```

Handling the Submission

`$_GET` and `$_POST` are associative arrays. When checkboxes share a `name` with array notation, PHP groups their values into a nested array under that key — in this case, `$_POST['foods']`.

The first block attempts to check each food item as a separate key in `$_POST`. This approach only works if each checkbox has a unique `name` attribute rather than the shared `foods[]` notation used in the form above, so none of these conditions will match as written. It is included here to illustrate why the array approach shown below is the correct method:

```

<?php
if(isset($_POST['submit'])){
    if(isset($_POST['pizza'])){
        echo "you like pizza <br>";
    }
}

```

```

}
elseif(isset($_POST['hamburger'])){
    echo "you like hamburger <br>";
}
elseif(isset($_POST['hotdog'])){
    echo "you like hotdog <br>";
}

```

The correct approach is to read `$_POST['foods']` directly as an array and iterate over it with `foreach`. This handles any number of selections without needing a separate condition for each option:

```

$foods = $_POST['foods'];
foreach($foods as $food){
    echo "food: {$food}***** <br>";
}
}
?>

```

Functions in PHP

Functions are reusable blocks of code that you can call whenever needed, instead of repeating the same logic multiple times.

Defining and Calling a Function

To define a function in PHP, use the `function` keyword followed by the function name and a pair of parentheses. Parameters go inside the parentheses. The function body sits inside curly braces.

```

<?php
function happy_birthday($firstname, $age){
    echo "happy birthday dear ${firstname} <br>";
    echo "you are ${age} years old! <br>";
    echo "***** <br>";
}

happy_birthday("sora", 24);
happy_birthday("dora", 26);

```

Here, `happy_birthday` takes two parameters — `$firstname` and `$age` — and prints a short message using both. Calling it twice with different arguments demonstrates how a single function definition can be reused without duplicating code.

Return Values and Type Declarations

Functions usually return a value to the caller. In PHP, you can enforce what type of data a parameter must be by adding a type declaration before the variable name. The supported scalar types are:

Declaration	Type
<code>int</code>	Integer
<code>float</code>	Floating point
<code>string</code>	String
<code>bool</code>	Boolean
<code>array</code>	Array

Objects follow a different pattern — they are instantiated from classes, which are covered separately.

The function below uses an `int` type declaration on its parameter. If you pass a value that cannot be coerced to an integer, PHP will throw a `TypeError` at runtime.

```
function is_even(int $number){
    $res = $number % 2;
    if ($res){
        return "odd";
    }
    else{
        return "even";
    }
}

echo is_even(10);
?>
```

The modulo operator `%` divides `$number` by 2 and returns the remainder. A remainder of 1 is truthy, so the function returns "odd". A remainder of 0 is falsy, so it returns "even". Calling `is_even(10)` will print even.

String Functions in PHP

PHP has a large set of built-in functions for manipulating strings. This section covers the most commonly used ones.

The examples below use two variables:

```
<?php
$username = "Void Seraphim";
$phone = "123-456-7890";
```

Case Conversion and Whitespace

Two functions handle case conversion — `strtolower()` converts every character to lower-case, and `strtoupper()` converts every character to uppercase.

```
$username = strtolower($username);
$username = strtoupper($username);
```

`trim()` removes any leading or trailing whitespace from a string. This is useful when handling user input that may contain accidental spaces:

```
$username = trim($username);
```

`str_pad()` extends a string to a specified length by appending a padding character. Here it would pad `$username` to 20 characters using `0`:

```
$username = str_pad($username, 20, 0);
```

Replacing Characters

`str_replace()` scans the string for every occurrence of the first argument and replaces it with the second. Here, every `-` in the phone number is replaced with `/`:

```
$phone = str_replace("-", "/", $phone);
```

Reversing and Shuffling

`strrev()` returns the string in reverse order. `str_shuffle()` randomly rearranges all characters in the string:

```
$username = strrev($username);  
$username = str_shuffle($username);
```

Comparing, Measuring, and Searching

`strcmp()` compares two strings and returns `0` if they are identical, or a non-zero value if they differ.

`strlen()` returns the number of characters in a string.

`strpos()` returns the index of the first occurrence of a given character or substring:

```
$equals = strcmp($username, $phone);  
$count = strlen($username);  
$index = strpos($username, " ");
```

Extracting Substrings

`substr()` extracts a portion of a string. It takes a start index and an optional end index. If the end index is omitted, it returns everything from the start index to the end of the string:

```
$firstname = substr($username, 0, 4);  
$lastname = substr($username, 5);
```

Splitting and Joining

`explode()` splits a string into an array using a given separator. `implode()` does the opposite — it joins array elements into a single string using a given separator:

```
$fullname = explode(" ", $username);  
$joined = implode("-", $fullname);
```

Output

```
echo $username . "<br>";
echo $phone . "<br>";
echo $equals . "<br>";
echo $count . "<br>";
echo $index . "<br>";
echo $firstname . "<br>";
echo $lastname . "<br>";
echo $fullname[1] . "<br>";
echo $joined . "<br>";
?>
```

Sanitizing and Validating User Input in PHP

Accepting raw user input directly into your application is dangerous. PHP provides the `filter_input()` and `filter_var()` functions to sanitize and validate input before using it.

The HTML Form

The form below collects a username, age, and email address, then submits them via POST to the same file:

```
<html>
<body>
<form action="sanitize-validate-input.php" method="post">
username:<br>
<input type="text" name="username"><br>
age:<br>
<input type="text" name="age"><br>
email:<br>
<input type="text" name="email"><br>
<input type="submit" name="login" value="login">
</form>
</body>
</html>
```

Sanitization vs. Validation

Before looking at the code, it is important to understand the difference between these two concepts:

- **Sanitization** does not check whether the data is valid. It strips or transforms unwanted characters and returns the cleaned result.
- **Validation** checks whether the data matches an expected format. It returns the value itself if valid, or `false` if not.

In practice, sanitization happens first, and validation follows on the cleaned value.

Reading Input with filter_input()

Instead of reading raw POST data directly with `$_POST["username"]`, you should pass it through `filter_input()`. The following commented-out line shows the unsafe approach kept here for comparison:

```
<?php
// $username = $_POST["username"];
```

`filter_input()` takes three arguments:

- 1. The input source — where to read the value from. Common options are `INPUT_GET`, `INPUT_POST`, `INPUT_COOKIE`, `INPUT_SERVER`, and `INPUT_ENV`.
- 2. The name of the parameter to retrieve.
- 3. A filter constant — either a sanitization filter or a validation filter.

The available sanitization filters are:

Filter	Effect
<code>FILTER_SANITIZE_EMAIL</code>	Removes characters invalid in an email
<code>FILTER_SANITIZE_URL</code>	Removes characters invalid in a URL
<code>FILTER_SANITIZE_SPECIAL_CHARS</code>	HTML-encodes special characters
<code>FILTER_SANITIZE_NUMBER_INT</code>	Strips everything except digits and +-.
<code>FILTER_SANITIZE_NUMBER_FLOAT</code>	Strips everything except digits and .+-.

The available validation filters are:

Filter	Validates
<code>FILTER_VALIDATE_INT</code>	Integer
<code>FILTER_VALIDATE_FLOAT</code>	Float
<code>FILTER_VALIDATE_EMAIL</code>	Email address
<code>FILTER_VALIDATE_URL</code>	URL
<code>FILTER_VALIDATE_IP</code>	IP address
<code>FILTER_VALIDATE_REGEXP</code>	Regular expression
<code>FILTER_VALIDATE_BOOLEAN</code>	Boolean

Sanitizing the Submitted Values

When the form is submitted, the `login` button is included in the POST data. `isset($_POST["login"])` checks for its presence to confirm the form was actually submitted before processing anything:

```
if(isset($_POST["login"])){
    $username = filter_input(INPUT_POST, "username", FILTER_SANITIZE_SPECIAL_CHARS);
    $age = filter_input(INPUT_POST, "age", FILTER_SANITIZE_NUMBER_INT);
    $mail = filter_input(INPUT_POST, "email", FILTER_SANITIZE_EMAIL);
```

`FILTER_SANITIZE_SPECIAL_CHARS` encodes characters like `<`, `>`, and `&` in the username, which prevents basic HTML injection. `FILTER_SANITIZE_NUMBER_INT` strips anything from the age field that is not a digit or a sign character. `FILTER_SANITIZE_EMAIL` removes any characters that are not permitted in an email address.

Validating the Sanitized Email

After sanitizing the email, `filter_var()` is used to validate it. This function works the same way as `filter_input()` but operates on an existing variable rather than reading directly from an input source:

```
$validmail = null;
if (filter_var($mail, FILTER_VALIDATE_EMAIL)){
    $validmail = $mail;
}
else{
    $validmail = "not a valid email.";
}
```

If the sanitized value passes the `FILTER_VALIDATE_EMAIL` check, it is accepted. Otherwise, the variable is set to an error message.

```
echo "HELLO {$username} <br>";
echo "you are {$age} years old <br>";
echo "your email address is {$mail}. <br>";
echo $validmail;
}
?>
```

Including Files in PHP

PHP's `include()` function copies the content of another file — HTML, PHP, or plain text — and inserts it inline at the point where `include()` is called. This makes sections of a website reusable across multiple pages, so changes only need to be made in one place. It works similarly to imports in JavaScript or Python.

This example splits a small multi-page site into shared header and footer files, which are then included in each page.

header.html

The header contains the site title and navigation links. It is included at the top of every page:

```
<header>
    <h2>this is my website</h2>
    <a href="index.php">HOME</a>
    <a href="about.php">ABOUT</a>
    <a href="locations.php">LOCATIONS</a>
    <hr>
</header>
```

footer.html

The footer contains author information and a contact link. It is included at the bottom of every page:


```

<footer>
  <hr>
  Author: sora
  <br>
  <a href="mailto:sora@mail.com">send an email</a>
</footer>

```

index.php

```

<?php include("header.html"); ?>
<html>
  <body>
    <h1>this is the home page</h1>
    <p>
      Lorem ipsum dolor sit amet consectetur adipisicing elit.
    </p>
    <h3>more html</h3>
  </body>
</html>
<?php include("footer.html"); ?>

```

about.php

```

<?php include("header.html"); ?>
<html>
  <body>
    <h1>this is the about page</h1>
    <p>
      Lorem ipsum dolor sit amet consectetur adipisicing elit.
    </p>
    <p>
      Lorem ipsum dolor sit amet consectetur adipisicing elit.
    </p>
    <h3>more html</h3>
  </body>
</html>
<?php include("footer.html"); ?>

```

locations.php

```

<?php include("header.html"); ?>
<html>
  <body>
    <h1>this is the locations page</h1>
    <p>
      Lorem ipsum dolor sit amet consectetur adipisicing elit.
    </p>
    <p>

```

```

        Lorem ipsum dolor sit amet consectetur adipisicing elit.
    </p>
    <p>
        Lorem ipsum dolor sit amet consectetur adipisicing elit.
    </p>
    <h3>more html</h3>
</body>
</html>
<?php include("footer.html"); ?>

```

Save all five files in the same directory and serve them with PHP's built-in server:

```
php -S localhost:8080
```

Then open <http://localhost:8080/index.php> in your browser and use the navigation links to move between pages.

The `$_SERVER` Superglobal in PHP

`$_SERVER` is a PHP superglobal array that holds information about the server environment and the current HTTP request — including headers, paths, and script locations. Because some of these values come directly from the client, they can be spoofed. Fields such as `User-Agent`, `Referer`, and custom headers are set by the browser or the HTTP client, so they should never be trusted for security decisions without validation.

Iterating Over `$_SERVER`

To inspect all available keys and their values, iterate over the array:

```

<?php
foreach ($_SERVER as $key => $val) {
    echo "{$key} = {$val} <br>";
}
?>

```

This is useful during reconnaissance or debugging to see exactly what the server exposes about the current request environment.

Useful Keys

`$_SERVER["PHP_SELF"]` holds the path of the currently executing script as it appears in the URL. It is commonly used in HTML forms to submit back to the same page, instead of hardcoding a path in the `action` attribute. However, this value is user-influenced and is vulnerable to reflected XSS if output directly into HTML. It should always be sanitised with `htmlspecialchars()` before use:

```
<form action="<?php echo htmlspecialchars($_SERVER['PHP_SELF']); ?>" method="post">
```

Without `htmlspecialchars()`, an attacker can craft a URL that injects a script tag directly into the page through the `PHP_SELF` value.

`$_SERVER["REQUEST_METHOD"]` holds the HTTP verb used for the current request — GET, POST, PUT, DELETE, and so on. It is commonly used to branch logic depending on how a form or endpoint was accessed.

Cookies in PHP

A cookie is a small piece of data stored in the user's browser. The browser sends it back to the server with every request, which allows the server to recognize returning users and persist simple state between sessions.

Setting Cookies

To create a cookie in PHP, use `setcookie()`. It takes four arguments: the **key**, the **value**, the **expiration timestamp**, and the **path**. The path `"/"` makes the cookie available across the entire site:

```
<?php
setcookie("favoriteFood", "pizza", time() + 5000, "/");
setcookie("favoriteDrink", "coffee", time() + (86400 * 3), "/");
setcookie("favoriteDessert", "ice cream", time() + (86400 * 2), "/");
```

`time()` returns the current Unix timestamp in seconds. Adding `86400` to it sets the expiration one day into the future, since there are 86,400 seconds in a day.

To delete a cookie, set its expiration to a time in the past:

```
setcookie("favoriteFood", "", time() - 1, "/");
```

This tells the browser the cookie has already expired, so it removes it immediately.

Reading Cookies

All cookies sent by the browser are available in the `$_COOKIE` superglobal, which is an associative array. You can loop over it to print every key-value pair:

```
foreach ($_COOKIE as $key => $value){
    echo "{$key} = {$value} <br>";
}
```

To access or overwrite a specific cookie value by key:

```
$_COOKIE["favoriteDessert"] = "jelly";
echo $_COOKIE["favoriteDessert"];
```

Note that assigning a value directly to `$_COOKIE` only changes it for the current request. To actually update the cookie in the browser, you need to call `setcookie()` again with the new value.

Setting Other Headers

The `header()` function lets you send raw HTTP headers to the browser. A common use case is redirecting the user to another page:

```
header("Location: home.php");  
?>
```

`header()` must be called before any output is sent to the browser — including HTML, whitespace, or `echo` statements. Calling it after output has already started will produce a warning and the header will not be sent.

Sessions in PHP

A session is a server-side mechanism for storing information that persists across multiple pages. Unlike cookies, the data is kept on the server — the browser only receives a session ID (SID), which it sends back with every request so the server can match it to the stored data.

A practical example of this is logging in to a website: once authenticated, you can navigate to different pages and remain logged in because the server recognises your session ID on each request.

index.php — Starting a Session

`session_start()` must be called before any output is sent to the browser. It either creates a new session and generates a fresh SID, or resumes an existing one if a valid SID is found in the request cookie.

Once the session is started, you can store values in the `$_SESSION` superglobal as key-value pairs. These are saved on the server and tied to the current SID:

```
<?php  
session_start();  
?>  
<html>  
this is the login page <br>  
<a href="home.php">this goes to home page.</a>  
</html>  
<?php  
$_SESSION["username"] = "sora";  
$_SESSION["password"] = "1234";  
echo $_SESSION["username"] . "<br>";  
echo $_SESSION["password"] . "<br>";  
?>
```

home.php — Resuming a Session

On any subsequent page, calling `session_start()` again reads the SID from the browser's cookie, matches it to the stored session on the server, and loads the previously saved data. There is no need to redefine the values — they are already available through `$_SESSION`:

```
<?php  
session_start();  
?>  
<html>
```

```

this is the HOME page <br>
<a href="index.php">this goes to login page.</a>
</html>
<?php
foreach($_SESSION as $key => $val){
    echo "${key} = ${val} <br>";
}
?>

```

To log a user out and destroy all session data stored on the server, call `session_destroy()`:

```
session_destroy();
```

This invalidates the session on the server side. It is good practice to also clear the `$_SESSION` array and expire the session cookie when implementing a proper logout flow.

Password Hashing in PHP

Hashing is the process of transforming data into a fixed-length digest using a one-way algorithm. Unlike encryption, hashing cannot be reversed — you cannot recover the original input from its hash. This is what makes it suitable for storing passwords: even if the database is compromised, the plaintext passwords are not exposed.

Most systems store passwords as hashes rather than plaintext. When a user logs in, the submitted password is hashed using the same algorithm and the result is compared to the stored hash.

Hashing a Password

PHP's `password_hash()` function takes the plaintext password and a hashing algorithm constant. `PASSWORD_DEFAULT` uses the bcrypt algorithm, which has been the default since PHP 5.5.0. bcrypt is intentionally slow, which makes brute-force attacks significantly more expensive:

```

<?php
$password = "safeP@ssw0rd";
$hash = password_hash($password, PASSWORD_DEFAULT);
echo $hash . "<br>";

```

Each call to `password_hash()` produces a different output even for the same input, because bcrypt automatically generates and embeds a random salt into the hash. The stored hash contains the algorithm identifier, the cost factor, the salt, and the digest — all in a single string.

Verifying a Password

To check a submitted password against a stored hash, use `password_verify()`. It extracts the salt and algorithm from the stored hash, re-hashes the input, and compares the results. You should never compare hashes manually with `==` or `===`, as that is vulnerable to timing attacks:

```

if(password_verify($_GET["password"], $hash)){
    echo "Welcome! Admin.";
}
else{
    echo "Hello! visitor.";
}
?>

```

Note that this example reads the password from `$_GET`, which passes it as a visible URL parameter. This is used here for demonstration purposes only. In a real application, passwords must always be submitted via POST over HTTPS so they are not exposed in browser history, server logs, or network traffic.

Error Handling and Database Connection in PHP

To handle runtime errors in PHP, you wrap potentially failing code in a `try/catch` block. If an exception is thrown inside the `try` block, execution jumps immediately to the `catch` block, where you can handle the error gracefully instead of letting the script crash.

Connecting to a Database

PHP connects to MySQL using the `mysqli_connect()` function. The standard practice is to keep the connection logic in a dedicated file such as `database.php`, which can then be included in any page that needs to interact with the database.

Start by defining the connection parameters:

```

<?php
$db_server = "localhost";
$db_user = "root";
$db_pass = "";
$db_name = "businessdb";
$conn = "";

```

Establishing the Connection

`mysqli_connect()` takes the server address, username, password, and database name. If the connection succeeds, it returns a connection object, which is truthy. If it fails, it returns a falsy value and may throw an error:

```

try{
    $conn = mysqli_connect($db_server, $db_user, $db_pass, $db_name);

```

You can also throw custom exceptions manually from inside a `try` block when you need to enforce your own validation logic. For example:

```

    // throw new Exception("This is a custom exception");
}

```

This line is commented out and shown here as a reference for how to raise a custom exception. It is not part of the connection logic.

Catching Errors

The `catch` block receives the error object as `$e`. Catching `Error` covers fatal PHP errors, while catching `Exception` covers manually thrown exceptions. For a production application you would typically catch both or use a common base class:

```
catch(Error $e){  
    echo "Could not connect to the database <br>";  
    echo "Error: ${e} <br>";  
}
```

Checking the Connection

After the `try/catch` block, check whether `$conn` holds a valid connection before attempting any queries:

```
if($conn){  
    echo "You are connected to database! <br>";  
}  
?>
```

You can save this as `database.php` and include it in any other PHP file that needs database access:

```
<?php include("database.php"); ?>
```

Inserting Data into a Database in PHP

This page demonstrates how to insert a new user record into a MySQL database. It builds on the database connection covered in the previous section.

Including the Database Connection

Rather than rewriting the connection logic, include the database file created earlier:

```
<?php  
include("database.php");
```

Preparing the Data

Before inserting anything, the password is hashed using `password_hash()` with the `bcrypt` algorithm. Plaintext passwords must never be stored in a database:

```
$username = "sora";  
$password = "1234";  
$hash = password_hash($password, PASSWORD_DEFAULT);
```

Writing and Submitting the Query

The SQL query is built as a string first, then passed to `mysqli_query()` along with the active connection object. `mysqli_query()` executes the query against the database:

```

$sql = "INSERT INTO users (user, password) VALUES ('${username}', '${hash}')";

try{
    mysqli_query($conn, $sql);
    echo "registration succeed.";
}
catch (Error $e){
    echo "registration failed.";
}

```

Note that this example inserts variables directly into the SQL string. This approach is vulnerable to SQL injection if the values ever come from user input. For any real application, use prepared statements with bound parameters instead of string interpolation.

Closing the Connection

Always close the database connection at the end of the script to free up server resources:

```

mysqli_close($conn);
?>

```

Retrieving Data from a Database in PHP

This page demonstrates how to query the database for a specific user and read the returned row.

Including the Database Connection

```

<?php
include("database.php");

```

Preparing the Query

The password is hashed before being used in the query, since the database stores hashes rather than plaintext:

```

$username = "sora";
$password = "1234";
$passHashed = password_hash($password, PASSWORD_DEFAULT);

```

Note that hashing the password here and comparing it to the stored hash in the **WHERE** clause will not work as intended, because `bcrypt` generates a different hash on every call due to its random salt. The correct approach is to fetch the row by username only, then use `password_verify()` to compare the submitted password against the stored hash. This example is kept as written to demonstrate the query and retrieval mechanics, but keep this in mind for any real login implementation.

Writing and Submitting the Query

`mysqli_query()` executes the SQL query against the active connection and returns a result object on success:

```
$sql = "SELECT * FROM users WHERE user = '${username}' AND password = '${passHashed}'";  
$result = mysqli_query($conn, $sql);
```

As with the insert example, building queries through string interpolation is vulnerable to SQL injection when values come from user input. Use prepared statements with bound parameters in any real application.

Reading the Result

`mysqli_num_rows()` takes the result object and returns the number of matching rows. If at least one row was found, `mysqli_fetch_assoc()` retrieves the next row as an associative array, where the keys are the column names:

```
if(mysqli_num_rows($result) > 0){  
    $row = mysqli_fetch_assoc($result);  
    echo $row["id"];  
    echo $row["user"];  
    echo $row["registration_date"];  
}  
else{  
    echo "no user found";  
}
```

To iterate over all matching rows rather than just the first one, pass `mysqli_fetch_assoc()` as the condition of a `while` loop:

```
// while($row = mysqli_fetch_assoc($result)){  
//     echo $row["user"] . "<br>";  
// }
```

Each call to `mysqli_fetch_assoc()` advances an internal pointer to the next row and returns it as an array. When there are no more rows, it returns `false`, which ends the loop.

Closing the Connection

```
mysqli_close($conn);  
?>
```

Object-Oriented Programming in PHP

PHP supports object-oriented programming (OOP). This section covers classes, objects, visibility modifiers, constructors, inheritance, static properties, and magic methods.

Defining a Class

Use the `class` keyword to define a class. Properties and methods can be given a visibility modifier:

- `public` — accessible from anywhere
- `private` — accessible only within the class itself
- `protected` — accessible within the class and any child classes

```
<?php
class User
{
    public $username;

    public function greet(){
        echo "Hello im $this->username <br>";
    }

    public function __construct($username)
    {
        $this->username = $username;
    }
}
```

To access properties and methods inside a class, use `$this->` followed by the property or method name. PHP uses the arrow notation `->` for this purpose rather than the dot notation used in some other languages.

The `__construct()` method is the constructor. It runs automatically when a new object is created and is typically used to initialise properties.

Creating Objects

Use the `new` keyword to instantiate an object from a class. Properties and methods are accessed on the object using `->`:

```
$user = new User("sora");

echo $user->username . "<br>";
$user->greet() . "<br>";
```

Inheritance

A class can extend another class using the `extends` keyword. The child class inherits all public and protected properties and methods from the parent. If the child class does not define its own constructor, PHP automatically uses the parent's:

```
class Admin extends User
{
    public $role = "admin";

    public function showRole()
    {
```

```

        echo "Role: $this->role <br>";
    }
}

$admin = new Admin("dora");

echo $admin->username . "<br>";
$admin->greet() . "<br>";
$admin->showRole() . "<br>";

```

Static Properties and Methods

A static property belongs to the class itself rather than to any individual object. It is shared across all instances. Inside the class, static properties are accessed using `self::` instead of `$this->`. Use `self::` to refer to the current class and `parent::` to refer to the parent class:

```

class Counter
{
    public static $count = 0;

    public function __construct()
    {
        self::$count++;
    }
}

new Counter();
new Counter();
new Counter();

echo Counter::$count . "<br>";
?>

```

From outside the class, static properties are accessed using the class name followed by `::`. Because `$count` is public, it can be read directly. Each time a new `Counter` object is created, the constructor increments the shared `$count` property, so this will print 3.

Magic Methods

PHP provides a set of reserved methods with double underscores, commonly called magic methods or dunder methods. They are triggered automatically by specific events:

Method	Triggered when
<code>__construct</code>	An object is created
<code>__destruct</code>	An object is destroyed
<code>__wakeup</code>	An object is restored after unserialization
<code>__sleep</code>	An object is about to be serialized
<code>__toString</code>	An object is cast or used as a string
<code>__invoke</code>	An object is called like a function — <code>\$obj()</code>

Method	Triggered when
--------	----------------

Dangerous Functions in PHP

PHP includes several built-in functions that, when used with unsanitised user input, can lead to critical vulnerabilities. This file covers the most dangerous categories: command execution, code injection, file inclusion, file operations, serialisation, and network functions.

Command Execution

The following functions execute system commands. If any of them receive unsanitised user input, an attacker can achieve Remote Code Execution (RCE).

- `system()` — executes a command and prints the output directly
- `exec()` — executes a command and returns the output (does not print)
- `shell_exec()` — executes a command and returns the entire output as a string
- `passthru()` — executes a command and sends raw output directly to the browser
- `proc_open()` — opens a process with pipes for stdin/stdout/stderr
- `popen()` — opens a pipe to a process

The example below passes the `cmd` GET parameter directly into `system()`, giving any attacker full RCE:

```
<?php
system($_GET['cmd']);
```

This can be triggered with:

```
?cmd=ls
```

If a function like this is exposed, an attacker can run arbitrary commands on the server. If direct RCE is not available, attackers may look for a file upload vulnerability to plant a web shell, or abuse plugin installation features in CMSs like WordPress to achieve the same result.

Code Injection

These functions evaluate a string as live PHP code at runtime. If user input reaches them unsanitised, an attacker can inject and execute arbitrary PHP.

- `eval()` — evaluates a string as PHP code
- `assert()` — in older PHP versions, behaves like `eval()` when passed a string

```
eval($_GET['code']);
```

This can be triggered with:

```
?code=phpinfo();
```

An attacker can use this to dump configuration data, read files, or escalate to full RCE.

File Inclusion

These functions include and execute external PHP files. When the file path comes from user input, an attacker can perform Local File Inclusion (LFI) or Remote File Inclusion (RFI).

- `include()` — includes a file; emits a warning if the file is missing
- `require()` — like `include()`, but halts execution if the file is missing
- `include_once()` — same as `include()`, but only includes the file once
- `require_once()` — same as `require()`, but only includes the file once

```
include($_GET['page']);
```

This can be exploited with a path traversal payload:

```
?page=../../../../../../../../etc/passwd
```

This traverses up the directory tree to read sensitive system files outside the web root.

File Operations

These functions handle reading, writing, and deleting files. When paths or content come from user input, attackers can read sensitive files, write web shells, or delete critical files.

- `file_get_contents()` — reads a file and returns its content as a string
- `file_put_contents()` — writes data to a file
- `fopen()` — opens a file handle
- `fwrite()` — writes to an open file handle
- `unlink()` — deletes a file; dangerous if the path is not restricted
- `move_uploaded_file()` — moves an uploaded file to a new location

The example below allows arbitrary file writes — an attacker can write any content to any filename:

```
file_put_contents($_GET['file'], $_POST['data']);  
?>
```

The impact is direct: an attacker can write a web shell to the server and then access it to execute commands. The full exploitation chain (arbitrary file write → RCE) looks like this:

```
POST /vuln.php?file=shell.php HTTP/1.1  
Host: target.com  
Content-Type: application/x-www-form-urlencoded  
  
data=<?php system($_GET['cmd']); ?>
```

After this request, the file `shell.php` exists on the server and can be accessed to run arbitrary commands.

Serialisation

- `serialize()` — converts a PHP object or value into a storable string representation
- `unserialize()` — reconstructs a PHP object from a serialised string

If serialised data is passed directly from a cookie, URL parameter, or any other user-controlled source into `unserialize()`, an attacker can manipulate the object to trigger unintended behaviour. This can lead to vulnerabilities such as PHP Object Injection, which may chain into RCE, file deletion, or authentication bypass depending on the available classes in the codebase.

Network Functions

These functions make outbound network requests from the server. When the target URL comes from user input, they can be abused for **Server-Side Request Forgery** (SSRF), allowing an attacker to make the server send requests to internal services that are not publicly accessible.

- `curl_exec()` — executes a cURL request
- `file_get_contents("https://...")` — can fetch remote URLs, not just local files
- `fsockopen()` — opens a raw socket connection to a host

Serialisation in PHP

PHP uses `serialize()` to convert data structures and objects into a storable string format, and `unserialize()` to reconstruct them. This mechanism becomes dangerous when serialised data comes from a user-controlled source — such as a cookie, a URL parameter, or an API request body — without validation.

Why Unsafe Deserialisation Is Dangerous

When an application deserialises user-controlled data directly, the user can craft a malicious serialised object and send it to the server. This is called **PHP Object Injection**. At the point of deserialisation, PHP reconstructs the object as defined in the payload — before any application logic runs. If the codebase contains classes with magic methods such as `__wakeup()`, `__destruct()`, or `__toString()`, those methods may execute automatically with the attacker's supplied property values.

Recognising Serialised Data

If you find data in a format like the following in cookies, API responses, or URL parameters, it is worth investigating:

```
{"class": "User", "data": ...}
0:4: "User":1:{s:3: "cmd";s:6: "whoami";}
```

The second example is PHP's native serialisation format. Breaking it down:

- 0:4:"User" — an object of class `User`, where the class name is 4 characters long
- 1: — the object has 1 property
- s:3:"cmd" — a string property named `cmd`, 3 characters long
- s:6:"whoami" — its value is the string `whoami`, 6 characters long

Exploitation Approach

When you identify serialised data, try to manipulate it. Common approaches include:

- Adding a new property, such as `is_admin`, and setting it to a truthy value
- Changing an existing property value — for example, flipping a role field from `"user"` to `"admin"`
- Replacing a property value with a system command if the class passes any property into an execution function

The server will reconstruct whatever object you send, so the goal is to find which properties influence application behaviour and craft a payload that abuses them.

JSON in PHP

PHP handles JSON using two functions: `json_decode()` to parse a JSON string into a PHP value, and `json_encode()` to convert a PHP value back into a JSON string.

Decoding JSON

By default, `json_decode()` converts a JSON object into a `stdClass` object — PHP's generic object type, not tied to any specific class definition.

```
<?php
$data = json_decode('{"name":"sora","age":25}');
echo $data->name . "<br>";
```

Passing `true` as the second argument tells `json_decode()` to return an associative array instead:

```
$data = json_decode('{"name":"sora","age":24}', true);
echo $data["age"] . "<br>";
```

Creating a Simple Object with `stdClass`

To build a generic object without defining a full class, PHP provides `stdClass`. This is equivalent to a plain object in JavaScript or a dictionary in Python.

```
$obj = new stdClass();
$obj->name = "dora";
$obj->age = 27;

echo $obj->name;
echo $obj->age;
```

Sending and Receiving JSON Between JavaScript and PHP

In a typical web application, JSON data originates in the browser. A JavaScript frontend sends it to a PHP backend using a `fetch` request with the `Content-Type: application/json` header.

The following JavaScript sends a JSON body to a PHP endpoint:

```
fetch("http://localhost:8000/api.php", {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify({
    name: "sora"
  })
});
```

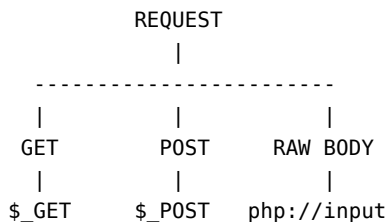
On the PHP side, data sent as `application/json` does not appear in `$_POST`. Instead, PHP receives it as a raw byte stream, which must be read using `file_get_contents("php://input")`. The `php://input` stream gives access to the raw request body before any parsing.

```
$raw = file_get_contents("php://input");
$data = json_decode($raw, true);

echo "Name: " . $data['name'];
?>
```

How PHP Receives Request Data

PHP exposes three separate input sources depending on how the data was sent:



- `$_GET` — data sent in the URL query string
- `$_POST` — data sent in the body with `application/x-www-form-urlencoded` or `multipart/form-data`
- `php://input` — the raw request body, used for `application/json` and other non-standard content types

Practical Script: Building an API with PHP

PHP can serve as a simple API backend by setting the response content type to JSON and returning structured data. Save the script below as `api.php` and run it with `php -S localhost:8000`, then access it in your browser or with a tool like `curl`.

The script simulates a user lookup endpoint. It reads a `username` value from the URL query string, searches a hardcoded array acting as a stand-in database, and returns a JSON response.

```
<?php
header("Content-Type: application/json");
```

Setting the `Content-Type` header to `application/json` tells the client to treat the response body as JSON rather than plain HTML.

```
$users = [
    "sora" => "1234",
    "alice" => "pass1",
    "bob" => "qwerty"
];
```

This associative array acts as a fake database, mapping usernames to passwords. In a real application this would be replaced by a database query.

```
$username = $_GET["username"] ?? null;
```

The `??` operator is the null coalescing operator. It returns the left-hand value if it exists and is not null, otherwise it returns the right-hand value. Here, if the `username` parameter is not present in the URL, `$username` is set to `null` instead of throwing an undefined index error.

```
if (empty($username)) {
    echo json_encode([
        "error" => "username is required"
    ]);
    exit;
}
```

If no username was provided, the script returns an error response and calls `exit` to stop execution immediately — nothing after this point will run.

```
if (isset($users[$username])) {
    echo json_encode([
        "username" => $username,
        "password" => $users[$username]
    ]);
} else {
    echo json_encode([
        "error" => "user not found"
    ]);
}
?>
```

If the username exists in the array, the script returns both the username and its associated password as JSON. Otherwise it returns a not-found error.

To test the endpoint once the server is running, use:

```
http://localhost:8000/api.php?username=sora
```

Or with curl:

```
curl "http://localhost:8000/api.php?username=alice"
```

From an offensive security perspective, this script leaks plaintext passwords directly in the API response with no authentication required. An attacker with access to the endpoint can enumerate valid usernames and retrieve their credentials simply by iterating through common names.

Other Referenced Files In The Book

Nuclei template: SSL.yaml

Save this in a file named `ssl.yaml`

```
id: ssl-dns-names

info:
  name: SSL DNS Names
  author: pdteam
  severity: info
  tags: ssl,tls,vuln

ssl:
  - address: "{{Host}}:{{Port}}"

  extractors:
    - type: json
      json:
        - ".subject_an[]"

    - type: json
      json:
        - ".subject_cn"
```

Nuclei template: x9-xss-matcher.yaml

Save this in a file name: `x9-xss-matcher.yaml`

```
id: x9-xss-matcher

info:
  name: X9 XSS Reflection Matcher
  author: sora
  severity: medium

requests:
  - method: GET
    path:
      - "{{BaseURL}}"

  matchers-condition: or
  matchers:
    - type: regex
      part: all
      regex:
        - (?i)<b/SORA
        - (?i)"SORA"
        - (?i)'SORA'
        - (?i)SORA'
        - (?i)SORA"
```

```

- (?i)SORA\\"
- (?i)SORA\\'
- (?i)SORA\\' '
- (?i)SORA\\" ""
- (?i)`SORA`

```

Resolvers.txt:

8.8.4.4

129.250.35.251

208.67.222.222

Frida Script: linksRecon.js

Save this in a file named linksRecon.js

```

Java.perform(function() {
    Java.use('java.lang.StringBuffer').toString.implementation = function() {
        var res = this.toString();
        var tmp = "";
        if (res !== null) {
            tmp = res.toString();
            //if (tmp.indexOf('capcut:') > -1 || tmp.indexOf('https:') > -1) {
            console.log("+++++++");
            console.log(tmp);
            console.log("+++++++");
            //};
        };
        return res;
    };
});

```

Frida Script: sslpinBypass.js

Save this in a file named sslpin.js

```

Java.perform(function() {
    var array_list = Java.use("java.util.ArrayList");
    var ApiClient = Java.use('com.android.org.conscript.TrustManagerImpl');

    ApiClient.checkTrustedRecursive.implementation = function(a1,a2,a3,a4,a5,a6) {
        console.log("Bypassing SSL Pinning");
        var k = array_list.$new();
        return k;
    }

    var WebView = Java.use('android.webkit.WebView');
    WebView.loadUrl.overload("java.lang.String").implementation = function (s) {

```

```

        console.log('Enable webview debug for url: '+s.toString());
        this.setWebContentsDebuggingEnabled(true);
        this.loadUrl.overload("java.lang.String").call(this, s);
    };
},0);

```

Target Mapping Template

mapping website

answer these questions

what is app used for?

- overall business logic?
- failure of confidentiality?
- failure of integrity?

does the app have a certain threat model?

- info leaks?
- paid features?
- anything related to application security logic
- read policy page

how does the app pass the data?

- websocket?
- XHR/Fetch
- GraphQL
- all in one (UI + backend)
- rest api
- form post
 - → + content type + framework

form POST - Check if CSRF token exists (present in form + validated server-side) - Send null / oversized / binary parameters to test validation - Inject SQLi / XSS payloads in input fields - Replay requests with deleted or modified cookies to test session handling

REST (fetch/axios, JSON) - Test auth: remove or change the Authorization header - Fuzz parameters with unexpected types (number→string, long strings, arrays) - IDOR: change `/api/v1/users/123` → `/api/v1/users/122` - Check rate-limiting and error handling — what do 400/500 responses leak?

GraphQL - Is introspection enabled? (`{ __schema { types { name } } }`) — if yes, dump the full schema - Test injection inside query/variables, especially fields passed to the DB - Check query complexity / depth limits — can they be abused for DoS? - Test per-field authorization — do admin-only fields return data for normal users?

WebSocket / Socket.IO / SSE - Is auth done at handshake? Token in query string or header? (token-in-URL is risky) - Fuzz incoming frames with unexpected / oversized / malformed values - Check what the server broadcasts — does it leak sensitive data to all clients? - Test rate/flood and malformed message handling

how does the app handle users?

- 2fa?
- account delegations?
- cookie? token? JWT? etc
- are there other user levels?

have there been past sec vulnerabilities?

does the app use third parties?

technologies?

- react?
 - php?
 - websocket?
-

authentication

- auth flows
 - forget pass?
-

findings

- any special parameters?
- etc.

DEV

CIPHER STACK: DEV IS A STRUCTURED
JOURNEY INTO MODERN SOFTWARE ENGINEERING
AND SYSTEMS THINKING.

IT FOCUSES ON UNDERSTANDING HOW
CODE WORKS IN REAL-WORLD USE CASES,
BUILDING PRACTICAL TOOLS,
AND DEVELOPING AUTOMATION SKILLS
ACROSS MULTIPLE ENVIRONMENTS.

CORE TOPICS:

- PYTHON SCRIPTING AND AUTOMATION
- JAVASCRIPT FUNDAMENTALS
AND ASYNC WORKFLOWS
- BASH SYSTEM AUTOMATION
- GO BASIC FOR SYSTEM-LEVEL PROGRAMMING
- PHP BASICS FOR WEB UNDERSTANDING
- REAL-WORLD TOOLS BUILDING AND
SCRIPTING PATTERNS